



BasicallyBASIC

Ryan Podzielny

CMPS 450: Senior Project

Professor Miller

Spring 2024

Table of Contents (sections are clickable to jump to)

README.....	3
A Brief Implementation Breakdown.....	3
Directory Structure.....	4
Installation and Running Instructions.....	6
Running.....	6
Adding to CLASSPATH.....	7
Optional VSCode Extension.....	12
User Manual.....	16
Running Modes.....	16
Syntax.....	17
Data Types and Assignment.....	19
Control Structures.....	20
Built-Ins.....	23
Errors.....	26
Code-Examples.....	27
Grammar.....	36
Design.....	39
Summary.....	39
Language Package.....	44
Lexer.....	46
Parser.....	47
Abstract Syntax Tree.....	49
Interpreter.....	54
Project Structure.....	58
Data Structures.....	60
Testing.....	62
Video.....	65
Conclusion and Summary.....	66
References and Resources.....	68
Design and Research.....	68
In Source.....	69
Miscellaneous.....	70
Source Code.....	71
BasicallyBASIC.....	72
Tests.....	263
VSCode Extension.....	276

README

BasicallyBASIC is an dynamically typed interpreted programming language based on syntax and semantics of the [BASIC](#) and [QuickBASIC](#) programming languages. It includes a command-line interface that allows you to run quick executable programs on a console, as well as a fully functional file 'runner' to run code from. Meaning the application is for programmers, so a background in computer science would be a good idea to have to use BasicallyBASIC.

In BasicallyBASIC, users can define and store variables, including integers, floats, booleans, strings, and lists, and perform a variety of operations on them. Logical comparisons and control flow are supported through if-else statements, while loops, and for loops, enabling the creation of complex programs. Functions are also possible and can be defined with arguments, making it easy to structure reusable code. The language also provides several built-in functions, such as printing to the console, reading files, checking variable types, measuring string and lists lengths, and many more.

There is an error messaging system to help provide users with a better understanding of where their code went wrong. When an error occurs, the system highlights the specific location and provides a description of the issue, guiding the user toward a solution.

Also, there is a VSCode extension available to have syntax highlighting and the ability to run BasicallyBASIC from an editor. If a user is looking for a better coding experience and environment, it is recommended that they use the extension so they can more easily debug and use the language.

More information on how to create valid code within BasicallyBASIC can be found within the User Manual section. It highlights the grammar design with Extended Backus-Naur Form, data types and variable assignment, built-in functions, etc. Everything you need to know to create a program in BasicallyBASIC can be found there. An installation guide can also be found under the Installation section.

A Brief Implementation Breakdown

Before getting started, here is some important information on how the underlying program works. This is to give a better understanding of why things are structured the way they are, and to hopefully fill you in on the basics of creating an interpreted language. It will hopefully give you some information to start with so you aren't completely dark. To start, the language consists of four major parts: the lexer, the parser, the abstract syntax tree and the Interpreter.

The lexer does what is known as [lexical analysis](#) which defines tokens for keywords, identifiers, literals and operators. Its function is to assign what are known as lexemes to their

proper token. For example, '+', '-', '*', and '/' are examples of binary operators that need to be assigned a value in order to parse.

The parser uses [recursive descent](#) which takes the tokens from the lexer and creates what is known as an [abstract syntax tree \(AST\)](#) that represents the program. This tree is derived from the language grammar ensures that the inputted user code is semantically and syntactically correct (see User Manual for more details on grammar). It is one of the most important parts of the language, as the interpreter uses the AST to perform and evaluate the operations.

Lastly, the interpreter using the generated AST, is where the code is actually evaluated. My implementation uses what is known as a tree-walking interpreter that can traverse the AST. In essence, it is the actual part of the project that is running the languages code.

Directory Structure

Now that you are caught up with what BasicallyBASIC is and are knowledgeable about its basic implementation, we need to ensure that you know where to find everything. Below is the directory structure of all the folders within the zip file:

```
├── BasicallyBasic
│   ├── prod
│   ├── src
│   │   ├── AbstractSyntaxTree
│   │   │   ├── Expressions
│   │   │   └── Statements
│   │   ├── Interpreter
│   │   ├── Language
│   │   ├── Lexer
│   │   ├── META-INF
│   │   └── Parser
│   └── test
│       ├── Cases
│       ├── Demo
│       └── Docs
├── External Libraries
├── Installation
├── VSCode Extension
│   ├── out
│   └── syntaxes
```


To start from the top, the ``BasicallyBasic`` directory holds all the code pertaining to the project, have three main directories within it. The ``prod`` folder contains the ``.jar`` file and is the actual production code that you can run. The ``src`` directory contains all the source code of the project, the directories inside of it are Java packages that contain related code close together for better file management. The sub-directory ``test`` holds all the testing code of the language, including the test cases to prove the language validity, the demo code from the video, and the examples found within the User Manual section.

Moving onto ``External Libraries``, it contains only one library that was needed to implement BasicallyBASIC. The library inside of it is used for Java's BigDecimal exponentiation, as Java's Number classes do not natively support it – see the Design section if curious about its implementation.

Moving slightly out of order, the ``VSCode Extension`` folder has all the source files relating to how the VSCode extension for BasicallyBASIC works. It contains a bunch of ``.json`` files and a JavaScript file - basically it's a bunch of settings to incorporate the BasicallyBASIC syntax into VSCode.

Lastly, the ``Installation`` directory contains all the files needed to actually run BasicallyBASIC. More notably, it contains the ``.jar`` file from ``prod`` and ``.vsix`` file to install the VSCode extension. Everything within this directory is for installing and running the program.

Now that you know where to find everything, the documentation below contains every specific installation instructure, language feature, and project implementation detail that you'll ever need to know. Each major section starts on its own page and can be found within the Table of Contents, so it should be clear on where to find a specific piece of information.

Installation and Running Instructions

Installing and using BasicallyBASIC is quite simple. Nothing serious is needed besides access to a console like macOS' Terminal, or Windows' Command Prompt and Java. A Java 20 or above instance is **needed** for it to run, which can be downloaded from Oracle's website [here](#).

After downloading and unzipping, you will be met with the following folders and files: BasicallyBasic, External Libraries, VSCode Extension, **Installation**, Grammar.txt, BasicallyBASICDocs.pdf and icon256x256.png. The highlighted ``Installation`` directory is what we are currently interested in. It contains two files: ``BasicallyBasic.jar`` and ``bb-1.0.0vsix``. The ``.jar`` is how will be running the program - you can keep it within this folder or copy it to another directory for ease of access.

Running

After installing Java 20 and locating the ``BasicallyBasic.jar``, we can run the program by opening a Terminal/Command-Prompt instance and run in two modes: from the command line interface (CLI) or running from a file. Both modes have the same capabilities, with the CLI being an interactive interface for quick calculations while running from a file lets you evaluate large chunks of code at once.

CLI Interpreter

To run from the command line and have your very own read evaluate print loop (REPL) you can type the following within the working directory of the ``.jar`` file:

```
`java -jar BasicallyBasic.jar`
```

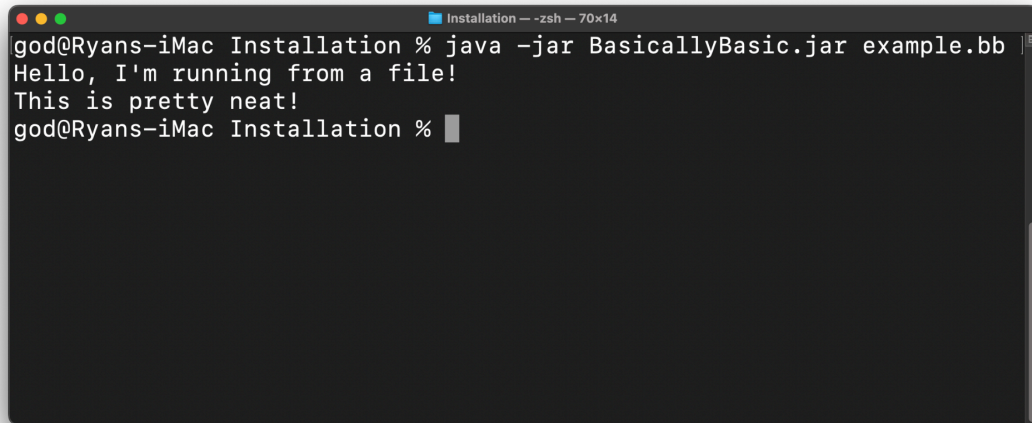
```
god@Ryans-iMac Installation % java -jar BasicallyBasic.jar
>>> 1 + 1
2
>>> (20 + 2) * 7
154
>>> LET duck = 20
20
>>> duck
20
>>> EXIT
god@Ryans-iMac Installation %
```

At any point you'd like to exit, simply press ``CTRL+C`` or type ``EXIT``.

Running a File

If you want to run a file, simply add an argument afterwards to the path of the file you want to run. This file **must** end with a `.bb`` extension, or it will not run. Unlike the CLI, it will not output any resulting value, only print statements which can be found in the language. For example, I added a simple `example.bb`` file within the same directory as the `.jar`` to run it:

```
`java -jar BasicallyBasic.jar example.bb`
```

A screenshot of a macOS terminal window titled "Installation -- zsh -- 70x14". The prompt is "god@Ryans-iMac Installation %". The user enters the command "java -jar BasicallyBasic.jar example.bb". The output is "Hello, I'm running from a file!" followed by "This is pretty neat!". The prompt returns to "god@Ryans-iMac Installation %".

```
god@Ryans-iMac Installation % java -jar BasicallyBasic.jar example.bb
Hello, I'm running from a file!
This is pretty neat!
god@Ryans-iMac Installation %
```

Adding to CLASSPATH

Optionally you can add the `.jar`` file to your CLASSPATH as an alias in both Unix based systems (Linux/macOS) and Windows for easier access to the language. This will allow you to run a BasicallyBASIC instance in your current working directory. If you do add the program to your CLASSPATH, it should be stored within a good location. For macOS, I recommend somewhere within your `Library`` directory, or `bin``, and for Windows storing within your `Program Files`` directory might be a good idea.

Unix Instructions

I will be going over Unix specifically under macOS as I do not have access to a Linux machine, however the setup will likely be identical. For macOS, their Terminal runs the Zsh shell, and requires access to the `~/ .zshrc`` profile (bash does something similar).

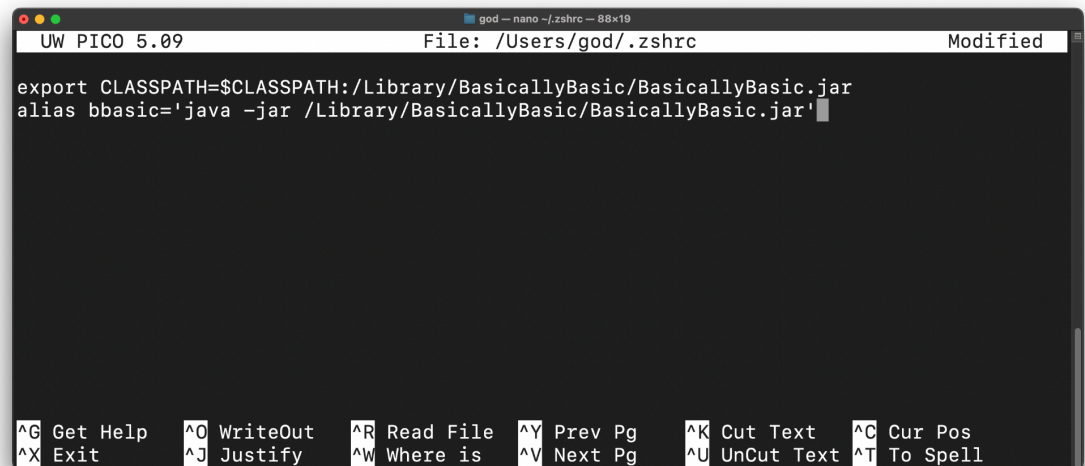
- I. Type ``nano ~/.zshrc`` to open the profile



- II. Enter the following within the file:

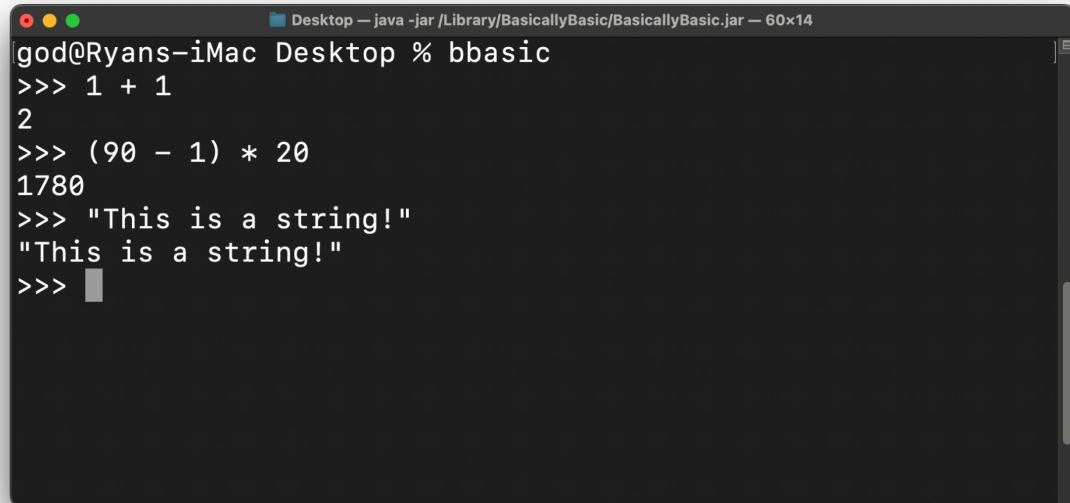
```
export CLASSPATH=$CLASSPATH:/path/to/.jar  
alias bbasic='java -jar /path/to/.jar'
```

Replacing `/path/to/.jar` to the actual path and name of the `.jar` file you've decided on. For me, I store it within my `/Library/BasicallyBasic` directory as its safe place. The alias name can be whatever you want, however it is **required** to be `bbasic` in order to run within VSCode.



- III. Press ``CTRL+X`` to exit, and hit ``y`` to save. Then relaunch your terminal and you should be set to run!

Once added to your CLASSPATH and made to an alias, we can run it from Terminal simply by typing ``bbasic`` to enter the CLI or ``bbasic <file path>`` to run from a file anywhere:

A screenshot of a macOS Terminal window. The title bar reads "Desktop — java -jar /Library/BasicallyBasic/BasicallyBasic.jar — 60x14". The prompt is "god@Ryans-iMac Desktop %". The user has entered "bbasic" and the prompt changes to ">>>". The user enters "1 + 1" and the output is "2". The user enters "(90 - 1) * 20" and the output is "1780". The user enters "This is a string!" and the output is "This is a string!". The prompt ">>>" is followed by a cursor.

```
god@Ryans-iMac Desktop % bbasic
>>> 1 + 1
2
>>> (90 - 1) * 20
1780
>>> "This is a string!"
"This is a string!"
>>> 
```

Windows Instructions

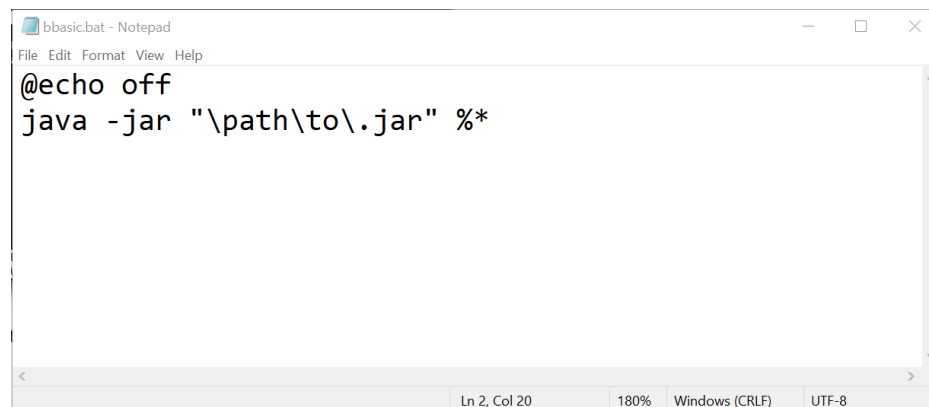
Windows is a completely different process than Unix, same result but twice the amount of steps.

- I. First open notepad, or any text editor, and type the following in it:

```
@echo off
```

```
java -jar "C:\path\to\.jar" %*
```

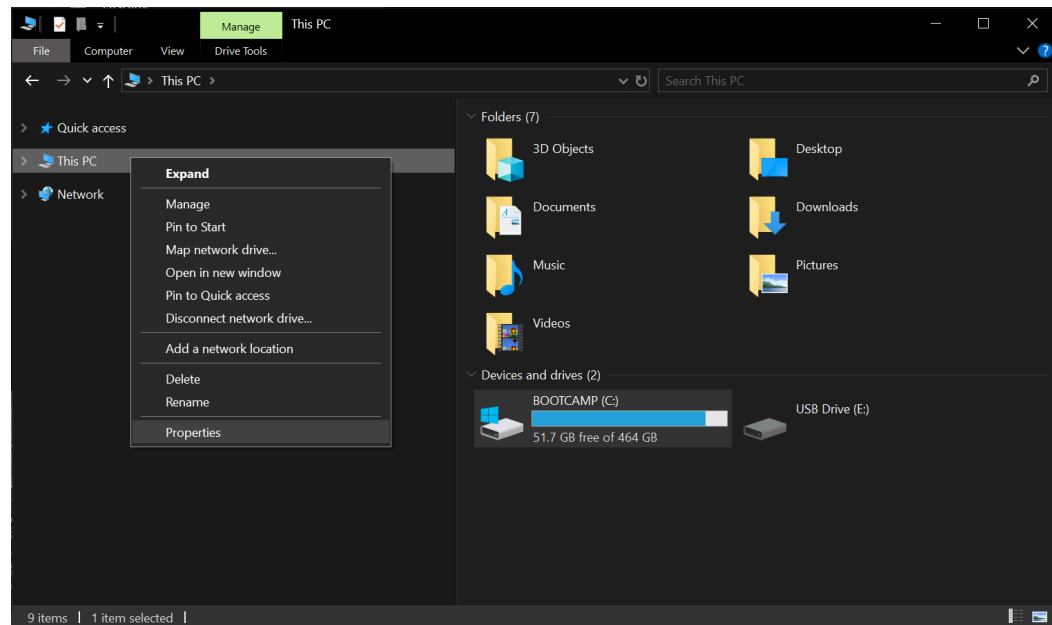
Replacing `\path\to\.jar` to the actual path and name of the `.jar` file you've decided on. For me, I store it within my `\Program Files\BasicallyBasic` directory.

A screenshot of a Notepad window titled "bbasic.bat - Notepad". The menu bar shows "File", "Edit", "Format", "View", and "Help". The text content is "@echo off" followed by "java -jar \"path\to\.jar\" %*" on the next line. The status bar at the bottom shows "Ln 2, Col 20", "180%", "Windows (CRLF)", and "UTF-8".

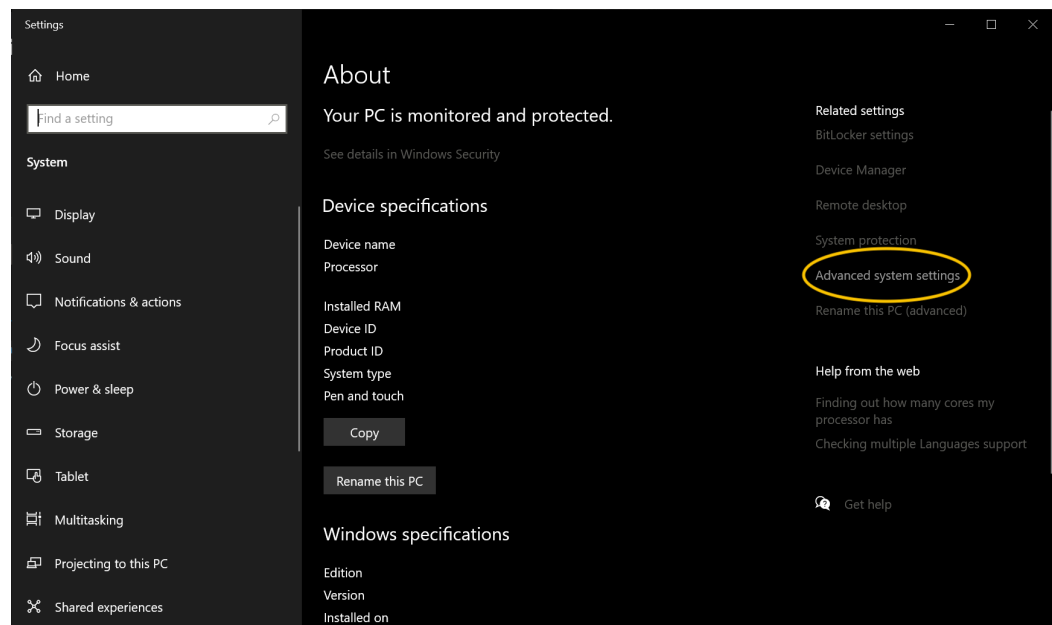
```
bbasic.bat - Notepad
File Edit Format View Help
@echo off
java -jar "path\to\.jar" %*
Ln 2, Col 20 180% Windows (CRLF) UTF-8
```

- II. Save the file as `bbasic.bat` - it can be any name with a `.bat` extension, but it is **required** to be `bbasic` in order to run within VSCode.

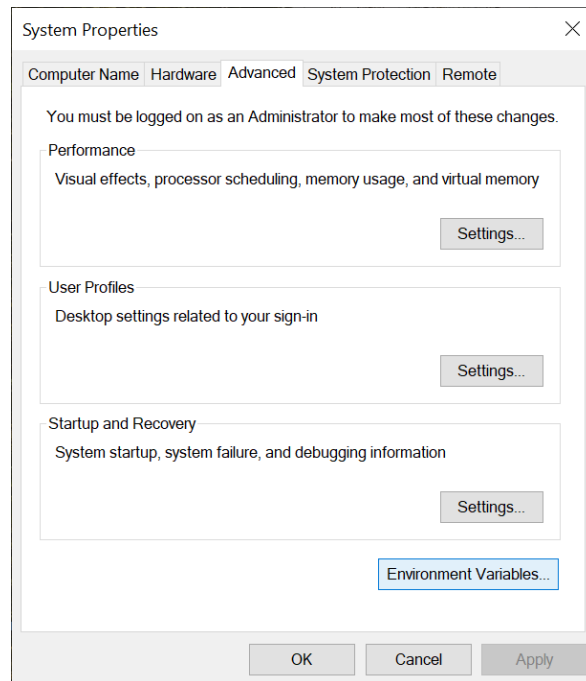
III. Open your window explorer and right click on **This PC** and hit **Properties**



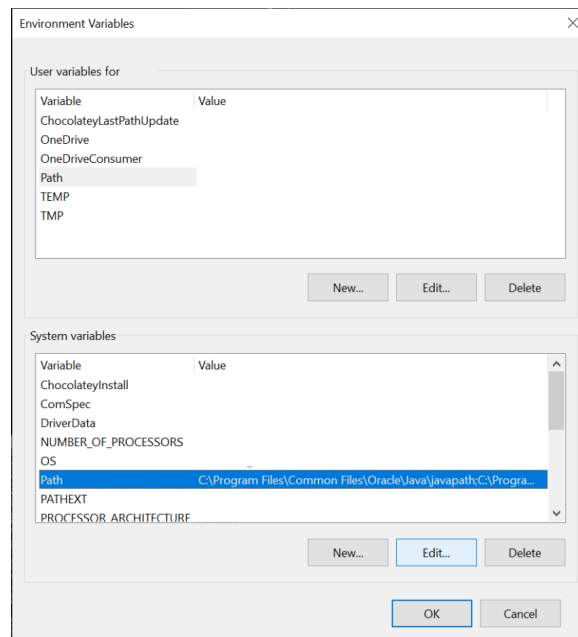
IV. This will open your system settings, from here hit **Advanced system settings** on the right (indicated by the yellow circle)



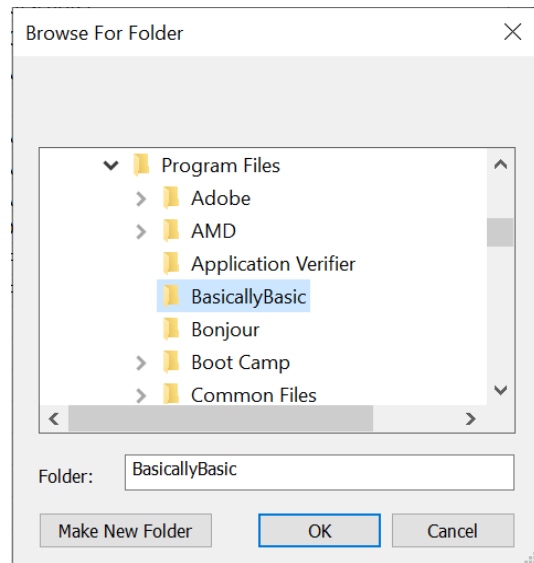
- V. This will open up a system properties window, from here hit `Environment Variables`



- VI. From here, under the `System variables` section, press `Path` and hit `Edit`



- VII. On the right side hit `Browse` and select the directory that contains the `.bat` file and hit `OK`. Again, I stored it within `\Program Files\BasicallyBasic`, but yours could be different.



Once added to your `CLASSPATH`, we can run it from anywhere with Command Prompt by simply typing ``bbasic`` to enter the CLI or ``bbasic <file path>`` to run from a file anywhere:

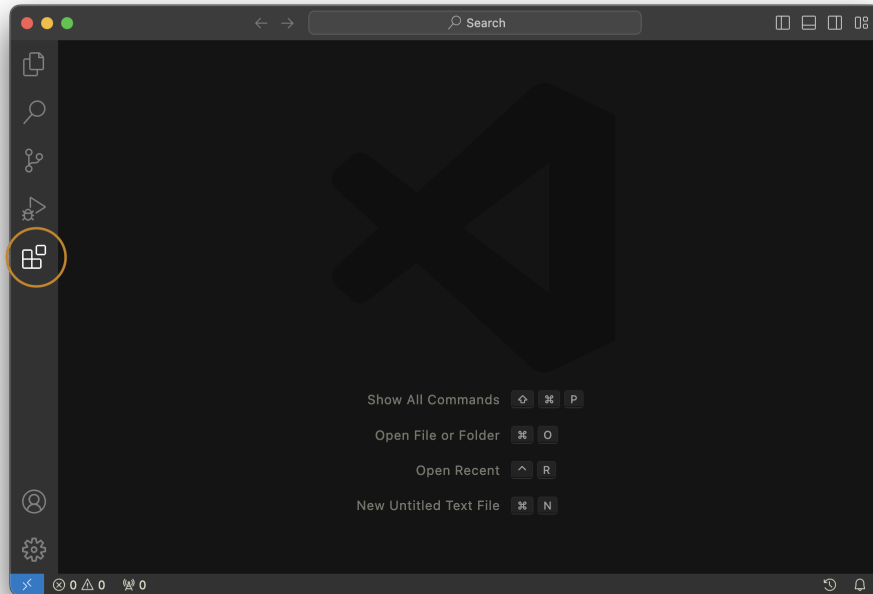
```
Command Prompt - bbasic
C:\Users\rypod\Desktop>bbasic
>>> 1 + 1
2
>>> "Running from the CLASSPATH in Windows!"
"Running from the CLASSPATH in Windows!"
>>> _
```

Optional VSCode Extension

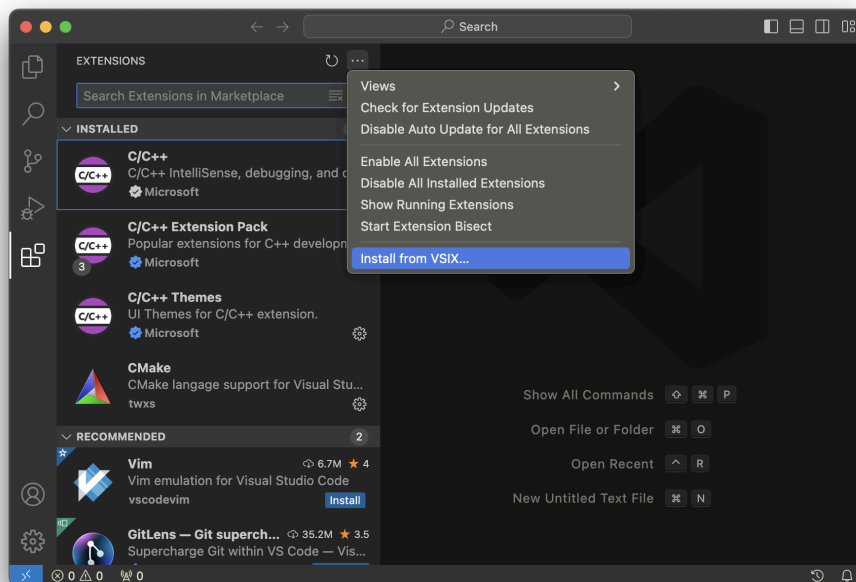
If you would like syntax highlighting and to be able to run from an editor (specifically VSCode), I made an extension that does just that! It is important to note, you **need** to have put the ``BasicallyBasic.jar`` file within your `CLASSPATH` and **must** have aliased it as ``bbasic`` or you will not be able to run files! It should be pretty obvious that you'll need VSCode for this to work; a download and installation guide can be found [here](#). There's no difference between Unix and Windows, so all steps will be the same for the operating system you are on.

Under the same directory `Installation` from the zip, there is a file called `bb-1.0.0.vsix`, make sure you know where the path is and then open up your VSCode and follow the instructions below:

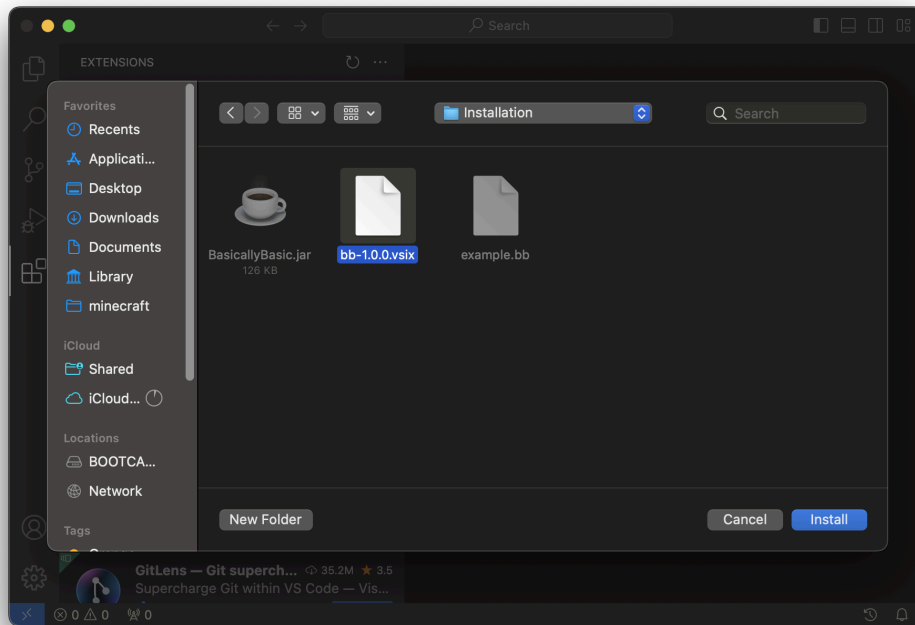
- I. Once opened, open the extensions tab on the left hand side (highlighted by the yellow circle)



- II. In the extensions menu, hit the three dots to the right and press `Install from VSIX`



- III. This will open up a window explorer, which you will need to navigate to the `bb-1.0.0.vsix` from the `Installation` directory, and hit install on the file. And that's it! It's really that simple.



Once installed, any `.bb` file extension will have syntax highlighting based on your theme of choice, and is able to be run directly from the VSCode editor! See below:

Code Without Syntax Highlighting

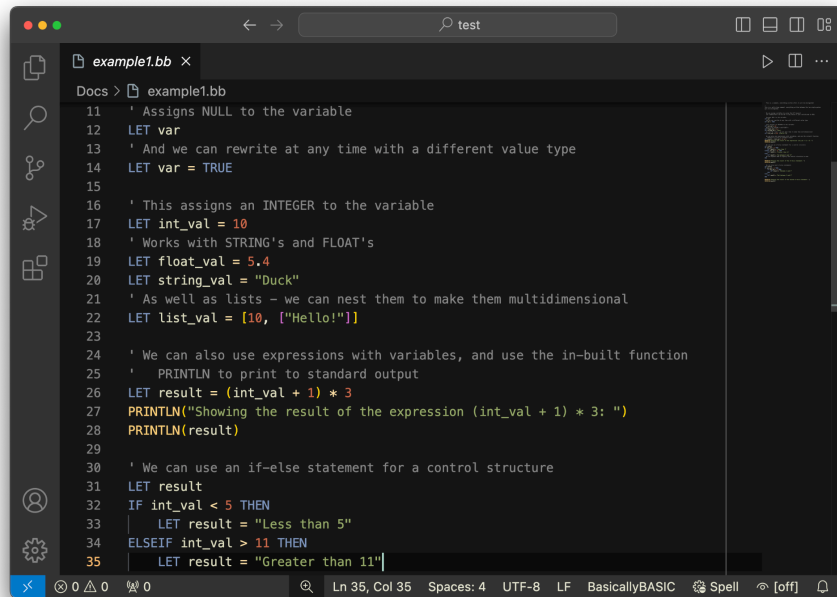
```
example1.bb - Edited
' Assigns NULL to the variable
LET var
' And we can rewrite at any time with a different value type
LET var = TRUE

' This assigns an INTEGER to the variable
LET int_val = 10
' Works with STRING's and FLOAT's
LET float_val = 5.4
LET string_val = "Duck"
' As well as lists - we can nest them to make them multidimensional
LET list_val = [10, ["Hello!"]]

' We can also use expressions with variables, and use the in-built function
' PRINTLN to print to standard output
LET result = (int_val + 1) * 3
PRINTLN("Showing the result of the expression (int_val + 1) * 3: ")
PRINTLN(result)

' We can use an if-else statement for a control structure
LET result
IF int_val < 5 THEN
    LET result = "Less than 5"
ELSEIF int_val > 11 THEN
    LET result = "Greater than 11"
```

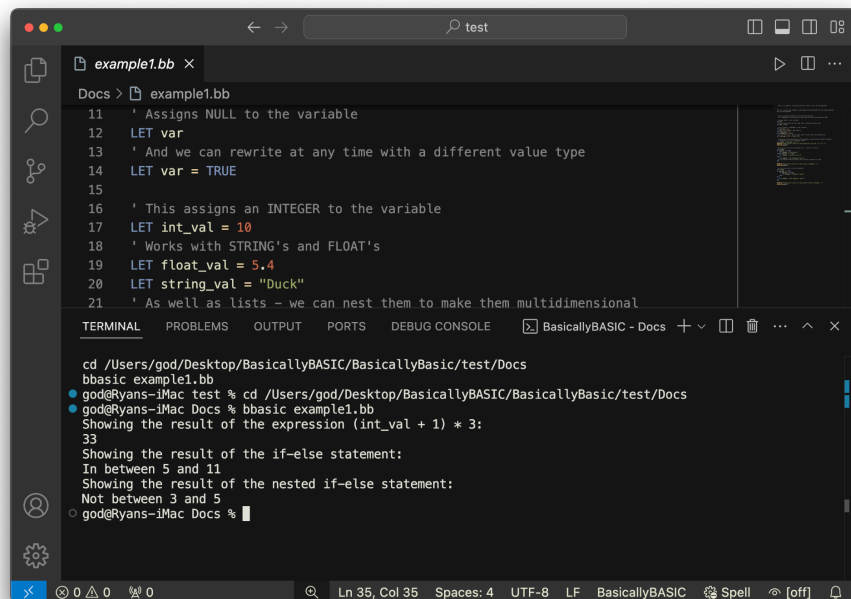
Code With Syntax Highlighting



The screenshot shows a code editor window titled 'example1.bb'. The code is written in BASIC and includes comments explaining various features like variable assignment, data types (INTEGER, STRING, LIST), and control structures (if-else). The code is syntax-highlighted, with keywords in blue, strings in red, and numbers in green. The editor has a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The status bar at the bottom shows 'Ln 35, Col 35', 'Spaces: 4', 'UTF-8', 'LF', 'BasicallyBASIC', 'Spell', and '[off]'.

```
11 ' Assigns NULL to the variable
12 LET var
13 ' And we can rewrite at any time with a different value type
14 LET var = TRUE
15
16 ' This assigns an INTEGER to the variable
17 LET int_val = 10
18 ' Works with STRING's and FLOAT's
19 LET float_val = 5.4
20 LET string_val = "Duck"
21 ' As well as lists - we can nest them to make them multidimensional
22 LET list_val = [10, ["Hello!"]]
23
24 ' We can also use expressions with variables, and use the in-built function
25 ' PRINTLN to print to standard output
26 LET result = (int_val + 1) * 3
27 PRINTLN("Showing the result of the expression (int_val + 1) * 3: ")
28 PRINTLN(result)
29
30 ' We can use an if-else statement for a control structure
31 LET result
32 IF int_val < 5 THEN
33   LET result = "Less than 5"
34 ELSEIF int_val > 11 THEN
35   LET result = "Greater than 11"
```

And notice the Run button at the top right? It actually works!



The screenshot shows the same code editor window, but now the terminal output is visible at the bottom. The terminal shows the command 'cd /Users/god/Desktop/BasicallyBASIC/BasicallyBasic/test/Docs' and the execution of 'bbasic example1.bb'. The output of the program is displayed, showing the result of the expression (int_val + 1) * 3, the result of the if-else statement, and the result of the nested if-else statement.

```
cd /Users/god/Desktop/BasicallyBASIC/BasicallyBasic/test/Docs
bbasic example1.bb
god@Ryans-iMac test % cd /Users/god/Desktop/BasicallyBASIC/BasicallyBasic/test/Docs
god@Ryans-iMac Docs % bbasic example1.bb
Showing the result of the expression (int_val + 1) * 3:
33
Showing the result of the if-else statement:
In between 5 and 11
Showing the result of the nested if-else statement:
Not between 3 and 5
god@Ryans-iMac Docs %
```

Once again, the VSCode extension for syntax highlighting is completely optional. But it makes the experience when coding in BasicallyBASIC 100 times better. Not only do we get syntax highlighting and running code snippets easily, it also allows you to use an editor attached for easier file management, better debugging and just a generally better experience.

User Manual

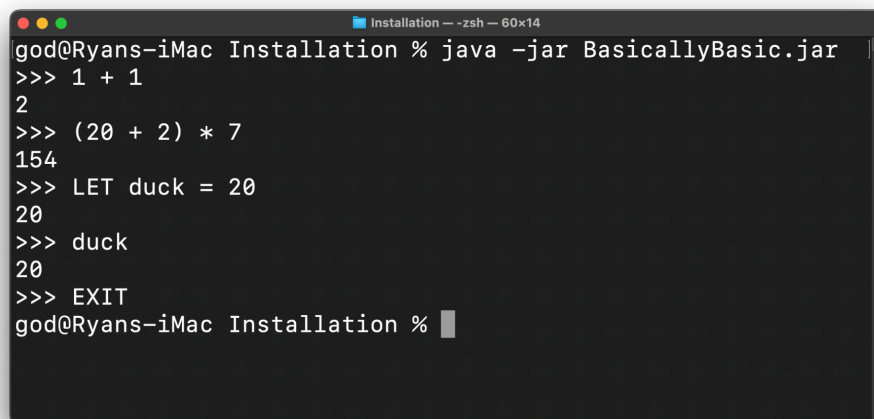
Now that you've installed BasicallyBASIC and can run the program, learning how to code in the syntax of the language is the next big step. In this section, everything you need to know from the syntax to grammar semantics can be found here. Specifically I'll retouch on running from the CLI and from a file, and look at the syntax, data types, control structures, built-in functions, errors, some code examples and the grammar of the language, so you know how to code in BasicallyBASIC.

It is quite lengthy, but just like any real language every aspect of it needs to be very well documented - with all specifics laid out in plain text. Users must be able to know how each aspect works, and how to actually use it effectively.

Running Modes

This is a **rehash of the Installation step** of how to run from the command-line interface or from a file, so if you've already read that part you can skip this part here. But just in case you forgot, there are two types of ways you can run and evaluate code. The first is from the command line. You do this by running the `.jar` file (from the alias in CLASSPATH or from `java -jar BasicallyBasic.jar`) and add no additional arguments. This will open up an interpreter state so you can enter expressions or statements as you wish. If you want to run from file, simply add the path as the first argument and hit enter. Below are screenshots to give a better understanding:

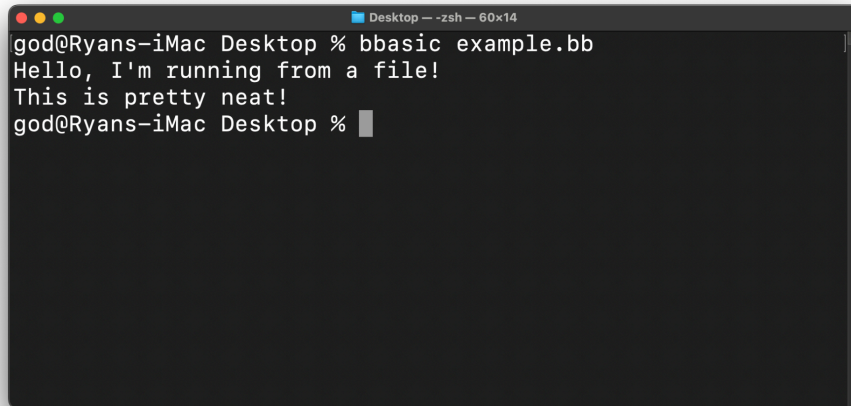
From the CLI

A screenshot of a terminal window titled "Installation - zsh - 60x14". The prompt is "god@Ryans-iMac Installation %". The user has entered the command "java -jar BasicallyBasic.jar". The terminal shows an interactive prompt ">>>". The user enters "1 + 1", and the terminal outputs "2". The user enters "(20 + 2) * 7", and the terminal outputs "154". The user enters "LET duck = 20", and the terminal outputs "20". The user enters "duck", and the terminal outputs "20". The user enters "EXIT", and the terminal outputs "god@Ryans-iMac Installation %".

```
god@Ryans-iMac Installation % java -jar BasicallyBasic.jar
>>> 1 + 1
2
>>> (20 + 2) * 7
154
>>> LET duck = 20
20
>>> duck
20
>>> EXIT
god@Ryans-iMac Installation %
```

The CLI will evaluate the expression and output the result back to you, and at any point you'd like to exit, simply press `CTRL+C` or type `EXIT`.

From a File

A terminal window titled "Desktop -- zsh -- 60x14" shows a user at the prompt "god@Ryans-iMac Desktop %". The user enters the command "bbasic example.bb". The program outputs two lines: "Hello, I'm running from a file!" and "This is pretty neat!". The prompt returns to "god@Ryans-iMac Desktop %".

```
god@Ryans-iMac Desktop % bbasic example.bb
Hello, I'm running from a file!
This is pretty neat!
god@Ryans-iMac Desktop %
```

Unlike the CLI it doesn't output the result of the expression back to you, rather executes and completes. To get output back to you you can use a print statement, like I did above.

Syntax

Below is a list of all the keywords, operators, punctuation, etc. that make up the language. They are the building blocks of how you make your code, just like any other language.

In terms of semantics BasicallyBASIC uses infix notation, with a left-to-right associativity for operators. The precedence of the operators can be found within the Grammar section, but in general it follows the C/C++ precedence table for the shared operations, which can be found [here](#).

Listed below is the syntax, in no particular order:

- Operators:
 - '+' - Addition operation (can add numbers to numbers, strings to strings, and lists to lists)
 - '-' - Subtraction operation
 - '*' - Multiplication operation
 - '/' - Division operation
 - '%' - Modulus operation
 - '^' - Power operation for exponents
 - '(' and ')' - Precedence operators and for argument holding in functions
 - '=' - Assignment operation
- Comparison Operators:
 - '==' - Equality comparison
 - '!=' - Not equal comparison

- `>` - Greater than comparison
- `>=` - Greater than or equal comparison
- `<` - Less comparison
- `<=` - Less than or equal comparison
- Logical Operators:
 - `AND` - Returns `TRUE` if both operands are true, `FALSE` otherwise
 - `OR` - Returns `TRUE` at least one operand is true
 - `NOT` - Returns `TRUE` if operand is false, `FALSE` if operand is true
- Punctuation:
 - `;` - Separates statements made on the same line (newline alternative)
 - `''` - Used to create a comment, code that will not be interpreted. If doubled up, it will create a multi-line comment, where it will end with another doubled up single quote.
 - `,` - Used to separate elements in a list and arguments in function definition
 - `.` - Used to create a floating point value
 - `[` and `]` - Used to create a list type
 - `:` - List/String accessor for an element at a certain index
 - `|` - List/String ranged access for an element (end element exclusive)
 - `""` - Used create a string type
- Keywords:
 - `LET` - Used to create variables and reassign their values
 - `IF` - Control structure depending of if comparison is true of false
 - `ELSEIF` - Can be added on to `IF` in order to avoid nested `IF` blocks
 - `ELSE` - Ending capture group to `IF`, if comparison results in false
 - `FOR` - Used to create for loop to execute multiple lines of code
 - `TO` - Acts as the range operator in `FOR`
 - `STEP` - Acts as how much a variable should iterate in `FOR`
 - `WHILE` - Used to create a while loop base on a certain comparison
 - `CONTINUE` - Goes to next iteration in `FOR` or `WHILE` loops
 - `BREAK` - Break out of entire loop for `FOR` or `WHILE`
 - `THEN` - Acts as a control word to execute code after a control structure
 - `DEF` - Used for function creation control structure
 - `RETURN` - Returns a value in a function
 - `END` - Ends a control structure or function when using multiline groups, like `FOR`, `WHILE`, `IF` blocks, `DEF`, etc.
- Built-In Values:
 - `TRUE`: Boolean value for true
 - `FALSE`: Boolean value for false

- ``NULL``: No value

Data Types and Assignment

There are seven data types in total: INTEGERS, FLOATs, BOOLEANs, STRINGs, LISTs, FUNCTIONs, and NULLs. Basically BASIC is not strongly/statically typed, meaning it uses dynamic typing in order to facilitate function.

To ensure correct usage of types, arithmetic operators, such as subtraction, division, modulus, etc. can only be used on number types (INTERGERS and FLOATs), with the only exception being addition which can be used on LISTs and STRINGs. Using addition on two lists will extend them, e.g. ``[1, 2] + [3, 4]`` becomes ``[1, 2, 3, 4]``. For STRINGs it does string concatenation, e.g. ``"Hello, " + "World!"`` becomes ``"Hello, World!"``. If you try to add two wrong types together, a type error will occur.

Implicit type conversion is only possible with INTEGERS and FLOATs. If an INTEGER is added to a FLOAT it will result in a float, however if an INTEGER is added to an INTEGER it will result in an INTEGER. For example, ``1 + 2.0`` becomes ``3.0``, while ``1 + 2`` becomes ``3``. In order to convert to a specific type, you'll need to use a built-in function: ``TO_<type>(<value>)``, which will throw an error if it cannot convert.

All types are also immutable, meaning that once created their original values can never be changed. This goes against most interpreted languages, where lists are usually mutable, but for the sake of simplicity and less confusion I decided that we will always return a copy when accessed for everything.

Types

- Numbers:
 - INTEGER - Any whole number, e.g. 10. No integer limit, can be as big as your memory for all you wish, so 290832038942830890 is completely valid.
 - FLOAT - Any whole number with one '.', e.g. 10.1. No upper bound or lower bound limit like INTEGERS, but all decimals round off at 16 decimal places because it is impossible to represent irrational numbers.
- Booleans:
 - TRUE - Boolean representation for true, uses ``TRUE`` keyword
 - FALSE - Boolean representation for false, uses ``FALSE`` keyword
- Composite:

Composite types are elements that can be indexed and subscripted. Accessing elements within composite types is done with the ``:`` and uses zero based indexing.

They can also be ranged accessed to get a substring or sublist with the ``|`` operator [check code examples for more details]. If the index is out of range it will throw an out of bounds error.

- STRING - A sequence of characters surrounded by `"`, e.g. `"DUCK"`
- LIST - A structure that contains elements, e.g. `[10, 20, "Duck"]`. These elements can be of any type, including another list. On creation the elements are surrounded by ``[`` and ``]``, and separated by ``,``.
- Functions:
Functions are types as most dynamically typed languages support it, so I felt that BasicallyBASIC should too. In essence, functions are callable blocks of code.
 - Built-ins - Functions that are a part of the language, e.g. ``PRINT``
 - User-Defined - Functions that a user defines, e.g. ``DEF work(<optional parameters>)``
 - They always return a value, when called. Even without a return statement or in a function like ``PRINT``, it'll return ``NULL`` in its place.
- No Value:
 - Represented with the ``NULL`` keywords, for when a user doesn't have a value to assign

Assignment

Assigning variables to these data types is done with the ``LET`` keyword. Specifically, ``LET`` must be followed by an identifier then a ``=`` operator and then the value. If it isn't we assume it has a ``NULL`` value. On reassignment the keyword ``LET`` must be used again. The only data type that cannot be declared with ``LET`` are functions, they use the ``DEF`` keyword to define them.

Listed plainly:

```
' Assigning with a value
LET <identifier> = <value>

' For a string:
LET <identifier> = "<value>"

' For a list:
LET <identifier> = [<value1>, <value2>, ...]

' Assigning with NULL
LET <identifier>
LET <identifier> = NULL
```


Control Structures

There are three types of control structures within the language: conditionals, loops and functions. Conditionals, as the name implies, handle comparisons and are implemented with if-else blocks. Loops, once again as the name implies, executes code either as a range iteration with for loops or conditional iteration with while. Loops can also use the keyword ``BREAK`` or ``CONTINUE`` to exit out of the control structure and go to the next iteration respectively. Functions are simply a block of code that we can call at a later date with parameters. It is encouraged that you consult the Grammar section for more specifics.

Newlines ``;`` or ```\n`` are integral to providing structure to these statements. Newlines are optional and ignored after the start of a statement (e.g. after ``THEN``), at the end of a file, or right before the ``END`` keyword. They are also required to split up expressions, e.g. ``1 + 2 3 * 1`` must either have a ``;`` or a ```\n`` between the 2 and the 3 in order for the expression to be valid. For example, the two expressions above must be written as ``1 + 2; 3 * 1`` in order to make the code unambiguous and parsable. It will throw a syntax error if not.

Each keyword after the conditional or loop statement will be followed by the keyword ``THEN`` to inform the parser which code should be executed if the condition goes into that specific block. ``THEN`` is not required after a function.

Below the control structures and how they work:

- Conditional Statements: ``IF``, ``ELSEIF``, ``ELSE``

The ``IF`` keyword starts the control structure group and can be followed by any number of ``ELSEIF`` keywords and only followed by one optional ``ELSE`` keyword. Each keyword besides ``ELSE`` must have a ``THEN`` keyword to inform the interpreter which code to execute if the condition is true. And the entire structure must have an ``END`` keyword to end the control block.

Listed plainly:

```
IF <condition> THEN
    <do work>
END
IF <condition> THEN
    <do work>
ELSE
    <do work>
END
```

```

IF <condition> THEN
    <do work>
ELSEIF <condition> THEN
    <do work>
END

```

- **Loops: `FOR`, `WHILE`**

- The `FOR` control block is to iterate over a range of numbers. It must be followed by an assignment of an INTEGER (e.g. `LET i = 0`, then a `TO` keyword, then an INTEGER, and lastly an `STEP` keyword which must be followed by an INTEGER.

Listed plainly:

```

FOR <assignment> TO <number> STEP <number> THEN
    <do work>
    BREAK[optional]
    CONTINUE[optional]
END

```

- The `WHILE` control block is to iterate based on a condition. The keyword must be followed by a conditional statement, and followed by the `THEN` keyword.

Listed plainly:

```

WHILE <comparison> THEN
    <do work>
    BREAK[optional]
    CONTINUE[optional]
END

```

- **User-Defined Functions:**

Functions are used as a way to repeat code and avoid code duplication. They are defined and created using the `DEF` keyword, can have optional arguments, and can be called any time after declaration. At any point within the code block the `RETURN` followed optionally by a value. Recursion is possible simply by calling it within the block. All functions must return a value, if no return statement is present, `NULL` will be returned implicitly (this goes for in-built functions as well).

Listed plainly:

```

DEF <identifier>(<arg1>[optional], <arg2>[optional], ...)
    <do work>
    RETURN[optional] <value>[optional]

```

END

Calling a function is straightforward, simply use the identifier used to define it with any arguments that may have been stated. A function must be called with the same number of parameters that the function was defined with, i.e. their arity must be the same.

Listed plainly:

```
<identifier>(<arg1>, <arg2>, ...)
```

If a function returns a function type, we can call that return function by surrounding the original call in parentheses.

Listed plainly:

```
((<identifier>(<arg1>, <arg2>, ...))(<arg1>, <arg2>, ...))
```

Built-Ins

Functions and values that provide extra utility to the language are listed below into their respective groups. These are extremely important to the language!

Functions

Built-in functions are integral to the language, but their operations do not fit under more than one keyword or statement. Hence why they are packaged as functions. Below is a list of all the functions that are currently implemented. Listed exhaustively so you know what in-builts are available. It lists what they do, what arguments they take and what they return.

- ``PRINT(value)``
 - One argument taken
 - Prints the value to standard output
 - Returns ``NULL``
- ``PRINTLN(value)``
 - One argument taken
 - Prints the value to standard output on its own line
 - Returns ``NULL``
- ``INPUT()``
 - No arguments taken
 - Gets the input from standard input
 - Returns the inputted value as a STRING
- ``IS_INTEGER(value)``
 - One argument taken
 - Checks if the value is an INTEGER
 - Returns ``TRUE`` if is an INTEGER, ``FALSE`` otherwise
- ``IS_FLOAT(value)``
 - One argument taken

- Checks if the value is an FLOAT
 - Returns `TRUE` if is an FLOAT, `FALSE` otherwise
- ``IS_NUMBER(value)``
 - One argument taken
 - Checks if the value is an INTEGER or FLOAT
 - Returns `TRUE` if is an INTEGER or FLOAT, `FALSE` otherwise
- ``IS_STRING(value)``
 - One argument taken
 - Checks if the value is a STRING
 - Returns `TRUE` if is a STRING `FALSE` otherwise
- ``IS_BOOLEAN(value)``
 - One argument taken
 - Checks if the value is a BOOLEAN type
 - Returns `TRUE` if is `TRUE` or `FALSE`, `FALSE` otherwise
- ``IS_LIST(value)``
 - One argument taken
 - Checks if the value is a LIST type
 - Returns `TRUE` if is a LIST, `FALSE` otherwise
- ``IS_FUNCTION(value)``
 - One argument taken
 - Checks if the value is a FUNCTION type
 - Returns `TRUE` if is a FUNCTION, `FALSE` otherwise
- ``IS_NULL(value)``
 - One argument taken
 - Checks if the value is a NULL type
 - Returns `TRUE` if is a NULL, `FALSE` otherwise
- ``TO_INTEGER(value)``
 - One argument taken
 - Converts a FLOAT, STRING, or INTEGER to its respective INTEGER value, e.g. 5.5 to 5 or "6" to 6
 - Returns an INTEGER value if it can convert, or will throw a type error if it can't
- ``TO_FLOAT(value)``
 - One argument taken
 - Converts a FLOAT, STRING, or INTEGER to its respective FLOAT value, e.g. 5 to 5.0 or "6.0" to 6.0
 - Returns an FLOAT value if it can convert, or will throw a type error if it can't
- ``TO_STRING(value)``
 - One argument taken

- Converts any type to a string, e.g. 5 to “5”, 6.6 to “6.6”, [1,2] to “[1, 2]”, PRINT to “FUNCTION: ‘PRINT’”, TRUE to “TRUE” or NULL to “NULL”
 - Returns the value converted to their respective STRING
- ``APPEND(list, value)``
 - Two arguments taken, first argument must be a LIST, second can be any type
 - Adds value to the end of the list argument
 - Returns the LIST with the appended value
- ``PREPEND(list, value)``
 - Two arguments taken, first argument must be a LIST, second can be any type
 - Adds value to the front of the list argument
 - Returns the LIST with the prepended value
- ``INSERT(list, value, index)``
 - Two arguments taken, first argument must be a LIST, second can be any type, third must be an INTEGER
 - Adds value to the specific index of the list, will error if out of bounds
 - Returns the LIST with the inserted value
- ``POP(list)``
 - One argument take, must be a LIST
 - Remove the last element of the LIST
 - Returns the LIST with the removed value
- ``REMOVE(list, index)``
 - Two arguments taken, first argument must be a LIST, second must be an INTEGER
 - Removes a value from at list at the specified index
 - Returns the LIST with the removed value, will error if out of bounds
- ``EXTEND(list1, list2)``
 - Two arguments taken, both must be LISTS
 - Adds both lists together
 - Returns the combined lists
- ``LENGTH(composite)``
 - One argument taken, must be a LIST or a STRING
 - Gets the length of the composite value
 - Returns the length of the value, will error if not a STRING or LIST
- ``READ(filepath)``
 - One argument taken, must be a STRING
 - Reads the file based on the filepath
 - Returns the contents of the file as a STRING
- ``WRITE(filepath, contents)``
 - Two arguments taken, both must be STRINGs

- Write to the file based on file path, with the contents lists. Overwrites the file or creates a new one if it doesn't exist.
- Returns ``NULL``
- ``RUN(filepath)``
 - One argument taken, must be a STRING
 - Run the selected file path as a BasicallyBASIC program. Used as an import for other files.
 - Returns ``NULL``

Values

Built-in values have already been listed within the Syntax subsection, but to rehash and put them all in one place. There are 3 built-in values: ``NULL`` (no value), ``TRUE`` (boolean true) and ``FALSE`` (boolean false). Used for convenience and leads to more explicit and clean code.

Errors

Errors are an important part of every language, and BasicallyBASIC is no exception. Any time an error occurs it means that there is something wrong with the user code and needs to be fixed in order to run - halting the program in the process. They can be classified into seven different types:

- ``Syntax`` - when code doesn't match the specified grammar, e.g. not having an ``END`` keyword after control statement
- ``Type`` - when trying to perform an operation that doesn't support the specific type, e.g. adding a STRING to an INTEGER
- ``Runtime`` - a general classification of something going wrong during evaluation, e.g. a function call not having the same number of arguments as the function parameters
- ``Arithmetic`` - when an operation cannot be performed during runtime, e.g. dividing by zero
- ``Index Out of Bounds`` - when trying to access a subscript not within zero to the length of the composite value, e.g. ``[0,1,2]:10`` is not accessible
- ``Undefined Identifier`` - when a user tries to access a value that they have not declared/instantiated
- ``Built-In`` - when something goes wrong calling an in-built function, e.g. passing an INTEGER type into a function that requires a LIST type

If at any point the language detects one of these errors, it will immediately halt and show the user what type of error it was, what line it occurred on, as well as a specific description on why the language errored. Below are a few errors that I purposefully caused within the CLI. The same errors would also appear if run from a file, it is just easier to show within the console.

```
god@Ryans-iMac ~ % bbasic
>>> LET value = 10
10
>>> value + undefined
Undefined Identifier Error at line 1: Undefined identifier 'undefined'!
      1 |    value + undefined
>>>
>>> IF TRUE THEN; PRINT("HI")
Syntax Error at line 1: Expected 'END' after a conditional!
      1 |    IF TRUE THEN
>>>
>>> 10 / 0
Arithmetic Error at line 1: Cannot '/' by 0!
      1 |    10 / 0
>>>
>>> "String" + 10
Type Error at line 1: Cannot perform '+' to types 'STRING' and 'INTEGER'!
      1 |    "String" + 10
>>> █
```

Obviously, this is not an exhaustive list of all the errors that are possible, doing so would be impossible, but it paints a picture that when something goes wrong the user will know about it. Follow the grammar, and everything should work flawlessly!

Code-Examples

Now to see BasicallyBASIC in action! As we can run from both files and from the command line interface, I'll have two subsections to describe both of them. They both can perform the same functionality, one is just limited to a single line. To note: not all built-in functions in the language are used here as creating examples for all of them would be extremely exhaustive and, in my opinion, not necessary. If you are curious about what a certain built-in function does, I suggest looking at the Built-In subsection.

Running From a File

Examples from running code from a file are listed below. The output is listed under each code segment, in order of the `PRINT` statements. I also used syntax highlighting, courtesy of the VSCode Extension, to make the code stand out better.

Example 1:

This shows off variable assignment, variable types and control structures through if-else statements:

```
' This is a comment, everything written after it is disregarded
```

```

''
This is a multi-line comment, everything written between the two
single quotes will be disregarded
''

' We can assign variables by using the LET keyword
' LET <identifier> can be any value or just initialized to NULL

' Assigns NULL to the variable
LET var

' And we can rewrite at any time with a different value type
LET var = TRUE

' This assigns an INTEGER to the variable
LET int_val = 10
' Works with STRINGS and FLOATs
LET float_val = 5.4
LET string_val = "Duck"
' As well as lists - we can nest them to make them multidimensional
LET list_val = [10, ["Hello!"]]

' We can also use expressions with variables, and use the in-built
function
' PRINTLN to print to standard output
LET result = (int_val + 1) * 3
PRINTLN("Showing the result of the expression (int_val+1)*3: ")
PRINTLN(result)

' We can use an if-else statement for a control structure
LET result
IF int_val < 5 THEN
    LET result = "Less than 5"
ELSEIF int_val > 11 THEN
    LET result = "Greater than 11"

```



```

ELSE
    LET result = "In between 5 and 11"
' We use keyword END to signify the control structure is over
END

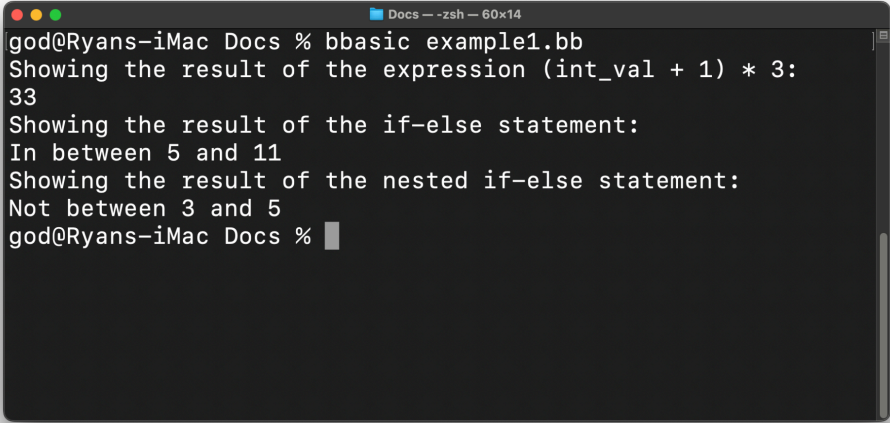
PRINTLN("Showing the result of the if-else statement: ")
PRINTLN(result)

' We can also nest if-else statements
LET result
IF int_val < 5 THEN
    IF int_val > 3 THEN
        LET result = "Between 3 and 5"
    END
ELSE
    LET result = "Not between 3 and 5"
END

PRINTLN("Showing the result of the nested if-else statement: ")
PRINTLN(result)

```

Output:



```

god@Ryans-iMac Docs % bbasic example1.bb
Showing the result of the expression (int_val + 1) * 3:
33
Showing the result of the if-else statement:
In between 5 and 11
Showing the result of the nested if-else statement:
Not between 3 and 5
god@Ryans-iMac Docs %

```

Example 2:

This example shows off lists and string indexing

```
' Like in example 1 we can assign a list to a variable
LET list_val = [1, 2, 3, 4, 5]

PRINTLN("Showing the list: ")
PRINTLN(list_val)

' We can access an element in the list by using a colon
PRINTLN("Element at index 0: ")
PRINTLN(list_val:0)
' We can also use the APPEND method to add items to a list
LET list_val = APPEND(list_val, 10)
' Remove an element from a list with the built-in method REMOVE
LET list_val = REMOVE(list_val, 0)

''

There are built-in methods for lists, and I encourage you to look at
the docs for more information on them
''

PRINTLN("Showing the list after appending/removing an element:")
PRINTLN(list_val)

' And we can also access a nested list
LET list_val2 = [1, [2, 3]]
PRINTLN("Showing the nested list access: ")
PRINT(list_val2:1:0)

PRINTLN("")
''

Strings operate on the same principle as lists for access, however
they don't have the same built-in functions
''

LET string_val = "Hello, World!"
PRINTLN("Showing the string: ")
```

```
PRINTLN(string_val)
```

```
PRINTLN("String at index 0: ")
```

```
PRINTLN(string_val:0)
```

```
''
```

We also have a ranged access operator for lists and strings, using '|'. This is an exclusive range, so `list:0|2` will return the elements at index 0 and 1

```
''
```

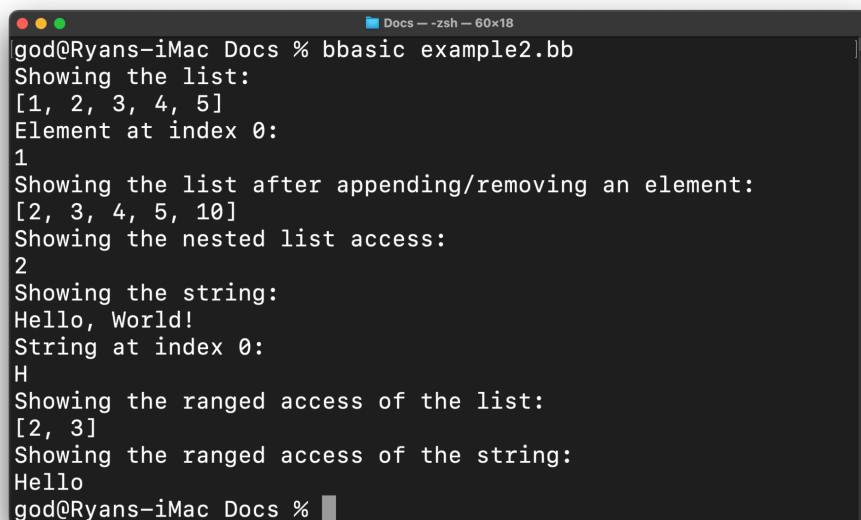
```
PRINTLN("Showing the ranged access of the list: ")
```

```
PRINTLN(list_val:0|2)
```

```
PRINTLN("Showing the ranged access of the string: ")
```

```
PRINTLN(string_val:0|5)
```

Output:

A terminal window titled 'Docs -- zsh -- 60x18' showing the execution of a program. The output is as follows:

```
god@Ryans-iMac Docs % bbasic example2.bb
Showing the list:
[1, 2, 3, 4, 5]
Element at index 0:
1
Showing the list after appending/removing an element:
[2, 3, 4, 5, 10]
Showing the nested list access:
2
Showing the string:
Hello, World!
String at index 0:
H
Showing the ranged access of the list:
[2, 3]
Showing the ranged access of the string:
Hello
god@Ryans-iMac Docs %
```

Example 3:

This example shows off for and while loops with lists

```
LET list_val = [2, 3, 4, 5, 10]
```

' We can access a lists/strings length with `LENGTH()`

```
LET length = LENGTH(list_val)
```

```

LET result = 0
''

Printing a list can be done with a for loop. We iterate from the
assignment start to the end value (after keyword TO) and step by the
value after the keyword STEP. This is inclusive of the end value, and
only only be used with INTEGERS.
''

FOR LET i = 0 TO length - 1 STEP 1 THEN
    LET result = result + list_val:i
END

PRINTLN("Showing the result of the for loop: ")
PRINTLN(result)

' We can also use a while loop to run on conditional statements
LET int_val = 10
WHILE int_val > 5 THEN
    LET int_val = int_val - 1
END

PRINTLN("Showing the result of the while loop: ")
PRINTLN(int_val)

' For loops and whiles also have the BREAK and CONTINUE keywords
FOR LET i = 0 TO 2 STEP 1 THEN
    IF (i == 1) THEN
        BREAK
    END
END

PRINTLN("Showing the result of the for loop with break:")
PRINTLN(i)

' Now for continue
LET i = 0
WHILE i < 3 THEN
    LET i = i + 1

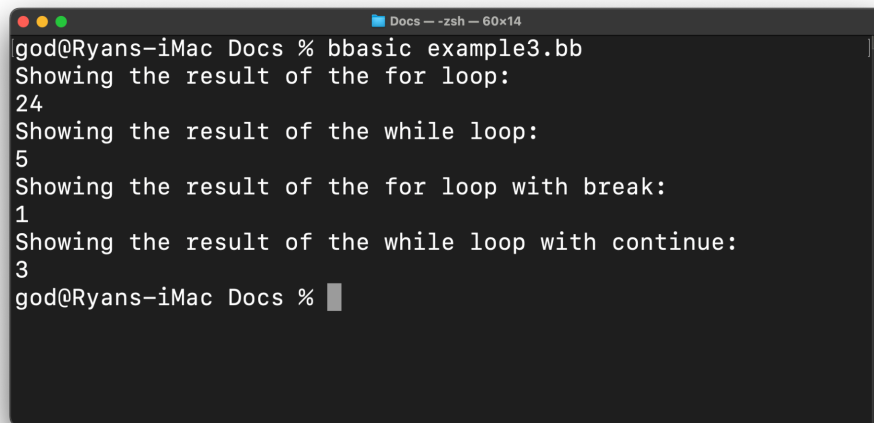
```

```

    IF (i == 2) THEN
        CONTINUE
    END
END
PRINTLN("Showing the result of the while loop with continue:")
PRINTLN(i)

```

Output:



A terminal window titled 'Docs -- zsh -- 60x14' showing the execution of 'bbasic example3.bb'. The output is as follows:

```

god@Ryans-iMac Docs % bbasic example3.bb
Showing the result of the for loop:
24
Showing the result of the while loop:
5
Showing the result of the for loop with break:
1
Showing the result of the while loop with continue:
3
god@Ryans-iMac Docs %

```

Example 4:

This example shows off functions

```

' We can use the keyword DEF to define a function
' It can contain any number of arguments separated by a comma
DEF add(value1, value2)
    ' Using return we can send the value back to caller
    RETURN value1 + value2
END

' We can use the in-built input function to get input from user
PRINTLN("Enter a INTEGER to add to 3: ")
LET in = INPUT()
' And we can call it at a later place
PRINTLN("Adding 3 to the input: ")
PRINTLN(add(TO_INTEGER(in), 3))

```

```

PRINTLN("")

' We can also use recursion
DEF countdown(value)
    ' Breakout of recursion
    IF value <= 0 THEN
        RETURN
    END

    PRINTLN(value)
    countdown(value - 1)
END

PRINTLN(value)
countdown(value - 1)
END

' We can use the TO_STRING() to convert a value to a string!
LET value = 5
PRINTLN("Counting down from: " + TO_STRING(value))
countdown(5)

PRINTLN("")

''
Using a semicolon we can break multi-line statements into one line.
These work with any statement within a file! They are treated as a
newline character, same as '\n'.
''

DEF sub(value1, value2); RETURN value1 - value2; END
PRINTLN("Subtracting 1 from 2: ")
PRINTLN(sub(1, 1))

PRINTLN("")

' Functions are treated as type and can be assigned to variables
LET func = add
PRINTLN("Using the function assigned to a variable: ")
PRINTLN(func(1, 2))

```

```
PRINTLN("")
```

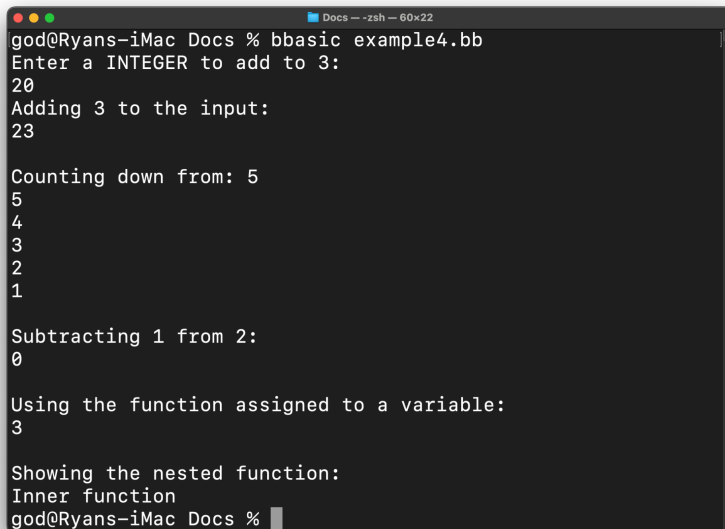
```
''
```

Lastly we can also nest functions, but they are only accessible within the scope they are defined in.

```
''
```

```
DEF outer()  
  DEF inner()  
    PRINTLN("Inner function")  
  END  
  inner()  
END  
PRINTLN("Showing the nested function: ")  
outer()
```

Output:

A terminal window titled 'Docs - zsh - 60x22' showing the execution of a program. The output is as follows:

```
god@Ryans-iMac Docs % bbasic example4.bb  
Enter a INTEGER to add to 3:  
20  
Adding 3 to the input:  
23  
  
Counting down from: 5  
5  
4  
3  
2  
1  
  
Subtracting 1 from 2:  
0  
  
Using the function assigned to a variable:  
3  
  
Showing the nested function:  
Inner function  
god@Ryans-iMac Docs %
```

Using the CLI

Everything that can be done in a file can also be done within the command line. When we type something in, which is determined by a newline, we respond with what it evaluates to. The interface will prompt you for input, shown as `>>>` and the result will be on a newline below it.

Example of evaluating expression, using variables and conditional statements

```
god@Ryans-iMac Docs % bbasic
>>> ' Evaluating expressions and creating/using values
>>> (1 + 2) * 10
30
>>> LET value = 10
10
>>> LET list = [10, 20, 30]
[10, 20, 30]
>>> APPEND(list, value)
[10, 20, 30, 10]
>>>
>>>
>>> ' Using if-else statements
>>> IF 10 > 5 THEN 9 END
9
>>> IF 10 == 11 THEN 10 ELSEIF 10 == 10 THEN 0 END
0
>>> IF 0 > 1 THEN 0 ELSE 1 END
1
>>> IF TRUE THEN; "Hello"; END
"Hello"
>>> █
```

Example of loops and functions

```
god@Ryans-iMac Docs % bbasic
>>> ' Using loops, we can use a semicolon to separate into newlines
>>> FOR LET i = 0 TO 2 STEP 1 THEN; PRINTLN("Hello"); END
Hello
Hello
Hello
>>> LET value = 2
2
>>> WHILE value == 2 THEN; LET value = value - 1; PRINTLN(value); END
1
>>>
>>>
>>> ' And we can do the same with functions
>>> DEF add(value1, value2); RETURN value1 + value2; END
>>> add(1, 2)
3
>>> █
```

Grammar

Below is the grammar of BasicallyBASIC in Extended Backus Naur Form (EBNF), in my own version at least. This is a way to describe the semantics, syntax and structure of the language all in one fell swoop. Using EBNF allows you to create context-free grammar easily; where anyone who knows how to understand it can create code for that language.

The basic gist is this: the items surrounded in angled brackets are called our 'rules', and define a structure for how the syntax should be laid out. They are also known as non-terminals, as anything that references these rules 'jump' to another part of the grammar - meaning when referenced, you can expect more information to be parsed from it. Items surrounded in single quotes are the literal representation in the language (in BasicallyBASIC they really represent the operation code of the string of characters parsed). These are terminals, because if represented, you know that's where parsing ends, there is nothing further that can be derived past those literal representations.

For example, ``<function>`` in the grammar below is non-terminal, because when referenced in ``<statement>`` we jump to the ``<function>`` rule and parse based on the grammar on the right hand side. While the non-terminal `IF` found within the rule ``<if-conditional>``, we expect the operation we are currently parsing to be the literal `IF` within the language. There are also some helper characters in EBNF like `\*` and `|` which basically reduces writing the same rule a bunch of times. If you are curious about the code implementation of this grammar, see the Parser subsection within Design.

Explaining how EBNF works is beyond the scope of this documentation, but if you are curious about how to read it to derive code from it, I suggest you look [here](#). Though not imperative to fully understand, it is important to at least know the basics of these rules, as BasicallyBASIC is created from them. In essence, the language follows these rules below to make meaning out of the input text (without getting too complicated). Anyways here is the grammar listed:

```
/* STATEMENTS */

<program>                :    ('NEWLINE'* <statement> 'NEWLINE'*)* 'EOF'

<statement>              :    <function>
                           | <if-conditional>
                           | <while-loop>
                           | <for-loop>
                           | <jump>
                           | <expression>

<function>               :
'DEF' 'IDENTIFIER' '(' ('IDENTIFIER' (',' 'IDENTIFIER')* )? ')' <body> 'END'

<if-conditional>         :    'IF' <expression> 'THEN' <body>
                           <elseif-conditional>*
                           <else-conditional>?
                           'END'

<elseif-conditional>     :    'ELSEIF' <expression> 'THEN' <body>
```

```

<else-conditional>      :   'ELSE' <body>

<while-loop>            :   'WHILE' <expression> 'THEN' <body> 'END'

<for-loop>              :
'FOR' <assignment> 'TO' <expression> 'STEP' <expression> 'THEN' <body> 'END'

<jump>                  :   'BREAK' | 'CONTINUE' | ('RETURN' <expression>?)

<body>                  :   <statement>*

/* EXPRESSIONS */

<expression>            :   <assignment>
                           | <logical>

<assignment>            :   'LET' 'IDENTIFIER' ('=' <expression>)?

<logical>                :   <equality> (('AND' | 'OR') <equality>)*

<equality>              :   <comparison> (('==' | '!=') <comparison>)*

<comparison>            :   <term> (('<' | '<=' | '>' | '>=') <term>)*

<term>                  :   <factor> (('+' | '-') <factor>)*

<factor>                :   <power> (('*' | '/' | '%') <power>)*

<power>                 :   <accessor> ('^' <accessor>)*

<accessor>              :   <unary> (':' <unary> ('|' <unary>)? )*

<unary>                 :   ('-' | '+' | 'NOT') <unary>
                           | <call>

<call>                  :   <atom> '(' ( <expression> (',' <expression>)* )? ')'
                           | <atom>

<atom>                  :   'IDENTIFIER'
                           | <literal>
                           | <group>
                           | <list>

<literal>               :   'INTEGER' | 'FLOAT' | 'STRING' | 'TRUE' | 'FALSE' | 'NULL'

<group>                 :   '(' <expression> ')'

<list>                  :   '[' ( <expression> (',' <expression>)* )? ']'

```

Design

Before getting into the details, I would like to explain how everything will be laid out in this section. First we will start with the summary of the overall implementation, it will generally avoid specifics like each class within each of the main parts, and attempt to paint a broad picture without too much of the technical details.

It will then break off in specific sections for the five main parts of the design. These subsections will explain their technical aspects, and it will also include a subsection of the structure of the project (i.e. classes and file structure) as well as a subsection for data structures. I use what are known Java packages within my code, which are essentially a way to group like code together, so each subsection will tackle each package separately.

Breaking the design section off in this way allows for a clearer explanation of the project itself. The summary, as the name entails, is good for getting a general grasp of flow of the implementation, while the specifics highlight the responsibility that each package has. Allowing for better maintainability in the future or for clarification on parts listed within the summary.

Summary

As I have hinted at before within the README and installation instructions, BasicallyBASIC is a Java program, running Java 20. The implementation of the language can generally be summed up into four main parts of flow: the Lexer, the Parser, the Abstract Syntax Tree (AST) and the Interpreter.

Below are the parts listed in the order that they are needed, i.e. which part needs to be called first to go to the next one.

- I. The **lexer** converts text into meaningful tokens that belong to certain categories. For example, the expression ``1 + 1`` would generate: ``INTEGER(1) PLUS INTEGER(1)``. Meaning it's responsible for 'lexical analysis' – the process of assigning tokens to the input – which breaks up the input into tokens that are readable by the parser.
- II. The **parser** then uses these tokens and ensures that they follow the correct semantics and syntax of the language grammar. For example, if we had the tokens ``INTEGER(1) PLUS INTEGER(2) INTEGER(1)``, the parser would attempt to assign semantically meaning to this, but wouldn't be able to. This is because ``INTEGER INTEGER`` does not make sense in the context of the grammar. If it finds an invalid syntax or invalid semantic structure, its job is to error to ensure it can be properly evaluated. If we receive a proper token structure, e.g. ``INTEGER(1) PLUS INTEGER(1)``, it then needs to generate the correct node that stores the information for the AST. The node

type depends on what operation is currently being done, for `INTEGER(1) PLUS INTEGER(1)`, our operator plus, is a binary operation (meaning it has two operands and an operator). So, if this token structure were to be found it would generate the binary node `+ 1 1`, where `+` is the root of the tree and `1` and `1` are its leaves - creating a big tree of data.

- III. The **abstract syntax tree or AST**, is what the parser attempts to generate and what I was referring to when, in the example, I said the `+` is the root of the tree and `1` and `1` are its leaves. In essence, the AST basically holds all the data that is relevant for evaluating the program.

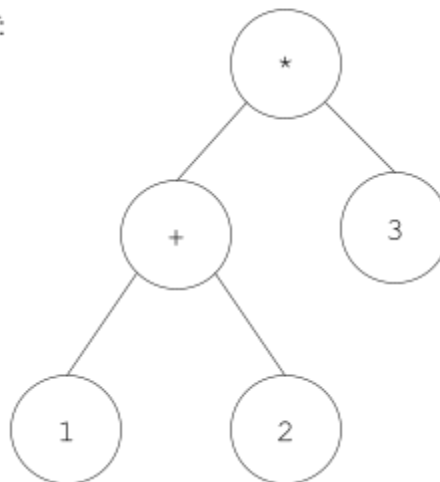
It contains structures for all types of nodes, like binary operations, group operations (e.g. `(3 + 3) - 1`) unary operations (e.g. `-1`), literals (e.g. a string `"STRING"`), any many more. And uses these nodes to generate a tree like structure in order to be evaluated in post-fix notation. Where the root node is the last operation to be evaluated, and the nodes closer to the leafs are the first ones to be evaluated. Below is an example of a valid AST that can be generated in BasicallyBASIC:

Input: `(1 + 2) * 3`

Tokens from Lexer: `RPAREN INTEGER(1) PLUS INTEGER(2) LPAREN MULTIPLY INTEGER(3)`

Nodes generated from Parser: `(* (+ 1 2) 3)`

Representation of AST:



Let's break this down: the parser generates a tree with multiplication (a binary operator node) as its root, with the left operand being a group node, which contains addition (a binary operator node) and the right being an integer literal. Why does it do this? Well the tree gets evaluated in postorder traversal, so bottom leftmost node first `1`, then `2`, then our `+`. Since we have a group expression we want to evaluate it first, so we need to stick it lower in our tree. Backing out to the root, we hit `3` then `*`. Meaning this wants the interpreter to evaluate `1 + 2` then evaluate its result (3) to the right hand operator of `3`.

So put into words what we've just seen, the AST puts the operations it wants the interpreter to evaluate first on the bottom most node (the leafs) and puts the operations that it wants to evaluate last at the top of the tree (the root). This way we can evaluate anything if the AST has a node for it. Hopefully that all made sense, but if it didn't I encourage you to look online for more information on ASTs. It's a relatively simple data structure in theory, but can be messy in practice. Your biggest take away should be: it's the structure of the program with **no** evaluation capability, it only stores how we want to evaluate code the user inputted later on.

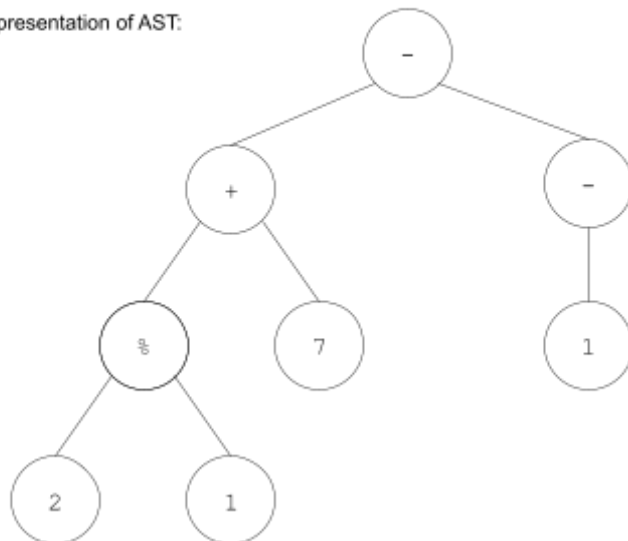
- IV. Now that we've done a quick lesson on the AST, we can move onto the **interpreter**. The AST and the interpreter are heavily intertwined, but serves two very distinct purposes: the AST is simply a data store for all the different types of nodes/structures that can be found in the syntax, while the interpreter evaluates the data within the AST to produce a result. If we take the example shown on the image in the AST part ('(1 + 2) * 3') and evaluate the tree ('(* (+ 1 2) 3)') in postorder travel, we evaluate the '+ 1 2' first, getting '3', then evaluate '* 3 3' and get the answer of '9'. The interpreter basically acts as the tree walker, evaluating expressions as it goes to produce the result. Here's an example of the interpreter in action:

Input: 2 % 1 + 7 - -1

Tokens from Lexer: INTEGER(2) MOD INTEGER(1) PLUS INTEGER(7) MINUS MINUS INTEGER(1)

Nodes generated from Parser: (- (+ (% 2 1) 7) (- 1))

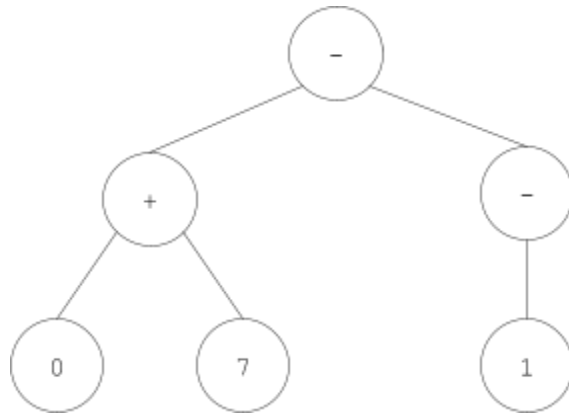
Representation of AST:



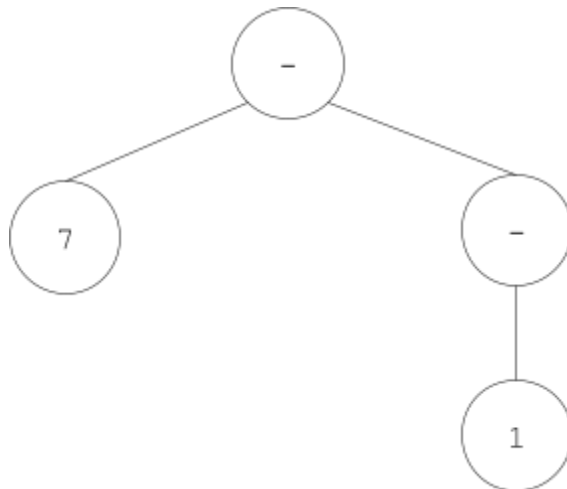
Breaking this down: since the '%' operator has higher precedence than addition or subtraction it is placed lower on the tree. We have left-to-right associativity so the first '+' is placed lower with its left node being the binary operation for '%' and right node being a literal for '7'. Then on the right side we have a unary minus, which has higher precedence than the binary subtraction, so we placed it lower on the tree. Finally at the

root node we have a binary subtraction between the '+' operator and the unary minus of '1'. Evaluating with post-order traversal we the steps:

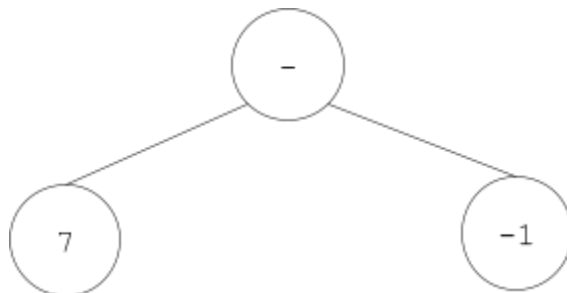
1. Evaluate '2 % 1' to get 0



2. Evaluate '0 + 7' to get 7



3. Evaluate '- 1' to get '-1'



4. Last evaluate '7 - -1' to get '8'

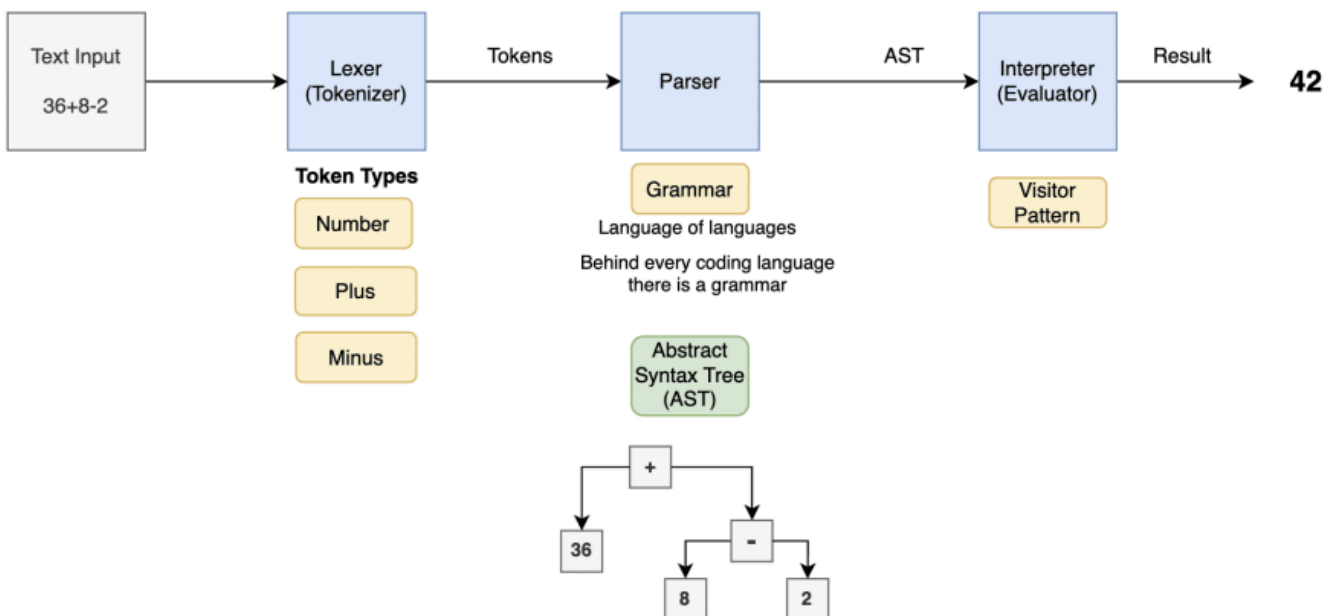


Though this may seem unnecessary to show out fully, this is basically how the design of the program works in its entirety. Though this is a simple example with just math operators, it is no different with actual syntax and statements. The biggest difference between the two statements like `LET` or `IF` is that they produce side effects as they evaluate, rather than a simple number.

To sum up what each part does in a nice little package:

- I. The **lexer** is responsible for lexical analysis of the input text, that is breaking up the input into tokens that are readable by the parser.
- II. The **parser** is responsible for taking the tokens generated by the lexer and ensuring that they follow the correct semantics and syntax listed within the grammar; creating nodes for the AST to hold on to.
- III. The **AST** is only responsible for holding the data associated with the parser, it contains no real logic - this is the job of the interpreter.
- IV. The **interpreter** takes the AST and evaluates it. In essence it performs actual calculations within the language.

Below is an image of the basic flowchart of how this gets implemented; curiosity of CodingWith-Adam in his article [“Introduction to Lexers, Parsers and Interpreters”](#) (see references for link).



That's basically how the entire program works (disregarding all the technical details of implementation). We take input, perform lexical analysis on it, parse it to ensure valid grammar, generate the AST and then evaluate it.

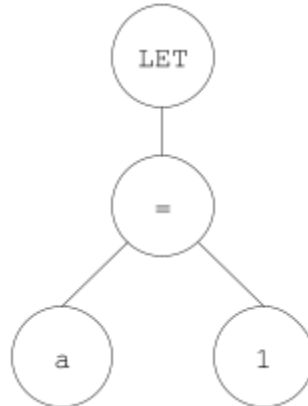
Here is one last example with an assignment statement to help solidify the design:

Input: `LET a = 1`

Tokens from Lexer: `LET IDENTIFIER(a) ASSIGNMENT INTEGER(1)`

Nodes generated from Parser: `(LET (= (IDENTIFIER a) 1))`

Representation of AST:



To break it down, we receive an assignment statement from the input, the lexer makes them into tokens, and the parser generates the tree to evaluate. I've extracted ``LET`` and ``=`` out to their own nodes for ease of representation. In the actual code, we'd simply have an assignment node which stores the identifier name and the expression to be assigned to it.

Here are the steps that the interpreter takes to evaluate it in postorder traversal. I've excluded images here as it's simple enough to walk through in text, and we are updating state so the tree is less relevant.

1. First we hit the bottom left node ``a`` and update it within a symbol table (which stores state) so we can resolve it later
2. We hit the right node ``1`` and assign it to ``a`` updating its value within the symbol table.
3. In most languages we'd be done here, ``a`` gets assigned to ``1`` and nothing happens. However BasicallyBASIC returns the value it gets assigned to, so ``LET a = 1`` actually evaluates to 1.

And that's the summary of the design. Like I've stated, I've disregarded the technical aspects and left them to the subsections below as they intrinsically muddy how you perceive the overall design structure. If skipping over the technical parts, the most important thing to take away is: given text, make tokens, put those tokens into an AST, and evaluate it post-order traversal.

Language Package

The **language package** is the package responsible for initializing BasicallyBASIC. Meaning it's in charge of getting the input from the user to run from the CLI or from a file, initializing the global symbol table for all functions and variables to reside in, initializing error reporting and registering the built-in functions.

It consists of three classes, below is their listed their responsibility and the actions that they provide towards the program:

- **Main.java** - the entry point of the program, responsible for creating the Lexer, Parser and Interpreter object and displaying the error message to the user if there is one. Below are the steps it takes to achieve this:
 - I. Decides whether or not to run from console or from a file based on user input
 - II. Initializes the global symbol table for variable and function states
 - III. Registers all the built-in functions defined in `BuiltInFunctions.java`
 - IV. Initializes the Error with the users input
 - V. Initializes the Lexer with the users input, and generates all tokens
 - VI. Takes the tokens from the Lexer and passes them to the Parser to generator the AST
 - VII. Sends the AST to the interpreter for evaluation, prints the result if in CLI run mode
 - VIII. Prints the error message if any of the above steps generated on based off user input
- **Error.java** - the class responsible for generating errors, both the message, type and line number they occur on. Is a throwable instance, so at any point a user makes a mistake any class can throw an Error with a descriptive message to inform the user. Facilities this with an enum with all the various error types. It is initialized with the user input and acts as the main debugging tool for the language. Not a large class, but vital to the language.
- **BuiltInFunctions.java** - responsible for registering built-in functions like ``PRINT``, ``LENGTH``, ``TO_STRING``, etc.
 - This class is essentially composed of static functions that the language needs which cannot operate on pure syntax in the grammar. For example, a user cannot use the standard output just based on syntax alone, so it needs a connect point, i.e. a function, that can do it for it - in this case it would be ``PRINT``.
 - It uses Java's functional interface to facilitate this need. In essence, it is a structure that a function must follow, returning a Value and accepting an ArrayList of arguments. Each in-built function has a static Java function counterpart that performs the action that the function is supposed to, and uses a functional interface to allow the function to be essentially passed around as an object, being called wherever it is needed. All of the functions defined get registered to the global symbol table, this action is done within `Main.java`.
 - It also creates its own expectation call ``BuiltInFunction Exception``, which is thrown within the Java defined function counterpart if anything goes wrong during execution. For example, if a user calls ``TO_INTEGER("hi")``, a

BuiltInFunction exception is thrown as it cannot convert the STRING to an INTEGER. It is then caught within the interpreter to be relayed with the line number and proper error message.

The package is quite simple, really acting as the language 'itself'. It's meant to get everything started and act as the driver behind everything.

Lexer

The **lexer package** is responsible for all things tokens and lexical analysis. Meaning it's in charge of breaking up the user input into tokens that assigns some meaning for the parser to work with. It is the first step in the interpreter process and is responsible for identifying the basic building blocks of the language.

For example, if the user inputs the text ``LET val = 10 + 2`` it is the lexer package's job to assign this text meaning. The inputted text will be transformed into the following tokens ``LET IDENTIFIER(val) ASSIGNMENT INTEGER(10) PLUS INTEGER(2)``. Each token records their specific value associated with it, the line number it was located on as the operation code it is. So if we look at ``val`` in our example the token value would be ``val``, its operation code would be ``IDENTIFIER`` and its line number would be ``1``.

There are several types of tokens, or operation codes, being keywords (so strings like ``LET``, ``IF``, ``FOR``, etc.), simple symbols (e.g. ``=``, or ``+``), shared symbols – items that have multiple characters in common – (e.g. ``>`` and ``>=``), complex – strings that have additional logic needed to parse, like strings – and no symbols – things like literals which can be represented in multiple ways (e.g. integers or floats).

Once finished lexing, the `Lexer.java` class will send the entire list of tokens back to the caller from the `GenerateAllTokens()` method. Main handles transferring this list to the Parser.

It consists of one enum class and two standard classes. They are listed with their respective responsibilities below:

- **OperationCode.java** - the enum class responsible for defining what type of token the currently parsed string is. Designated as a class that stores the enum type for each piece of grammatical syntax, as noted above. In essence, we define the types of tokens here and there representation within the language. For example, the ``LESS_THAN`` enum gets represented as ``<`` in the language and the operation code enum marks that down for us. It uses a hashmap to store all keywords for constant time lookup, as well hash map for all simple symbols, once again for constant time lookup and just a general easier time for lexing instead of long if-else chains for checking a a single character.

- **Token.java** - this class acts more of a struct than anything - but structs are not a thing within Java. There is no logic attached to it, it really acts as a data store to represent the tokens we generate in the Lexer. Is responsible for holding the Value that was lexed, the line number the lexeme was found on as well as OperationCode enum type, to in a way give it a type for future parsing. For example, a literal integer token might be stored something like this: `OperationCode = LITERAL, lineNumber = 10, Value = 20`.
- **Lexer.java** - the driver of breaking up the user inputted string into actual tokens, i.e. performs the actual lexical analysis.
 - The main job of this class is to loop over every single character to find what the string generates. It will remove all whitespace and comments it comes across and they are not important pieces of grammar, then look at the current character it is on to see what value is matched based on the representation in the language from the OperationCode enum. For example if we run into the character `>` we know it's a `LESS\_THAN` token type. Or if we run into `"` we know it's a string and we need to parse all the contents of it until we hit another `"` character.
 - This class also makes all the literal types into Values we can work with later on in the Interpreter. For example, if we run into an integer `10`, it will parse it to the Java equivalent integer. Same with floats, strings, null's and booleans.
 - Whenever the class hits a value we cannot parse, it will throw a syntax error. This also goes for unclosed strings or multiline comments. And as we are moving through the input text, we record the line number based on the newline character we defined in the syntax, so we can have better error reporting.

To sum up this package, we handle all the lexical analysis and define the building blocks of the language here. Generating a list of tokens for the parser to generate into an AST.

Parser

The **parser package** is responsible for ensuring that the tokens generated by the Lexer conform to the grammar of BasicallyBASIC, and transforming them into an AST for later evaluation by the Interpreter. The grammar rules the parser follows can be found in the Grammar subsection of the User Manual, where it highlights the specific syntactic structures allowed in the language.

My parser uses recursive descent parsing, meaning it breaks down the input tokens recursively according to the grammar's hierarchical rules. Each grammar rule corresponds to one or more methods within the parser, and each method will return a node in the AST that represents the corresponding syntactic construct.

Here is a snippet of part of the grammar for a factor to help further explain:

```
<factor> : <power> (('*' | '/' | '%') <power>)*
```

The left side denotes the name of the rule and the right side is how to parse it, enforcing the terminals and non-terminals of the EBNF grammar. When parsed we will return a BinaryOperator node for the AST as the factor rule contains all binary operations (multiplication, division and modulus). [Once again, this is mentioned with the Grammar subsection of the User Manual, so if you are confused on what EBNF grammar is, and what terminals and nonterminals are, please look at that section.]

With recursive descent we actually enforce the rule as code. So the left-hand side ``factor`` becomes the function `Factor()`, with the body of it reflecting the rules it has. In general this is how we reflect that body in code:

- A terminal (surrounded with single quotes), the direct operation code we need to match. So for example ``*`` in ``factor`` would be a terminal, if we do not see that token when parsing, we know we have an error.
- A nonterminal (surrounded by angle brackets), is another rule, and it is represented by a function, just like how ``factor`` has its own function `Factor()`. We call this function recursively (hence the name recursive descent) until we hit a terminal. So, the ``<power>`` would have a corresponding `Power()` function to call in code.
- A ``|`` symbol found in the EBNF becomes an if or switch statement. For example, once again looking at ``factor`` we see that ``*``, ``/`` and ``%`` are all value operators after the ``<power>`` rule.
- A ``*`` or ``+`` symbol is used to denote zero or more of and at least one or more of respectively. For example within ``<factor>`` we can chain multiple ``<power>`` rules together zero or more times, as long as there is ``*``, ``/`` or ``%`` separating them.
- Lastly, the ``?`` symbol is just a simple if statement, denoting this rule is optional. There are none of these symbols within ``<factor>`` but the symbol does appear in function arguments, where we can have an optional amount of arguments defined. So the ``?`` would appear after each expression rule, e.g. ``(' ( <expression> (',' <expression>)* )? ')``.

Now let's implement the ``<factor>`` rule in some pseudo code to show you how it works!

```
BinaryOperator Function(){  
    // Call the power rule as shown in grammar  
    power = Power()  
    // While next token is *, / or % continue to call power  
    while m_currToken == (MULTIPLY || DIVIDE || MODULUS) {  
        // Eat/advance on the token to ensure we got the right one  
        Advance(m_currToken)
```

```

        // Our old power becomes the left hand side to eval first
        // With the right hand side becomes the right hand side
        power = new BinaryOperator(power, m_currToken, Power())
    }
    // And put the binary operator node into the AST
    // This basically puts the node into the callers node
    return power;
}

```

This is basically what we do for each grammar rule, where we follow the grammar definition and create the corresponding AST node based on what the grammar calls for. There are some simplifications being done here, and things get more complicated when the rules get more complicated but this is the general structure for parsing each rule.

After parsing a statement, e.g. an if conditional, we add it to a list of nodes that quite literally is the program that the user created. We pass this off to the Interpreter to then evaluate the tree.

The package itself only has one class that handles all of this logic. It doesn't need more as all it's doing is going through each token and making them into an AST node. Here it is:

- **Parser.java** - Responsible for generating the AST based on BasicallyBASICS grammar rules. If at any point the user's input doesn't match these rules a error will be thrown to inform them of what they need to fix.

To sum up this package, the parser ensures that the tokens provided by the Lexer conform to the language's grammar and organizes them into an AST. This AST is then passed to the Interpreter for evaluation.

Abstract Syntax Tree

The **abstract syntax tree (AST) package** is a tree structure that holds the entirety of the program within itself. It is responsible solely for holding the data associated with each operation, either it be an expression or a statement. To put it simply, it is a collection of structs that represent the program as a tree. No actual logic is performed here, only storing data. So, it's not really a class, but rather a collection of 'stuff' that makes it up.

In BasicallyBASIC, the AST follows the Visitor Design Pattern to facilitate this implementation. In simple terms, the pattern separates the representation of the AST nodes from the actual operations that can be performed on those nodes. Meaning it has no methods, and like I said, is just for storing data. This pattern allows for cleaner and more modular code, as it decouples the logic for traversing and processing the AST from the structure of the AST itself.

We can define operations on a set of AST nodes without changing the classes of those nodes themselves. Instead of having logic inside each AST node to handle interpretation, evaluation, or any other operation, a separate "Visitor" class is used to encapsulate that logic. Each node in the AST knows how to accept a visitor, and the visitor knows how to perform operations on the node through a technique called "Double Dispatch." This mechanism basically sends out a function call to different concrete functions based on the runtime types of the two nodes involved in the call.

STOP! This pseudo-definition, though informative, throws a lot of terms around that some may not understand if they never encountered it before. If you did, I applaud you, and you can probably skip the next few steps. As the visitor design pattern is not so commonly used in the majority of programs it is understandable that people won't get it right off the bat. I had this same issue when learning it myself, so I'll explain the terms in how it's implemented in BasicallyBASIC:

1. **AST Nodes:** each type of node (such as BinaryOperator node, Literal node, etc.) has an Accept() method that takes a Visitor as a parameter. The Accept() method then calls the corresponding visit method on the visitor, passing this as the argument.

The BinaryOperator node looks something like this

```
public class BinaryOperator extends Node {
    // Other methods/constructors...
    public Value Accept(NodeVisitor a_visitor) {
        visitor.VisitBinaryOperator(this);
    }
}
```

2. **NodeVisitor Interface:** This interface is a template that defines the visit methods for each node type in the AST. Meaning each node has a corresponding visitor function attached to it. These methods define what should happen when a specific type of node is encountered. As it is an interface any class can implement it, but is forced to use the methods defined within the interface.

For example, here's how the Binaryoperator node and Literal nodes would be defined

```
public interface NodeVisitor {
    Value VisitBinaryOperator(BinaryOperator a_node);
    Value VisitLiteral(Literal a_node);
    // Other node types...
}
```

3. **Concrete Visitors:** You can create different visitor implementations for different tasks as it is an interface. However, in BasicallyBASIC we just need it for the interpreter class to evaluate it.

For example, here what the the Interpreter would look like with the BinaryOperator node

```
public class Interpreter implements NodeVisitor {  
    public void visitBinaryOpNode(BinaryOperator a_node) {  
        // Recursively visit the left and right child nodes  
        Value left = a_node.GetLeftExpression().Accept(this);  
        Value rhs = a_node.GetRightExpression().Accept(this);  
        // Perform binary operation based on the operator...  
    }  
}
```

Basically what happens here is, the concrete method called the .Accept() on the left and right expressions (postorder traversal) which are nodes themselves. As we accept them, they recursively call .Visit on the interpret object (used with this) which then in turn calls .Accept() on the nodes that it has. It continues down this chain until we hit a terminal node, i.e. one that returns a Value instead of calling .Accept() again.

Hopefully that cleared things up! It took a while for me to get it myself, but after I did it helped me understand why this design pattern is the most optional route to go, instead of something like deeply inherited polymorphism. Which makes the node not only store the data but perform its operations on itself as well. If you are still a bit confused, you may want to think about it in math terms: $node_x$ calls $function_x$ of the visitor.

Now to list the nodes that BasicallyBASIC incorporates so you know how it all works. These nodes are separated into two distinct categories: expressions and statements. Expressions are strictly for returning values, while statements are for changing the state of the program or controlling the flow of it, e.g. updating the SymbolTable or doing a conditional statement. Once again I call them structs as they only represent values, but they need to be implemented as classes in Java.

Expression Structs

These are all the expression node types – nodes result in a value – and a brief explanation of what they store within BasicallyBASIC:

- **Assignment** - responsible for storing the variable as a token and the expression the variable will be assigned to. Created by ``LET <identifier> = <expression>`` or ``LET <identifier>`` within the parser. For example, ``LET val = 10`` would be a valid operator node.

- **BinaryOperator** - represents a binary operation – an expression that has two operands and an operator – and is responsible for holding the left and right expression as well as the operator token. Created by ``<expression> <operator> <expression>`` within the parser. For example ``1 + 2`` or ``1 * 3`` would become valid binary operator nodes.
- **Group** - represents a group of expressions (e.g. `(1 + 2)`) (or a parenthesized expression) and is responsible for holding that expression. Is created by ``(<expression>)`` in the parser.
- **Identifier** - represents a variable/function and is created when an identifier is encountered within the parser. For example ``val`` would become an identifier node.
- **Index** - responsible for the index access of a list or string value. There are two types you can store, normal subscript access (accessing one element) and ranged subscript access (accessing multiple indices at once). Created by ``<identifier>:<index>`` for normal subscript, and ``<identifier>:<start-index>|<end-index>`'. So, ``list:0`` or ``list:0|2`` would both become index nodes.
- **ListExpression** - represents a list of expressions, e.g. `[1 + 2, "hello", (3 * 4)]`, meaning it is the unevaluated list before transformation to a list Value. Is created from ``[<expression>, <expression>, ...]`` in the parser.
- **Literal** - represents a literal value (e.g. 1, true, "hello") - created when a literal is encountered within the parser.
- **UnaryOperator** - is a unary operation, e.g. -1 or NOT TRUE. Created by ``<operator> <expression>`` in the parser.

And that's all for expressions! I know there are a lot of them, and we still have yet to do statements, but we need a way to represent the data for each one, and this is really the only way in OOP. I'll explain more on that later, but for right now we can move onto the statement structs.

Statement Structs

These are all the statement node types – nodes result in a state change or control flow – and a brief explanation of what they store within BasicallyBASIC:

- **Body** - represents a body/block of statements attached to some other statement. For example, the body of this if statement `IF TRUE THEN PRINT("HI") END` is `PRINT("HI")`. Stores the statements within a List.
- **Call** - represents the call of a function, created by `<function>(<arguments>)`. For example `LENGTH("HELLO")` would be turned into a call node.
- **For** - is the for loop statement, and is created from `FOR <assignment> TO <endValue> STEP <increment> THEN <body> END`. Holds assignment expression, the increment and body - basically everything that the for loop needs to work. For example `FOR LET i = 0 TO 10 STEP 1 THEN PRINT(i) END` would have the assignment as `LET i = 0`, `10` as the end value, `1` as the increment and `PRINT(i)` as the body.
- **Function** - represents a function as both a value and a built-in for the language. It is created from the `DEF <identifier>(<parameters>) <body> END` syntax during parsing. This class holds the function's name, its parameters, the count of parameters, whether the function is built-in or user-defined, the body of the function, and, in the case of built-in functions, the associated operation. Each function is expected to return a value, and if none is provided, a NULL value will be returned by default.
- **IfElseifElse** - represents an if-elseif-else chain, created from the syntax `IF <condition> THEN <body> ELSEIF <condition> THEN <body> ELSE <body> END` during parsing. Responsible for storing the conditions and corresponding bodies of the chain. The else and elseif parts are optional, but there must be at least one `IF` condition at the time of creation.
- **Jump** - represents a jump out of a specific code block, handling types such as break, continue, and return. It holds the type of jump and allows other nodes to catch and process it as needed. Exception control flow is used to exit the current block and return control to the point where the jump was initiated.
- **While** - a while loop, created during parsing when a `WHILE <condition> THEN <body> END` structure is encountered. It is responsible for storing the loop's condition and its body. For example, `WHILE TRUE THEN CONTINUE END` has the condition of `TRUE` and body of `CONTINUE`.

There is also a struct called **ConditionalField**, but is not actually a node within the AST, rather a way to express similarities between the `IfElseIfElse` node and the `While` node. It stores a condition and the body of the field. So for example, ``IF TRUE THEN PRINT("HI") END`` would have the condition of ``TRUE`` and the body of ``PRINT("HI")``. And you can see how similar it is for the `While` node where ``WHILE TRUE THEN BREAK END`` has the condition of ``TRUE`` and body of ``BREAK``. So instead of repeating the same structure twice, it was easier to make it as its own struct.

Now that was a lot of information about how the AST is designed and implemented! Hopefully it wasn't too boring. Before moving on to the interpreter, I want to talk about why this visitor design pattern was chosen with the creation of 16 structs. It may seem crazy to do this, especially in an object oriented language where each class is expected to do more than just store data, but there is a good reason for it.

Tree classes are not 'owned' by any specific entity; their primary purpose is to store data. Each node is designed only to hold data and should not be responsible for its own evaluation—that's the interpreter's responsibility. This is why the visitor design pattern is used: it allows the nodes to focus on data storage, while the evaluation logic is handled by the interpreter, with communication facilitated via double dispatch. Though it may seem unconventional to have around twenty classes solely for data without methods, this approach is better suited to handling tree structures, as traditional OOP principles don't align neatly with them.

To put it in a way that I found made sense, the visitor design pattern attempts to mimic a more functional programming approach. Where everything isn't as rigidly defined as an object, rather exists as its 'own thing'. Basically what I am trying to get at is, OOP is not suited for this type of job. Perhaps that is on me for choosing Java over something more functional, but there are also parts of the language that are really well suited for OOP, like the `Lexer` and the `Parser`. Those things work perfectly, updating state has a ton of helper functions and methods - it is the principle of object oriented programming.

To sum it up, the visitor design pattern was chosen to allow the AST structure to remain separate from the evaluation part of the language. It also allows for modularity and extensibility, as to add a new node, you simply extend it from the `Node` abstract class and define a visitor function - no need to change 100 classes like you would need to with deep inheritance. It also reduces the complexity in nodes, each node doesn't need to know the entire process and the actions of other nodes, rather it can simply have one purpose.

And that's the abstract syntax tree package. Man that was a lot of information to put out, but needed to understand how BasicallyBASIC works. Hopefully you haven't fallen asleep after that.

Interpreter

Here is our last and final package, the **interpreter package**. It is responsible for maintaining scope and evaluating expression - with a sprinkle of data. This class implements the NodeVisitor interface, previously discussed within the AST design section. This is why it's called a 'tree-walker' interpreter as it walks through the abstract syntax *tree*.

It implements this functionality with three very important classes, which are listed below with the respective responsibilities:

- **Value.java** - this is how we dynamically store our types: INTEGER, FLOAT, STRING, BOOLEAN, LIST, NULL, and FUNCTION. It is responsible for not just storing these values, but also providing useful methods not just for the implementation but for the user code as well. In essence it acts as a wrapper class for our language data types from Java's data types.
 - This class's main responsibility is to represent data type values as Java types. It uses an enum to determine the type and several constructors (one for each type there is listed above) to facilitate this. It uses the Java Object class as a way to represent more than one type on a single instance - similar to how you can represent anything in C/C++ as a void pointer. Below is a list of our types represented as java types, and why certain aspects were chosen for it:
 - INTEGER: We represent our ints as BigIntegers in Java, which is a class that allows for extremely large numbers. Meaning we don't have overflow in our language, you can represent any value as large as you want. So 23490782374982739, is valid in BasicallyBASIC.
 - FLOAT: For our floats, we use the BigDecimal object in Java, similar to the BigInteger class, it allows us to represent extremely large numbers. We cap our precision at 16 decimal places as we cannot represent irrational numbers. So, 12318349018034802938.2934892390 is valid in BasicallyBASIC.
 - STRING: 1 to 1 conversion to a Java string, nothing more to it.
 - BOOLEAN: 1 to 1 conversion to a Java primitive boolean, though we represent true and false in all caps: `TRUE` and `FALSE`.
 - LIST: we take advantage of Java's List 'generic' to incorporate this. Basically our list value is a List of Value objects, so we can put any single type of value in it. For example, [1, 2, "hello"] is valid even though it contains different types.

- NULL: 1 to 1 conversion to Java null, but we show it as ``NULL``.
- FUNCTION: we use our own Function object defined as an AST node. It makes it easier to both call and store instead of extrapolating out the function name and body into Value.
- In BasicallyBASIC everything is immutable. Lists are copied, strings are copied, everything is copied. This was an intentional decision to not update types like lists and have unexpected results - or at least results you can't easily track. Originally I was planning on making lists references, but taking a look at Python which does this, I have often found myself overwriting data accidentally because I forgot that lists are references. To clear up this confusion, everything will be returned as a copy (besides a function type which must pass its reference in order to be called).
- Value supports a ton of helper functions and methods to allow for easier comparison and type conversion. Specifically it will implicitly cast an INTEGER to a FLOAT when requested as a FLOAT and vice versa. It also provides a pretty printing function so objects like lists will appear as `[1, 2, 3]` instead of `[Value@refid, Value@refid, Value@refid]`. And it will also properly compare values, even with floating point arithmetic. For example, ``.1 + .2 == .3`` actually evaluates to ``TRUE`` instead of ``FALSE``.
- **SymbolTable.java** - how we store the state of our program. Responsible for saving variables in functions to be used at a later point within user code.
 - Our symbol table is actually just a giant hashmap with a bit of nested objects to make static scoping possible. We have two members that do this, a `m_outerscope` which is a SymbolTable object and a `m_currentscope` which is a hashmap. On first initialization, create a new instance without any arguments, acting as the global scope. When we enter something like a function, which has its own scope, we create a new instance, passing the outerscope as a constructor argument and which gets saved to `m_outerscope`, and creates a new hashmap for our `m_currentscope`. Meaning, when we want to access an identifier, we can first look in `m_currentscope`, and if we don't find it we can chain up from `m_outerscope` to see if it is defined.

Typing recursion out like this in plain text can be hard to understand, so here is an example:

```
// Define out global scope
SymbolTable global = new SymbolTable()
// We enter a function, so we need to create its own scope
Symboltable current = new SymbolTable(global)
```

```
// Now when we want to access a new value
current.GetIdentifier("a");
// It will try to get the current scope, and if it isn't
// in that scope, it will then try to find an outerscope.
// Eventually it'll hit global if undefined and throw
// and error to inform the user.
```

- With our current implementation for BasicallyBASIC, we cannot change the variables outside the current scope. For example, if we defined a variable “duck” outside of a function then entered it, we cannot edit and change this value with side effects. The only thing we can do is access the value it has. In essence it is a read only variable if not directly within the current scope. This is done on purpose to avoid dynamic binding at runtime, it creates a confusing mess when we claim to be statically bound - so it's read only.
- **Interpreter.java** - finally onto the actual computation within our language. Though extraordinarily important for actual functionality there isn't much to it that we haven't already described within the AST. This class implements the NodeVisitor interface, meaning it will have a visitor function for each of the node types.
 - This class is responsible for evaluating the AST (the user code) and ensuring that the runtime evaluation is correct and that the operations that we perform on types has type safety and isn't illegal. We typically evaluate the AST through postorder travel, so lowest and leftist most nodes first, then we climb up. And as for type safety, this class will catch if you try to do something like ``"HELLO" + 1``, throwing a type error if encountered.
 - The actual evaluation for each node can be found within the source code, and the design of it is pretty straight forward based on the node type. For example, `VisitAssignment()` within the Interpreter assigns a value to an identifier in the symbol table - pretty cut and dry. With the visitor design pattern implemented on the AST, the Interpreter itself has no intrinsic design, rather it follows the NodeVisitors pattern instead. So evaluating every node is trivial, just `.Accept()` on the structure of the node and evaluate based on the node type you are in, simple.
 - The most notable thing in the interpreter which is not a part of the AST is the way it evaluates all numbers as FLOATs, then converting back to INTEGERs if both operands are INTERGERS to begin with. This is done to save on if-elseif-else chains for number types which just aren't needed. Java's BigDecimal also holds the set of all whole numbers - so it saves a lot of unnecessary checking.

To sum it up in the most basic way to put it, the interpreter is responsible for keeping state with the symbol table, evaluating expressions into a Value object. It provides helper functions for those Values not for easier maintainability, but for a better user experience as well. When

being evaluated within the Interpreter class, it gets all the statements and nodes generated from the Parser does the evaluation and ensures no errors created in the process. It is the actual brain of the language, and uses the AST visitor design pattern as its guide for evaluation.

Project Structure

Each class and their respective responsibility have already been listed thoroughly throughout each package, but are rehashed here for a more concise, specific spot to access. As a reminder, a Java package is a way to group related/similar code with one another. I will also list the file/directory structure of the source code for a better understanding of where everything is.

Classes

I will list them exhaustively here with a brief description of what they do for easier understanding and overall border view of the design. If you are looking for more specifics on each class, I recommend looking at the package they were a part of within the design in order to gain a better understanding. If any class acts as a structure, interface or is an enum, they will be listed as such.

- **Language Package** - initializes the language
 - Main - entry point and initializer of the program
 - Error - error class that generates the error type and message for user
 - BuiltInFunctions - defines built-ins functions and puts them in global scope
- **Lexer Package** - handles all lexical analysis
 - OperatorCode Enum - defines the representation of the syntax
 - Token Struct - holds the operation code, line number and value as a token
 - Lexer - breaks apart input into a list of tokens
- **Parser Package** - parses the tokens
 - Parser - generates the AST and ensures valid language semantics in grammar based on the tokens from Lexer
- **Abstract Syntax Tree Package** - holds all nodes as structs for data
 - Node - an abstract class that each node will inherit from
 - NodeVisitor Interface - template for the interpreter to follow for dual dispatch
 - Expressions Package: Assignment, BinaryOperator, Group, Identifier, Index, ListExpression, Literal, and UnaryOperator structs
 - Statements Package: Body, Call, For, Function, IfElseifElse, ConditionalField, Jump, and While structs
- **Interpreter Package** - evaluates the program holding its state

- Value - holds all the datatypes in a nice neat package
- SymbolTable - acts as the scope for our language to reference variables and functions (holds the state)
- Interpreter - evaluates the AST and produces a result

In terms of classes interacting with each other, there are only a few that do so. More specifically: Token, OperationCode enum class, Value and Error are found throughout pretty much the entire program.

Tokens are needed in the Lexer, Parser and Interpreter as they contain vital information about the line number for error reporting and the value it contains. The OperationCode enum has similar properties where we need them in the Lexer for tokens, in the parser for ensuring correct grammar, and in the interpreter for evaluation (specifically for binary and unary operations). This is also the case for the Value class, we need them for literals in the Lexer, node creation in the parser, and evaluation within the Interpreter.

Errors should be clear why they are needed everywhere, they happen every stage in the code. Either it be syntax errors, runtime errors, arithmetic errors, etc. Errors need to be recorded, so it exists everywhere.

There are multiple interactions between the AST nodes for the parser and the interpreter, which should be pretty self explanatory. We need to generate nodes in the parser to build the actual tree, and we need the nodes in the interpreter to evaluate.

Other classes such as SymbolTable pop up within a few places, but are really only needed within Main, BuiltInFunctions and the Interpreter. For Main, we need to initialize the scope and give it to the interpreter and built-in functions. While in the Interpreter we need to put variables and functions within the scope. And as for BuiltInFucntions, we need a way to register them for later access.

In total there are 5 packages, 9 classes, 1 enum class, 1 interface class, and 18 structs (which must be represented as classes in Java). Most of the structs are contained within the AST as there is no generic way to represent each node type without sacrificing readability. We need a way to represent structures like binary operations and in OOP there is no good way to do that.

File Structure

As the package/classes are directly related to the source code's file structure, I felt it best to group them together. This way everything is neatly placed where it is supposed to be, and allows for easier mentality and extendibility when we need to, e.g. adding a new node to our AST. Below is the tree for the source code directory:

```

├── AbstractSyntaxTree
│   ├── Expressions
│   │   ├── Assignment.java
│   │   ├── BinaryOperator.java
│   │   ├── Group.java
│   │   ├── Identifier.java
│   │   ├── Index.java
│   │   ├── ListExpression.java
│   │   ├── Literal.java
│   │   └── UnaryOperator.java
│   ├── Node.java
│   ├── NodeVisitor.java
│   └── Statements
│       ├── Body.java
│       ├── Call.java
│       ├── ConditionalField.java
│       ├── For.java
│       ├── Function.java
│       ├── IfElseifElse.java
│       ├── Jump.java
│       └── While.java
├── Interpreter
│   ├── Interpreter.java
│   ├── SymbolTable.java
│   └── Value.java
├── Language
│   ├── BuiltInFunctions.java
│   ├── Error.java
│   └── Main.java
├── Lexer
│   ├── Lexer.java
│   ├── OperationCode.java
│   └── Token.java
├── META-INF
│   └── MANIFEST.MF
└── Parser
    └── Parser.java

```

As you can see the packages and classes are directly related to the file structure. The most notable exception to this is the `META-INF` directory which contains the `MANIFEST.MF` file. This is directly responsible for generating production code in Java, containing the version of

the project as well as the Java version it needs to run. It is not something that I created, rather something is produced when compiling for release.

Data Structures

Most of the data structures, like the classes, have already been discussed within each package. I will list every single data structure I used here for ease of reference and understanding.

- **Lists/Vectors** - Lists, or ArrayLists as Java likes to call them, are vectors in C++. They are used pretty much everywhere within the code, from storing all the tokens generated from Lexer, storing all the statements within the parser, the actual representation of our LIST type, and the arguments of functions. They are pretty standard, and have a bit more functionality and flexibility compared to arrays.
- **Sets/EnumSets** - Sets are used specifically within the parser package to make easy changes to the grammar. Since each specific rule found in EBNF usually contains multiple operation codes, it's easier to group them together within a set. For example addition and subtraction have the same level of precedence within our grammar, and instead of checking each type hard coded within the parser, we use a set to simply call `.containsKey()` for constant time lookup and easier readability. We also store them in the parser as static constants to easily swap out values while not recreating the sets multiple times within the recursive descent.
- **HashMaps** - There are two spots where hashmaps are used, within the `OperationCode` enum class for storing keywords and simple symbols, as well as our structure for the `SymbolTable`. In `OperationCode` it makes it easier to store and lookup keywords in constant time then checking an array. And in the `SymbolTable` it is the perfect way to retrieve/put identifiers within the scope. As BasicallyBASIC allows for redeclaring and initializing variables, the hashmap is the perfect data structure. For example, storing the variable ``var`` simply puts it into the current hashmap as the scope, it's really that simple. All this is made better with constant time lookup!
- **Abstract Syntax Tree** - It should have been pretty obvious that we used a tree structure to store our program (specifically an abstract syntax tree), but in case it wasn't here it is mentioned. This stores every single operation type and structure of them for easy parsing and evaluation. It really is the backbone of BasicallyBASIC.

Testing

Testing the validity of the language was a very important part to make sure everything works correctly. As BasicallyBASIC has a lot of parts, I needed to unit test after a new module was added, and regression tested when any new module changed state or interacted with other modules. This was not done formally with a proper testing suite, as it would have likely doubled the time to create the project, but rather informally after adding a function. Mainly I ran a bunch of cases within the main function of each class (as Java can allow for multiple entry points) and made sure that when I added something, or changed something it worked properly.

I also ran the program through the debugger a multitude of times (probably close to a thousand times) to ensure state and functionality was working. This alone saved me from countless bugs and helped me truly understand what was going wrong. In fact, after I confirmed the Lexer was properly assigning tokens based on the input test, it was pretty easy to track them being parsed and then evaluated. I simply could run the program from the CLI and see if anything was going wrong. Meaning any bugs that I found were easily fixed.

Integration testing was also a big deal, especially between the Lexer and the Parser, and the Parser and the Interpreter. I needed to make sure that what the Lexer handed off to the Parser was correct, or the whole program would fail. The same thing goes for the handoff between the Parser and the interpreter. A lot of time was spent making sure the large modules connected seamlessly.

Once the base language was able to lex, parse and evaluate I simply booted up a Python instance to ensure that expected results matched a language that had been tested thoroughly. Meaning every expression, every operator, every control statement has been directly compared to another language in order to make sure the basics of any programming language works flawlessly.

Lastly, I used the language to test itself. I created around 250 test cases that test almost every single part of the language within the language itself. Meaning every data type, every built-in function, every and every operation within the language has been tested. It's sort of meta, using the language to test the language, but surprisingly it works really well and saves me the trouble of writing Java code. How it works is they all connect to the ``libTest.bb`` script through the ``RUN`` in-built function, and call a function that accepts the test message, the result of the operation, and a hard coded expected result that we expect it to be after executing. These test cases can be found within ``BasicallyBasic/test/Cases`` directory, and found within the Source Code section of this document. I'll also post the results of each category of tests below to show it working.

Values

```
Cases -- -zsh -- 60x17
Expected Result: NULL
PASSED: ✓

Testing Assignment LET g
Result: NULL
Expected Result: NULL
PASSED: ✓

Testing Function PRINT
Result: FUNCTION 'PRINT'
Expected Result: FUNCTION 'PRINT'
PASSED: ✓

Results:
Passed: 29
Failed: 0
god@Ryans-iMac Cases %
```

Expressions

```
Cases -- -zsh -- 60x17
Expected Result: FALSE
PASSED: ✓

Testing NOT TRUE
Result: FALSE
Expected Result: FALSE
PASSED: ✓

Testing NOT FALSE
Result: TRUE
Expected Result: TRUE
PASSED: ✓

Results:
Passed: 80
Failed: 0
god@Ryans-iMac Cases %
```

Statements

```
Cases -- -zsh -- 60x17
Expected Result: 1
PASSED: ✓

Testing For with continue
Result: 2
Expected Result: 2
PASSED: ✓

Testing Nested for
Result: 18
Expected Result: 18
PASSED: ✓

Results:
Passed: 15
Failed: 0
god@Ryans-iMac Cases %
```

Functions

```
Expected Result: 7
PASSED: ✓

Testing EarlyReturn()
Result: 5
Expected Result: 5
PASSED: ✓

Testing NestedIf()
Result: TRUE
Expected Result: TRUE
PASSED: ✓

Results:
Passed: 10
Failed: 0
god@Ryans-iMac Cases %
```

Built-Ins

```
Expected Result: 20
PASSED: ✓

Testing RUN(libRUN.bb)
Result: NULL
Expected Result: NULL
PASSED: ✓

Testing From libRUN.bb: add(1, 2)
Result: 3
Expected Result: 3
PASSED: ✓

Results:
Passed: 81
Failed: 0
god@Ryans-iMac Cases %
```

I know showing passed test results is not needed, but it shows that BasicallyBASIC is a language that we can actually do things in. These results not only mean all the functionality works, but also that writing code to do something is possible.

Video

Below is a link to the demo video. It both shows off the program/language, but also explains some stuff within the user manual. Quite lengthy, but in my humble opinion, worth the watch.

<https://youtu.be/8hUhX38OJDk>

Conclusion and Summary

Overall, I learned a lot more than I ever expected to. It was most definitely a challenge, and there were points where it was extremely frustrating and stressful, but worth the journey.

Starting out with my first project of an SQL clone, it was a needed experience to in a way humble me (no better way to put it). I was extremely mistaken on how hard it was to create a database language with good functionality. Spending roughly four months learning everything I could about trying to implement SQL, about making a relational database as well as a functional language made me realize that certain things I just cannot do, at least not in a time span that I was okay with. SQL is a very complicated language in terms of grammar and syntax, and probably would have taken years to figure it out. When I realized that I just didn't know how to do it, I think I was on my fourth parser rewrite and just had enough at that point to call it quits and do something that was still challenging but more reasonable with my capabilities.

That's where BasicallyBASIC was born. I still knew I wanted to write my own language, relational databases was something I was curious about, but the bulk of the enjoyment when working on the SQL clone came from designing my own syntax and trying to evaluate it, rather than try to completely mimic another one. Yes, my grammar is based off of QuickBASIC/BASIC but there were a lot of liberties that I took on my own and that's where I learned the most. I wouldn't call myself an expert on language design, but I definitely gained a bunch of experience on the topic and just a lot of experience on problem solving in general.

Working at my own pace, needing to research different topics and constantly iterating through different ideas to see which ones work the best, was the highlight of working on this project. The most valuable thing I learned was simply just learning in general. It meant learning how to research, learning how to figure out complicated problems, learning different design patterns in software and learning how to make something. All that and without direct supervision, which was a great feeling and is invaluable to being in computer science.

In terms of the project, there are still a few things that I want to tackle and simply make better, but for time constraints I just can't. Specifically with error reporting and debugging. Right now it only displays the line and the error message with a description, which is more than good enough to get by and use the language, but it can be better. Having something like a stack trace to show where something went wrong in recursion, or point out the specific piece of text that made the program error would be a nice addition. Making a debugger to also walk through files would be a dream, making finding bugs thousands of times easier.

With that being said, knowing where I can add on and improve, I'm still extremely happy at what I've produced and what I've learned. This was the largest project I've ever made, being

around ten thousand lines, and exceeds a lot of my own expectations of what I've originally intended. I'm grateful for the learning experience, even if it meant having to essentially restart after the SQL clone debacle. I hope to get more experience like the kind I got when working on this project, because it most definitely made me a better programmer and better problem solver.

References and Resources

The references, and help that I received when researching and writing code for BasicallyBASIC can all be found here. They are split up into two major sections: **Design and Research**, which influenced how I wrote my code and designed it, but not directly used for exact code recreation. And **In Source**, which are resources that helped write code within my source files. There is a third for miscellaneous sources that don't really fit into either category. Of course there is just general knowledge from experience while being here at Ramapo!

Design and Research

The two most major and important sources for learning how to write an Interpreter came from:

- <https://ruslanspivak.com/lbasi-part1/>
- <https://craftinginterpreters.com/introduction.html>

Both were pivotal in learning how to actually write an interpreter and what design patterns to use. Specifically, for the recursive descent parser and the visitor design pattern for storing the abstract syntax tree and evaluating it in a maintainable way. Without these sources, my code would have deep layers of inheritance and unnecessary polymorphism, so I am grateful for the information they provided. Not only did they provide design suggestions, but also just taught me the steps involved as well as the resources I'd need to build my own language.

After reading those two texts, they led me to researching a plethora of different topics that all helped me understand how interpreters work. Below is a list of topics and their respective links on how I learned more on them:

- Interpreter and their Design Patterns:
 - https://en.wikipedia.org/wiki/Interpreter_pattern
- Abstract Syntax Trees:
 - https://en.wikipedia.org/wiki/Abstract_syntax_tree
 - <https://dev.to/balapriya/abstract-syntax-tree-ast-explained-in-plain-english-1h38>
- Lexing and Parsing:
 - https://en.wikipedia.org/wiki/Recursive_descent_parser
 - <https://home.adelphi.edu/~siegfried/cs372/372l5.pdf>
 - https://www.cs.rochester.edu/u/nelson/courses/csc_173/grammars/
 - https://www.cs.rochester.edu/u/nelson/courses/csc_173/grammars/parsing.html
 - <https://www.geeksforgeeks.org/recursive-descent-parser/>
 - https://en.wikipedia.org/wiki/Lexical_analysis
 - <https://dev.to/codingwithadam/introduction-to-lexers-parsers-and-interpreters-with-chevrotain-5c7b>
- Visitor Design and Dual Dispatch:
 - <https://www.youtube.com/watch?v=UQP5XqMqtqQ>
 - <https://www.baeldung.com/java-visitor-pattern>

- <https://www.geeksforgeeks.org/interfaces-in-java/>
- <https://medium.com/codex/compound-pattern-series-visitor-and-interpreter-part-1-31dc22140bf0>
- <https://stackoverflow.com/a/67821899>
- https://en.wikipedia.org/wiki/Separation_of_concerns
- <https://www.youtube.com/watch?v=hxGOiiR9ZKg>
- Understanding BNF/EBNF:
 - <https://problets.org/courses/opl/syntax/representation.html> (who knew Professor Kumar would come in handy here)
 - https://en.wikipedia.org/wiki/Chomsky_hierarchy
 - https://en.wikipedia.org/wiki/Formal_grammar
 - https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form
- Syntax and Grammar Rules of QuickBASIC/BASIC
 - <https://www.qbasic.net/en/reference/qb11/overview.htm>
 - <https://youtu.be/seM9SqTsRG4?si=jjd38kEyMlqxbY-D>
 - https://en.cppreference.com/w/c/language/operator_precedence
 - <https://en.wikipedia.org/wiki/BASIC>

In Source

Any help that I received either it be from my IDE suggestions or directly from the web is prefaced within the source code files as ``Help received: <link> or IDE suggestion`` within the code. It is as close as possible to the actual code that needed referencing, and should be straightforward on how I used it. Not all links to external sources are copied 1:1 (in fact most aren't), but they definitely helped in what I was trying to write, so I believe they deserved being cited. Below you'll find the topic the source was used for and the links to each source:

- Hash Maps and Sets:
 - <https://stackoverflow.com/questions/6802483/how-to-directly-initialize-a-hashmap-in-a-literal-way>
 - https://www.w3schools.com/java/ref_hashmap_getordefault.asp
 - <https://www.baeldung.com/java-initialize-hashset>
 - <https://www.baeldung.com/java-immutable-set>
 - <https://stackoverflow.com/questions/7996499/creating-hashtable-as-final-in-java>
- Files and Paths:
 - <https://www.geeksforgeeks.org/java-program-to-read-a-file-to-string/>
 - https://www.w3schools.com/java/java_files_read.asp
 - <https://stackoverflow.com/a/50948472>
 - https://www.w3schools.com/java/java_files_create.asp
 - <https://stackoverflow.com/a/23221771>
- Abstract Classes:

- <https://www.geeksforgeeks.org/abstract-classes-in-java/>
- Suppliers:
 - <https://www.baeldung.com/java-callable-vs-supplier>
- Numbers - BigInteger, BigDecimal, MathContext, and External Library for BigDecimal power function:
 - <https://docs.oracle.com/javase/8/docs/api/java/lang/Number.html>
 - <https://stackoverflow.com/a/9898528>
 - <https://www.geeksforgeeks.org/biginteger-equals-method-in-java/>
 - <https://stackoverflow.com/a/34677687>
 - <https://docs.oracle.com/javase/8/docs/api/java/math/BigInteger.html>
 - <https://docs.oracle.com/javase/8/docs/api/java/math/BigDecimal.html>
 - <https://stackoverflow.com/a/4591216>
 - <https://docs.oracle.com/javase/8/docs/api/java/math/MathContext.html>
 - <https://github.com/eobermuhlner/big-math> (used the as an external library as there are no in-built exponents for BigDecimal which is needed to facilitate the power operation).
- Strings and Regex:
 - <https://www.geeksforgeeks.org/stringbuilder-delete-in-java-with-examples/>
 - <https://www.geeksforgeeks.org/substring-in-java/>
 - <https://stackoverflow.com/a/11230042>
 - <https://www.geeksforgeeks.org/java-string-join-examples/>
 - <https://regex101.com>
 - https://www.w3schools.com/java/ref_string_lastindexof.asp
- Lists:
 - [https://docs.oracle.com/javase/7/docs/api/java/util/List.html#subList\(int,%20int\)](https://docs.oracle.com/javase/7/docs/api/java/util/List.html#subList(int,%20int))
 - <https://stackoverflow.com/questions/189559/how-do-i-join-two-lists-in-java>
 - <https://stackoverflow.com/a/32227674>

Miscellaneous

Below are items that need to be cited, but are not a part of the main code nor really influenced design:

- Syntax Highlighting and VSCode Extension:
 - https://www.youtube.com/watch?v=cHQo26fdx_o
 - <https://code.visualstudio.com/api/get-started/your-first-extension>
 - <https://code.visualstudio.com/download>
- Logo for BasicallyBASIC was Generated here:
 - <https://fontmeme.com/>
- Java Download to Run Program:
 - <https://www.oracle.com/java/technologies/downloads>

Source Code

Table of Contents (sections are clickable to jump to)

BasicallyBASIC.....	71
Language Package.....	71
Main.java.....	71
Error.java.....	76
BuiltInFunctions.java.....	79
Lexer Package.....	107
Lexer.java.....	107
OperationCode.java.....	124
Token.java.....	128
Parser Package.....	130
Parser.java.....	130
AbstractSyntaxTree Package.....	167
Node.java.....	167
NodeVisitor.java.....	168
Expressions Package.....	169
Statements Package.....	184
Interpreter Package.....	202
Interpreter.java.....	202
SymbolTable.java.....	248
Value.java.....	251
Tests.....	262
Cases Directory.....	262
libTest.bb.....	262
libRUN.bb.....	262
values.bb.....	263
expressions.bb.....	264
functions.bb.....	270
built-ins.bb.....	272
Demo Directory.....	275
VSCoDe Extension.....	280
extension.js.....	280
bbasic.tmLanguage.json.....	280
language-configuration.json.....	282
package.json.....	283

BasicallyBASIC

Language Package

Main.java

```
///  
/// Main - The entry point for the program. It initializes the global scope and interpreter for the language and  
/// registers all built-in functions. Responsible for driving the program from the console (CLI) or from a file.  
///  
  
package Language;  
  
import static Lexer.OperationCode.*;  
  
import AbstractSyntaxTree.Node;  
import Interpreter.Interpreter;  
import Interpreter.Value;  
import Interpreter.SymbolTable;  
import Parser.Parser;  
import Lexer.Lexer;  
import Lexer.Token;  
  
import java.io.IOException;  
import java.nio.file.Files;  
import java.nio.file.Path;  
import java.util.List;  
import java.util.Scanner;  
  
public class Main {  
    /* PUBLIC */  
  
    /** Constants **/  
  
    // Easily changeable constants for the language  
    public static final String FILE_EXTENSION = ".bb";           // File extension for our language  
    public static final String CONSOLE_PROMPT = ">>> ";        // Input prompt for the console  
    public static final String EXIT_PROMPT = "EXIT";           // Exit prompt so user can quit out  
  
    /** Main **/  
  
    /**/  
    /*  
    Language.Main.Main.Main()  
  
    NAME  
  
    Main - entry point for the program.
```

SYNOPSIS

```
public static void main(String[] args);
    args    -> the arguments passed in from the command line.
```

DESCRIPTION

The main function is the entry point for the program. It initializes the global scope and interpreter for the language and registers the built-in functions. It then checks the arguments passed in from the command line and runs the program accordingly; no arguments runs the program from the console (CLI), one argument runs the program from a file, and any other number means improper usage.

RETURNS

Nothing - it's the main function.

```
*/
/**/
public static void main(String[] args) {
    // Initialize the global scope and interpreter
    SymbolTable globalScope = new SymbolTable();
    m_interpreter = new Interpreter(globalScope);

    // Register built-in functions (done at construction)
    BuiltInFunctions.InitializeFunctions(globalScope);

    switch (args.length) {
        // 0 arguments means run from the console
        case 0: RunFromConsole(); return;
        // 1 argument means run from a file, try to run it and catch an IO error if can't read
        case 1:
            try { RunFromFile(args[0]); }
            catch (IOException e) { System.out.println("An error occurred reading the file!"); }
            return;

        // Any other number of arguments is improper usage
        default: System.out.println("Arguments: 0 for console, 1 for file");
    }
}
/* public static void main(String[] args); */

/* PRIVATE */

/** Members **/

private static Interpreter m_interpreter;    // Interpreter for evaluating

/** Private Helper Functions **/
```

```
/**/
/*
Language.Main.RunFromConsole()
```

NAME

RunFromConsole - runs the program from the console (CLI).

SYNOPSIS

```
public static void RunFromConsole();
```

DESCRIPTION

The function runs the program from the console, otherwise known as a CLI or REPL. It scans each line of input from the user and runs it through the interpreter. If the user enters the exit prompt, or CTRL+D the program will exit. It outputs the result of the expression if there is one, i.e. the Value is not our NULL.

RETURNS

Nothing - it's a void function.

```
*/
/**/
public static void RunFromConsole() {
    Scanner scanner = new Scanner(System.in);
    String input;

    // Scan each line until the user exits
    while (true) {
        System.out.print(CONSOLE_PROMPT);

        // If user enters CTRL+D, we need to break out of the loop
        if (!scanner.hasNextLine()) { break; }
        input = scanner.nextLine();

        // If user can use key bind to exit any console, but an exit prompt is also nice
        if (input == null || input.equals(EXIT_PROMPT)) { break; }

        // If we resulted in a value, i.e. from an expression, show the user it as it's a CLI
        Value result = Run(input);
        // Only print if there is a non-null (both Java and our NULL) result
        if (result != null && !result.IsNull()) { System.out.println(result.FormatValueToString()); }
    }
}
/* public static void RunFromConsole(); */

/**/
/*
Language.Main.RunFromFile()
```

NAME

RunFromFile - runs the program from a file.

SYNOPSIS

```
public static void RunFromFile(String a_filePath);
    a_filePath    -> the string of the path to the file to run.
```

DESCRIPTION

The function runs the program from a file. It checks if the file has the correct extension and if it exists. If it does, it reads the entire file into a string and runs it through the interpreter. It does not care about the value returned as it's a file and not the CLI, so it outputs only the print statements done by the user.

RETURNS

Nothing - it's a void function.

```
*/
/**/
public static void RunFromFile(String a_filePath) throws IOException {
    // Check if right extension
    if (!a_filePath.endsWith(FILE_EXTENSION)) {
        System.out.println("Invalid file extension. Must be a " + FILE_EXTENSION + " file!");
        return;
    }

    // Check if file exists - can't run if it doesn't
    // Help received: https://stackoverflow.com/questions/50948361/adding-sub-directory-to-an-existing-path
    Path path = Path.of(System.getProperty("user.dir") + "/" + a_filePath);
    if (!Files.exists(path)) {
        System.out.println("File '" + a_filePath + "' not found!");
        return;
    }

    // Read entire file into a string and run it - we don't care about the value return as it's a file not the CLI
    // Help Received: https://www.geeksforgeeks.org/java-program-to-read-a-file-to-string/
    try { Run(Files.readString(path).trim()); }
    catch (IOException e) { System.out.println("An error occurred reading the file!"); }
}

/* public static void RunFromFile(String a_filePath); */

/** Private Helper Functions **/

/**/
/*
Language.Main.Run()
```

NAME

Run - runs the program from a string.

SYNOPSIS

```
private static Value Run(String a_input);
    a_input    -> the string of the program to run.
```

DESCRIPTION

The function runs the program from a string, only executing the program if there is content in the string and is not just whitespace. It lexes the input into tokens, parses the tokens into an abstract syntax tree, and evaluates the tree (or program). It will also initialize our error class to our input for displaying our errors.

If at any point we generate a language error, it prints it out immediately. If an unexpected error occurs, it means there is a bug and the developer should be notified.

RETURNS

Value - the result of the program, or Value.NULL if there was an error.

```
*/
/**/
// Want to print out the stack trace as we don't have a proper logger
@SuppressWarnings("CallToPrintStackTrace")
private static Value Run(String a_input) {
    // Don't bother parsing if there is nothing there
    if (a_input.isBlank()) { return Value.NULL; }

    // Error gets a static copy of the input to display the error message, instead of having it in each class
    Error.InitializeInput(a_input);

    try {
        // Lex the input into tokens
        Lexer lexer = new Lexer(a_input);
        List<Token> tokens = lexer.GenerateAllTokens();

        // Comment or whitespace, don't bother parsing
        if (tokens.isEmpty() || tokens.get(0).GetOpcode() == EOF) { return Value.NULL; }

        // Parse tokens into an abstract syntax tree
        Parser parser = new Parser(tokens);
        List<Node> program = parser.ParseProgram();

        // Evaluate the tree (or program)
        return m_interpreter.Interpret(program);
    }
    // If at any point we generate a language error, print it out immediately
    catch (Error inBUILTError) { System.out.print(inBUILTError.GetMessage()); }
    // If unexpected error occurs (a bug), print it out for developer to see
```

```

        catch (Exception javaError) {
            System.out.println("An unexpected error occurred!");
            javaError.printStackTrace();
        }

        // In case we don't have a value to return (we errored out)
        return Value.NULL;
    }
    /* private static Value Run(String a_input); */
}

```

Error.java

```

///
/// Error - Used for error handling in the language. This is used to throw exceptions when an error occurs in the users
/// code, such as a syntax error, type error, runtime error, etc. This is used to inform the user of what went
wrong
/// and where it went wrong.
///

package Language;

import Lexer.OperationCode;

public class Error extends RuntimeException {
    /* PUBLIC */

    /** Constants */

    // Using an enum instead of polymorphism to avoid having classes with nothing in them. The only thing that changes
    // is the name/type of error.
    public enum ErrorType {
        SYNTAX_ERROR          ("Syntax"),           // Error in the syntax of the language
        TYPE_ERROR            ("Type"),             // Data type is not correct
        RUNTIME_ERROR         ("Runtime"),          // Something went wrong during execution
        ARITHMETIC_ERROR       ("Arithmetic"),       // Arithmetic operation failed
        OUT_OF_BOUNDS_ERROR    ("Index Out of Bounds"), // List/String was accessed out of bounds
        UNDEFINED_IDENTIFIER   ("Undefined Identifier"), // Identifier was not found in the symbol table
        BUILT_IN_ERROR        ("Built-In Function"); // Error in a built-in function

        // Label for the error type - used as constructor to avoid unnecessary switch statements
        ErrorType(String a_label) { LABEL = a_label + " Error"; }
        private final String LABEL; // The label for the error type when displaying to the user
    }

    /** Constructors */

    /**/
    /*
    Language.Error.Error()
    */

```


NAME

Error - constructor for the error class. Should be thrown.

SYNOPSIS

```
Error(String a_message, Error.ErrorType a_errorType, int a_lineNumber);
    a_message      -> the message to display to the user.
    a_errorType    -> the type of error that occurred.
    a_lineNumber   -> the line number the error occurred on.
```

DESCRIPTION

Initializes the error message, error type, and line number of the error. Specifically used for throwing exceptions when an error occurs in the user code. Based on what error it is, line number and error message, it generates an error so the user can have some idea of what went wrong and where.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Error(String a_message, ErrorType a_errorType, int a_lineNumber) {
    m_errorMessage = a_message;
    m_errorType    = a_errorType;
    m_lineNumber   = a_lineNumber;
    GenerateErrorMessage();
}
/* Error(String a_message, Error.ErrorType a_errorType,
    int a_lineNumber); */

/** Accessors **/

// Gets the error message to display to the user
public String GetMessage() {
    return m_errorMessage;
}

// Gets the input text for error reporting
public static String GetInput() {
    return m_text;
}

/** Methods **/

// Initializes the input text for error reporting - static as we don't need to store multiple instances of the text
public static void InitializeInput(String a_text) {
    m_text = a_text;
}
```

```

}

public static void SaveState(Error a_error) {

}

/* PRIVATE */

/** Members */

private static String m_text;           // The text the user entered, static as we don't need multiple copies
private String m_errorMessage;         // The error message to display to the user
private final ErrorType m_errorType;   // The type of error that occurred (e.g. syntax, type, runtime, etc.)
private int m_lineNumber;              // The line number the error occurred on

/** Private Helper Functions */

/**/
/*
Language.Error.GenerateErrorMessage()

NAME

    GenerateErrorMessage - generates the error message to display to the
    user.

SYNOPSIS

    private void GenerateErrorMessage();

DESCRIPTION

    Generates the error message to display to the user. It splits the text
    into lines and then shows the error message, the line number and the
    line itself, to point where the error occurred. Message is split up with
    the '\n' and ';' characters to make it easier to read.

RETURNS

    Nothing - it's a void function.

*/
/**/
private void GenerateErrorMessage() {
    // Help received: https://docs.oracle.com/javase/8/docs/api/java/lang/StringBuilder.html
    StringBuilder message = new StringBuilder();

    // Split the current text into lines
    String[] lines = m_text.split("\n|" + OperationCode.NEWLINE.SYMBOL);

    // In case the line number is out of bounds, just use last line. Will only occur if last characters in string

```

```

//      are the new line ones (.split() removes trailing empty strings).
if (m_lineNumber > lines.length) { m_lineNumber = lines.length; }

// Show the error message
message.append("\n")
        .append(m_errorType.LABEL)
        .append(" at line ")
        .append(m_lineNumber)
        .append(": ")
        .append(m_errorMessage)
        .append("!\n");

// Show the line where the error occurred
message.append("\n\t")
        .append(m_lineNumber)
        .append(" |\t")
        .append(lines[m_lineNumber - 1])
        .append(" \n\n");

m_errorMessage = message.toString();
}
/* private void GenerateErrorMessage(); */
}

```

BuiltInFunctions.java

```

///
/// Built-In Functions - Functions that are built into the language that provide utility that is not accessible through
/// the languages' syntax. For example PRINT() and PRINTLN() are built-in functions that print to the console, but
/// if they were build into the language it would mean each one would need it own node and syntax to call it. This
/// way it can be generalized and called like any other function. This class is a collection of built-in functions
/// and are initialized within the global scope of the program. All built-ins must return a Value and take a list
/// of
/// arguments.
///
package Language;

import static Language.Main.RunFromFile;

import AbstractSyntaxTree.Statements.Function;
import Interpreter.SymbolTable;
import Interpreter.Value;

import java.io.IOException;
import java.math.BigDecimal;
import java.math.BigInteger;
import java.nio.file.*;
import java.util.List;
import java.util.Scanner;

```

```

public class BuiltInFunctions {
    /* PUBLIC */

    // This functional interface can define a Java function that takes a list of arguments and returns a Value. It,
    //     allows us to pass a Java function into a parameter so it can be resolved in the SymbolTable. Though each
    //     built-in has a specific number of arguments, and the list makes it less clear, it makes it easier to call
    //     and resolve the built-in. Done to allow for flexibility in how many arguments a function can take, so if we
    //     wanted to add a function with 20 arguments we don't need to create a new interface to handle it. It does
    //     make it harder to know how many arguments a function takes in their definition, but the flexibility is worth
    //     the tradeoff. Regardless, arity is checked beforehand they can't be called with wrong number of arguments.
    // Help received: https://www.geeksforgeeks.org/functional-interfaces-java/
    public interface BuiltIn {
        Value Execute(List<Value> arguments);
    }

    // Similar to the Jump node, we have a custom exception for built-ins that allow us to notify the place where the
    //     function was called that something went wrong. This is useful as it allows us to break out of the current
    //     recursive call to get the line to be more descriptive in the error message. This is a simple class that only
    //     holds a message, so it doesn't need its own file and can be nested inside the BuiltInFunctions.
    public static class BuiltInFunctionException extends RuntimeException {
        public BuiltInFunctionException(String a_message) { m_message = a_message; }
        public String GetMessage() { return m_message; }
        public final String m_message;
    }

    /** Public Utility Methods */

    /**/
    /*
    Language.BuiltInFunctions.InitializeFunctions()

```

NAME

InitializeFunctions - initializes all the built-in functions for the language, putting them in the global scope.

SYNOPSIS

```

public static void InitializeFunctions(SymbolTable a_globalScope);
    a_globalScope -> the global scope of the program.

```

DESCRIPTION

Initializes all the built-in functions for the language, putting them in the global scope. This is done so that they can be accessed by the interpreter when a function is called. Each function must be registered here for the interpreter to know about it. And each function created must specify the parameter count and the Java function it uses to execute.

For example the PRINT() function is registered here with a parameter count of 1 and the Java function BuiltInFunctions::PRINT() that takes a

list of arguments (but can only use the first one) and prints it to the console.

RETURNS

Nothing - it's a void function.

```
*/
/**/
public static void InitializeFunctions(SymbolTable a_globalScope) {
    m_globalScope = a_globalScope;

    // Register all the built-in functions: Usage is RegisterFunction(name, parameter count, function)
    RegisterFunction("PRINT",      1, BuiltInFunctions::PRINT);
    RegisterFunction("PRINTLN",    1, BuiltInFunctions::PRINTLN);
    RegisterFunction("INPUT",      0, BuiltInFunctions::INPUT);
    RegisterFunction("IS_INTEGER", 1, BuiltInFunctions::IS_INTEGER);
    RegisterFunction("IS_FLOAT",   1, BuiltInFunctions::IS_FLOAT);
    RegisterFunction("IS_NUMBER", 1, BuiltInFunctions::IS_NUMBER);
    RegisterFunction("IS_STRING",  1, BuiltInFunctions::IS_STRING);
    RegisterFunction("IS_BOOLEAN", 1, BuiltInFunctions::IS_BOOLEAN);
    RegisterFunction("IS_LIST",    1, BuiltInFunctions::IS_LIST);
    RegisterFunction("IS_FUNCTION", 1, BuiltInFunctions::IS_FUNCTION);
    RegisterFunction("IS_NULL",    1, BuiltInFunctions::IS_NULL);
    RegisterFunction("TO_INTEGER",  1, BuiltInFunctions::TO_INTEGER);
    RegisterFunction("TO_FLOAT",    1, BuiltInFunctions::TO_FLOAT);
    RegisterFunction("TO_STRING",   1, BuiltInFunctions::TO_STRING);
    RegisterFunction("APPEND",      2, BuiltInFunctions::APPEND);
    RegisterFunction("PREPEND",    2, BuiltInFunctions::PREPEND);
    RegisterFunction("INSERT",     3, BuiltInFunctions::INSERT);
    RegisterFunction("POP",        1, BuiltInFunctions::POP);
    RegisterFunction("REMOVE",     2, BuiltInFunctions::REMOVE);
    RegisterFunction("EXTEND",     2, BuiltInFunctions::EXTEND);
    RegisterFunction("LENGTH",     1, BuiltInFunctions::LENGTH);
    RegisterFunction("READ",       1, BuiltInFunctions::READ);
    RegisterFunction("WRITE",      2, BuiltInFunctions::WRITE);
    RegisterFunction("RUN",        1, BuiltInFunctions::RUN);
}
/* public static void InitializeFunctions(SymbolTable a_globalScope); */

/* PRIVATE */

/** Members **/

private static SymbolTable m_globalScope;    // The global scope of the program

/** Function Registry **/

/**/
/*
Language.BuiltInFunctions.RegisterFunction()
```

NAME

RegisterFunction - registers a built-in function with the global scope.

SYNOPSIS

```
private static void RegisterFunction(String a_name, int a_parameterCount,
                                     BuiltIn a_function);
    a_name           -> the name of the function.
    a_parameterCount -> the number of parameters the function takes.
    a_function       -> the built-in Java function to call.
```

DESCRIPTION

Registers a built-in function with the global scope so a user can call it at a later time. This is done by creating a new Function object with the name, parameter count, and built-in function to call, then transformed into a value to put in our SymbolTable.

This is a helper function for InitializeFunctions() that allows us to avoid code duplication and keep the code clean.

RETURNS

Nothing - it's a void function.

```
*/
/**/
private static void RegisterFunction(String a_name, int a_parameterCount, BuiltIn a_function) {
    Function function = new Function(a_name, a_parameterCount, a_function);
    m_globalScope.PutIdentifier(a_name, new Value(function));
}
/* private static void RegisterFunction(String a_name, int a_parameterCount,
                                     BuiltIn a_function); */

/** Built-Ins - All must return a Value, should be NULL if no return. All built-ins should have method headers! **/

/**/
/*
Language.BuiltInFunctions.PRINT()
```

NAME

PRINT - prints a Value to the console. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value PRINT(List<Value> a_argument);
    a_argument -> the Value to print.
```

DESCRIPTION

Prints a Value to the console without a newline. Only uses 1 argument.

RETURNS

A NULL Value object.

```
*/
/**/
private static Value PRINT(List<Value> a_argument) {
    Value value = a_argument.get(0);
    System.out.print(GetPrintableValue(value));
    return Value.NULL;
}
/* private static Value PRINT(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.PRINTLN()
```

NAME

PRINTLN - prints a Value to the console with a newline. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value PRINTLN(List<Value> a_argument);
a_argument -> the value to print.
```

DESCRIPTION

Prints a Value to the console with a newline. Only uses 1 argument.

RETURNS

A NULL Value object.

```
*/
/**/
private static Value PRINTLN(List<Value> a_argument) {
    Value value = a_argument.get(0);
    System.out.println(GetPrintableValue(value));
    return Value.NULL;
}
/* private static Value PRINTLN(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.INPUT()
```

NAME

INPUT - gets input from the user. DOESN'T TAKE ANY ARGUMENTS.

SYNOPSIS

```
private static Value INPUT(List<Value> NONE);  
    NONE -> no arguments.
```

DESCRIPTION

Gets input from the user and returns it as a string. Not cast as it is users responsibility to convert it to the appropriate type.

RETURNS

A String Value object with the user's input.

```
*/  
/**/  
private static Value INPUT(List<Value> NONE) {  
    Scanner scanner = new Scanner(System.in);  
    String input = scanner.nextLine();  
  
    // Up to user to convert to appropriate type - more about user agency than anything  
    return new Value(input);  
}  
/* private static Value INPUT(List<Value> NONE); */  
  
/**/  
/*  
Language.BuiltInFunctions.IS_INTEGER()
```

NAME

IS_INTEGER - checks if a Value is an integer. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_INTEGER(List<Value> a_argument);  
    a_argument -> the Value to check.
```

DESCRIPTION

Checks if a value is an integer. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our integer.

```
*/  
/**/  
private static Value IS_INTEGER(List<Value> a_argument) {  
    Value value = a_argument.get(0);
```



```

        return new Value(value.IsInteger());
    }
    /* private static Value IS_INTEGER(List<Value> a_argument); */

```

```

/**/
/*
Language.BuiltInFunctions.IS_FLOAT()

```

NAME

IS_FLOAT - checks if a Value is a float. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```

private static Value IS_FLOAT(List<Value> a_argument);
    a_argument -> the Value to check.

```

DESCRIPTION

Checks if a Value is a float. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our float.

```

*/
/**/
private static Value IS_FLOAT(List<Value> a_argument) {
    Value value = a_argument.get(0);
    return new Value(value.IsFloat());
}
/* private static Value IS_FLOAT(List<Value> a_argument); */

```

```

/**/
/*
Language.BuiltInFunctions.IS_NUMBER()

```

NAME

IS_NUMBER - checks if a Value is a number. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```

private static Value IS_NUMBER(List<Value> a_argument);
    a_argument -> the Value to check.

```

DESCRIPTION

Checks if a Value is a number, i.e. if the Value is our integer or float. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is an integer or float.

```
*/  
/**/  
private static Value IS_NUMBER(List<Value> a_argument) {  
    Value value = a_argument.get(0);  
    return new Value(value.IsNumber());  
}
```

```
/**/  
/*  
Language.BuiltInFunctions.IS_STRING()
```

NAME

IS_STRING - checks if a Value is a string. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_STRING(List<Value> a_argument);  
a_argument -> the Value to check.
```

DESCRIPTION

Checks if a Value is a string. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our string.

```
*/  
/**/  
private static Value IS_STRING(List<Value> a_argument) {  
    Value value = a_argument.get(0);  
    return new Value(value.IsString());  
}  
/* private static Value IS_STRING(List<Value> a_argument); */
```

```
/**/  
/*  
Language.BuiltInFunctions.IS_BOOLEAN()
```

NAME

IS_BOOLEAN - checks if a Value is a boolean. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_BOOLEAN(List<Value> a_argument);  
a_argument -> the Value to check.
```

DESCRIPTION

Checks if a Value is a boolean. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our boolean.

```
*/
/**/
private static Value IS_BOOLEAN(List<Value> a_argument) {
    Value value = a_argument.get(0);
    return new Value(value.IsBoolean());
}
/* private static Value IS_BOOLEAN(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.IS_LIST()
```

NAME

IS_LIST - checks if a Value is a list. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_LIST(List<Value> a_argument);
    a_argument -> the Value to check.
```

DESCRIPTION

Checks if a Value is a list. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our list.

```
*/
/**/
private static Value IS_LIST(List<Value> a_argument) {
    Value value = a_argument.get(0);
    return new Value(value.IsList());
}
/* private static Value IS_LIST(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.IS_FUNCTION()
```

NAME

IS_FUNCTION - checks if a Value is a function. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_FUNCTION(List<Value> a_argument);  
    a_argument -> the Value to check.
```

DESCRIPTION

Checks if a Value is a function. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is our function.

```
*/  
/**/  
private static Value IS_FUNCTION(List<Value> a_argument) {  
    Value value = a_argument.get(0);  
    return new Value(value.IsFunction());  
}  
/* private static Value IS_FUNCTION(List<Value> a_argument); */  
  
/**/  
/*  
Language.BuiltInFunctions.IS_NULL()
```

NAME

IS_NULL - checks if a Value is null. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value IS_NULL(List<Value> a_argument);  
    a_argument -> the Value to check.
```

DESCRIPTION

Checks if a Value is null. Only uses 1 argument. Need to check the type of the Value - standard across all languages.

RETURNS

A Boolean Value object, if the Value is null.

```
*/  
/**/  
private static Value IS_NULL(List<Value> a_argument) {  
    Value value = a_argument.get(0);  
    return new Value(value.IsNull());  
}  
}
```

```
/* private static Value IS_NULL(List<Value> a_argument); */
```

```
/**/
```

```
/*
```

```
Language.BuiltInFunctions.TO_INTEGER()
```

NAME

TO_INTEGER - converts a Value to an integer. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value TO_INTEGER(List<Value> a_argument);  
a_argument -> the Value to convert.
```

DESCRIPTION

Converts a Value to an integer if it can be converted. For example, if the Value is a string "5" it will convert it to the integer 5, or if it is a float 5.6 it will convert it to the integer 5. Lists, booleans, functions, and NULLs cannot be converted to integers - and an exception will be thrown if it is one of these values (or if a bad string). Only uses 1 argument.

RETURNS

An integer Value object, if it was able to convert.

```
*/
```

```
/**/
```

```
private static Value TO_INTEGER(List<Value> a_argument) {  
    Value value = a_argument.get(0);  
  
    // Convert to number than to integer  
    BigDecimal convertedValue = ConvertToNumber(value, Interpreter.Value.DataType.INTEGER);  
    return new Value(convertedValue.toBigInteger());  
}  
/* private static Value TO_INTEGER(List<Value> a_argument); */
```

```
/**/
```

```
/*
```

```
Language.BuiltInFunctions.TO_FLOAT()
```

NAME

TO_FLOAT - converts a Value to a float. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value TO_FLOAT(List<Value> a_argument);  
a_argument -> the Value to convert.
```

DESCRIPTION

Converts a Value to a float if it can be converted. For example, if the Value is a string "5.6" it will convert it to the float 5.6, or if it is an integer 5 it will convert it to the float 5.0. Lists, booleans, functions, and NULLs cannot be converted to floats - and an exception will be thrown if it is one of these values (or if a bad string). Only uses 1 argument.

RETURNS

A float Value object, if it was able to convert.

```
*/
/**/
private static Value TO_FLOAT(List<Value> a_argument) {
    Value value = a_argument.get(0);

    // Convert to BigDecimal, then into our FLOAT value
    BigDecimal convertedValue = ConvertToNumber(value, Interpreter.Value.DataType.FLOAT);
    return new Value(convertedValue);
}
/* private static Value TO_FLOAT(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.TO_STRING()
```

NAME

TO_STRING - converts a Value to a string. ONLY TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value TO_STRING(List<Value> a_argument);
    a_argument -> the Value to convert.
```

DESCRIPTION

Converts a Value to a string. For example, if the Value is an integer 5 it will convert it to the string "5", or if it is a float 5.6 it will convert it to the string "5.6". Lists will be 'pretty printed' as something like [1, 2, "hello"]. Booleans will be shown as "TRUE" or "FALSE" and NULL as "NULL". Functions will be shown as "FUNCTION <name>" where <name> is the function name. Strings on the other hand will not be converted to their 'pretty print'. Only uses 1 argument.

RETURNS

A string Value object of the 'pretty printed' Value.

```
*/
/**/
```

```
private static Value TO_STRING(List<Value> a_argument) {
    Value value = a_argument.get(0);

    // We don't want to format strings (don't want to add quotes)
    if (value.IsString()) { return value; }

    // Everything else can be formatted
    return new Value(value.FormatValueToString());
}
/* private static Value TO_STRING(List<Value> a_argument); */
```

/**/

/*

Language.BuiltInFunctions.APPEND()

NAME

APPEND - appends a Value to the end of a list. TAKES 2 ARGUMENTS, FIRST ARGUMENT MUST BE A LIST.

SYNOPSIS

```
private static Value APPEND(List<Value> a_arguments);
    a_arguments -> the list and value to append (0 is list, 1 is value)
```

DESCRIPTION

Appends a Value to the end of a list. The first argument must be a list and the second argument is the value to append. If the first argument is not a list, an exception will be thrown. The original list is not modified, rather the new list is created with the value appended to the end. Only uses 2 arguments.

RETURNS

A new list Value object with the Value appended to the end.

*/

/**/

```
private static Value APPEND(List<Value> a_arguments) {
    int size = MakeRawList(a_arguments).size();
    return InsertValue(a_arguments, new Value(BigInteger.valueOf(size)));
}
/* private static Value APPEND(List<Value> a_arguments); */
```

/**/

/*

Language.BuiltInFunctions.PREPEND()

NAME

PREPEND - prepends a Value to the beginning of a list. TAKES 2 ARGUMENTS, FIRST ARGUMENT MUST BE A LIST.

SYNOPSIS

```
private static Value PREPEND(List<Value> a_arguments);  
    a_arguments -> the list and value to prepend (0 is list, 1 is value)
```

DESCRIPTION

Prepends a Value to the beginning of a list. The first argument must be a list and the second argument is the value to prepend. If the first argument is not a list, an exception will be thrown. The original list is not modified, rather the new list is created with the value prepended to the beginning. Only uses 2 arguments.

RETURNS

A new list Value object with the Value prepended to the beginning.

```
*/  
/**/  
private static Value PREPEND(List<Value> a_arguments) {  
    // Insert at beginning, so index is 0  
    return InsertValue(a_arguments, new Value(BigInteger.ZERO));  
}  
/* private static Value PREPEND(List<Value> a_arguments); */  
  
/**/  
/*  
Language.BuiltInFunctions.INSERT()
```

NAME

INSERT - inserts a Value at a specified index in a list. TAKES 3 ARGUMENTS, FIRST ARGUMENT MUST BE A LIST AND THE THIRD ARGUMENT MUST BE THE INDEX TO INSERT AT.

SYNOPSIS

```
private static Value INSERT(List<Value> a_argument);  
    a_arguments -> the list, value to insert, and index to insert at  
    (0 is list, 1 is value, 2 is index)
```

DESCRIPTION

Inserts a Value at a specified index in a list. The first argument must be a list, the second argument is the value to insert, and the third argument is the index to insert at. If the first argument is not a list, an exception will be thrown. If the third argument is not an integer, an exception will also be thrown - it also must be in bounds of the length of the list.

The original list is not modified, rather the new list is created with the value inserted at the specified index. Only uses 3 arguments.

RETURNS

A new list Value object with the Value inserted at the specified index.

```
*/
/**/
private static Value INSERT(List<Value> a_arguments) {
    // Insert at specified index (third argument)
    return InsertValue(a_arguments, a_arguments.get(2));
}
/* private static Value INSERT(List<Value> a_argument); */

/**/
/*
Language.BuiltInFunctions.POP()
```

NAME

POP - removes the last element from a list. TAKES 1 ARGUMENT, WHICH MUST BE A LIST.

SYNOPSIS

```
private static Value POP(List<Value> a_argument);
a_argument -> the list to remove the last element from.
```

DESCRIPTION

Removes the last element from a list. The argument must be a list. If it is not a list, an exception will be thrown. The original list is not modified, rather the new list is created without the last element. Only uses 1 argument.

RETURNS

A new list Value object with the last element removed, if it can.

```
*/
/**/
private static Value POP(List<Value> a_argument) {
    // Ensure argument is a list and remove it from the end
    Value list = a_argument.get(0);
    EnforceArgumentType(Value.DataType.LIST, list.GetDataType(), 0);
    int endIndex = list.AsList().size() - 1;

    // Return the list without the last element
    return RemoveValue(a_argument, new Value(BigInteger.valueOf(endIndex)));
}
/* private static Value POP(List<Value> a_argument); */

/**/
/*
```

Language.BuiltInFunctions.REMOVE()

NAME

REMOVE - removes an element from a list at a specified index. TAKES 2 ARGUMENTS, FIRST ARGUMENT MUST BE A LIST AND THE SECOND ARGUMENT MUST BE THE INDEX TO REMOVE AT.

SYNOPSIS

```
private static Value REMOVE(List<Value> a_argument);  
    a_arguments -> the list and index to remove at (0 is list, 1 is  
        index)
```

DESCRIPTION

Removes an element from a list at a specified index. The first argument must be a list and the second argument is the index to remove at. If the first argument is not a list, an exception will be thrown. If the second argument is not an integer, an exception will also be thrown - it also must be in bounds of the length of the list.

The original list is not modified, rather the new list is created without the element at the specified index. Only uses 2 arguments.

RETURNS

A new list Value object with the element removed at the specified index.

```
*/  
/**/  
private static Value REMOVE(List<Value> a_arguments) {  
    // Remove at specified index (second argument) and return the list  
    return RemoveValue(a_arguments, a_arguments.get(1));  
}  
/* private static Value REMOVE(List<Value> a_argument); */  
  
/**/  
/*  
Language.BuiltInFunctions.EXTEND()
```

NAME

EXTEND - extends a list by adding all elements from another list. TAKES 2 ARGUMENTS, BOTH MUST BE LISTS.

SYNOPSIS

```
private static Value EXTEND(List<Value> a_arguments);  
    a_arguments -> the list to extend and the list to add (0 is list to  
        extend, 1 is list to add)
```

DESCRIPTION

Extends a list by adding all elements from another list. Both arguments must be lists. If either argument is not a list, an exception will be thrown. The original list is not modified, rather the new list is created with all the elements from the second list added to the end of the first list. This operation is the '+' operator on the two lists, but as a built-in function. Only uses 2 arguments.

RETURNS

A new list Value object with all elements from the second list added to the end of the first list, if it can.

```
*/
/**/
private static Value EXTEND(List<Value> a_arguments) {
    Value leftList = a_arguments.get(0);
    Value rightList = a_arguments.get(1);

    // Ensure both arguments are lists
    EnforceArgumentType(Value.DataType.LIST, leftList.GetDataType(), 0);
    EnforceArgumentType(Value.DataType.LIST, rightList.GetDataType(), 1);

    // Add all elements from right list to left list and return
    // Help received: https://stackoverflow.com/questions/1128723/how-to-join-two-lists-in-java
    List<Value> rawList = leftList.AsList();
    rawList.addAll(rightList.AsList());
    return new Value(rawList);
}
/* private static Value EXTEND(List<Value> a_arguments); */

/**/
/*
Language.BuiltInFunctions.LENGTH()
```

NAME

LENGTH - gets the length of a string or list. TAKES 1 ARGUMENT.

SYNOPSIS

```
private static Value LENGTH(List<Value> a_argument);
    a_argument -> the string or list to get the length of.
```

DESCRIPTION

Gets the length of a string or list. The argument must be a string or list. If it is not a string or list, an exception will be thrown. Only uses 1 argument.

RETURNS

An integer Value object with the length of the string or list.

```

*/
/**/
private static Value LENGTH(List<Value> a_argument) {
    Value value = a_argument.get(0);

    // Help received: IDE suggestion to use switch expression
    int length = switch (value.GetDataType()) {
        case STRING -> value.AsString().length();
        case LIST -> value.AsList().size();

        // Can only get length of strings and lists (only time we check for 2 types, so not in a helper)
        default ->
            throw new BuiltInFunctionException("Argument must be a '" + Value.DataType.STRING + "'" +
                " or '" + Value.DataType.LIST + "'");
    };

    return new Value(BigInteger.valueOf(length));
}
/* private static Value LENGTH(List<Value> a_argument); */

```

/**/

/*

Language.BuiltInFunctions.READ()

NAME

READ - reads the content of a file. TAKES 1 ARGUMENT, MUST BE A STRING.

SYNOPSIS

```

private static Value READ(List<Value> a_argument);
    a_argument -> the file name to read.

```

DESCRIPTION

Reads the content of a file. The argument must be a string. If it is not a string, an exception will be thrown. If the file is not found, or an error occurs while reading, an exception will be thrown. Only uses 1 argument.

RETURNS

A string Value object with the content of the file.

*/

/**/

```

private static Value READ(List<Value> a_argument) {
    // Get the path based off the file name
    Path path = MakeReadPath(a_argument.get(0));

    // Try to read the file and return the content
    try { return new Value(Files.readString(path)); }

```

```

        catch (IOException ignored) { throw new BuiltInFunctionException("Could not read file '" + path + "'"); }
    }
    /* private static Value READ(List<Value> a_argument); */

```

```
/**/
```

```
/*
```

```
Language.BuiltInFunctions.WRITE()
```

NAME

WRITE - writes content to a file. TAKES 2 ARGUMENTS, BOTH MUST BE STRINGS.

SYNOPSIS

```

private static Value WRITE(List<Value> a_arguments);
    a_arguments -> the file name to write to and the content to write.

```

DESCRIPTION

Writes content to a file. The first argument must be a string with the file name and the second argument must be a string with the content to write. If either argument is not a string, an exception will be thrown. If the file cannot be written to, an exception will be thrown. Only uses 2 arguments.

RETURNS

A NULL Value object.

```
*/
```

```
/**/
```

```

private static Value WRITE(List<Value> a_arguments) {
    // First argument is file name, second is content - both should be strings
    Value fileName = a_arguments.get(0);
    EnforceArgumentType(Value.DataType.STRING, fileName.GetDataType(), 0);
    Value content = a_arguments.get(1);
    EnforceArgumentType(Value.DataType.STRING, content.GetDataType(), 1);

    // Get the file path and write the content to it
    Path filePath = Paths.get(fileName.AsString());

    // Write the content to the file (overwrites if the file already exists) if we can
    // Help received: https://stackoverflow.com/a/23221771
    try { Files.writeString(filePath, content.AsString()); }
    catch (NoSuchFileException error) { throw new BuiltInFunctionException("Dir '" + filePath + "' not found"); }
    catch (IOException error) { throw new BuiltInFunctionException("Could not write to file '" + filePath + "'"); }

    return Value.NULL;
}
/* private static Value WRITE(List<Value> a_arguments); */

/**/

```

```
/*  
Language.BuiltInFunctions.RUN()
```

NAME

RUN - runs a file. TAKES 1 ARGUMENT, MUST BE A STRING.

SYNOPSIS

```
private static Value RUN(List<Value> a_argument);  
    a_argument -> the file name to run.
```

DESCRIPTION

Runs a .bb program file. Any functions or variables defined in the file will be added to the global scope. Extremely useful for importing large amounts of code. The argument must be a string, an exception will be thrown if not. If the file is not found, does not have a '.bb' extension, or an error occurs while running, an exception will be thrown to notify the user. Only uses 1 argument.

RETURNS

A NULL Value object.

```
*/  
/**/  
private static Value RUN(List<Value> a_argument) {  
    // Get the file path from first argument (filename)  
    Path filePath = MakeReadPath(a_argument.get(0));  
    EnforceFileExtension(filePath);  
  
    // Save the current file we are on  
    String previousText = Error.GetInput();  
  
    // Use our Main class to run the file since it does exactly what we want (we don't care about the return value)  
    try { RunFromFile(filePath.toString()); }  
    catch (IOException error) { throw new BuiltInFunctionException("Could not run file '" + filePath + "'"); }  
  
    // Reset our errors  
    Error.InitializeInput(previousText);  
  
    return Value.NULL;  
}  
/* private static Value RUN(List<Value> a_argument); */  
  
/** Private Helper Function for Built-Ins **/  
  
/**/  
/*  
Language.BuiltInFunctions.GetPrintableValue()
```

NAME

GetPrintableValue - gets a printable string from a Value.

SYNOPSIS

```
private static String GetPrintableValue(Value a_value);  
    a_value -> the Value to get a printable string from.
```

DESCRIPTION

Gets a printable string from a Value. If the Value is a string, it will return the string to avoid quotes. If it is not a string, it will return the formatted string of the Value for pretty print. This is a helper function for the PRINT() and PRINTLN() built-in functions.

RETURNS

A string that is printable from the Value.

```
*/  
/**/  
private static String GetPrintableValue(Value a_value) {  
    if (a_value.IsString()) { return a_value.AsString(); }  
    else { return a_value.FormatValueToString(); }  
}  
  
/**/  
/*  
Language.BuiltInFunctions.ConvertToNumber()
```

NAME

ConvertToNumber - converts a Value to a specified number type.

SYNOPSIS

```
private static BigDecimal ConvertToNumber(Value a_value,  
                                           Value.DataType a_toType);  
    a_value -> the Value to convert.  
    a_toType -> the type to convert to.
```

DESCRIPTION

Converts a Value to a specified number type. The Value must be an integer, float, or string that can be converted to a number. If it is not one of these types, an exception will be thrown. If the string is not a valid number, an exception will also be thrown. This is a helper function for the TO_INTEGER() and TO_FLOAT() built-in functions.

Uses regex to check if the string is a valid number. If it is, it will convert it to a BigDecimal and then to the specified number type.

RETURNS

A BigDecimal object which was the Value converted to a Java number.

```
*/
/**/
private static BigDecimal ConvertToNumber(Value a_value, Value.DataType a_toType) {
    Value.DataType type = a_value.GetDataType();
    switch(type) {
        case INTEGER, FLOAT: return a_value.AsFloat();

        // Can only convert strings that are integers or floats, falls to default if not
        // Help received: https://regex101.com
        case STRING:
            boolean isValidNumber = a_value.AsString().matches("^-?\\d+(\\.\\d+)?$");
            if (isValidNumber) { return new BigDecimal(a_value.AsString()); }

            // Can only convert: integer, float, string everything else is invalid (can't reach ZERO, Java doesn't know)
            default: throw new BuiltInFunctionException("Cannot convert '" + type + "' to '" + a_toType + "'");
    }
}
/* private static BigDecimal ConvertToNumber(Value a_value, Value.DataType a_toType); */

/**/
/*
Language.BuiltInFunctions.MakeReadPath()
```

NAME

MakeReadPath - creates a Path object from a file name to read.

SYNOPSIS

```
private static Path MakeReadPath(Value a_fileName);
    a_fileName -> the file name to read.
```

DESCRIPTION

Creates a Path object from a file name to read. The file name must be a string. If it is not a string, an exception will be thrown. If the file does not exist, an exception will be thrown. This is a helper function for the READ() and RUN() built-in functions.

RETURNS

A Path object to the file to read, if it can.

```
*/
/**/
private static Path MakeReadPath(Value a_fileName) {
    // Ensure argument is a string
    EnforceArgumentType(Value.DataType.STRING, a_fileName.GetDataType(), 0);

    // Get file and attach to current directory
```



```

    Path path = Paths.get(a_fileName.AsString());
    EnforceFileExists(path);

    return path;
}
/* private static Path MakeReadPath(Value a_fileName); */

/*
 *      List Helper - Inserting and removing elements from a list
 */

/**/
/*
Language.BuiltInFunctions.InsertValue()

```

NAME

InsertValue - inserts a Value at a specified index in a list.

SYNOPSIS

```

private static Value InsertValue(List<Value> a_arguments,
                                Value a_index);
    a_arguments -> the list, value to insert, and index to insert at
                  (0 is list, 1 is value, 2 is index)
    a_index      -> the index to insert at.

```

DESCRIPTION

Inserts a Value at a specified index in a list. The first argument must be a list, the second argument is the value to insert, and the third argument is the index to insert at. If the first argument is not a list, an exception will be thrown. If the third argument is not an integer, an exception will also be thrown - it also must be in bounds of the length of the list. Helper function for the INSERT() and APPEND() built-in.

The original list is not modified, rather the new list is created with the value inserted at the specified index.

RETURNS

A new list Value object with the Value inserted at the specified index.

```

*/
/**/
private static Value InsertValue(List<Value> a_arguments, Value a_index) {
    // Get the raw list and index - ensures that the list is a list and the index is an integer and in range
    List<Value> rawList = MakeRawList(a_arguments);
    int index = MakeIndex(a_index, rawList.size());

    // Insert at that index and return
    rawList.add(index, a_arguments.get(1));
}

```

```

        return new Value(rawList);
    }
    /* private static Value InsertValue(List<Value> a_arguments,
                                        Value a_index); */

/**/
/*
Language.BuiltInFunctions.RemoveValue()

```

NAME

RemoveValue - removes an element from a list at a specified index.

SYNOPSIS

```

private static Value RemoveValue(List<Value> a_arguments,
                                Value a_index);
    a_arguments -> the list and index to remove at (0 is list, 1 is
                    index)
    a_index      -> the index to remove at.

```

DESCRIPTION

Removes an element from a list at a specified index. The first argument must be a list and the second argument is the index to remove at. If the first argument is not a list, an exception will be thrown. If the second argument is not an integer, an exception will also be thrown - it also must be in bounds of the length of the list. Helper function for the REMOVE() and POP() built-in.

The original list is not modified, rather the new list is created without the element at the specified index.

RETURNS

A new list Value object with the element removed at the specified index.

```

*/
/**/
private static Value RemoveValue(List<Value> a_arguments, Value a_index) {
    // Get the raw list and index - ensures that the list is a list and the index is an integer and in range
    List<Value> rawList = MakeRawList(a_arguments);
    int index = MakeIndex(a_index, rawList.size() - 1);

    // Remove at that index and return
    rawList.remove(index);
    return new Value(rawList);
}
/* private static Value RemoveValue(List<Value> a_arguments,
                                    Value a_index); */

/**/
/*

```

Language.BuiltInFunctions.MakeRawList()

NAME

MakeRawList - creates a raw list (Value to List<Value>).

SYNOPSIS

```
private static List<Value> MakeRawList(List<Value> a_arguments);  
a_arguments -> the list Value to convert.
```

DESCRIPTION

Creates a raw list from a list Value. The first argument must be a list. If it is not a list, an exception will be thrown. Helper function for the InsertValue() and RemoveValue(). Ensures that the list is a list and avoids code duplication.

RETURNS

A raw list of Value objects from the list Value.

```
*/  
/**/  
private static List<Value> MakeRawList(List<Value> a_arguments) {  
    // First argument must be a list  
    Value list = a_arguments.get(0);  
    EnforceArgumentType(Value.DataType.LIST, list.GetDataType(), 0);  
    return list.AsList();  
}  
/* private static List<Value> MakeRawList(List<Value> a_arguments); */  
  
/**/  
/*  
Language.BuiltInFunctions.MakeIndex()
```

NAME

MakeIndex - creates an integer index from a Value.

SYNOPSIS

```
private static int MakeIndex(Value a_index, int a_listSize);  
a_index    -> the index Value to convert.  
a_listSize -> the size of the list.
```

DESCRIPTION

Ensures that the index is an integer and in bounds of the list size. If not these things, an exception will be thrown. Lists in Java are capped at around 2 billion, so we must convert a Java int to handle. Is the helper function for the InsertValue() and RemoveValue(). Ensures that the index is usable.

RETURNS

An integer index from the Value.

```
*/
/**/
private static int MakeIndex(Value a_index, int a_listSize) {
    // Ensure 3rd argument (index) is an integer
    EnforceArgumentType(Value.DataType.INTEGER, a_index.GetData(), 2);

    // Ensure index is in range
    int index = a_index.AsInteger().intValue();
    EnforceIndexBounds(index, a_listSize);
    return index;
}
/* private static int MakeIndex(Value a_index, int a_listSize); */

/*
 *      Error Handling - Ensuring proper types, conversions, and bounds
 */

/**/
/*
Language.BuiltInFunctions.EnforceArgumentType()
```

NAME

EnforceArgumentType - ensures that a Value is of a specified type.

SYNOPSIS

```
private static void EnforceArgumentType(Value.DataType a_expected,
                                         Value.DataType a_actual,
                                         int a_argumentIndex);

a_expected      -> the expected Value type.
a_actual        -> the actual Value type.
a_argumentIndex -> the index of the argument when called.
```

DESCRIPTION

Ensures that a Value is of a specified type. If it is not, an exception will be thrown. Ensures that the Value is the expected type when called allowing for more descriptive error messages.

RETURNS

Nothing - is a void function.

```
*/
/**/
```

```
private static void EnforceArgumentType(Value.DataType a_expected, Value.DataType a_actual, int a_argumentIndex) {
    if (a_expected != a_actual) {
        throw new BuiltInFunctionException("Invalid argument (index " + a_argumentIndex + ") for function!" +
            " Expected '" + a_expected + "' but got '" + a_actual + "'");
    }
}
/* private static void EnforceArgumentType(Value.DataType a_expected,
    Value.DataType a_actual,
    int a_argumentIndex); */
```

```
/**/
/*
Language.BuiltInFunctions.EnforceIndexBounds()
```

NAME

EnforceIndexBounds - ensures that an index is in bounds of a list.

SYNOPSIS

```
private static void EnforceIndexBounds(int a_index, int a_size);
    a_index -> the index to check.
    a_size  -> the size of the list.
```

DESCRIPTION

Ensures that an index is in bounds of a list. If it is not, an exception will be thrown. Ensures that the index is in range of the list size when called allowing for more descriptive error messages.

RETURNS

Nothing - is a void function.

```
*/
/**/
private static void EnforceIndexBounds(int a_index, int a_size) {
    if (a_index < 0 || a_index > a_size) {
        throw new BuiltInFunctionException("Index " + a_index + " out of range of usable length " + a_size);
    }
}
/* private static void EnforceIndexBounds(int a_index, int a_size); */

/**/
/*
Language.BuiltInFunctions.EnforceFileExists()
```

NAME

EnforceFileExists - ensures that a file exists.

SYNOPSIS

```
private static void EnforceFileExists(Path a_path);  
    a_path -> the file path to check.
```

DESCRIPTION

Ensures that a file exists. If it does not, an exception will be thrown.
Ensures that the file exists when called allowing for more descriptive error messages.

RETURNS

Nothing - is a void function.

```
*/  
/**/  
private static void EnforceFileExists(Path a_path) {  
    if (!Files.exists(a_path)) {  
        throw new BuiltInFunctionException("File '" + a_path + "' does not exist");  
    }  
}  
/* private static void EnforceFileExists(Path a_path); */  
  
/**/  
/*  
Language.BuiltInFunctions.EnforceFileExtension()
```

NAME

EnforceFileExtension - ensures that a file has a specified extension.

SYNOPSIS

```
private static void EnforceFileExtension(Path a_path);  
    a_path -> the file path to check.
```

DESCRIPTION

Ensures that a file has a specified extension. If it does not, an exception will be thrown as we can only guarantee that the file can run from a '.bb' extension. Ensures that the file has the correct extension when called allowing for more descriptive error messages.

RETURNS

Nothing - is a void function.

```
*/  
/**/  
private static void EnforceFileExtension(Path a_path) {  
    if (!a_path.toString().endsWith(Main.FILE_EXTENSION)) {  
        throw new BuiltInFunctionException("Invalid file extension. Must be a " + Main.FILE_EXTENSION + " file!");  
    }  
}  
}
```

```

    /* private static void EnforceFileExtension(Path a_path); */
}

```

Lexer Package

Lexer.java

```

///
/// Lexer - Breaks apart the input string into Tokens that are manageable for the parser, i.e. performs lexical
///      analysis. It is the first step in the compilation process, and is responsible for identifying the basic
///      building blocks of the language.
///

package Lexer;

import static Lexer.OperationCode.*;
import static Language.Error.*;

import Language.Error;
import Interpreter.Value;

import java.math.BigDecimal;
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class Lexer {
    /* PUBLIC */

    /** Constants */

    // Escape characters that can reside in strings (defined here so we can add or remove easily)
    // Help received: https://stackoverflow.com/questions/6802483/how-to-directly-initialize-a-hashmap-in-a-literal-way
    public static final Map<Character, Character> VALID_ESCAPES = Map.of(
        'n', '\n',
        't', '\t',
        '\\', '\\',
        '\"', '\"',
    );

    // Floating point number separator (different in some parts of world, best to make it a constant)
    public static final char DECIMAL = '.';

    /** Constructors */

    /**/

```

```
/**  
Lexer.Lexer.Lexer()
```

NAME

Lexer - constructor for the Lexer class.

SYNOPSIS

```
Lexer(String a_input);  
    a_input    -> The input string (program) to be parsed by the
```

DESCRIPTION

Initializes the Lexer with the input string to be parsed. It will keep track of the line number, the current character being parsed, and the index of the character in the input string.

RETURNS

Nothing - it's a constructor.

```
*/  
/**/  
public Lexer(String a_input) {  
    m_input = a_input;  
  
    // Line numbers are not 0 indexed  
    m_lineNumber = 1;  
  
    m_textIndex = 0;  
    m_currentChar = m_input.charAt(m_textIndex);  
}  
/* Lexer(String a_input); */
```

```
/** Methods **/
```

```
/**/  
/*  
Lexer.Lexer.GenerateAllTokens()
```

NAME

GenerateAllTokens - Generates all Tokens from the input string.

SYNOPSIS

```
public List<Token> GenerateAllTokens();
```

DESCRIPTION

Breaks apart all the input string into their respective Tokens that are

manageable for the parser. It will add each Token to a list until it reaches the end of input string, marked by the OperationCode EOF. If at any point it cannot parse a character, it will throw a syntax error for the user to be informed.

RETURNS

Returns a list of Tokens that were generated from the input string. Storing the operation code, value associated with it, and the line number it was found on for error reporting.

```
*/
/**/
public List<Token> GenerateAllTokens() {
    List<Token> Tokens = new ArrayList<>();

    // Add each Token until we are done with the input
    while (true) {
        Token Token = GenerateNextToken();
        Tokens.add(Token);

        // Break out once done
        if (Token.GetOpcode() == EOF) { break; }
    }

    return Tokens;
}
/* public List<Token> GenerateAllTokens(); */

/* PRIVATE */

/** Members **/

private final String m_input;      // String to be parsed
private int m_lineNumber;          // Line number of the current Token
private int m_textIndex;           // Position of the current character in input string
private char m_currentChar;        // Current character being parsed

/** Private Helper Functions **/

/**/
/*
Lexer.Lexer.GenerateNextToken()
```

NAME

GenerateNextToken - Generates the next Token from input string.

SYNOPSIS

```
private Token GenerateNextToken();
```

DESCRIPTION

Generates the next Token from the input string, and assign it their respective operation code, value associated with it, and the line number based on the character it is currently on. It will skip whitespace and comments, and if it cannot parse a character, it will throw a syntax error for the user to be informed. If the end of the input is reached, it will end, marking it with the OperationCode EOF.

Parsing is done by the label of the operation code, and the character we are currently on. For example, if a character is a letter, it will be an identifier or key, if it is a digit, it will be a number, etc. Based on the complexity of the operation code, it will call a helper function to generate the Token.

RETURNS

The Token associated with the current character in the input string.

```
*/
/**/
private Token GenerateNextToken() {
    // If there is whitespace (besides newlines), skip it - we don't care about it
    RemoveWhitespace();
    // Or if we have a comment, we don't want to parse it
    // Comment - skip all characters after symbol (SYMBOL: ' for line, '' for multi-line)
    RemoveComment();

    // If we are at the end of the input, then we are done
    if (IsEndOfInput()) { return new Token(EOF, m_lineNumber); }

    /* Multiple character Tokens */
    // Identifier - see if keyword, if not its variable (in regex: [a-zA-Z_][a-zA-Z0-9_]*)
    if (IsValidStartIdentifier(m_currentChar)) { return MakeIdentifier(); }
    // Number - check there are digits separated by 1 decimal (in regex: ?[0-9]+(\.[0-9]+)?)
    if (IsPotentialDigit(m_currentChar)) { return MakeNumber(); }
    // String - anything in between quotes (SYMBOL: ", in regex: \".*\")
    if (m_currentChar == QUOTE.SYMBOL) { return MakeString(); }
    // Not equal - two character symbol (SYMBOL: !=)
    if (IsNotEqualSign(m_currentChar)) { return MakeSimpleSymbol(INEQUALITY); }
    // Newline - user inputted newline break (SYMBOL: \n and ;)
    if (IsNewline(m_currentChar)) { return MakeSimpleSymbol(NEWLINE); }

    /* Operation codes that share symbols, e.g. = and == */
    // Assignment: =      Equality: ==
    if (m_currentChar == ASSIGNMENT.SYMBOL) { return MakeSharedSymbol(ASSIGNMENT, EQUALITY); }
    // Less than: <      Less than or equal: <=
    if (m_currentChar == LESS_THAN.SYMBOL) { return MakeSharedSymbol(LESS_THAN, LESS_THAN_EQUAL); }
    // Greater than: >      Greater than or equal: >=
    if (m_currentChar == GREATER_THAN.SYMBOL) { return MakeSharedSymbol(GREATER_THAN, GREATER_THAN_EQUAL); }
    /* Single character Tokens that do not share characters, i.e. simple symbols */
    if (SIMPLE_SYMBOLS.containsKey(m_currentChar)) { return MakeSimpleSymbol(SIMPLE_SYMBOLS.get(m_currentChar)); }
}
```

```

    // Could not parse character - so we can't do anything with it
    throw new Error("Invalid character, could not parse: '" + m_currentChar + "'",
        ErrorType.SYNTAX_ERROR,
        m_lineNumber);
}
/* private Token GenerateNextToken(); */

```

```

/*
 *      Making characters into Tokens
 */

```

```

/**/
/*
Lexer.Lexer.MakeIdentifier()

```

NAME

MakeIdentifier - Generates a Token for an identifier or keyword.

SYNOPSIS

```
private Token MakeIdentifier();
```

DESCRIPTION

Generates a Token for an identifier or keyword based on the current character in the input string. It will keep track of the characters that are letters, underscores, or digits, and if it is a keyword (which can be found in the KEYWORD map), it will assign the operation code to that keyword. If it is TRUE, FALSE, or NULL, it will assign the Java representable value to the Token.

RETURNS

The Token associated with the identifier or keyword in the input string.

```

*/
/**/
private Token MakeIdentifier() {
    // Stopping execution here as it is a problem with the call, not the lexer
    assert IsValidStartIdentifier(m_currentChar) : "Character must start with a letter or underscore";

    StringBuilder identifier = new StringBuilder();

    // The next characters are either letters, underscores or digits, but not whitespace
    do {
        identifier.append(m_currentChar);
        Advance();
    } while (!IsEndOfInput() && (IsValidStartIdentifier(m_currentChar) || Character.isDigit(m_currentChar)));

    // If the string is in the set of keywords, then it can't be an identifier

```

```

// Help received: https://www.w3schools.com/java/ref_hashmap_getordefault.asp
OperationCode opcode = KEYWORDS.getDefault(identifier.toString(), IDENTIFIER);

// Check if TRUE, FALSE, or NULL as we need to represent as a Java value
if (IsBooleanOpcode(opcode)) { return MakeBoolean(opcode); }
if (opcode == OperationCode.NULL) { return MakeNull(opcode); }

return new Token(opcode, new Value(identifier.toString()), m_lineNumber);
}
/* private Token MakeIdentifier(); */

/**/
/*
Lexer.Lexer.MakeBoolean()

```

NAME

MakeBoolean - Generates a Token for TRUE or FALSE with the Java representable value.

SYNOPSIS

```

private Token MakeBoolean(OperationCode a_opcode);
    a_opcode    -> A 'TRUE' or 'FALSE' operation code which we can use
                  to determine the value to assign to the Token

```

DESCRIPTION

Generates a Token for TRUE or FALSE based on the operation code passed in. Assigning the Java representable value to our own Value.

RETURNS

The Token and Value associated with the TRUE or FALSE operation code.

```

*/
/**/
private Token MakeBoolean(OperationCode a_opcode) {
    // Ensure it is TRUE or FALSE
    assert IsBooleanOpcode(a_opcode) : "Opcode must be TRUE or FALSE";

    // Return with the Java representable of a boolean
    return new Token(a_opcode, new Value(a_opcode == TRUE), m_lineNumber);
}
/* private Token MakeBoolean(OperationCode a_opcode); */

/**/
/*
Lexer.Lexer.MakeNull()

```

NAME

MakeNull - Generates a Token for NULL with the Java representable

value.

SYNOPSIS

```
private Token MakeNull(OperationCode a_opcode);  
    a_opcode    -> A 'NULL' operation code which we can use to determine  
        the value to assign to the Token
```

DESCRIPTION

Generates a Token for NULL based on the operation code passed in.
Assigning the Java representable value to our own Value.

RETURNS

The Token with the Value.NULL associated with the NULL operation code.

```
*/  
/**/  
private Token MakeNull(OperationCode a_opcode) {  
    // Ensure it is NULL  
    assert (a_opcode == OperationCode.NULL) : "Opcode must be NULL";  
  
    // Make it our NULL value  
    return new Token(OperationCode.NULL, Value.NULL, m_lineNumber);  
}  
/* private Token MakeNull(OperationCode a_opcode); */  
  
/**/  
/*  
Lexer.Lexer.MakeNumber()
```

NAME

MakeNumber - Generates a Token for a number with the Java
representable value (BigDecimal and BigInteger).

SYNOPSIS

```
private Token MakeNumber();
```

DESCRIPTION

Generates a Token for a number based on the current character in the
input string. While we have a digit or decimal point, we will keep
appending to the number. We count the decimal points to ensure it is
when we generate the value, we know if it is a float or integer. If it
has one decimal point, it is a float (we use Java's BigDecimal to
represent it), otherwise it is an integer(using BigInteger to represent
it). Using these classes as we don't want to overflow in our language.

If the number is not a digit or decimal point, it will throw a syntax
error for the user to be informed. If the number is a decimal point

without any digits, it will also throw a syntax error.

RETURNS

The Token, with a Value that is either an INTEGER or FLOAT.

```
*/
/**/
private Token MakeNumber() {
    // Can't have a number if not a digit or decimal
    assert (IsPotentialDigit(m_currentChar)) : "Character must be a digit or decimal point";

    // Keep track if number is a float or integer
    int decimalCount = 0;

    // Create a string for our presumably multi-digit number
    StringBuilder builder = new StringBuilder();

    // While we have digits, append to result and continue with input
    while (!IsEndOfInput() && IsPotentialDigit(m_currentChar)) {
        if (m_currentChar == DECIMAL) { decimalCount++; }

        builder.append(m_currentChar);
        Advance();
    }
    String number = builder.toString();

    // Guard clauses before casting - ensures proper number syntax
    EnsureValidNumber(m_currentChar, number, decimalCount);

    // If we have a decimal point, then we have a float, otherwise it's an integer. Using BigDecimal and BigInteger
    // as we don't overflow in our language.
    // Help received: IDE suggestion for ternary
    Value value = (decimalCount == 1) ? new Value(new BigDecimal(number)) : new Value(new BigInteger(number));
    return new Token(LITERAL, value, m_lineNumber);
}
/* private Token MakeNumber(); */

/**/
/*
Lexer.Lexer.MakeString()
```

NAME

MakeString - Generates a Token with a value of string within quotes.

SYNOPSIS

```
private Token MakeString();
```

DESCRIPTION

If we get a character that is the quote symbol, we will generate loop

through each character until we hit a quote symbol again. Within the inner contents of the two quotes, we will append to a string builder and keep track of escape characters (using the map `VALID_ESCAPES`).

If our string does not have a closing quote, it will throw a syntax error for the user to be informed. It also throws a syntax error if there is an invalid escape character.

RETURNS

The Token with a Value of the string within the quotes, e.g. "Hello".

```
*/
/**/
private Token MakeString() {
    // Can't create a string Token if we don't have a quote - improper call
    assert (m_currentChar == QUOTE.SYMBOL) : "Character must be a quote";

    // Go past first quote
    Advance();

    // Parse the actual values within the string, saving the start for better error reporting
    int startLine = m_lineNumber;
    String innerContents = ProcessStringContents();

    // If last character is not a quote, then it's an error
    if (m_currentChar != QUOTE.SYMBOL) {
        throw new Error("String not closed",
                        ErrorType.SYNTAX_ERROR,
                        startLine);
    }

    // Go past last quote
    Advance();

    return new Token(LITERAL, new Value(innerContents), m_lineNumber);
}
/* private Token MakeString(); */

/**/
/*
Lexer.Lexer.MakeSharedSymbol()
```

NAME

`MakeSharedSymbol` - Generates a Token for a shared symbol (a symbol that shares characters with another symbol).

SYNOPSIS

```
private Token MakeSharedSymbol(OperationCode a_firstOpcode,
                               OperationCode a_secondOpcode);
    a_firstOpcode    -> The first operation code that shares the symbol,
```

which should only be one character long and must be the first character of the second opcode.
a_secondOpcode -> The second operation code that shares the symbol with the first opcode, which should be two characters long where its first character should be the same as the first opcode.

DESCRIPTION

Generates a Token for a shared symbol based on the current character in the input string. If the second character is present, then we have the second opcode, otherwise we have the first opcode. It will advance past the symbol/label, however long it may be. For example, if the symbol is '=', then it will be either '=' or '=='. If it is '==', then it will advance past the second '='.

RETURNS

The Token associated with the symbol in the input string, e.g. = or ==.

```

*/
/**/
private Token MakeSharedSymbol(OperationCode a_firstOpcode, OperationCode a_secondOpcode) {
    /* GUARD CLAUSES */
    // We only have symbols that share first character, where one opcode is a single char and the other is two.
    // No need in checking for longer symbol lengths in current implementation
    assert (a_firstOpcode.LABEL.length() == 1 && a_secondOpcode.LABEL.length() == 2) : "Incorrect opcode lengths";
    // The first opcode symbol, must be the first character - if not it's a problem with the call
    assert (a_firstOpcode.SYMBOL == a_secondOpcode.LABEL.charAt(0)) : "Opcode label must share first character";
    // First characters must also be the same
    assert (m_currentChar == a_firstOpcode.SYMBOL) : "Character must be the first character of the opcode";

    // Assume the first opcode is the final opcode
    OperationCode finalOpcode = a_firstOpcode;

    // If the second character is present, then we have the second opcode
    if (LookAhead() == a_secondOpcode.LABEL.charAt(1)) {
        finalOpcode = a_secondOpcode;
        Advance();
    }

    Advance();

    return new Token(finalOpcode, m_lineNumber);
}
/* private Token MakeSharedSymbol(OperationCode a_firstOpcode,
                                   OperationCode a_secondOpcode); */

/**/
/*
Lexer.Lexer.MakeSimpleSymbol()

```

NAME

MakeSimpleSymbol - Generates a Token for a simple symbol (a symbol that does not share characters with another symbol).

SYNOPSIS

```
private Token MakeSimpleSymbol(OperationCode a_opcode);
    a_opcode    -> The operation code that is a simple symbol, which
                  does not share characters with another symbol.
```

DESCRIPTION

Generates a Token for a simple symbol based on the current character in the input string. It will advance past the symbol/label, however long it may be. For example, if the symbol is '+', then it will advance past it, or '!='.

RETURNS

The Token associated with the symbol in the input string, e.g. '+'.

```
*/
/**/
private Token MakeSimpleSymbol(OperationCode a_opcode) {
    // Advance past the symbol/label, however long it may be
    for (int index = 0; index < a_opcode.LABEL.length(); index++) { Advance(); }

    return new Token(a_opcode, m_lineNumber);
}
/* private Token MakeSimpleSymbol(OperationCode a_opcode); */

/*
 *      Helping parse the input
 */

/**/
/*
Lexer.Lexer.RemoveComment()
```

NAME

RemoveComment - Removes comments from the input string.

SYNOPSIS

```
void RemoveComment();
```

DESCRIPTION

If we are on a comment symbol, we will remove the comment from the input string. If it is a multi-line comment, we will remove everything until we reach the end of the comment. If it is a single line comment, we will remove everything until we reach the end of the line (which is denoted

by both a newline character and a semicolon).

If the multi-line comment is not closed, it will throw a syntax error for the user to be informed.

RETURNS

Nothing - just avoids the comment in the input string.

```
*/
/**/
private void RemoveComment() {
    // Not a comment, move on
    if (m_currentChar != COMMENT.SYMBOL) { return; }

    // Multi-line comment (''), only need to check LookAhead() once as we know the first character is a comment
    if (LookAhead() == COMMENT.SYMBOL) {
        // Save the line number for better error reporting
        int startLine = m_lineNumber;

        // Skip the first two characters
        Advance(); Advance();

        // Advance till we reach the end of the comment or file
        while (!IsEndOfInput()) {
            if (m_currentChar == COMMENT.SYMBOL && LookAhead() == COMMENT.SYMBOL) { break; }
            Advance();
        }

        // If not closed, should inform user - would be annoying if nothing ran, and they didn't know why
        if (IsEndOfInput()) {
            throw new Error("Multi-line comment not closed",
                           ErrorType.SYNTAX_ERROR,
                           startLine);
        }

        // Skip the last two characters
        Advance(); Advance();

        // Remove any whitespace or comments after the multi-line comment (in case we have chained multi-lines
        //      comments we need to ensure we remove all of them, hence the recursive call).
        RemoveWhitespace();
        RemoveComment();
    }
    // Single line comment (')
    else { while (!IsEndOfInput() && !IsNewline(m_currentChar)) { Advance(); } }
}
/* private void RemoveComment(); */

/**/
/*
Lexer.Lexer.RemoveWhitespace()
```

NAME

RemoveWhitespace - Removes/avoids whitespace from the input string.

SYNOPSIS

```
private void RemoveWhitespace();
```

DESCRIPTION

If we are on a whitespace character, that is not a newline we will skip it. We need to keep newlines as they are used to separate statements in the parser.

RETURNS

Nothing - just avoids the whitespace in the input string.

```
*/  
/**/  
private void RemoveWhitespace() {  
    // Continue passed all whitespace besides newlines as they are needed for statement separation in parser  
    while (!IsEndOfInput() && (Character.isWhitespace(m_currentChar) && !IsNewline(m_currentChar))) {  
        Advance();  
    }  
}  
/* private void RemoveWhitespace(); */  
  
/**/  
/*  
Lexer.Lexer.ProcessStringContents()
```

NAME

ProcessStringContents - Processes the contents withing the quotes of a string.

SYNOPSIS

```
private String ProcessStringContents();
```

DESCRIPTION

Builds out the raw characters within the quotes of a string. It will keep track of escape characters, and if it is a valid escape character (found in the VALID_ESCAPES map), it will append the actual character to the string. If it is not a valid escape character, it will throw a syntax error for the user to be informed.

It will continue until it reaches the end of the input string or the end of the quote.

RETURNS

A string of raw characters within the quotes of the input text.

```
*/
/**/
private String ProcessStringContents() {
    // Separated from MakeString() as logic was getting too complex to keep in one function
    StringBuilder contents = new StringBuilder();

    // Keep track of if we are on an escape character (meaning previous character is '\\')
    boolean isEscape = false;
    // While we have characters and we haven't reached the end quote
    while (!IsEndOfInput() && (m_currentChar != QUOTE.SYMBOL || isEscape)) {
        // If we are on an escape character, and it exists in our valid characters add it to the string
        if (isEscape) {
            EnsureValidEscape(m_currentChar);
            contents.append(VALID_ESCAPES.get(m_currentChar));
            isEscape = false;
        }
        // We are at an escape character, so for next iteration our character is an escape
        else if (m_currentChar == '\\') { isEscape = true; }
        // Otherwise its normal, so add the character to the string
        else { contents.append(m_currentChar); }

        Advance();
    }

    return contents.toString();
}
/* private String ProcessStringContents(); */

/**/
/*
Lexer.Lexer.EnsureValidEscape()
```

NAME

EnsureValidEscape - Ensures the escape character is valid.

SYNOPSIS

```
private void EnsureValidEscape(char a_char);
    a_char -> An escape character that needs to against our valid
               escape characters in the language.
```

DESCRIPTION

Ensures the escape character is valid, and if it is not, it will throw a syntax error for the user to be informed.

RETURNS

Nothing - just ensures the escape character is valid.

```

*/
/**/
private void EnsureValidEscape(char a_char) {
    // If the escape character is not valid, then it's an error (in another function for readability)
    if (!VALID_ESCAPES.containsKey(a_char)) {
        throw new Error("Invalid escape character",
            ErrorType.SYNTAX_ERROR,
            m_lineNumber);
    }
}
/* private void EnsureValidEscape(char a_char); */

/**/
/*
Lexer.Lexer.EnsureValidNumber()

```

NAME

EnsureValidNumber - Ensures the number is valid.

SYNOPSIS

```

void EnsureValidNumber(char a_char, String a_numberToBe,
                        int a_decimalCount);
a_char          -> The current character that is being parsed.
a_numberToBe     -> The number as a string that is being parsed.
a_decimalCount   -> The count of decimal points in the number.

```

DESCRIPTION

Ensures the number is valid, and if it is not, it will throw a syntax error for the user to be informed. It will check if the number is a decimal point without any digits, if the number is followed by a letter, or if the number has more than one decimal point. Need to ensure the number is valid before we cast it to a Java representable value.

RETURNS

Nothing - just ensures the number is valid.

```

*/
/**/
private void EnsureValidNumber(char a_char, String a_numberToBe, int a_decimalCount) {
    // Only time a decimal point can exist is if it is number (no classes in our language)
    if (a_numberToBe.equals(Character.toString(DECIMAL))) {
        throw new Error("Decimal point cannot exist without digits",
            ErrorType.SYNTAX_ERROR,
            m_lineNumber);
    }
    // Numbers cannot be followed by a letter - similar to most languages
    if (IsValidStartIdentifier(a_char)) {
        throw new Error("Digit cannot be followed by a letter",

```

```

        ErrorType.SYNTAX_ERROR,
        m_lineNumber);
    }
    // A number that is .23.2 is not an actual number
    if (a_decimalCount > 1) {
        throw new Error("Number cannot have more than one decimal point",
            ErrorType.SYNTAX_ERROR,
            m_lineNumber);
    }
}
/* private void EnsureValidNumber(char a_char, String a_numberToBe,
    int a_decimalCount); */

```

```

/*
 *      Moving through the input
 */

```

```

/**/
/*
Lexer.Lexer.Advance()

```

NAME

Advance - Advances to the next character in the input string.

SYNOPSIS

```
private void Advance();
```

DESCRIPTION

Advances our m_currentChar to the next character in the input string. If the character is a newline ('\n' or ';'), it will increment the line number. If we are at the end of the input, it will not advance to avoid errors.

RETURNS

Nothing - just advances to the next character in the input string.

```

*/
/**/
private void Advance() {
    // Newline means new line number, otherwise just move to next character
    if (IsNewline(m_currentChar)) { m_lineNumber++; }
    m_textIndex++;

    // Ran out of input, so we are done
    if (IsEndOfInput()) { return; }

    m_currentChar = m_input.charAt(m_textIndex);
}

```

```
/* private void Advance(); */
```

```
/**/
```

```
/*
```

```
Lexer.Lexer.LookAhead()
```

NAME

LookAhead - Peeks at the next character in the input string.

SYNOPSIS

```
char LookAhead();
```

DESCRIPTION

Peeks at the next character in the input string for multi-character symbols. If the next character is out of bounds, it will return the null character to avoid errors.

RETURNS

The next character in the input string, or the null character if it is out of bounds.

```
*/
```

```
/**/
```

```
private char LookAhead() {
```

```
    // Peek at the next character in the input string
```

```
    int peekPosition = m_textIndex + 1;
```

```
    // Next char is out of bounds return null character to avoid errors
```

```
    if (peekPosition > m_input.length() - 1) { return '\0'; }
```

```
    return m_input.charAt(peekPosition);
```

```
}
```

```
/* private char LookAhead(); */
```

```
/*
```

```
 *      Character Checking - Seeing what type of character we are on. Ease of use functions that make the code more  
 *      readable and easier to understand. Short enough to not need function headers.
```

```
*/
```

```
// Check to see if we are at the end of the input string
```

```
private boolean IsEndOfInput() {
```

```
    return m_textIndex == m_input.length();
```

```
}
```

```
// See if the character is a newline ';' or '\n' - need ';' for CLI
```

```
private boolean IsNewline(char a_char) {
```

```
    return a_char == NEWLINE.SYMBOL || a_char == '\n';
```

```
}
```

```

// Check if character is alphabetic or an underscore (for snake case), and not a digit
private boolean IsValidStartIdentifier(char a_char) {
    return Character.isAlphabetic(a_char) || a_char == '_';
}

// Check if character is '!=' and only supports not equal as a two character symbol, report it if longer
private boolean IsNotEqualSign(char a_char) {
    assert INEQUALITY.LABEL.length() == 2 : "Not equal sign must be two characters";
    return (a_char == INEQUALITY.LABEL.charAt(0)) && (LookAhead() == INEQUALITY.LABEL.charAt(1));
}

// See if character is a digit - Character.isDigit() does a lot more than just ASCII digits, so need to make our own
private boolean IsPotentialDigit(char a_char) {
    return (a_char >= '0' && a_char <= '9') || a_char == DECIMAL;
}

// Check if the opcode is TRUE or FALSE
private boolean IsBooleanOpcode(OperationCode a_opcode) {
    return a_opcode == TRUE || a_opcode == FALSE;
}
}

```

OperationCode.java

```

///
/// OperationCode Enum - Used to represent the operation codes of the language, specifically for easier lexing and
/// parsing. This enum contains all the operators and operands that are used in the language.
///

package Lexer;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

// Language keywords and symbols were found from the QuickBASIC syntax from:
// https://www.qbasic.net/en/reference/qb11/overview.htm
public enum OperationCode {
    // Format: OPCODE("Label", SymbolType)

    /* Arithmetic Operators */
    ADD ("+", SymbolType.SIMPLE),
    SUBTRACT ("-", SymbolType.SIMPLE),

    MULTIPLY ("*", SymbolType.SIMPLE),
    DIVIDE ("/", SymbolType.SIMPLE),
    MODULO ("%", SymbolType.SIMPLE),

    POWER ("^", SymbolType.SIMPLE),

```



```

LPARENTHESIS      ("(",      SymbolType.SIMPLE),
RPARENTHESIS      (")",      SymbolType.SIMPLE),

ASSIGNMENT         ("=",      SymbolType.SHARED),

/* Logical, Equality and Comparison Operators */
EQUALITY           ("==",     SymbolType.SHARED),
INEQUALITY         ("!=",     SymbolType.COMPLEX),      // Complex since '!' is never used anywhere else
LESS_THAN         ("<",       SymbolType.SHARED),
LESS_THAN_EQUAL    ("<=",     SymbolType.SHARED),
GREATER_THAN       (">",      SymbolType.SHARED),
GREATER_THAN_EQUAL(">=",     SymbolType.SHARED),

/* Punctuation */
NEWLINE           ("; ",      SymbolType.SIMPLE),        // End of line - can be both '\n' and ';'
COMMENT           ("'",       SymbolType.COMPLEX),       // Comments, doubled (') are used for multi-lines

QUOTE              ("\" ",    SymbolType.COMPLEX),       // String creation

LBRACKET           ("[ ",     SymbolType.SIMPLE),        // List start
RBRACKET           ("] ",     SymbolType.SIMPLE),        // List end
COMMA              (", ",     SymbolType.SIMPLE),        // Element separation in lists/functions
ACCESSOR           (": ",     SymbolType.SIMPLE),        // List element access
SUBSCRIPT          ("| ",     SymbolType.SIMPLE),        // Subscript in element access, e.g. [1,2,3]:0|2

/* Keywords */
// Conditionals
AND                ("AND",     SymbolType.KEYWORD),
OR                 ("OR",      SymbolType.KEYWORD),
NOT                ("NOT",     SymbolType.KEYWORD),

// Variable management
LET                ("LET",     SymbolType.KEYWORD),

// Control flow
IF                 ("IF",      SymbolType.KEYWORD),
ELSEIF             ("ELSEIF",   SymbolType.KEYWORD),
ELSE               ("ELSE",     SymbolType.KEYWORD),

// Loop management
FOR                ("FOR",     SymbolType.KEYWORD),
TO                 ("TO",      SymbolType.KEYWORD),
STEP              ("STEP",     SymbolType.KEYWORD),
WHILE              ("WHILE",    SymbolType.KEYWORD),
CONTINUE           ("CONTINUE", SymbolType.KEYWORD),
BREAK              ("BREAK",    SymbolType.KEYWORD),

THEN               ("THEN",     SymbolType.KEYWORD),      // Used to separate condition from block

// Function management
DEF                ("DEF",     SymbolType.KEYWORD),       // Function definition
RETURN             ("RETURN",   SymbolType.KEYWORD),

```

```

END                ("END",      SymbolType.KEYWORD),    // End of block

/* Primitive Data Types */
TRUE               ("TRUE",     SymbolType.KEYWORD),
FALSE              ("FALSE",    SymbolType.KEYWORD),
NULL               ("NULL",     SymbolType.KEYWORD),    // Null value - declared but not initialized
LITERAL            ("LITERAL",  SymbolType.NO_SYMBOL),  // Literal value, e.g. 1, "Hello", TRUE

/* Management */
IDENTIFIER         ("IDENTIFIER", SymbolType.COMPLEX),
EOF                ("End of File", SymbolType.COMPLEX);

/** Constants **/

// Each opcode is lexed differently, some opcodes have special rules, others can be lexed easily. The symbol type is
// only used for lexing, however to keep information on literal value in one place it is stored here. Can parse
// each symbol to determine type, but is very complicated and messy - best to assign at creation.
public enum SymbolType {
    SIMPLE,        // Symbols that are a one character, do not share with other opcode, and no special rules
    COMPLEX,       // More than one character and may have special rules, e.g. comments
    SHARED,        // Symbols that share characters with other opcodes, e.g. assignment and equals
    NO_SYMBOL,     // Has no exact representation in language
    KEYWORD        // Reserved words that need to be lexed as a whole
}

public final String LABEL;           // Literal representation in the language (needed for lexing)
public final char SYMBOL;           // Character representation of the operator, if it has one
public final SymbolType SYMBOL_TYPE; // The type of symbol the opcode is

// The keywords and simple symbols are stored to easily lex (reduces if-else chains). Complex and no symbol opcodes
// don't need a map as their lexing is not a simple comparison. Used as static constants as to not overload
// enum with methods (and makes it easier to access/initialize).
// Help received: https://www.baeldung.com/java-initialize-hashset
public static final Map<String, OperationCode> KEYWORDS;
public static final Map<Character, OperationCode> SIMPLE_SYMBOLS;
// Help received: https://stackoverflow.com/questions/7996499/creating-hashtable-as-final-in-java
static {
    // I'm aware of code duplication, but no good way to avoid in this case due to different types within the maps.
    // Generics can't be used as char and string due to different methods.

    // Need to temporary maps as they need to be final to not be reassigned.
    Map<Character, OperationCode> tempSimpleSymbols = new HashMap<>();
    Map<String, OperationCode> tempKeywords = new HashMap<>();

    // Go through each opcode and add it to their respective maps
    for (OperationCode opcode : OperationCode.values()) {
        if (opcode.SYMBOL_TYPE == SymbolType.KEYWORD) { tempKeywords.put(opcode.LABEL, opcode); }
        else if (opcode.SYMBOL_TYPE == SymbolType.SIMPLE) { tempSimpleSymbols.put(opcode.SYMBOL, opcode); }
    }

    // Make it so they can't be changed as java passes objects by reference
    SIMPLE_SYMBOLS = Collections.unmodifiableMap(tempSimpleSymbols);

```

```

        KEYWORDS = Collections.unmodifiableMap(tempKeywords);
    }

    /** Constructors **/

    /**
     *
     * Lexer.OperationCode.OperationCode()
     *
     * NAME
     *
     *     OperationCode - constructor for each enum value.
     *
     * SYNOPSIS
     *
     *     OperationCode(String a_label, OperationCode.SymbolType a_symbolType);
     *         a_label      -> the literal representation of the opcode in the
     *             language.
     *         a_symbolType -> the type of symbol the opcode is (simple, complex,
     *             shared, no symbol, keyword).
     *
     * DESCRIPTION
     *
     *     It initializes the label, symbol, and symbol type of the opcode. The
     *     label is the literal representation of the opcode in the language, if
     *     it has no symbol however, the label acts as a name to identify the
     *     operation. The symbol is the character representation of the operator,
     *     if it's too complex to have a symbol, it is set to a null character.
     *     The symbol type is the type of symbol the opcode is, which is used for
     *     lexing and putting the opcode in the correct category.
     *
     * RETURNS
     *
     *     Nothing - it's a constructor.
     *
     */
    /**
     *
     * OperationCode(String a_label, SymbolType a_symbolType) {
     *     LABEL = a_label;
     *
     *     // If the label is a single character, use it as the symbol - easier when lexing
     *     // Help received: IDE suggestion for ternary
     *     SYMBOL = (a_label.length() == 1) ? a_label.charAt(0) : '\0';
     *     SYMBOL_TYPE = a_symbolType;
     * }
     *
     * OperationCode(String a_label, OperationCode.SymbolType a_symbolType) */
}

```

Token.java

```
///
/// Token - Represents a token in the input file, storing the operation code to inform parser of what to do with it,
///         value associated with it when found, and line number it was found on for error reporting. Acts more as a struct
///         rather than a class, as it only contains data and state changing methods. Without it, we would have to pass
///         around multiple values in a list or array, which would be less readable and harder to manage.
///

package Lexer;

import Interpreter.Value;

public final class Token {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * Lexer.Token.Token()
     *
     * NAME
     *
     * Token - constructor for the token class.
     *
     * SYNOPSIS
     *
     * Token(OperationCode a_opcode, Value a_value, int a_lineNumber);
     *     a_opcode      -> the operation code of the token.
     *     a_value       -> the value associated with the token.
     *     a_lineNumber  -> the line number the token was found on.
     *
     * DESCRIPTION
     *
     * Initializes the operation code, value, and line number of the token.
     * Specifically used for operation codes that have a specific value
     * associated with them, e.g. numbers, identifiers, etc.
     *
     * RETURNS
     *
     * Nothing - it's a constructor.
     */
    /**
     *
     * public Token(OperationCode a_opcode, Value a_value, int a_lineNumber) {
     *     m_opcode = a_opcode;
     *     m_value = a_value;
     *     m_lineNumber = a_lineNumber;
     * }
     *
     * /* Token(OperationCode a_opcode, Value, int a_lineNumber) */
     *
     * /**
     */
}
```

```
/*  
Lexer.Token.Token()
```

NAME

Token - constructor for the token class.

SYNOPSIS

```
Token(OperationCode a_opcode, int a_lineNumber);  
    a_opcode      -> the operation code of the token.  
    a_lineNumber  -> the line number the token was found on.
```

DESCRIPTION

Initializes the operation code, value, and line number of the token. Specifically used for operation codes that don't have a specific value associated with them, e.g. ADD, SUB, MUL, DIV, etc. Will just pass the label of the operation code as the value.

RETURNS

Nothing - it's a constructor.

```
*/  
/**/  
public Token(OperationCode a_opcode, int a_lineNumber) {  
    m_opcode = a_opcode;  
    // No value will exist for certain operation codes, so just pass their label as value  
    m_value = new Value(a_opcode.LABEL);  
    m_lineNumber = a_lineNumber;  
}  
/* Token(OperationCode a_opcode, int a_lineNumber) */  
  
/** Accessors **/  
  
// Gets the operation code of the token  
public OperationCode GetOpcode() {  
    return m_opcode;  
}  
  
// Gets the value associated with the token  
public Value GetValue() {  
    return m_value;  
}  
  
// Gets the line number the token was found on for error reporting  
public int GetLineNumber() {  
    return m_lineNumber;  
}
```

```

/* PRIVATE */

/** Members **/

private final OperationCode m_opcode; // The operation code of the token (e.g. ADD, SUB, MUL, DIV, etc.)
private final Value m_value; // The value of the token - will be string if token is operator
private final int m_lineNumber; // The line number the token was found on
}

```

Parser Package

Parser.java

```

///
/// Parse - Responsible for parsing the tokens from the lexer into an Abstract Syntax Tree (AST), that the Interpreter
/// can evaluate. We use recursive descent to parse the tokens and build the AST, using EBNF grammar rules to guide
/// us. This mean our grammar (found in the Grammar.txt file and within the comments of the functions) is directly
/// translated into code. We also have error checking to ensure the tokens match the grammar rules, and if not, we
/// throw an error to inform the user.
///

package Parser;

import AbstractSyntaxTree.Statements.ConditionalField;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.Expressions.*;
import AbstractSyntaxTree.Expressions.Assignment;
import AbstractSyntaxTree.Statements.*;
import AbstractSyntaxTree.Statements.For;
import AbstractSyntaxTree.Statements.While;
import Language.Error;
import Language.Error.ErrorType;
import Interpreter.Value;
import Lexer.OperationCode;
import Lexer.Token;

import java.util.ArrayList;
import java.util.EnumSet;
import java.util.List;
import java.util.Set;
import java.util.function.Supplier;

public class Parser {
    /* PUBLIC */

    /** Constructors **/

    /**/

```

```
/*
Parser.Parser()
```

NAME

Parser - constructor for the parser.

SYNOPSIS

```
Parser(List<Token> a_tokens);
    a_tokens    -> the list of tokens to parse from the lexer.
```

DESCRIPTION

Initializes the list of tokens and sets up the state of the parser. Specifically, we set our nest flags to 0 and set our current token to the first token in the list. These tokens will be parsed to create the Abstract Syntax Tree (AST).

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Parser(List<Token> a_tokens) {
    m_tokens = a_tokens;
    m_currentTokenIndex = 0;
    m_currentToken = m_tokens.get(m_currentTokenIndex);
    m_savedTokenState = m_currentToken;

    // Not in any nesting structures yet
    m_ifElseNest = 0;
    m_loopNest = 0;
    m_functionNest = 0;
}
/* Parser(List<Token> a_tokens); */

/** Methods */

// Driver function to parse the program and return the list of nodes
public List<Node> ParseProgram() {
    return Program();
}

/* PRIVATE */

/** Members */

private final List<Token> m_tokens;    // List of tokens to parse from Lexer
private Token m_savedTokenState;       // Saves the state of the token before consuming
```

```

private Token m_currentToken;           // Current token being parsed
private int m_currentTokenIndex;        // The index of the current token within m_tokens

// Flags to help determine positional statements, e.g. break or else, which can't be accessed in certain contexts.
// They count the number of nests its currently in to determine if those statements are allowed (needs to be
// able to detect nested value, hence why not just boolean).
private int m_ifElseNest;               // See if we are in if-else statement or not
private int m_loopNest;                 // Whether we are in a loop or not
private int m_functionNest;             // Parsing inside a function or not

/** Constants **/

// Each static set of operation codes are used to determine what operations are allowed in the current context when
// parsing. They are declared as static constants to avoid creating new sets each time they are used, so
// if we are deep within a nested structure, we don't have to create new sets each time we check the context.
// Done to reduce code duplication, e.g. binary operators all have 2 operands but need to result in a different
// evaluation based on the operator at a specific precedence level. Put here to easily change if needed. See
// grammar rules for more information, as each set corresponds to a different level of the grammar.
private static final Set<OperationCode> BODY_TERMINATORS = EnumSet.of(
    OperationCode.END, OperationCode.EOF,
    OperationCode.ELSEIF, OperationCode.ELSE);

private static final Set<OperationCode> LOGICAL_OPERATORS = EnumSet.of(
    OperationCode.AND, OperationCode.OR);
private static final Set<OperationCode> EQUALITY_OPERATORS = EnumSet.of(
    OperationCode.EQUALITY, OperationCode.INEQUALITY);
private static final Set<OperationCode> COMPARISON_OPERATORS = EnumSet.of(
    OperationCode.LESS_THAN, OperationCode.LESS_THAN_EQUAL,
    OperationCode.GREATER_THAN, OperationCode.GREATER_THAN_EQUAL);
private static final Set<OperationCode> TERM_OPERATORS = EnumSet.of(
    OperationCode.ADD, OperationCode.SUBTRACT);
private static final Set<OperationCode> UNARY_OPERATORS = EnumSet.of(
    OperationCode.ADD, OperationCode.SUBTRACT, OperationCode.NOT);
private static final Set<OperationCode> POWER_OPERATORS = EnumSet.of(
    OperationCode.POWER);
private static final Set<OperationCode> FACTOR_OPERATORS = EnumSet.of(
    OperationCode.MULTIPLY, OperationCode.DIVIDE, OperationCode.MODULO);

/** Private Helper Functions **/

/*
 * Statements - Operations that have side effects
 */

/**/
/*
Parser.Program()

NAME

Program - parses the program from the list of tokens.

```


SYNOPSIS

```
private List<Node> Program();
```

DESCRIPTION

Parses the program from the list of tokens. The program is a list of statements, which have optional newlines between. It will continue to parse statements until it reaches the end of the file - designated by the EOF operation code.

RETURNS

List<Node> - the list of statements that make up the program.

```
*/
/**/
private List<Node> Program() {
    /*
     * Grammar Rules
     * <program>          : ('NEWLINE'* <statement> 'NEWLINE'*)* 'EOF'
     */

    // A program is just a bunch of statements
    List<Node> program = new ArrayList<>();

    // Make sure if user entered just a bunch of newlines we don't parse them
    IgnoreNewlines();

    // While we have not reached the end of the file, parse the statements
    while (!IsEndOfFile()) {
        program.add(Statement());

        // Ignore newlines between statements
        IgnoreNewlines();
    }

    return program;
}
/* private List<Node> Program(); */

/**/
/*
Parse.Statement()
```

NAME

Statement - parses a statement from the list of tokens.

SYNOPSIS

```
private Node Statement();
```

DESCRIPTION

This function parses a statement from the inputted tokens. It can be a function definition, an if-elseif-else conditional, a while loop, a for loop, a jump control statement, or an expression. It will error if a positional statement (e.g. BREAK outside a loop or ELSEIF not in IF chain). It will also ensure that a newline is present after a statement unless it is the end of the file or the next token ends a body.

RETURNS

A Node object that represents the statement parsed.

```
*/
/**/
private Node Statement() {
    /*
    * Grammar Rules
    * <statement>          :   <function>
    *                       |   <if-conditional>
    *                       |   <while-loop>
    *                       |   <for-loop>
    *                       |   <jump>
    *                       |   <expression>
    */

    // If we detect a statement that has a positional context, e.g. break or else with no if body, show a detailed
    // error message. Can be caught within Atom(), but this is much more descriptive.
    ErrorOnPositionalStatements();

    // Help received: IDE suggestion for switch expression
    Node statement = switch (m_currentToken.GetOpcode()) {
        // These statements don't require a newline after them, it's pretty clear where they end
        case DEF -> Function();
        case IF -> IfConditional();
        case WHILE -> WhileLoop();
        case FOR -> ForLoop();
        case BREAK, CONTINUE, RETURN -> Jump();

        // Not a statement start, so it must be an expression
        default -> Expression();
    };

    // Newline only optional at EOF, END, ELSEIF, or ELSE as it's unambiguous where the statement ends
    if (!BODY_TERMINATORS.contains(m_currentToken.GetOpcode())) { ForceNewline(); }

    return statement;
}
/* private Node Statement(); */

/**/
/*
```

Parser.Function()

NAME

Function - parses a function definition if token starts with 'DEF'.

SYNOPSIS

```
private Function Function();
```

DESCRIPTION

When a statement start with the keyword 'DEF' we are in a function definition. 'DEF' must be followed by an identifier for the function name, a list of parameters which must be 'IDENTIFIER' operation codes, and the body of the function. The function body is ended by the 'END' keyword. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Function node object that represents the function definition.

```
*/
/**/
private Function Function() {
    /*
     * Grammar Rules
     * <function>          :  'DEF' 'IDENTIFIER' '(' ('IDENTIFIER' (',' 'IDENTIFIER')* )? ')' <body> 'END'
     */

    MarkInFunction();
    Advance(OperationCode.DEF);

    // Save where we started
    int functionLineNumber = m_savedTokenState.GetLineNumber();

    Advance(OperationCode.IDENTIFIER);
    String functionName = m_savedTokenState.GetValue().AsString();

    // Get the parameters for the function
    List<Node> parameters = ParseArguments(OperationCode.LPARENTHESIS, OperationCode.RPARENTHESIS,
                                           this::ParseIdentifier);

    // Cast them to Identifiers. ParseArguments() has same structure for calls, lists and function definitions, but
    // the only thing that changes is what is "consumes" as the arguments. We need identifiers in the
    // function definition, so if we can't cast there is a huge problem.
    List<Identifier> identifiers = new ArrayList<>();
    for (Node parameter : parameters) { identifiers.add((Identifier) parameter); }

    // Get the body of the function
    Body body = Body();
}
```

```

    // Need to ensure we end the function
    ExpectEndOfBody("function definition", functionLineNumber);
    MarkOutFunction();

    return new Function(functionName, identifiers, body);
}
/* private Function Function(); */

/**/
/*
Parser.IfConditional()

```

NAME

IfConditional - parses an if-elseif-else conditional chain statement.

SYNOPSIS

```
private IfElseifElse IfConditional();
```

DESCRIPTION

When a statement starts with the keyword 'IF' we are in an if-elseif-else chain. After the 'IF' keyword, we expect an expression to evaluate, followed by the 'THEN' keyword. If these are not met an error is thrown. The body of the if statement is then parsed. If there are any 'ELSEIF' tokens, they are parsed in a loop until we either hit an 'ELSE' token or the 'END' token of the conditional. These conditions are stored within a list of ConditionalField objects, which will store the condition and body of the conditional.

For example, this would be a valid if-elseif-else chain:
 'IF x < 10 THEN PRINT(x) ELSEIF x < 20 THEN PRINT(x) ELSE PRINT(x) END'.

The if-elseif-else chain can also have an 'ELSE' token, which will be parsed if it is present. Regardless of the chain, 'END' token is always required to end the conditional. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

An IfElseifElse Node that represents the if-elseif-else chain is return. It will contain a list of ConditionalField objects that store the chain, if it is length 1 it is just an if statement, if it is more than 1 it is an else-if chain. If it had an else statement, it will be stored in the IfElseifElse object as well.

```

*/
/**/
private IfElseifElse IfConditional() {
    /*
    * Grammar Rules

```

```

* <if-conditional>      :   'IF' <expression> 'THEN' <body>
*                        <elseif-conditional>*
*                        <else-conditional>?
*                        'END'
*/

// Conditions for the if-elseif-else chain. If size is one, only if, more than one is else-if chain.
MarkInConditional();
List<ConditionalField> a_conditions = new ArrayList<>();

/* IF */
Advance(OperationCode.IF);

// Save line number for error message at runtime
int ifLineNumber = m_savedTokenState.GetLineNumber();

// Parse the condition and body
Node ifCondition = Expression();
Advance(OperationCode.THEN);
Body ifBody = Body();

// Add the if condition
a_conditions.add(new ConditionalField(ifCondition, ifBody, ifLineNumber));

/* ELSE-IF */
// While we have an else-if chain, add their conditions and bodies
while (m_currentToken.GetOpCode() == OperationCode.ELSEIF) {
    a_conditions.add(ElseIfConditional());
}

/* ELSE */
// Check if we have an else statement. Conditional constructor will handle empty/null check of elseifConditions,
//     elseifBodies, and elseBody.
// Help received: IDE suggestion for ternary
IfElseifElse ifElseBody = m_currentToken.GetOpCode() == OperationCode.ELSE ?
    new IfElseifElse(a_conditions, ElseConditional()) :
    new IfElseifElse(a_conditions);

// Need to ensure we end the conditional body
ExpectEndOfBody("conditional", ifLineNumber);
MarkOutConditional();

// Conditional constructor will handle empty/null check of elseifConditions, elseifBodies, and elseBody
return ifElseBody;
}
/* private IfElseifElse IfConditional(); */

/**/
/*
Parser.ElseIfConditional()

NAME

```

ElseIfConditional - parses an else-if conditional statement.

SYNOPSIS

```
private ConditionalField ElseIfConditional();
```

DESCRIPTION

When a statement starts with the keyword 'ELSEIF' we are in an else-if conditional. After the 'ELSEIF' keyword, we expect an expression that represents the condition to evaluate, followed by the 'THEN' keyword. An 'ELSEIF' keyword can only be parsed if it is attached to an already existing if-elseif-else chain. If these are not met an error is thrown. For example, this would be a valid else-if statement: 'ELSEIF x < 10 THEN PRINT(x) END'.

At any point if we hit an 'ELSE' or 'END' token, we end that elseif conditional and return the condition and body. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A ConditionalField object that represents the else-if conditional. It will store the condition and body of the conditional.

```
*/
/**/
private ConditionalField ElseIfConditional() {
    /*
     * Grammar Rules
     * <elseif-conditional>      :   'ELSEIF' <expression> 'THEN' <body>
     */

    Advance(OperationCode.ELSEIF);

    // Save line number for error message at runtime
    int elseifLineNumber = m_savedTokenState.GetLineNumber();

    // Parse the condition and else-if body
    Node elseifCondition = Expression();
    Advance(OperationCode.THEN);
    Body elseifBody = Body();

    return new ConditionalField(elseifCondition, elseifBody, elseifLineNumber);
}
/* private ConditionalField ElseIfConditional(); */

/**/
/*
Parser.ElseConditional()
```

NAME

ElseConditional - parses an else after an if-elseif chain.

SYNOPSIS

```
private Body ElseConditional();
```

DESCRIPTION

When a statement starts with the keyword 'ELSE' we are in an else, so parse its body. An 'ELSE' keyword can only be parsed if it is attached to an already existing if-elseif-else chain. If these are not met an error is thrown. For example, this would be a valid else statement: 'ELSE PRINT("No conditions met") END'.

If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Body object that represents the body of the else statement.

```
*/
/**/
private Body ElseConditional() {
    /*
     * Grammar Rules
     * <else-conditional>      :   'ELSE' <body>
     */

    Advance(OperationCode.ELSE);

    // Get the body of code for the else statement
    return Body();
}

/**/
/*
Parser.WhileLoop()
```

NAME

WhileLoop - parses a while loop statement.

SYNOPSIS

```
private While WhileLoop();
```

DESCRIPTION

When a statement starts with the keyword 'WHILE' we are in a while loop. Mark our nested loop flag to indicate we are in a loop, so we can parse break and continue statements. Our 'WHILE' keyword must be followed by an expression which evaluates to a boolean, followed by the 'THEN'

keyword. If these are not met an error is thrown. The body of the while loop is then parsed. For example, this would be a valid while loop:
'WHILE x < 10 THEN PRINT(x); LET x = x - 1 END'.

The loop is ended by the 'END' keyword. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A While Node that represents the data from the while loop statement.

```

*/
/**/
private While WhileLoop() {
    /*
    * Grammar Rules
    * <while-loop>          :   'WHILE' <expression> 'THEN' <body> 'END'
    */

    MarkInLoop();
    Advance(OperationCode.WHILE);

    // Save line number for error message at runtime
    int whileLineNumber = m_savedTokenState.GetLineNumber();

    // Parse the condition and body with
    Node condition = Expression();
    Advance(OperationCode.THEN);
    Body body = Body();

    // Need to ensure we end the loop
    ExpectEndOfBody("while loop", whileLineNumber);
    MarkOutLoop();

    return new While(new ConditionalField(condition, body, whileLineNumber));
}
/* private While WhileLoop(); */

/**/
/*
Parser.ForLoop()

```

NAME

ForLoop - parses a for loop statement.

SYNOPSIS

```
private For ForLoop();
```

DESCRIPTION

When a statement starts with the keyword 'FOR' we are in a for loop. Mark our nested loop flag to indicate we are in a loop, so we can parse break and continue statements. Our 'FOR' keyword must be followed by an assignment, e.g. 'FOR LET i = 10', the 'TO' keyword, an expression for the end value, the 'STEP' keyword followed by an expression for the increment value. It must be followed by the 'THEN' keyword, and the body of the loop. For example, this would be a valid for loop: 'FOR LET i = 0 TO 10 STEP 1 THEN PRINT(i) END'.

The loop is ended by the 'END' keyword. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A For Node that represents the data from the for loop statement.

```

*/
/**/
private For ForLoop() {
    /*
     * Grammar Rules
     * <for-loop>           :   'FOR' <assignment> 'TO' <expression> 'STEP' <expression> 'THEN' <body> 'END'
     */

    MarkInLoop();
    Advance(OperationCode.FOR);

    // Save line number for error message at runtime
    int forLineNumber = m_savedTokenState.GetLineNumber();

    // 'FOR' must be followed by an assignment
    Assignment assignment = Assignment();

    // Get the ending value for the loop
    Advance(OperationCode.TO);
    Node endValue = Expression();

    // See how much to increment by (our step)
    Advance(OperationCode.STEP);
    Node incrementValue = Expression();

    // Get the body of code for the loop
    Advance(OperationCode.THEN);
    Body loopBody = Body();

    // Need to ensure we end the loop
    ExpectEndOfBody("for loop", forLineNumber);
    MarkOutLoop();

    return new For(assignment, endValue, incrementValue, loopBody, forLineNumber);
}
/* private For ForLoop(); */

```

```
/**/
/*
Parser.Jump()
```

NAME

Jump - parses a jump control statement, e.g. break, continue, return.

SYNOPSIS

```
private Jump Jump();
```

DESCRIPTION

When a statement starts with the keyword 'BREAK', 'CONTINUE', or 'RETURN' we have a jump control statement. Break and continue can only exist without any expression on the same line, while return can have an expression to return. If these are not met an error is thrown. If no expression is found next to a 'RETURN' keyword, our NULL Value is returned.

It will also error if we are not inside a loop or function when the positional tokens are found. Meaning, 'BREAK' and 'CONTINUE' can only be found inside a loop, and 'RETURN' can only be found inside a function. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Jump Node object that represents the jump control statement parsed.

```
*/
/**/
private Jump Jump() {
    /*
    * Grammar Rules
    * <jump>                :  'BREAK' | 'CONTINUE' | ('RETURN' <expression>?)
    */

    // Ensure we are inside a loop before allowing loop control statements
    ErrorOnPositionalStatements();

    OperationCode opcode = m_currentToken.GetOpcode();
    switch(opcode) {
        // Record that we found a jump or continue
        case BREAK, CONTINUE:
            Advance(opcode);
            return new Jump(m_savedTokenState);
        // Can be 'RETURN' with or without an expression
        case RETURN:
            Advance(opcode);
            // No expression to return, so return null
```

```

        if (IsNewline()) { return new Jump(m_savedTokenState); }
        return new Jump(m_savedTokenState, Expression());

        // Should not be able to reach here, but if we do, it's a syntax error
    default:
        throw new Error("Expected a jump control statement, but got '" + opcode.LABEL + "'",
                        ErrorType.SYNTAX_ERROR,
                        m_savedTokenState.GetLineNumber());
    }
}
/* private Jump Jump(); */

/**/
/*
Parser.Body()

```

NAME

Body - parses a body of code, which is a list of statements.

SYNOPSIS

```
private Body Body();
```

DESCRIPTION

A body of code is a list of statements. It will continue to parse statements until it reaches the end of the file, or a token that ends a body (ELSEIF, ELSE, END, or EOF). The body of code can be empty, so it will return an empty list if no statements are found. At any point, if the statement that is recursively called does not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Body object storing the list of statements that make up the body of code.

```

*/
/**/
private Body Body() {
    /*
     * Grammar Rules
     * <body>           :   <statement>*
     */

    // Any newlines before a new body can be ignored
    IgnoreNewlines();

    List<Node> statements = new ArrayList<>();
    // While there is nothing that signals the end of the body, keep parsing statements
    while (!BODY_TERMINATORS.contains(m_currentToken.GetOpcode())) {
        statements.add(Statement());
    }
}

```

```

        // Ignore newlines between statements
        IgnoreNewlines();
    }

    return new Body(statements);
}
/* private Body Body(); */

/*
 *   Expressions - Operations that will result in a value when evaluated
 */

/**/
/*
Parser.Expression()

```

NAME

Expression - parses an expression from the list of tokens.

SYNOPSIS

```
private Node Expression();
```

DESCRIPTION

This function parses an expression from the inputted tokens. An expression can be an assignment, a logical operation, an equality operation, a comparison operation, a term operation, a factor operation, or a power operation. These rules are chain downwards (hence recursive decent parsing) and are evaluated if their respective tokens are found. It will continue to parse the expression until it reaches the 'EOF' token, or a token that ends an expression. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Node object that represents the expression parsed.

```

*/
/**/
private Node Expression() {
    /*
     * Grammar Rules
     * <expression>          :   <assignment>
     *                       |   <logical>
     */

    if (m_currentToken.GetOpcode() == OperationCode.LET) { return Assignment(); }

    return Logical();
}

```

```

}
/* private Node Expression(); */

```

```

/**/
/*
Parser.Assignment()

```

NAME

Assignment - parses an assignment statement.

SYNOPSIS

```
private Assignment Assignment();
```

DESCRIPTION

When a statement starts with the keyword 'LET' we are in an assignment. 'LET' must be followed by an identifier, which is the variable we are assigning to. If there is an '=' token after the identifier, we are assigning a value to the variable. If there is no '=', we are declaring the variable with our NULL Value. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

An Assignment Node object that represents the assignment statement. Stores the variable name and the expression to assign to it (if there is one).

```

*/
/**/
private Assignment Assignment() {
    /*
    * Grammar Rules
    * <assignment>          :  'LET' 'IDENTIFIER' ('=' <expression>)?
    */

    Advance(OperationCode.LET);
    Advance(OperationCode.IDENTIFIER);

    // Save the identifier as the variable to assign to
    Token variable = m_savedTokenState;

    // If we are declaring here, we need to have an '=' sign to assign a value
    if (m_currentToken.GetOpcode() == OperationCode.ASSIGNMENT) {
        Advance(OperationCode.ASSIGNMENT);
        return new Assignment(variable, Expression());
    }

    // End of file or line, so no expression to assign and we have a null value
    if (IsEndOfFile() || IsNewline() || IsSeparator()) { return new Assignment(variable, new Literal(Value.NULL)); }
}

```

```

// If something follows the identifier, not being EOF or newline, it means it's an expression (e.g.
//      'LET x + 3 = 2'). Thrown here to have a more descriptive error message.
throw new Error("Expected a complete assignment statement, but got " +
    "" + m_currentToken.GetOpcode().LABEL + "",
    ErrorType.SYNTAX_ERROR,
    m_savedTokenState.GetLineNumber());
}

```

```

/* private Assignment Assignment(); */

```

```

/**/

```

```

/*

```

```

Parser.Logical()

```

NAME

Logical - parses a logical operation.

SYNOPSIS

```

private Node Logical();

```

DESCRIPTION

When parsing a logical operation, we are looking for 'AND' or 'OR' operators. These operators are used to combine two expressions together and evaluate them. We call the ParseBinaryOperation function to parse the logical operation, if the equality operation is followed by the 'AND' or 'OR' operators we have a logical operation. If those keywords are not found, then we just have an equality operation (descending until we hit the base case).

For example 'x == 10 AND y == 20' or 'x == 10 OR y == 20' would be a valid logical operation.

RETURNS

A Node object that represents the logical operation parsed.

```

*/

```

```

/**/

```

```

private Node Logical() {

```

```

    /*

```

```

        * Grammar Rules

```

```

        * <logical>                :   <equality> (('AND' | 'OR') <equality>)*

```

```

        */

```

```

        return ParseBinaryOperation(Equality(), LOGICAL_OPERATORS, this::Equality);

```

```

    }

```

```

/* private Node Logical(); */

```

```

/**/

```

```

/*

```

```

Parser.Equality()

```

NAME

Equality - parses an equality operation.

SYNOPSIS

```
private Node Equality();
```

DESCRIPTION

When parsing an equality operation, we are looking for '==' or '!=' operators. These operators are used to compare two expressions together and see if they are equal or not. We call the ParseBinaryOperation function to parse the equality operation, if the token after comparison is followed by the '==' or '!=' operators we have an equality operation. If those keywords are not found, then we just have a comparison operation (descending until we hit the base case).

For example 'x == 10' or 'x != 10' would be a valid equality operation.

RETURNS

A Node object that represents the equality operation parsed.

```
*/
/**/
private Node Equality() {
    /*
     * Grammar Rules
     * <equality>           :   <comparison> (('==' | '!=') <comparison>)*
     */

    return ParseBinaryOperation(Comparison(), EQUALITY_OPERATORS, this::Comparison);
}
/* private Node Equality(); */

/**/
/*
Parser.Comparison()
```

NAME

Comparison - parses a comparison operation.

SYNOPSIS

```
private Node Comparison();
```

DESCRIPTION

When parsing a comparison operation, we are looking for '<', '<=', '>', or '>=' operators. These operators are used to compare two expressions

together and see if they are less than, less than or equal to, greater than, or greater than or equal to. We call the ParseBinaryOperation function to parse the comparison operation, if the token after the term is followed by the above symbols we have a comparison operation. If those keywords are not found, then we just have a term operation - still descending until we hit the base case.

For example 'x < 10' or 'x >= 10' would be a valid comparison operation.

RETURNS

A Node object that represents the comparison operation parsed.

```
*/
/**/
private Node Comparison() {
    /*
    * Grammar Rules
    * <comparison>          :   <term> (('<' | '<=' | '>' | '>=') <term>)*
    */

    return ParseBinaryOperation(Term(), COMPARISON_OPERATORS, this::Term);
}
/* private Node Comparison(); */

/**/
/*
Parser.Term()
```

NAME

Term - parses a term operation.

SYNOPSIS

```
private Node Term();
```

DESCRIPTION

When parsing a term operation, we are looking for '+' or '-' operators. These operators are used to add or subtract two expressions together, and can be found above the factor operation as they are of lower precedence. We call the ParseBinaryOperation function to parse the term operation, if the token after factor is followed by the '+' or '-' operators we have a term operation. If those keywords are not found, then we just have a factor operation - still descending until we hit the base case.

For example 'x + 10' or 'x - 10' would be a valid term operation.

RETURNS

A Node object that represents the term operation parsed.


```

*/
/**/
private Node Term() {
    /*
    * Grammar Rules
    * <term>           :   <factor> (('+' | '-' ) <factor>)*
    */

    return ParseBinaryOperation(Factor(), TERM_OPERATORS, this::Factor);
}
/* private Node Term(); */

```

```

/**/
/*
Parser.Factor()

```

NAME

Factor - parses a factor operation.

SYNOPSIS

```
private Node Factor();
```

DESCRIPTION

When parsing a factor operation, we are looking for '*', '/', or '%' operators. These operators are used to multiply, divide, or modulo two expressions together, and can be found above the power operation as they are of lower precedence. We call the ParseBinaryOperation function to parse the factor operation, if the factor operation is followed by the above symbols we have a factor operation.

For example 'x * 10' or 'x / 10' would be a valid factor operation. As well as 'x % 10' for modulo.

RETURNS

A Node object that represents the factor operation parsed.

```

*/
/**/
private Node Factor() {
    /*
    * Grammar Rules
    * <factor>           :   <power> (('*' | '/' | '%') <power>)*
    */

    return ParseBinaryOperation(Power(), FACTOR_OPERATORS, this::Power);
}
/* private Node Factor(); */

```

```
/**/
/*
Parser.Power()
```

NAME

Power - parses a power operation.

SYNOPSIS

```
private Node Power();
```

DESCRIPTION

When parsing a power operation, we are looking for the '^' operator. This operator is used to raise the left expression to the power of the right expression. It can be found above the accessor call as it's of lower precedence. We call the ParseBinaryOperation function to parse the power operation, if the accessor operation is followed by the '^' operator we have a power operation. If not found, we just have an accessor operation - still descending until we hit the base case.

For example 'x ^ 10' or '2 ^ 1.1' would be a valid power operation.

RETURNS

A Node object that represents the power operation parsed.

```
*/
/**/
private Node Power() {
    /*
    * Grammar Rules
    * <power>           :   <accessor> ('^' <accessor>)*
    */

    return ParseBinaryOperation(Accessor(), POWER_OPERATORS, this::Accessor);
}
/* private Node Power(); */

/**/
/*
Parser.Accessor()
```

NAME

Accessor - parses an accessor operation; which accesses a string or list at a specific index or range.

SYNOPSIS

```
private Node Accessor();
```

DESCRIPTION

When parsing an accessor operation, we are looking for ':'. First we parse the unary operation, which is the value to be accessed, and if it is followed by a ':', we are accessing a specific index. This must be followed by another unary operation, which is the index to access at.

If we find a '|', we are accessing a range of indices. The unary before the '|' is the starting index, and the unary after the '|' is the ending index. If the tokens do not match the grammar rules, an error is thrown to inform the user. The '|' is optional, so we can have a single index accessor or a ranged index accessor. And it can be chained together, so we can have multiple accessors in a row.

If no ':' is found, we just have a unary operation - still descending until we hit the base case. For example 'x:0' or 'x:0|2' would be a valid accessor operation.

RETURNS

A Node object that represents the accessor operation parsed.

```
*/
/**/
private Node Accessor() {
    /*
     * Grammar Rules
     * <accessor>          :   <unary> (':' <unary> ('|' <unary>)? ) *
     */

    // First unary is the item to be accessed (if we find a colon)
    Node rootUnary = Unary();

    // While we have a colon, continue parsing for more unary operators (the indices)
    while (m_currentToken.GetOpcode() == OperationCode.ACCESSOR) {
        Advance(OperationCode.ACCESSOR);

        // Next unary is the index (could be starting one if ranged based subscript)
        Node index = Unary();

        // If we have a ranged based subscript with '|', parse the next unary operation for ending index
        if (m_currentToken.GetOpcode() == OperationCode.SUBSCRIPT) {
            Advance(OperationCode.SUBSCRIPT);
            // Get the ending index
            Node endIndex = Unary();

            // Create the ranged subscript accessor, putting the root as the value to be accessed
            rootUnary = new Index(rootUnary, m_savedTokenState, index, endIndex);
        }
        // No ranged subscript, so only one index
        else { rootUnary = new Index(rootUnary, m_savedTokenState, index); }
    }
}
```

```

        return rootUnary;
    }
    /* private Node Accessor(); */

    /**/
    /*
    Parser.Unary()

```

NAME

Unary - parses a unary operation.

SYNOPSIS

```
private Node Unary();
```

DESCRIPTION

When parsing a unary operation, we are looking for '+', '-', or 'NOT'. If we hit any of these tokens, it means we are performing a unary operation on the next expression. As they can be chained together, e.g. '---3', we call the Unary function recursively to parse the next unary operation. If we do not have any of those tokens, it means we have a call operation (or an atom if not a function call).

For example '-3' or 'NOT TRUE' would be a valid unary operation.

RETURNS

A Node object that represents the unary operation parsed.

```

*/
/**/
private Node Unary() {
    /*
    * Grammar Rules
    * <unary>          :  ('-' | '+' | 'NOT') <unary>
    *                  | <call>
    */

    // Check if we have a unary operations, i.e. '+', '-', 'NOT'
    if (UNARY_OPERATORS.contains(m_currentToken.GetOpcode())) {
        Advance(m_currentToken.GetOpcode());

        // Fetch another unary after the unary operator
        return new UnaryOperator(m_savedTokenState, Unary());
    }

    // If we don't have a unary operator, just return the call (or atom)
    return Call();
}
/* private Node Unary(); */

```

```
/**/
/*
Parser.Call()
```

NAME

Call - parses a call operation.

SYNOPSIS

```
private Node Call();
```

DESCRIPTION

When parsing a call operation, we are looking for a function call. A function call is an atom followed by a parenthesis, which contains the arguments for the function. If we find a parenthesis, we parse the arguments for the function call. It can be 0 or more arguments, and they are separated by a comma. If the tokens do not match the grammar rules, an error is thrown to inform the user.

If no parenthesis is found, it means we just have an atom. For example 'print("Hello")' or 'add(3, 2)' would be a valid call operation. While 'x' or '3' would be a valid atom.

RETURNS

A Node object that represents the call operation parsed.

```
*/
/**/
private Node Call() {
    /*
    * Grammar Rules
    * <call>          :  <atom> '(' ( <expression> (',' <expression>)* )? ')'
    *                  |  <atom>
    */

    Node atom = Atom();

    if (m_currentToken.GetOpcode() == OperationCode.LPARENTHESIS) {
        // Line number of the call for error message at runtime (current to get the LPARENTHESIS)
        int lineNumber = m_currentToken.GetLineNumber();

        // Parse the arguments for the call and return the call node
        List<Node> arguments = ParseArguments(OperationCode.LPARENTHESIS, OperationCode.RPARENTHESIS,
                                             this::Expression);
        return new Call(atom, arguments, lineNumber);
    }

    // If we don't have a call, just return the atom
    return atom;
}
```

```

}
/* private Node Call(); */

```

```

/**/
/*
Parser.Atom()

```

NAME

Atom - parses an atom operation; the base case of the expression parsing and recursive descent.

SYNOPSIS

```
private Node Atom();
```

DESCRIPTION

When parsing an atom operation, we are looking for an identifier, a literal, a group, or a list. If we find an identifier, it means we are looking at a variable, if we find a literal, it means we are looking at 'INTEGER', 'FLOAT', 'STRING', 'TRUE', 'FALSE', or 'NULL'. If we find a group, it means we are looking at an expression surrounded by parenthesis. If we find a list, it means we are looking at a list of expressions surrounded by brackets. These are the items with the highest precedence in the expression parsing.

For example 'x', '3', '(3 + 2)', or '[1, 2, 3]' would be a valid atom. Or "Hello" or 'TRUE' would be a valid literal.

As we are at the end of the expression parsing, if we do not hit a match here it means we've encountered bad syntax. An error is thrown to inform the user of their mistake, but as we can expect an extremely long list of values - we just say we expected a complete expression.

RETURNS

A Node object that represents the atom operation parsed.

```

*/
/**/
private Node Atom() {
    /*
    * Grammar Rules
    * <atom>           :   'IDENTIFIER'
    *                  |   <literal>
    *                  |   <group>
    *                  |   <list>
    */

    // Help received: IDE suggestion for switch expression
    return switch (m_currentToken.GetOpcode()) {
        // Literals

```

```

    case LITERAL, TRUE, FALSE, NULL -> Literal();
    // Variables
    case IDENTIFIER -> ParseIdentifier();
    // Parenthesized expressions
    case LPARENTHESIS -> Group();
    // Lists as literal
    case LBRACKET -> List();

    // Not matching the grammar, we expected to finish an expression at this point. So error is more general
    // than simply expecting a particular token.
    default ->
        throw new Error("Expected a complete expression, but got " +
            "\"" + m_currentToken.GetOpcode().LABEL + "\"",
            ErrorType.SYNTAX_ERROR,
            m_savedTokenState.GetLineNumber());
};
}
/* private Node Atom(); */

```

```

/**/
/*
Parser.Literal()

```

NAME

Literal - parses a literal operation; which can be an 'INTEGER', 'FLOAT', 'STRING', 'TRUE', 'FALSE', or 'NULL'.

SYNOPSIS

```
private Literal Literal();
```

DESCRIPTION

When parsing a literal operation, we are looking for a value with a literal representation, i.e. an actual value.

For example '3', '3.14', '"Hello"', 'TRUE', 'FALSE', or 'NULL' would be a valid literal.

RETURNS

A Literal Node object that represents the value parsed.

```

*/
/**/
private Literal Literal() {
    /*
    * Grammar Rules
    * <literal>          :  'INTEGER' | 'FLOAT' | 'STRING' | 'TRUE' | 'FALSE' | 'NULL'
    */

    // Advance current token and return the saved literal

```

```

        Advance(m_currentToken.GetOpcode());
        return new Literal(m_savedTokenState.GetValue());
    }
    /* private Literal Literal(); */

```

```

/**/
/*
Parser.Group()

```

NAME

Group - parses a group operation; which is an expression surrounded by parenthesis.

SYNOPSIS

```
private Group Group();
```

DESCRIPTION

When parsing a group operation, we are looking for an expression that is within parenthesis. This is used to indicate the order of operations, e.g. `'(3 + 2) * 5'`. We parse the expression inside the parenthesis and return it. Meaning we expect a left parenthesis, an expression, and then a right parenthesis. If the tokens do not match the grammar rules, an error is thrown to inform the user.

RETURNS

A Group Node object that represents the expression inside the group.

```

*/
/**/
private Group Group() {
    /*
    * Grammar Rules
    * <group>           :  '(' <expression> ')'
    */

    // Groups are surrounded by parenthesis to indicate order of operations, e.g. (3 + 2) * 5
    // Advance out inner and outer parenthesis and parse the expression inside
    Advance(OperationCode.LPARENTHESIS);
    Node expression = Expression();
    Advance(OperationCode.RPARENTHESIS);

    return new Group(expression);
}
/* private Group Group(); */

/**/
/*
Parser.List()

```


NAME

List - parses a list operation; which is a list of expressions surrounded by brackets.

SYNOPSIS

```
private ListExpression List();
```

DESCRIPTION

When parsing a list operation, we are looking for a list of expressions that are within brackets. First we expect a left bracket, then we parse an expression, and if there is a comma, we parse more expressions - we do this until we reach the right bracket. A list can have no expressions within it, so it can be empty. If the tokens do not match the grammar rules, an error is thrown to inform the user.

For example '[1, 2, 3]' or '["Hello", "World"]' would be a valid list, and '[]' would be an empty list. We are index 0 based, so the first element is at index 0.

RETURNS

A ListExpression Node object that represents the list of expressions inside the brackets.

```
*/
/**/
private ListExpression List() {
    /*
     * Grammar Rules
     * <list>           :   '[' ( <expression> (',' <expression>)* )? ']'
     */

    // When parsing we can have lists with expressions in them, so we need to parse them as such. E.g. [1 + 2, "hi"]
    List<Node> expressions = ParseArguments(OperationCode.LBRACKET, OperationCode.RBRACKET, this::Expression);

    return new ListExpression(expressions);
}
/* private ListExpression List(); */

/*
 *      Helping Parse - Avoiding code duplication and make parsing cleaner in grammar functions
 */

/**/
/*
Parser.ParseBinaryOperation()
```

NAME

ParseBinaryOperation - parses a binary operation; e.g. addition, subtraction, multiplication, division, etc.

SYNOPSIS

```
private Node ParseBinaryOperation(Node a_leftExpression,
                                  Set<OperationCode> a_operators,
                                  Supplier<Node> a_rightExpressionFunction);
a_leftExpression      -> the left expression of the binary
                        operation.
a_operators           -> the set of operators that are allowed
                        for the specific binary operation.
a_rightExpressionFunction -> the function to call to get the right
                        expression. This is a supplier function to allow for recursive
                        calls of different functions, e.g. Logical(), Comparison(),
                        Term(), etc.
```

DESCRIPTION

When parsing a binary operation, we are looking for an operation that combines two expressions together. This can be addition, subtraction, multiplication, division, power, etc. This helper function is called by all the binary operation parsing functions (logical, equality, comparison, term, factor, power) to parse the binary operation. As they all are binary expressions and have the same rules, we can avoid code duplication by allowing them to pass in the right expression as a lambda function. This function will then parse the binary operation according to the rules of the specific binary operation.

It is encouraged to look at the grammar of a specific binary operation to see what set of operators are allowed, and what the left and right expressions are.

For example '3 + 2' and '3 - 2' would be a valid term operation, where the right function is the factor operation. And '3 * 2' and '3 / 2' would be a valid factor operation, where the right function is the power operation. Once again, as binary operations all share the same structure for their code, it makes sense to allow a function to be passed in to get the right expression.

RETURNS

A Node object that represents the binary operation parsed.

```
*/
/**/
private Node ParseBinaryOperation(Node a_leftExpression, Set<OperationCode> a_operators,
                                  Supplier<Node> a_rightExpressionFunction) {
    // Help received: https://www.baeldung.com/java-callable-vs-supplier for the right expression function

    // While we have a valid operator, continue parsing
    // This can repeat, e.g. having 3 terms "3 + 3 - 1", so we absorb the operator and term in a loop
    while (a_operators.contains(m_currentToken.GetOpcode())) {
```

```

        // Advance to move to next token
        Advance(m_currentToken.GetOpcode());

        // Left term becomes the root of the tree/subtree
        // Calling the right expression will cause a recursive call to create the Abstract Syntax Tree (AST),
        a_leftExpression = new BinaryOperator(a_leftExpression, m_savedTokenState, a_rightExpressionFunction.get());
    }

    // Return our root of the tree/subtree
    return a_leftExpression;
}

/* private Node ParseBinaryOperation(Node a_leftExpression,
                                     Set<OperationCode> a_operators,
                                     Supplier<Node> a_rightExpressionFunction); */

/**/
/*
Parser.ParseArguments()

```

NAME

ParseArguments - parses a list of arguments for a function call, a list, or a function definition.

SYNOPSIS

```

private List<Node> ParseArguments(OperationCode a_startEncloser,
                                OperationCode a_endEncloser,
                                Supplier<Node> a_expressionFunction);
a_startEncloser      -> the opening token that encloses the
                        arguments, e.g. '(' for function calls, '[' for lists.
a_endEncloser        -> the closing token that encloses the
                        arguments, e.g. ')' for function calls, ']' for lists.
a_expressionFunction -> the function to call to get the
                        expression for the argument. This is a supplier function to
                        allow for different parsing functions to be called, e.g.
                        ParseIdentifier() for function definitions, and Expression() for
                        function calls and lists.

```

DESCRIPTION

When parsing arguments for a function call, a list, or a function definition, we are looking for a list of expressions that are separated by a comma. This function is called by the Call(), List(), and FunctionDefinition() functions to parse the arguments. As they all have the same rules for parsing arguments, we can avoid code duplication by allowing them to pass in the expression as a lambda function. This function will then parse the arguments according to the rules of the specific operation.

It is encouraged to look at the grammar of a specific operation to see what the start and end enclosers are, and what the expression for the argument is.

For example 'print("Hello", "World")' would be a valid function call, and '[1, 2, 3]' would be a valid list. And 'function add(x, y) { ... }' would be a valid function definition. As arguments all share the same structure for their code, it makes sense to allow a function to be passed in to get the expression/identifier for the argument. Similar to how the ParseBinaryOperation function works.

RETURNS

A list of Node objects that represents the arguments parsed.

```

*/
/**/
private List<Node> ParseArguments(OperationCode a_startEncloser, OperationCode a_endEncloser,
                                Supplier<Node> a_expressionFunction) {

    // Help received: https://www.baeldung.com/java-callable-vs-supplier for the expression function

    // Arguments are surrounded by parenthesis/brackets
    Advance(a_startEncloser);

    // The arguments are a list of expressions, so we need to parse them as such (if there are any)
    List<Node> arguments = new ArrayList<>();

    // If not closed, parse the arguments
    if (m_currentToken.GetOpcode() != a_endEncloser) {
        arguments.add(a_expressionFunction.get());

        // While we have a comma, continue parsing for more arguments
        while (IsSeparator()) {
            Advance(OperationCode.COMMA);
            // Allowed to have a newline between arguments if they get long
            IgnoreNewlines();
            arguments.add(a_expressionFunction.get());
        }
    }

    // Close the end parenthesis/bracket
    Advance(a_endEncloser);

    return arguments;
}

/* private List<Node> ParseArguments(OperationCode a_startEncloser,
                                    OperationCode a_endEncloser,
                                    Supplier<Node> a_expressionFunction); */

/**/
/*
Parser.ParseIdentifier()

```

NAME

ParseIdentifier - parses an identifier operation; which is a variable.

SYNOPSIS

```
private Identifier ParseIdentifier();
```

DESCRIPTION

When parsing an identifier operation, we are looking for a variable. In essence, it is just a name that represents a variable. It can be a function name, a variable name, or a parameter name. If we call this function, we are expecting an identifier token - so throw an error if we do not find one.

For example 'x', 'add', or 'name' would be a valid identifier.

RETURNS

An Identifier object that represents the variable parsed.

```
*/
/**/
private Identifier ParseIdentifier() {
    // Just a variable, so return it
    Advance(OperationCode.IDENTIFIER);
    return new Identifier(m_savedTokenState.GetValue().AsString(), m_savedTokenState.GetLineNumber());
}
/* private Identifier ParseIdentifier(); */

/*
 *      Management - Moving through tokens ensuring they are correct
 */

/**/
/*
Parser.Advance()
```

NAME

Advance - moves to the next token in the list of tokens, if our current token is the expected token.

SYNOPSIS

```
private void Advance(OperationCode a_opcode);
    a_opcode    -> the expected token that we are on.
```

DESCRIPTION

Before advancing to the next token, we are checking if the current token is the expected token. If it is not, we throw an error to inform the user of their mistake. If it is the expected token, we move to the next token

in the list of tokens.

For example, if we are expecting a 'POWER' token, we call this function with 'POWER' as the expected token and throw an error if we do not find it. If we do, we are free to move to the next token.

RETURNS

No return value - function is void.

```
*/
/**/
private void Advance(OperationCode a_opcode) {
    // If the current token is not the expected token, throw an error
    if (m_currentToken.GetOpcode() != a_opcode) {
        throw new Error("Expected '" + a_opcode.LABEL + "', but got " +
            "'" + m_currentToken.GetOpcode().LABEL + "'",
            ErrorType.SYNTAX_ERROR,
            m_savedTokenState.GetLineNumber());
    }

    // Move to the next token
    SetNextToken();
}
/* private void Advance(OperationCode a_opcode); */

/**/
/*
Parser.SetNextToken()
```

NAME

SetNextToken - moves to the next token in the list of tokens, regardless of end of file.

SYNOPSIS

```
private void SetNextToken();
```

DESCRIPTION

Before moving to the next token, we need to save the current token - so update the saved token state member. This is done to keep track of the tokens position within the line, making it easier to throw errors. This We also need to check if we've reached the end of the file, as we cannot move past the last token. If we have not reached the end of the file, we move to the next token in the list of tokens.

Function keep track of the state of the parser, moving on to the next token when required.

RETURNS

No return value - function is void.

```
*/
/**/
private void SetNextToken() {
    // Save the current token and move to the next token, regardless of EOF
    m_savedTokenState = m_currentToken;

    // Check to see if we have no more tokens to consume (EOF must be the last token)
    if (!IsEndOfFile()) {
        m_currentTokenIndex++;
        m_currentToken = m_tokens.get(m_currentTokenIndex);
    }
}
/* private void SetNextToken(); */
```

```
/*
 *      Error Handling - Ensuring the grammar is followed correctly
 */
```

```
/**/
/*
Parser.ErrorOnPositionalStatements()
```

NAME

ErrorOnPositionalStatements - throws an error if a positional statement is found outside its allowed context.

SYNOPSIS

```
private void ErrorOnPositionalStatements();
```

DESCRIPTION

When parsing a positional statement, we are looking for 'ELSEIF', 'ELSE' for conditional control, 'BREAK', 'CONTINUE', 'TO', 'STEP' for loop control, and 'RETURN' for function control. These statements are only allowed in specific contexts, e.g. 'ELSEIF' and 'ELSE' are only allowed in a conditional body, 'BREAK' and 'CONTINUE' are only allowed in loops, 'TO' and 'STEP' are only allowed in for loops, and 'RETURN' is only allowed within functions. We need to check throw and error if they exist outside their allowed context - which is what this function handles.

For example, if we find a 'BREAK' when our loop nest is 0, we throw an error. Same goes for 'ELSEIF' and 'ELSE' when our conditional nest is 0, and 'RETURN' when our function nest is 0. Very important to ensure the grammar is followed correctly and the user knows about it.

Also, it creates the message for the error depending on the statement found and the context it is found in - to make it extra clear.

RETURNS

No return value - function is void.

```
*/
/**/
private void ErrorOnPositionalStatements() {
    OperationCode opcode = m_currentToken.GetOpcode();
    // Error flag, if we've detected a positional statement outside its allowed context
    boolean isError = false;
    String message = "" + opcode.LABEL + " statement is not allowed outside of a ";

    // Check if the opcode is a positional statement, and if it is, check if it is allowed in the current context.
    // If not, set error flag to true and append the message to the error message.
    switch (opcode) {
        // Conditional control
        case ELSEIF, ELSE:
            if (IsNotInConditional()) { message += "conditional body"; isError = true; }
            break;
        // Loop control
        case BREAK, CONTINUE, TO, STEP:
            if (IsNotInLoop()) { message += "loop"; isError = true; }
            break;
        // Function control
        case RETURN:
            if (IsNotInFunction()) { message += "function"; isError = true; }
            break;
        // Body control for loops and conditionals
        case THEN:
            if (IsNotInConditional() && IsNotInLoop()) { message += "conditional/loop body"; isError = true; }
            break;
        case END:
            if (IsNotInConditional() && IsNotInLoop() && IsNotInFunction()) { message += "body"; isError = true; }
            break;

        // Not a positional statement, so no error can be thrown
        default: return;
    }

    // Throw an error if we got one
    if (isError) {
        throw new Error(message,
            ErrorType.SYNTAX_ERROR,
            m_savedTokenState.GetLineNumber());
    }
}

/* private void ErrorOnPositionalStatements(); */

/**/
/*
Parser.ExpectEndOfBody()

NAME
```


ExpectEndOfBody - throws an error if the end of a body for a specific body type is not found.

SYNOPSIS

```
private void ExpectEndOfBody(String a_bodyType, int startLine);
    a_bodyType      -> the type of body we are expecting to end, e.g.
                      'conditional', 'loop', 'function' as a string.
    startLine       -> the line number where the body started, to give
                      more context to the error message.
```

DESCRIPTION

When we are at the end of a body, we expect an 'END' token to close the body. This is to ensure the grammar is followed correctly and nothing is ambiguous. If we do not find an 'END' token when expected, we throw an error to inform the user of their mistake. We also give the line number where the body started to give more context to the error message.

RETURNS

No return value - function is void.

```
*/
/**/
private void ExpectEndOfBody(String a_bodyType, int startLine) {
    // If we are at the end of the body, we expect an 'END' token
    if (m_currentToken.GetOpcode() != OperationCode.END) {
        throw new Error("Expected '" + OperationCode.END.LABEL + "' after a " + a_bodyType,
            ErrorType.SYNTAX_ERROR,
            startLine);
    }
    Advance(OperationCode.END);
}
/* private void ExpectEndOfBody(String a_bodyType, int startLine); */

/**/
/*
Parser.ForceNewline()
```

NAME

ForceNewline - throws an error if a newline is not found when expected.

SYNOPSIS

```
private void ForceNewline();
```

DESCRIPTION

After a statement, which isn't followed by an 'EOF' or 'END', we expect a newline to separate the statements. This is to ensure the grammar is

followed correctly and nothing is ambiguous. If we do not find a newline when expected, we throw an error to inform the user of their mistake.

For example, if we find a 'RETURN' statement, we expect a newline to separate it from the next statement. If we do not find a newline, we really don't know how to interpret the next statement. For example 1 + 1 2 doesn't make sense, but 1 + 1; 2 does - they are two separate expressions.

RETURNS

No return value - function is void.

```
*/
/**/
private void ForceNewline() {
    // Newlines can be 'n' or ';' and they do not have a direct label. This ensures we are expecting a newline.
    if (!IsNewline()) {
        throw new Error("Expected a newline, but got '" + m_currentToken.GetOpcode().LABEL + "'",
            ErrorType.SYNTAX_ERROR,
            m_savedTokenState.GetLineNumber());
    }
    Advance(OperationCode.NEWLINE);
}
/* private void ForceNewline(); */

// Ignore newlines when they are not significant in the grammar, i.e. skip all until we hit a non-newline token
private void IgnoreNewlines() {
    while (IsNewline()) { Advance(OperationCode.NEWLINE); }
}

// Check if the current token is the end of the file (checked multiple times and more descriptive, so function it)
private boolean IsEndOfFile() {
    return m_currentToken.GetOpcode() == OperationCode.EOF;
}

// See if the current token is a newline (made into a function as more descriptive, and we check multiple times)
private boolean IsNewline() {
    return m_currentToken.GetOpcode() == OperationCode.NEWLINE;
}

// See if current token is a comma for separating arguments in function calls, lists, etc.
private boolean IsSeparator() {
    return m_currentToken.GetOpcode() == OperationCode.COMMA;
}

/*
 * Nesting - Keeping track of how deep we are in certain structures. Separated into functions to ensure
 * readability, as having m_ifElseNest++ is not very clear, but MarkInConditional() is much more
 * descriptive in code. Also ensures the members are being used properly. We need to use integers
 * instead of booleans as if we are doubly nested, we will mark false when we are still in an outer
 * structure. No method headers as they are self-explanatory functions.
 */
```

```

*/

// Marking when we are in a conditional (if-elseif-else)
private void MarkInConditional(){
    m_ifElseNest++;
}
// Marking when we are in a loop (for, while)
private void MarkInLoop() {
    m_loopNest++;
}
// Marking when we are in a function
private void MarkInFunction() {
    m_functionNest++;
}

// Marking when we are out of a conditional
private void MarkOutConditional() {
    if (m_ifElseNest > 0) { m_ifElseNest--; }
}
// Marking when we are out of a loop
private void MarkOutLoop() {
    if (m_loopNest > 0) { m_loopNest--; }
}
// Marking when we are out of a function
private void MarkOutFunction() {
    if (m_functionNest > 0) { m_functionNest--; }
}

// Checking if we are not in a conditional
private boolean IsNotInConditional() {
    return m_ifElseNest <= 0;
}
// Checking if we are not in a loop
private boolean IsNotInLoop() {
    return m_loopNest <= 0;
}
// Checking if we are not in a function
private boolean IsNotInFunction() {
    return m_functionNest <= 0;
}
}

```

AbstractSyntaxTree Package

Node.java

```

///
/// Node - An abstract class for the nodes in the Abstract Syntax Tree. It mainly serves as a structure for data. It
/// seems odd in OOP to have a class with only one method to override (similar to all the nodes that inherit from

```

```

/// it), but it is necessary for the visitor design pattern. Tree classes are 'owned' by any single thing, so they
/// merely need to exist just to store data. Each node should not be responsible for both storing data and
/// evaluating itself, the evaluation part is the interpreters job. This why the visitor design pattern is used, to
/// be able to store data in the nodes and put the evaluation in the interpreter with communication through double
/// dispatch. Seems odd to have roughly twenty classes with only data and no methods, but OOP doesn't really fit
/// well with trees, simply put.
///

package AbstractSyntaxTree;

import Interpreter.Value;

// Help received: https://www.geeksforgeeks.org/abstract-classes-in-java/ for abstract classes in Java
abstract public class Node {

    // This is a part of the visitor design pattern (see NodeVisitor.java)
    public abstract Value Accept(NodeVisitor a_visitor);

}

```

NodeVisitor.java

```

///
/// Node Visitor Interface - Using the Visitor Design Pattern to communicate between the nodes and the interpreter. As
/// a node is only responsible for storing structured data for each operation type, and the interpreter is
/// responsible for evaluating the nodes, we need an interface to bind them together. Each node has an Accept
/// method that takes a visitor, and the visitor has a Visit method for each node type. This way, the interpreter
/// can call Accept on the root node, and the nodes will call the appropriate Visit method on the visitor. This
/// approach, known as double dispatch, allows the nodes to store data and the interpreter to evaluate the nodes in
/// a clean, loosely coupled, and not in a horrible polymorphic mess.
///

// The visitor design pattern was completely new to me, and I've never heard of it before, a lot of research was needed
// It's more of a functional programming style, which I'm not used to. Because of this I wanted to ensure I give proper
// credit to the sources (in code as well as in the docs) that helped me understand what it is and how to use them.
//
// Idea to use visitor pattern came from sources that describe implementing an interpreter:
// https://craftinginterpreters.com/introduction.html
// https://ruslanspivak.com/lsbasi-part1/
// https://medium.com/codex/compound-pattern-series-visitor-and-interpreter-part-1-31dc22140bf0
// Help received for what visitor patterns are and how to implement in code:
// https://www.youtube.com/watch?v=UQP5XqMqtqQ
// https://www.geeksforgeeks.org/interfaces-in-java/
// https://www.baeldung.com/java-visitor-pattern
// https://stackoverflow.com/a/67821899
// https://www.geeksforgeeks.org/visitor-design-pattern/
// For why I shouldn't be method overloading the inherited Node classes:
// https://en.wikipedia.org/wiki/Separation\_of\_concerns
// https://en.wikipedia.org/wiki/Interpreter\_pattern
// https://www.youtube.com/watch?v=hxG0iir9ZKg

```

```

package AbstractSyntaxTree;

import AbstractSyntaxTree.Expressions.*;
import AbstractSyntaxTree.Expressions.Assignment;
import AbstractSyntaxTree.Statements.*;
import AbstractSyntaxTree.Statements.For;
import AbstractSyntaxTree.Statements.While;
import Interpreter.Value;

public interface NodeVisitor {
    /*
     *      Statements
     */

    Value VisitFunction(Function a_function);
    Value VisitIfElseifElse(IfElseifElse a_conditional);
    Value VisitWhile(While a_while);
    Value VisitFor(For a_for);
    Value VisitJump(Jump a_jump);
    Value VisitBody(Body a_body);

    /*
     *      Expressions
     */

    Value VisitAssignment(Assignment a_assignment);
    Value VisitBinaryOperator(BinaryOperator a_binary);
    Value VisitIndex(Index a_index);
    Value VisitUnaryOperator(UnaryOperator a_unary);
    Value VisitCall(Call a_call);
    Value VisitIdentifier(Identifier a_identifier);
    Value VisitLiteral(Literal a_literal);
    Value VisitGroup(Group a_group);
    Value VisitListExpression(ListExpression a_listExpression);
}

```

Expressions Package

Assignment.java

```

///
/// Assignment Node - Represents and holds data for an assignment operation in the Abstract Syntax Tree. Created by
///      'LET <identifier> = <expression>' or 'LET <identifier>' in the parser. Responsible for holding the variable
///      and the expression being assigned to the variable.
///
package AbstractSyntaxTree.Expressions;

```

```

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Lexer.Token;

public class Assignment extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Expressions.Assignment.Assignment()
     *
     * NAME
     *
     * Assignment - constructor for the assignment node.
     *
     * SYNOPSIS
     *
     * Assignment(Token a_variable, Node a_expression);
     *     a_variable      -> the name of the variable being assigned to.
     *     a_expression    -> the expression being assigned to the variable.
     *
     * DESCRIPTION
     *
     * Initializes the variable name and expression being assigned to the
     * variable. Specifically used for creating an assignment node in the
     * Abstract Syntax Tree. For example 'LET x = 1' would be an assignment
     * with 'x' as the variable name and '1' as the expression being assigned.
     *
     * RETURNS
     *
     * Nothing - it's a constructor.
     *
     */
    /**
     * public Assignment(Token a_variable, Node a_expression) {
     *     m_variable = a_variable;
     *     m_expression = a_expression;
     * }
     * /* Assignment(Token a_variable, Node a_expression); */
     *
     * /** Accessors */
     *
     * // Get the name of the variable being assigned to
     * public String GetVariableName() {
     *     return m_variable.GetValue().AsString();
     * }
     *
     * // Get the expression being assigned to the variable

```

```

public Node GetExpression() {
    return m_expression;
}

/** Methods */

// Accept a visitor and have it visit the assignment expression
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitAssignment(this);
}

/* PRIVATE */

/** Members */

private final Token m_variable;    // The name of the variable being assigned to
private final Node m_expression;   // The expression (soon to be value) being assigned to the variable
}

```

BinaryOperator.java

```

///
/// Binary Operator Node - Represents a binary operation in the Abstract Syntax Tree. Is created from
///     '<expression> <operator> <expression>' and is responsible for holding the left and right expressions as well as
///     the operator token.
///

package AbstractSyntaxTree.Expressions;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Lexer.Token;

public class BinaryOperator extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Expressions.BinaryOperator.BinaryOperator()
     *
     * NAME
     *
     * BinaryOperator - constructor for the binary operator node.
     *
     * SYNOPSIS
     */
}

```

```

BinaryOperator(Node a_leftExpression, Token a_token, Node a_rightExpression);
    a_leftExpression    -> the left side of the binary operation.
    a_token              -> the operator token (e.g. '+', '-', '*', '/')
    a_rightExpression    -> the right side of the binary operation.

```

DESCRIPTION

Initializes the left expression, operator token, and right expression of the binary operation. Specifically used for creating a binary operator node in the Abstract Syntax Tree. For example '1 + 2' would be a binary operation with '1' as the left expression, '+' as the operator token, and '2' as the right expression.

RETURNS

Nothing - it's a constructor.

```

*/
/**/
public BinaryOperator(Node a_leftExpression, Token a_token, Node a_rightExpression) {
    m_leftExpression = a_leftExpression;
    m_token = a_token;
    m_rightExpression = a_rightExpression;
}
/* BinaryOperator(Node a_leftExpression, Token a_token,
    Node a_rightExpression); */

/** Accessors */

// Get the left expression of the binary operation
public Node GetLeftExpression() {
    return m_leftExpression;
}

// Get the operator token of the binary operation
public Token GetToken() {
    return m_token;
}

// Get the right expression of the binary operation
public Node GetRightExpression() {
    return m_rightExpression;
}

/** Methods */

// Accept a visitor and have it visit the binary operator expression
@Override
public Value Accept(NodeVisitor a_visitor) {

```



```

        return a_visitor.VisitBinaryOperator(this);
    }

    /* PRIVATE */

    /** Members */
    private final Node m_leftExpression;    // The left side of the binary operation
    private final Token m_token;            // The operator token (e.g. '+', '-', '*', '/')
    private final Node m_rightExpression;    // The right side of the binary operation
}

```

Group.java

```

///
/// Group Node - Represents a group of expressions (e.g. (1 + 2)) (or a parenthesized expression). Is created from
/// '(<expression>)' in the parser. Responsible for holding the expression inside the group for precedence.
///

package AbstractSyntaxTree.Expressions;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

public class Group extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Expressions.Group.Group()
     *
     * NAME
     *
     * Group - constructor for the group node.
     *
     * SYNOPSIS
     *
     * Group(Node a_expression);
     *     a_expression    -> the expression inside the group.
     *
     * DESCRIPTION
     *
     * Initializes the expression inside the group. Specifically used for
     * creating a group node in the Abstract Syntax Tree. For example '(1 + 2)'
     * would be a group with '1 + 2' as the expression inside the group.
     */
    /**
     *
     * public Group(Node a_expression) {
     */

```

```

        m_expression = a_expression;
    }
    /* Group(Node a_expression); */

    /** Accessors **/

    // Get the expression inside the group
    public Node GetExpression() {
        return m_expression;
    }

    /** Methods **/

    // Accept a visitor and return the value it evaluates to
    @Override
    public Value Accept(NodeVisitor a_visitor) {
        return a_visitor.VisitGroup(this);
    }

    /* PRIVATE */

    /** Members **/

    private final Node m_expression;    // The expression inside the group (e.g. (1 + 2) -> 1 + 2)
}

```

Identifier.java

```

///
/// Identifier Node - Represents a variable/function. Is created from the parser when a variable is encountered, e.g.
/// 'x' or 'y'. Responsible for holding the name of the variable/function and the line number where it is accessed.
///

package AbstractSyntaxTree.Expressions;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

public class Identifier extends Node {
    /* PUBLIC */

    /** Constructors **/

    /**/
    /*
    AbstractSyntaxTree.Expressions.Identifier.Identifier()

```

NAME

Identifier - constructor for the identifier node.

SYNOPSIS

```
Identifier(String a_variableName, int a_lineNumber);  
    a_variableName -> the name of the variable.  
    a_lineNumber   -> the line number the variable was accessed on.
```

DESCRIPTION

Initializes the name of the variable and the line number where the variable was accessed. Specifically used for creating an identifier node in the Abstract Syntax Tree. For example 'x' would be an identifier.

RETURNS

Nothing - it's a constructor.

```
*/  
/**/  
public Identifier(String a_variableName, int a_lineNumber) {  
    m_variableName = a_variableName;  
    m_lineNumber = a_lineNumber;  
}  
/* Identifier(String a_variableName, int a_lineNumber); */  
  
/** Accessors **/  
  
// Get the name of the variable  
public String GetVariableName() {  
    return m_variableName;  
}  
  
// Get the line number where the variable was used  
public int GetLineNumber() {  
    return m_lineNumber;  
}  
  
/** Methods **/  
  
// Accept a visitor and visit the identifier node  
@Override  
public Value Accept(NodeVisitor a_visitor) {  
    return a_visitor.VisitIdentifier(this);  
}  
  
/* PRIVATE */  
  
/** Members **/
```

```

private final String m_variableName;    // Name of the variable/identifier
private final int m_lineNumber;        // The line number where the variable is accessed
}

```

Index.java

```

///
/// Index Node - Represents an index access of a variable, list or string. Is created from '<identifier>:<index>' which
/// is a normal index access, or '<identifier>:<start-index>|<end-index>' which is a ranged access. Responsible for
/// holding the atom being accessed, the index being accessed and whether it is a ranged access. Normal index gets
/// the element from the string/list (or accessee), while ranged access is getting a range of elements from the
/// accessee.
///

package AbstractSyntaxTree.Expressions;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Lexer.Token;

public class Index extends Node {
    /* PUBLIC */

    /** Constructors */

    // Not using polymorphism because the two constructors are different enough that it would be confusing to have
    // multiple inherited classes for a simple data store. If it were to get more complex, then it would make sense
    // to do this, but small classes like this are better off with simple constructors.

    /**/
    /*
    AbstractSyntaxTree.Expressions.Index.Index()

    NAME

        Index - constructor for the index node (normal index access).

    SYNOPSIS

        Index(Node a_accessee, Token a_token, Node a_index);
        a_accessee -> the atom being accessed, e.g. the variable name,
                        list or string.
        a_token     -> the ':' operator token (used for reporting errors).
        a_index     -> the subscript value of where to access, e.g. the
                        index in a list.

    DESCRIPTION

        Initializes the accessee, token, and index of the index access. Used for
        creating index nodes with no ranged access. For example 'list:1' would

```

be an index access with 'list' as the accessee, ':' as the operator token, and '1' as the index.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Index(Node a_accessee, Token a_token, Node a_index) {
    m_accessee = a_accessee;
    m_token = a_token;

    m_index = a_index;

    // Make start and end indices same as index, so they are not null
    m_isRanged = false;
    m_startIndex = a_index;
    m_endIndex = a_index;
}
/* Index(Node a_accessee, Token a_token, Node a_index); */

/**/
/*
AbstractSyntaxTree.Expressions.Index.Index()
```

NAME

Index - constructor for the index node (ranged index access).

SYNOPSIS

```
Index(Node a_accessee, Token a_token, Node a_startIndex, Node a_endIndex);
    a_accessee      -> the atom being accessed, e.g. the variable name,
                      list or string.
    a_token         -> the ';' operator token (used for reporting
                      errors).
    a_startIndex    -> the starting index of where to access from.
    a_endIndex      -> the ending index of where to access to.
```

DESCRIPTION

Initializes the accessee, token, start index, and end index of the index access. Used for creating index nodes with ranged access. For example 'list:1|3' would be an index access with 'list' as the accessee, ':' as the operator token, '1' as the start index, and '3' as the end index.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
```

```

public Index(Node a_accessee, Token a_token, Node a_startIndex, Node a_endIndex) {
    m_accessee = a_accessee;
    m_token = a_token;

    m_isRanged = true;
    m_startIndex = a_startIndex;
    m_endIndex = a_endIndex;

    // Make index same as start index, so it is not null
    m_index = a_startIndex;
}
/* Index(Node a_accessee, Token a_token, Node a_startIndex,
    Node a_endIndex); */

/** Accessors */

// Get the atom being accessed, e.g. the variable name, list or string
public Node GetAccessee() {
    return m_accessee;
}

// Get the operator token of the index access
public Token GetToken() {
    return m_token;
}

// Check if the index access is ranged
public boolean IsRanged() {
    return m_isRanged;
}

// Get the index value of where to access, if not ranged
public Node GetIndex() {
    ConfirmIsNotRanged();
    return m_index;
}

// Get the start index of where to access, if ranged
public Node GetStartIndex() {
    ConfirmIsRanged();
    return m_startIndex;
}

// Get the end index of where to access, if ranged
public Node GetEndIndex() {
    ConfirmIsRanged();
    return m_endIndex;
}

/** Methods */

```

```

// Accept a visitor and have it visit the index expression
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitIndex(this);
}

/* PRIVATE */

/** Members */

private final Node m_accessee;    // The atom being accessed, e.g. the variable name, list or string
private final Token m_token;      // The ':' or '|' operator token (used for reporting errors)

private final Node m_index;       // The subscript value of where to access, e.g. the index in a list

private final Node m_startIndex;  // For ranged access, the starting index of where to access from
private final Node m_endIndex;    // For ranged access, the ending index of where to access to
private final boolean m_isRanged; // Whether we are accessing a range of values

/** Private Helper Function */

/*
 *      Guard Clause - Check if the index is ranged
 */

// Make sure to not improperly access the index
private void ConfirmIsRanged() {
    assert m_isRanged : "Accessor is not ranged, use GetIndex() instead";
}

// Make sure you use start and end indices for ranged access
private void ConfirmIsNotRanged() {
    assert !m_isRanged : "Accessor is ranged, use GetStartIndex() and GetEndIndex() instead";
}
}

```

ListExpression.java

```

///
/// List Expression Node - Represents a list of expressions, e.g. [1 + 2, "hello", (3 * 4)], meaning it is the
///     unevaluated list before transformation to a list Value. Is created from '<expression>, <expression>, ...]'
///     in the parser.
///
package AbstractSyntaxTree.Expressions;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

```

```

import java.util.Collections;
import java.util.List;

public class ListExpression extends Node {
    /* PUBLIC */

    /** Constructors */

    /**/
    /*
AbstractSyntaxTree.Expressions.ListExpression.ListExpression()

NAME

    ListExpression - constructor for the list expression node.

SYNOPSIS

    ListExpression(List<Node> a_expression);
        a_expression    -> the list of expressions inside the list.

DESCRIPTION

    Initializes the list of expressions inside the list. It serves as a way
    to store unevaluated lists, For example , '[1 + 2, "hello", (3 * 4)]'
    would be a list expression with '1 + 2', '"hello"', and '3 * 4' as the
    expressions inside the list, and would eventually be evaluated to a
    list Value of [3, "hello", 12].

RETURNS

    Nothing - it's a constructor.

    */
    /**/
    public ListExpression(List<Node> a_expression) {
        // Make sure the list can't be added to or modified later on (for encapsulation)
        m_expressions = Collections.unmodifiableList(a_expression);
    }
    /* ListExpression(List<Node> a_expression); */

    /** Accessors */

    // Get the list of expressions inside the list
    public List<Node> GetExpressions() {
        return m_expressions;
    }

    /** Methods */

```



```

// Accept a visitor and evaluate all the expressions in the list
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitListExpression(this);
}

/* PRIVATE */

/** Members */

private final List<Node> m_expressions; // The list of expressions inside the list, e.g. [1 + 2, "hello", (3 * 4)]
}

```

Literal.java

```

///
/// Literal Node - Represents a literal value (e.g. 1, true, "hello"). Is created from the parser when a literal is
/// encountered (integer, float, string, boolean or null).
///

```

```

package AbstractSyntaxTree.Expressions;

```

```

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

```

```

public class Literal extends Node {
    /* PUBLIC */

    /** Constructors */

    /**/
    /*
    AbstractSyntaxTree.Expressions.Literal.Literal()

    NAME

```

```

    Literal - constructor for the literal node.

```

SYNOPSIS

```

    Literal(Value a_value);
    a_value    -> the value of the literal.

```

DESCRIPTION

```

    Initializes the value of the literal. Specifically used for creating a
    literal node in the Abstract Syntax Tree. For example '1' would be a
    literal with the Value Object of 1.

```

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Literal(Value a_value) {
    m_value = a_value;
}
/* Literal(Value a_value); */

/** Accessors **/

// Get the value of the literal
public Value GetValue() {
    return m_value;
}

/** Methods **/

// Accept a visitor and evaluate the literal (just returns the value)
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitLiteral(this);
}

/* PRIVATE */

/** Members **/

private final Value m_value;    // The value of the literal (e.g. and integer, boolean, etc.)
}
```

UnaryOperator.java

```
///
/// Unary Operator Node - Represents a unary operation, e.g. -1 or NOT TRUE in the Abstract Syntax Tree. Is created by
///      '<operator> <expression>' and is responsible for holding the expression and the operator in token form.
///
package AbstractSyntaxTree.Expressions;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Lexer.Token;

public class UnaryOperator extends Node {
```

```

/* PUBLIC */

/** Constructors */

/**/
/*
AbstractSyntaxTree.Expressions.UnaryOperator.UnaryOperator()

NAME

    UnaryOperator - constructor for the unary operator node.

SYNOPSIS

    UnaryOperator(Token a_token, Node a_expression);
        a_token          -> the operator token (e.g. '-', 'NOT').
        a_expression     -> the expression the operator is applied to.

DESCRIPTION

    Initializes the operator token and the expression the operator is
    applied to. Used for creating a unary operator node in the Abstract
    Syntax Tree. For example '-1' would be a unary operation with '-' as the
    operator token and '1' as the expression the operator is applied to.

RETURNS

    Nothing - it's a constructor.

*/
/**/
public UnaryOperator(Token a_token, Node a_expression) {
    m_token = a_token;
    m_expression = a_expression;
}
/* UnaryOperator(Token a_token, Node a_expression); */

/** Accessors */

// Get the operator token
public Token GetToken() {
    return m_token;
}

// Get the expression the operator is applied to
public Node GetExpression() {
    return m_expression;
}

/** Methods */

```

```

// Accept a visitor and evaluate the unary operation
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitUnaryOperator(this);
}

/* PRIVATE */

/** Members */

private final Token m_token;        // The operator token (e.g. '-', '!')
private final Node m_expression;    // The expression the operator is applied to
}

```

Statements Package

Body.java

```

///
/// Body Node - Represents a block of statements attached to a statement, e.g. the statements inside an if-else chain.
///      No exact representation for parser, but is created after a loop, if-elseif-else chain or a function.
///      Responsible for holding a list of statements.
///

package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

import java.util.List;

public class Body extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Statements.Body.Body()
     *
     * NAME
     *
     *      Body - constructor for the body node.
     *
     * SYNOPSIS
     *
     *      Body(List<Node> a_statements);
     */
}

```

a_statements -> the list of statements in the body.

DESCRIPTION

Holds a list of statements to get executed. Specifically used for if-elseif-else chains, loops, and functions in the Abstract Syntax Tree. Important for grouping multiple statements together.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Body(List<Node> a_statements) {
    m_statements = a_statements;
}
/* Body(List<Node> a_statements); */

/** Accessors **/

// Get all the statements in the body
public List<Node> GetStatements() {
    return m_statements;
}

/** Methods **/

// Accept a visitor and visit/execute all the statements in the body
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitBody(this);
}

/* PRIVATE */

/** Members **/

private final List<Node> m_statements;    // The statements in the body
}
```

Call.java

```
///
/// Call Node - Represents the call of a function in the Abstract Syntax Tree. Is created by '<function>(<arguments>)'
/// and is responsible for holding the function to be called, the arguments being passed to the function, and the
/// line the call occurs on. When evaluated it will create a new scope for the function to exist in and initialize
/// the arguments passed to the function based on the Functions parameter names.
///
```

```

package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

import java.util.List;

public class Call extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Statements.Call.Call()
     *
     * NAME
     *
     * Call - constructor for the call node.
     *
     * SYNOPSIS
     *
     * Call(Node a_callee, List<Node> a_arguments, int a_lineNumber);
     *     a_callee      -> the function to be called.
     *     a_arguments    -> the arguments being passed to the function.
     *     a_lineNumber   -> the line number where the function is called.
     *
     * DESCRIPTION
     *
     * Initializes the function to be called, the arguments being passed to the
     * function, and the line number where the function is called. Specifically
     * used for creating a call node in the Abstract Syntax Tree. For example
     * 'print("hello")' would be a call with 'print' as the function to be
     * called and '"hello"' as the argument being passed to the function.
     *
     * RETURNS
     *
     * Nothing - it's a constructor.
     *
     */
    /**
     *
     */
    public Call(Node a_callee, List<Node> a_arguments, int a_lineNumber) {
        m_callee = a_callee;
        m_arguments = a_arguments;
        m_lineNumber = a_lineNumber;
    }
    /* Call(Node a_callee, List<Node> a_arguments, int a_lineNumber); */

    /** Accessors */

```

```

// Returns the function to be called
public Node GetCallee() {
    return m_callee;
}

// Returns the arguments being passed to the function
public List<Node> GetArguments() {
    return m_arguments;
}

// Returns the line number where the function is called
public int GetLineNumber() {
    return m_lineNumber;
}

/** Methods */

// Accept a visitor and evaluate the call
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitCall(this);
}

/* PRIVATE */

/** Members */

private final Node m_callee;           // The function to be called (must be identifier when evaluated)
private final List<Node> m_arguments;   // The arguments being passed to the function
private final int m_lineNumber;         // The line number where the function is called
}

```

ConditionalField.java

```

///
/// Conditional Field - THIS IS NOT A NODE, rather a way to reuse code for the IfElseifElse and While loop. Both of
/// these nodes have a condition and a body, so this class is used to store that information. This class is not to
/// be evaluated by the interpreter, but rather act similar to a struct in C++ (Java doesn't have structs).
///

package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Node;

public class ConditionalField {
    /* PUBLIC */

    /** Constructors */
}

```

```
/**/
/*
AbstractSyntaxTree.Statements.ConditionalField.ConditionalField()
```

NAME

ConditionalField - constructor for the conditional field.

SYNOPSIS

```
ConditionalField(Node a_condition, Body a_body, int a_lineNumber);
    a_condition    -> the condition for the statement (if or while).
    a_body         -> the body of the if statement.
    a_lineNumber   -> the line number the if statement was found on.
```

DESCRIPTION

Initializes the condition for the statement (if or while), the body of the if statement, and the line number the if statement was found on. It acts more a struct for the similarities found in condition like statements. For example 'IF x == 5 THEN; PRINT("HELLO")' would be a conditional field with 'x == 5' as the condition, 'PRINT("HELLO")' as the body, and the line number where the if statement was found.

Helps reduce code duplication between the IfElseifElse and While nodes.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public ConditionalField(Node a_condition, Body a_body, int a_lineNumber) {
    m_condition = a_condition;
    m_body = a_body;
    m_lineNumber = a_lineNumber;
}
/* ConditionalField(Node a_condition, Body a_body, int a_lineNumber); */

/** Accessors **/

// Get the condition for the statement
public Node GetCondition() {
    return m_condition;
}

// Gets the body of the conditional statement
public Node GetBody() {
    return m_body;
}
```



```

// Gets the line number where the statement was found
public int GetLineNumber() {
    return m_lineNumber;
}

/* PRIVATE */

/** Members */

private final Node m_condition;           // The condition for the if statement
private final Node m_body;               // The body of the if statement
private final int m_lineNumber;          // The line number the if statement was found on
}

```

For.java

```

///
/// For Loop Node - Represents a for loop statement. Is created from the parser when a for loop statement is
/// found 'FOR <assignment> TO <endValue> STEP <increment> THEN <body> END'. Responsible for holding the assignment
/// variable for the start, the end value to get to, the increment/step value, and the body of the loop.
/// Responsible for executing a block of code a certain number of times.
///

```

```
package AbstractSyntaxTree.Statements;
```

```

import AbstractSyntaxTree.Expressions.Assignment;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

```

```

public class For extends Node {
    /* PUBLIC */

```

```
    /** Constructors */
```

```
    /**/
```

```
    /*
```

```
    AbstractSyntaxTree.Statements.For.For()
```

```
    NAME
```

```
        For - constructor for the for loop node.
```

```
    SYNOPSIS
```

```

    For(Assignment a_assignment, Node a_endValue, Node a_increment,
        Body a_body, int a_lineNumber);
    a_assignment    -> the assignment for the loop, e.g. 'LET i = 0'.
    a_endValue       -> the end value to get to, e.g. '10'.
    a_increment      -> what to increment the variable by, e.g. '1'.

```

a_body -> loop body.
a_lineNumber -> the line number the for loop was found on.

DESCRIPTION

Initializes the assignment, end value, increment, and body of the for loop. Specifically used for creating a for loop node in the Abstract Syntax Tree. For example 'FOR LET i = 0 TO 10 STEP 1 THEN ... END' would be a for loop with the assignment 'LET i = 0', end value '10', increment '1', and the body of the loop.

RETURNS

Nothing - it's a constructor.

```
*/  
/**/  
public For (Assignment a_assignment, Node a_endValue, Node a_increment, Body a_body, int a_lineNumber) {  
    m_assignment = a_assignment;  
    m_endValue = a_endValue;  
    m_increment = a_increment;  
    m_body = a_body;  
    m_lineNumber = a_lineNumber;  
}  
/* For(Assignment a_assignment, Node a_endValue, Node a_increment,  
    Body a_body, int a_lineNumber); */  
  
/** Accessors **/  
  
// Get the assignment for the for loop (e.g. 'LET i = 0')  
public Assignment GetAssignment() {  
    return m_assignment;  
}  
  
// Get the end value to get to (e.g. '10')  
public Node GetEndValue() {  
    return m_endValue;  
}  
  
// Get what to increment the variable by (e.g. '1')  
public Node GetIncrement() {  
    return m_increment;  
}  
  
// Get the body of the loop  
public Body GetBody() {  
    return m_body;  
}  
  
// Get the line number the for loop was found on  
public int GetLineNumber() {  
    return m_lineNumber;  
}
```

```

}

/** Methods */

// Accept a visitor and evaluate the for loop
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitFor(this);
}

/* PRIVATE */

/** Members */

private final Assignment m_assignment;    // The assignment for the for loop, e.g. 'LET i = 0'
private final Node m_endValue;           // The end value to get to, i.e. '10'
private final Node m_increment;          // What to increment the variable by, e.g. '1'
private final Body m_body;               // Loop body to execute
private final int m_lineNumber;          // The line number the for loop was found on
}

```

Function.java

```

///
/// Function Node - Represents a function, both as within the Abstract Syntax Tree and as a Value. It is created from
///     'DEF <identifier>(<parameters>) <body>' in the parser. Responsible for holding the function name, parameters,
///     parameter count, whether it is a built-in function, the body of the function, and the builtin function to call.
///     It is pivotal to the language as it allows for user-defined functions and built-in functions, and allows code
///     to be reused and organized. Each function is expected to return a Value, and if it doesn't, it will return our
///     NULL Value.
///
package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Expressions.Identifier;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Language.BuiltInFunctions.BuiltIn;

import java.util.List;

public class Function extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     */
}

```

AbstractSyntaxTree.Statements.Function.Function()

NAME

Function - constructor for a user-defined function.

SYNOPSIS

```
Function(String a_functionName, List<Identifier> a_parameters,
        Body a_body);
    a_functionName -> the name of the function.
    a_parameters   -> the parameters (identifiers) of the function.
    a_body         -> the body of the function.
```

DESCRIPTION

Initializes the name of the function, the parameters of the function, and the body of the function. For example 'DEF add(x, y) x + y' would be a function with 'add' as the name, 'x' and 'y' as the parameters, and 'x + y' as the body of the function.

This is for user definitions, so we need to store the names that the user has chosen for the parameters, as well as the body of the function the user created.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Function(String a_functionName, List<Identifier> a_parameters, Body a_body) {
    m_functionName = a_functionName;
    m_parameters = a_parameters;
    m_parameterCount = a_parameters.size();

    // User-defined functions don't have a built-in function to call
    m_isBuiltIn = false;
    m_body = a_body;
    m_function = null;
}
/* Function(String a_functionName, List<Identifier> a_parameters,
    Body a_body); */

/**/
/*
AbstractSyntaxTree.Statements.Function.Function()
```

NAME

Function - constructor for a built-in function.

SYNOPSIS

```

Function(String a_functionName, int a_parameterCount,
        BuiltIn a_function);
    a_functionName -> the name of the function.
    a_parameterCount-> the number of parameters the function takes.
    a_function      -> the built-in function to call. Is an interface
        from the BuiltInFunctions class that uses Java code for its
        implementation.

```

DESCRIPTION

Initializes the name of the function, the number of parameters the function takes, and the built-in function to call. For example PRINT() is a function with 1 parameter, and calls BuiltInFunctions.PRINT() to execute.

This is for built-in functions, so we don't need to store the names of the parameters or the body of the function, just the number of parameters and the built-in function to call so set the body and parameters to null and set the isBuiltIn flag to true.

```

*/
/**/
public Function(String a_functionName, int a_parameterCount, BuiltIn a_function) {
    m_functionName = a_functionName;
    m_parameterCount = a_parameterCount;
    m_function = a_function;

    // Built-in functions don't have parameter names or a body
    m_isBuiltIn = true;
    m_body = null;
    m_parameters = null;
}
/* Function(String a_functionName, int a_parameterCount,
    BuiltIn a_function); */

/** Accessors **/

// Returns the name of the function
public String GetFunctionName() {
    return m_functionName;
}

// Gets the parameters of the function if it is a user-defined function
public List<Identifier> GetParameters() {
    ConfirmUserDefined();
    return m_parameters;
}

// Gets the number of parameters the function takes (or arity)
public int GetParameterCount() {
    return m_parameterCount;
}

```

```

}

// Returns whether the function is a built-in function or not
public boolean IsBuiltIn() {
    return m_isBuiltIn;
}

// Gets the built-in function to call if is indeed a built-in function
public BuiltIn GetBuiltInFunction() {
    ConfirmBuiltIn();
    return m_function;
}

// Gets the body of the function if it is a user-defined function
public Body GetBody() {
    ConfirmUserDefined();
    return m_body;
}

/** Methods */

// Accept a visitor and evaluate the function, will need to check the flag to see if it is a built-in function
// before running
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitFunction(this);
}

/* PRIVATE */

/** Members */

private final String m_functionName;           // The name of the function
private final List<Identifier> m_parameters;   // The parameters of the function (all identifiers)
private final int m_parameterCount;           // Number of parameters the function takes
private final boolean m_isBuiltIn;             // Whether the function is a built-in function
private final Body m_body;                     // The body of the function
private final BuiltIn m_function;              // The built-in function (its 'body')

/** Private Helper Functions */

/*
 *      Correct Function Value - So developers don't do something they shouldn't
 */

// Used if the function is user-defined and doesn't have a built-in function to access
public void ConfirmUserDefined() {
    assert !m_isBuiltIn : "Function '" + m_functionName + "' +
        "is a built-in function, no parameters names or body to access.";
}

```

```

// Used if the function is built-in and doesn't have parameters or a body to access
public void ConfirmBuiltIn() {
    assert m_isBuiltIn : "Function '" + m_functionName + "'" +
        "is a user-defined function, no built-in function to access.";
}
}

```

IfElseifElse.java

```

///
/// If-Elseif-Else Node - Represents an if-elseif-else chain. Is created from the parser from
/// 'IF <condition> THEN <body> ELSEIF <condition> THEN <body> ELSE <body> END'. Responsible for holding the
/// conditions and bodies of the if-elseif-else chain, and will evaluate first true condition's body, or the else.
/// An else is not required, nor are the else-ifs, but there must be at least one if on creation.
///

```

```

package AbstractSyntaxTree.Statements;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

```

```

import java.util.List;

```

```

public class IfElseifElse extends Node {
    /* PUBLIC */

```

```

    /** Constructors */

```

```

    /**/

```

```

    /*

```

```

    AbstractSyntaxTree.Statements.IfElseifElse.IfElseifElse()

```

```

    NAME

```

```

        IfElseifElse - constructor for the if-elseif-else node.

```

```

    SYNOPSIS

```

```

        IfElseifElse(List<ConditionalField> a_conditions);
            a_conditions    -> the conditions and bodies for the ifs and
                               else-ifs. Must contain at least one condition.

```

```

    DESCRIPTION

```

```

        Initializes the conditions and bodies for the if-elseif-else chain.
        This constructor is used when there is no else body, setting the
        m_hasElse flag to false and the body to null. The a_conditions list
        will be 1 if there is an if, or more than one if there are else-ifs. If
        the conditions list is empty, an assertion will be thrown.

```

Easier to create another constructor for the else body than to create a separate Node for the else body as they are not two separate entities. Rather a different state of the same entity.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public IfElseifElse(List<ConditionalField> a_conditions) {
    ConfirmValidField(a_conditions);
    m_conditions = a_conditions;

    // No else, so set the else body to null
    m_hasElse = false;
    m_elseBody = null;
}
/* IfElseifElse(List<ConditionalField> a_conditions); */

/**/
/*
AbstractSyntaxTree.Statements.IfElseifElse.IfElseifElse()
```

NAME

IfElseifElse - constructor for the if-elseif-else node.

SYNOPSIS

```
IfElseifElse(List<ConditionalField> a_conditions, Body a_elseBody);
    a_conditions    -> the conditions and bodies for the ifs and
                        else-ifs. Must contain at least one condition.
    a_elseBody      -> the body for the else.
```

DESCRIPTION

Initializes the conditions and bodies for the if-elseif-else chain. This contractor is used when a else is present on the chain, setting the m_hasElse flag to true and the body to a_elseBody.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public IfElseifElse(List<ConditionalField> a_conditions, Body a_elseBody) {
    ConfirmValidField(a_conditions);
    m_conditions = a_conditions;

    // There is an else, so set the else body
    m_hasElse = true;
```



```

        m_elseBody = a_elseBody;
    }
    /* IfElseifElse(List<ConditionalField> a_conditions, Body a_elseBody) */

    /** Accessors **/

    // Returns the conditions and bodies for the ifs and else-ifs
    public List<ConditionalField> GetConditions() {
        return m_conditions;
    }

    // Returns the body for the else
    public Body GetElseBody() {
        ConfirmHasElse();
        return m_elseBody;
    }

    // Returns whether there is an else statement attached to the chain
    public boolean HasElse() {
        return m_hasElse;
    }

    /** Methods **/

    // Accept a visitor and evaluate the if-elseif-else chain
    @Override
    public Value Accept(NodeVisitor a_visitor) {
        return a_visitor.VisitIfElseifElse(this);
    }

    /* PRIVATE */

    /** Members **/

    private final List<ConditionalField> m_conditions;    // The conditions and bodies for the ifs and else-ifs
    private final boolean m_hasElse;                    // Whether there is an else
    private final Body m_elseBody;                      // The body for the else (if there is one)

    /** Private Helper Functions **/

    /*
     *      Confirm Conditional State - So developers don't do something they shouldn't
     */

    // Confirm that we have at the very least an if condition
    private void ConfirmValidField(List<ConditionalField> a_conditions) {
        assert !a_conditions.isEmpty() : "Conditional must have at least one condition";
    }

```

```

    // Confirm that there is an else body before trying to access it
    private void ConfirmHasElse() {
        assert HasElse() : "No else body exists for this conditional";
    }
}

```

Jump.java

```

///
/// Jump Node - Represents a jump out of a specific body. Responsible for holding the type of jump (break, continue,
///     return) and allowing another node to catch it and deal with it where it needs to. It uses exception control
///     flow to break out of the current body and go back to where the jump was used. If it is a return, it will also
///     give the value to catch block.
///

package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Expressions.Literal;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;
import Lexer.OperationCode;
import Lexer.Token;

public class Jump extends Node {
    /* PUBLIC */

    // Custom exceptions for breaking, continuing, and returning. As we nest deep into the interpreter with recursive
    // calls, we need a way to break out of the current body and go back to where the break, and exceptions do
    // that perfectly. Having a simple try catch at a point like inside a loop, allows us to avoid having to pass
    // a boolean up the chain to break out of the loop. Java does all the work for us.
    // Since these are simple classes that only hold one value, there is no need for them to have their own file and
    // can be nested inside the Jump class for better organization. No method headers are needed, as they simply
    // hold the line number it was used on or hold the value to return. Breaks are just line numbers, while returns
    // contain the value to return.
    public static class BreakException extends RuntimeException {
        public BreakException(int a_lineNumber) { m_lineNumber = a_lineNumber; }
        public int GetLineNumber() { return m_lineNumber; }
        private final int m_lineNumber;
    }
    public static class ContinueException extends RuntimeException {
        public ContinueException(int a_lineNumber) { m_lineNumber = a_lineNumber; }
        public int GetLineNumber() { return m_lineNumber; }
        private final int m_lineNumber;
    }
    public static class ReturnException extends RuntimeException {
        public ReturnException(Value a_value) { m_value = a_value; }
        public Value GetValue() { return m_value; }
        private final Value m_value;
    }
}

```

```
/** Constructors **/
```

```
/**/
```

```
/*
```

```
AbstractSyntaxTree.Statements.Jump.Jump()
```

NAME

Jump - constructor a jump without an expression

SYNOPSIS

```
Jump(Token a_jumpType);  
a_jumpType    -> the type of jump.
```

DESCRIPTION

Initializes the type of jump and the expression to our NULL Value.
This constructor is used when there is no expression required for the
jump, like an empty return, a break or continue.

RETURNS

Nothing - it's a constructor.

```
*/
```

```
/**/
```

```
public Jump(Token a_jumpType) {  
    m_jumpType = a_jumpType;  
    // Default to a literal with no value if no expression is provided  
    m_expression = new Literal(Value.NULL);  
}
```

```
/* Jump(Token a_jumpType); */
```

```
/**/
```

```
/*
```

```
AbstractSyntaxTree.Statements.Jump.Jump()
```

NAME

Jump - constructor for a jump with an expression. For return.

SYNOPSIS

```
Jump(Token a_jumpType, Node a_expression);  
a_jumpType    -> the type of jump.  
a_expression  -> the expression to return.
```

DESCRIPTION

Initializes the type of jump and the expression to return. This
constructor is used when there is an expression required for the jump,

like a return.

RETURNS

Nothing - it's a constructor.

```
*/
/**/
public Jump(Token a_jumpType, Node a_expression) {
    m_jumpType = a_jumpType;
    m_expression = a_expression;
}
/* Jump(Token a_jumpType, Node a_expression); */

/** Accessors **/

// Get the type of jump (break, continue, return)
public OperationCode GetJumpType() {
    return m_jumpType.GetOpcode();
}

// Get the line number the jump occurs on
public int GetLineNumber() {
    return m_jumpType.GetLineNumber();
}

// Get the expression to return if it is return, otherwise it'll be our NULL wrapped in a Literal node
public Node GetExpression() {
    return m_expression;
}

/** Methods **/

// Accept a visitor and throw the appropriate exception
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitJump(this);
}

/* PRIVATE */

/** Members **/

private final Token m_jumpType;    // The type of jump (break, continue, return)
private final Node m_expression;    // The expression to return (if return) or our NULL if no expression
}
```

While.java

```

///
/// While Loop Node - Represents a while loop. Is created from the parser when a while loop is encountered,
/// 'WHILE <condition> THEN <body> END'. Responsible for holding the condition to check for, and the body of the
/// loop. Evaluates the condition and executes the body of the loop while the condition is true within Interpreter.
///

package AbstractSyntaxTree.Statements;

import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.NodeVisitor;
import Interpreter.Value;

public class While extends Node {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * AbstractSyntaxTree.Statements.While.While()
     *
     * NAME
     *
     * While - constructor for the while loop node.
     *
     * SYNOPSIS
     *
     * While(ConditionalField a_field);
     *     a_field    -> the field that stores the condition, body and line
     *                   number of the while loop.
     *
     * DESCRIPTION
     *
     * Initializes the condition to check for and the body of the loop. For
     * example 'WHILE x < 5 THEN BREAK END' would be a while loop with the
     * condition 'x < 5' and 'BREAK' as the body.
     *
     * RETURNS
     *
     * Nothing - it's a constructor.
     *
     */
    /**/
    public While(ConditionalField a_field) {
        m_condition = a_field.GetCondition();
        m_body = a_field.GetBody();
        m_lineNumber = a_field.GetLineNumber();
    }
    /* While(ConditionalField a_field); */

    /** Accessors */

```

```

// Get the condition to check for (e.g. x < 5)
public Node GetCondition() {
    return m_condition;
}

// Get the body of the loop
public Node GetBody() {
    return m_body;
}

// Get the line number where WHILE was found
public int GetLineNumber() {
    return m_lineNumber;
}

/** Methods */

// Accept a visitor and attempt to evaluate the while loop
@Override
public Value Accept(NodeVisitor a_visitor) {
    return a_visitor.VisitWhile(this);
}

/* PRIVATE */

/** Members */

private final Node m_condition;    // The condition to check for
private final Node m_body;         // The body of the loop
private final int m_lineNumber;    // The line number the loop started on
}

```

Interpreter Package

Interpreter.java

```

///
/// Interpreter - Responsible for interpreting the Abstract Syntax Tree made within the Parser. In essence, it performs
/// the calculations and operations that the user has written in the code, using the visitor pattern from the Node
/// Visitor interface to traverse the tree and perform the operations. Errors are a part of calculations, so they
/// are thrown as when they occur (e.g. incorrect types, division by zero, etc.).
///
package Interpreter;

import AbstractSyntaxTree.Statements.*;

```

```

import Interpreter.Value.DataType;
import AbstractSyntaxTree.NodeVisitor;
import AbstractSyntaxTree.Node;
import AbstractSyntaxTree.Expressions.*;
import AbstractSyntaxTree.Statements.For;
import AbstractSyntaxTree.Statements.While;
import Language.BuiltInFunctions;
import Lexer.OperationCode;
import Lexer.Token;
import Language.Error;
import Language.Error.ErrorType;

import ch.obermuhlner.math.big.BigDecimalMath;
import java.math.BigDecimal;
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.function.BiFunction;
import java.util.function.BiPredicate;
import java.util.function.IntPredicate;

```

```

public class Interpreter implements NodeVisitor {

```

```

    /* PUBLIC */

```

```

    /** Constructors */

```

```

    /**/

```

```

    /*

```

```

    Interpreter.Interpreter()

```

```

    NAME

```

```

        Interpreter - constructor for the interpreter.

```

```

    SYNOPSIS

```

```

        Interpreter(SymbolTable a_globalScope);
        a_globalScope    -> the global scope of the program.

```

```

    DESCRIPTION

```

```

        Initializes the current scope of the interpreter to the global scope, so
        if when we define a function, or declare a variable, we have somewhere
        to put it.

```

```

    RETURNS

```

```

        Nothing - it's a constructor.

```

```

    */

```

```

    /**/

```

```

public Interpreter(SymbolTable a_globalScope) {
    m_currentScope = a_globalScope;
}
/* Interpreter(SymbolTable a_globalScope); */

```

/** Methods **/

```

/**/
/*
Interpreter.Interpret()

```

NAME

Interpret - interprets the Abstract Syntax Tree (AST) generated by the parser, evaluating the nodes it comes across.

SYNOPSIS

```

public Value Interpret(List<Node> a_tree);
    a_tree    -> the list of nodes/statements that make up the AST.

```

DESCRIPTION

Interprets the AST by traversing the tree and interpreting the nodes it in order of the list. It will evaluate the nodes and return the result of the last evaluated node. This function is the driver for the actual evaluation of the code.

At any point in the traversal when we have a runtime error (e.g. incorrect types arithmetic, or division by zero), it will be thrown as an error immediately at the node it occurs. Done to inform the user. Break and continue exceptions are caught and reported as runtime errors here if not caught within parser - it is not a syntax error as they were defined in a loop, but not in the correct context at runtime.

RETURNS

The result of the last evaluated node. If it is a statement, like a while loop, it will return our version of NULL as everything must return a value in our language.

```

*/
/**/
public Value Interpret(List<Node> a_tree) {
    Value result = Value.NULL;

    // Traverse the tree and interpret the nodes
    for (Node node : a_tree) {
        try { result = node.Accept(this); }

        // Very specific problem when a function gets defined in a loop and then outside of it. Is a runtime error
        // as it's syntactically correct, but the function cannot jump out of a loop that isn't being executed.
    }
}

```



```

        catch (Jump.BreakException error) { ErrorOnJump("Break", error.GetLineNumber());}
        catch (Jump.ContinueException error) { ErrorOnJump("Continue", error.GetLineNumber());}
    }

    // Return the last evaluated result
    return result;
}
/* public Value Interpret(List<Node> a_tree); */

```

```

/*
 *      Visiting Statements - Changing state and performing actions
 */

```

```

/**/
/*
Interpreter.VisitFunction()

```

NAME

VisitFunction - binds a function definition to the current scope.

SYNOPSIS

```

public Value VisitFunction(Function a_function);
    a_function    -> the function node to visit.

```

DESCRIPTION

Binds the function to the current scope of evaluation, so it can be called later on in. Responsible for user defined functions, not built-ins. Simply put, we add the function name and the corresponding function to the current scope.

For example, 'DEF add(x, y); x + y END' would bind the function 'add' to the symbol table, storing the parameters and body of the function.

RETURNS

Our NULL Value - everything must evaluate to something in our language, but as a statement, we don't give random information.

```

*/
/**/
@Override
public Value VisitFunction(Function a_function) {
    // Add the function to the global scope
    m_currentScope.PutIdentifier(a_function.GetFunctionName(), new Value(a_function));

    // Don't want any value returned from a function definition - done to avoid output in CLI
    return Value.NULL;
}
/* public Value VisitFunction(Function a_function); */

```

```
/**/
/*
Interpreter.VisitIfElseifElse()
```

NAME

VisitIfElseifElse - visits an if-elseif-else conditional statement, executing the first body that evaluates to true.

SYNOPSIS

```
public Value VisitIfElseifElse(IfElseifElse a_conditional);
    a_conditional    -> the if-elseif-else conditional statement to
        visit.
```

DESCRIPTION

Evaluates the if-elseif-else conditional chain by checking each condition in order. As soon as one condition evaluates to true (which we find by visiting the expression), the body of that condition is executed and the result is returned. This expression must be a BOOLEAN Value, if it isn't, a type error is thrown.

Lastly, if no conditions are met, and there is an else body, it is executed. If no else, then no statement will be executed.

For example, 'IF x < 5 THEN 1 END' would return the Value of 1.

RETURNS

The last resulted Value from the executed body of the condition that was met, or the else body if no conditions were met. If no else body, then NULL Value gets returned - everything must evaluate to something.

```
*/
/**/
@Override
public Value VisitIfElseifElse(IfElseifElse a_conditional) {
    // Loop through all the conditions until we hit a true (if one). If size of list is one it is just if, otherwise
    // it is an elseif chain. Doesn't matter regardless as the first true will be executed.
    for (ConditionalField field : a_conditional.GetConditions()) {
        // Make sure the condition is a boolean, if not, type error
        Value condition = field.GetCondition().Accept(this);
        EnforceGenericType(DataType.BOOLEAN, condition, field.GetLineNumber());

        // If the condition is true, execute the body and return the result
        if (condition.AsBoolean()) { return field.GetBody().Accept(this); }
    }

    // No conditions were met, if there is an else body, execute it
    if (a_conditional.HasElse()) { return a_conditional.GetElseBody().Accept(this); }
```

```

    // No conditions were met and no else body, return our NULL value
    return Value.NULL;
}
/* public Value VisitIfElseifElse(IfElseifElse a_conditional); */

```

```

/**/
/*
Interpreter.VisitWhile()

```

NAME

VisitWhile - visits and evaluates a while loop.

SYNOPSIS

```

public Value VisitWhile(While a_while);
    a_while    -> the while loop to visit and evaluate.

```

DESCRIPTION

Evaluates the condition of the while loop, and if it is true, executes the body of the loop. This process is repeated until the condition is no longer true. If a break is encountered (through a Jump BreakException), the loop is stopped, and if a continue statement is encountered (through a Jump ContinueException), the current iteration gets skipped. The condition must always be a BOOLEAN Value, if it isn't, a type error gets thrown.

For example, 'WHILE x < 5 THEN PRINT(x); x = x + 1 END' would print: '01234'.

RETURNS

Our NULL Value - everything must evaluate to something in our language, but as a statement it doesn't have any meaningful information to return.

```

*/
/**/
@Override
public Value VisitWhile(While a_while) {
    // Evaluate the condition and ensure it is a boolean
    Value condition = a_while.GetCondition().Accept(this);
    EnforceGenericType(DataType.BOOLEAN, condition, a_while.GetLineNumber());

    while (condition.AsBoolean()) {
        // Execute the body of the while loop
        try { a_while.GetBody().Accept(this); }
        // If a break is encountered, stop the loop
        catch (Jump.BreakException e) { break; }
        // Just want to stop current iteration (will evaluate condition again after)
        catch (Jump.ContinueException ignored) { }
    }
}

```

```

        // Update the condition for the next iteration, ensure it is still a boolean
        condition = a_while.GetCondition().Accept(this);
        EnforceGenericType(DataType.BOOLEAN, condition, a_while.GetLineNumber());
    }

    // Don't want any value returned from a while loop - done to avoid output in CLI
    return Value.NULL;
}
/* public Value VisitWhile(While a_while); */

/**/
/*
Interpreter.VisitFor()

```

NAME

VisitFor - visits and evaluates a for loop.

SYNOPSIS

```

public Value VisitFor(For a_for);
    a_for    -> the for loop to visit and evaluate.

```

DESCRIPTION

Evaluates the assignment, end value and increment of the for loop. These Values must all be of type INTEGER, and if they aren't a type error is thrown. The loop will then iterate from the assignment to the end value (inclusive) by the increment. Steps/Increments can be negative, but it can result in an infinite loop. In order to facilitate this, we calculate the distance between the start and end values and iterate that many times.

Similar to the while loop, if a break is encountered (through a Jump BreakException), the loop is stopped, and if a continue statement is encountered (through a Jump ContinueException), the current iteration gets skipped.

For example, 'FOR i = 1 TO 5 THEN PRINT(i) END' would print '12345'. And 'FOR i = 5 TO 1 STEP -1 THEN PRINT(i) END' would print '54321'.

RETURNS

Our NULL Value - everything must evaluate to something in our language, but as a statement it doesn't have any meaningful information to return.

```

*/
/**/
@Override
public Value VisitFor(For a_for) {
    // Get the assignment, end value and increment for the for loop
    Value assignment = a_for.GetAssignment().Accept(this);
    Value endValue = a_for.GetEndValue().Accept(this);

```

```

Value increment = a_for.GetIncrement().Accept(this);

// Ensure they are all integers. Could allow floats, but I find it is more confusing. For loops are generally
//      used to iterate a certain whole number of times, and it muddies the water to say 1.5 to 5.5.
EnforceForLoopTypes(assignment, endValue, increment, a_for.GetLineNumber());

// Convert all to integers here - easier to work with than calling AsInteger() every time
String variableName = a_for.GetAssignment().GetVariableName();
BigInteger index = assignment.AsInteger();
BigInteger step = increment.AsInteger();
// The loopIterations is just the distance between the start and end values, regardless of direction
BigInteger loopIterations = index.subtract(endValue.AsInteger()).abs();

// Loop until the index is above distance inclusive (will be infinite if step is opposite direction or 0)
// Help received: https://stackoverflow.com/a/34677687
while (index.compareTo(loopIterations) <= 0) {
    // Update to the actual value, will change if overridden in block - similar to fortran
    m_currentScope.PutIdentifier(variableName, new Value(index));

    try { a_for.GetBody().Accept(this); }
    // If a break is encountered, stop the loop
    catch (Jump.BreakException e) { break; }
    // Just want to stop current iteration
    catch (Jump.ContinueException ignored) { }

    // Increment the index by the step
    index = index.add(step);
}

// Don't want any value returned from a for loop - done to avoid output in CLI
return Value.NULL;
}
/* public Value VisitFor(For a_for); */

/**/
/*
Interpreter.VisitBody()

```

NAME

VisitBody - visits and evaluates a block of statements.

SYNOPSIS

```

public Value VisitBody(Body a_body);
    a_body    -> the block of statements to visit and evaluate.

```

DESCRIPTION

Evaluates each statement in the block of statements by traversing down each statement found within our List of nodes. The result of the Last Value is returned

For example, 'IF TRUE THEN PRINT("HELLO"); PRINT("!") END', where the body is the two print statements, would print 'HELLO!'.

RETURNS

The result of the last evaluated statement in the body. Helpful for conditional statements in the CLI.

```
*/
/**/
@Override
public Value VisitBody(Body a_body) {
    Value result = Value.NULL;

    // Traverse down each statement in the block
    for (Node statement : a_body.GetStatements()) { result = statement.Accept(this); }

    // Return the result of the last statement (helpful for conditional statements in CLI)
    return result;
}
/* public Value VisitBody(Body a_body); */

/**/
/*
Interpreter.VisitJump()
```

NAME

VisitJump - visits a jump statement (break, continue, return), throwing their respective exceptions.

SYNOPSIS

```
public Value VisitJump(Jump a_jump);
    a_jump    -> the jump statement to visit.
```

DESCRIPTION

Throws a Jump Exception based on the type of jump statement. If it is a break, it will throw a BreakException, if continued, a ContinueException, and if it is a return, it will throw a ReturnException with the Value of the expression. We throw exceptions as control flow as we are likely deeply nested in the tree, and it is easier to catch and handle them then try to return a value up the tree and check the condition there.

For example, 'WHILE TRUE THEN BREAK; PRINT("HI") END' would stop the loop before getting to the print statement.

RETURNS

Nothing - it throws an exception, so it doesn't return anything. If a jump statement is not one of the three, an IllegalStateException is thrown because we should never be here without a valid jump type.

```

*/
/**/
@Override
public Value VisitJump(Jump a_jump) {
    // Help received: IDE suggestion for switch expression
    switch (a_jump.GetJumpType()) {
        case BREAK -> throw new Jump.BreakException(a_jump.GetLineNumber());
        case CONTINUE -> throw new Jump.ContinueException(a_jump.GetLineNumber());
        // Jump back to the caller with the value
        case RETURN -> throw new Jump.ReturnException(a_jump.GetExpression().Accept(this));

        // Should never be able to happen, report a real error
        default -> throw new IllegalStateException("Invalid jump type: " + a_jump.GetJumpType().LABEL);
    }
}
/* public Value VisitJump(Jump a_jump); */

```

```

/*
 *      Visiting Expressions - Performing operations and getting a value
 */

```

```

/**/

```

```

/*
Interpreter.VisitAssignment()

```

NAME

VisitAssignment - visits and evaluates an assignment statement.

SYNOPSIS

```

public Value VisitAssignment(Assignment a_assignment);
    a_assignment    -> the assignment statement to visit and bind to the
                      current scope.

```

DESCRIPTION

Evaluates the expression of the assignment statement and binds the result in the current scope we are in. The value of the expression is returned, as some languages do not return anything on assignment, but we do. If the variable is already in the scope, it will be overridden.

For example, 'LET x = 5 + 1' would first evaluate '5 + 1' and then bind the result to 'x', so 'x' would be bound to '6'.

RETURNS

The value of the expression that was assigned to the variable.

```

*/

```

```

/**/
@Override
public Value VisitAssignment(Assignment a_assignment) {
    // Get the value of the expression
    Value value = a_assignment.GetExpression().Accept(this);

    // Assign the value to the variable
    m_currentScope.PutIdentifier(a_assignment.GetVariableName(), value);

    // Some languages don't return anything on assignment, but we are different, return the value
    return value;
}
/* public Value VisitAssignment(Assignment a_assignment); */

```

```

/**/
/*
Interpreter.VisitBinaryOperator()

```

NAME

VisitBinaryOperator - visits and evaluates a binary operation.

SYNOPSIS

```

public Value VisitBinaryOperator(BinaryOperator a_binary);
    a_binary    -> the binary operation to visit and evaluate.

```

DESCRIPTION

Evaluates the left and right expressions of the binary operation and performs the operation. The result of the operation is returned. If the operation is not valid (e.g. adding a string and a number), a type error is thrown. If the operation is division or modulo, and the right value is 0, a division by zero error is thrown.

Done by traversing down the left expression, then the right, and then passing the value off to the helper function to actually perform the operation.

For example, '1 + 2' would return the Value of 3, and '5 / 0' would throw a division by zero error.

RETURNS

The result of the binary operation.

```

*/
/**/
@Override
public Value VisitBinaryOperator(BinaryOperator a_binary) {
    // Traverse down left to the value, then right (post-order traversal)
    Value leftValue = a_binary.GetLeftExpression().Accept(this);
    Value rightValue = a_binary.GetRightExpression().Accept(this);
}

```



```

        // Perform the binary operation (1 + 1, 2 * 3, etc.)
        return EvaluateBinaryOperator(leftValue, a_binary.GetToken(), rightValue);
    }
    /* public Value VisitBinaryOperator(BinaryOperator a_binary); */

    /**/
    /*
    Interpreter.VisitIndex()

```

NAME

VisitIndex - visits and evaluates an index operation.

SYNOPSIS

```

public Value VisitIndex(Index a_index);
    a_index    -> the index operation to visit and evaluate.

```

DESCRIPTION

Evaluates the accessee and index of the index operation. If the index is ranged, it evaluates the start and end indices. The result of the operation is returned. If the index is out of bounds, an index out of bounds error is thrown.

For example, 'list:1' would return the Value at index 1 in the list, and 'list:1|3]' would return a list Value for the indices at 1 and 2 in the list (not inclusive).

RETURNS

The result of the index operation. So the element at a specific index or a list of elements between two indices.

```

*/
/**/
@Override
public Value VisitIndex(Index a_index) {
    // Traverse down the accessee expression and the index expression
    Value accesseeValue = a_index.GetAccessee().Accept(this);

    // If ranged deal with starting and ending indices
    if (a_index.IsRanged()) {
        Value startIndex = a_index.GetStartIndex().Accept(this);
        Value endIndex = a_index.GetEndIndex().Accept(this);
        return AccessRangedSubscript(accesseeValue, startIndex, endIndex, a_index.GetToken().GetLineNumber());
    }

    Value index = a_index.GetIndex().Accept(this);
    return AccessIndexSubscript(accesseeValue, index, a_index.GetToken().GetLineNumber());
}
/* public Value VisitIndex(Index a_index); */

```

```
/**/
/*
Interpreter.VisitUnaryOperator()
```

NAME

VisitUnaryOperator - visits and evaluates a unary operation.

SYNOPSIS

```
public Value VisitUnaryOperator(UnaryOperator a_unary);
    a_unary    -> the unary operation to visit and evaluate.
```

DESCRIPTION

Evaluates the expression of the unary operation and performs the operation. The result of the operation is returned. If the operation is not valid (e.g. adding a string and a number), a type error is thrown.

Done by traversing down the expression and then passing the value off to the helper function to actually perform the operation.

For example, '-1' would return the Value of -1, and 'NOT TRUE' would return the Value of FALSE.

RETURNS

The result of the unary operation.

```
*/
/**/
@Override
public Value VisitUnaryOperator(UnaryOperator a_unary) {
    // Traverse down the expression to get the value
    Value value = a_unary.GetExpression().Accept(this);

    // Perform unary operation (+1, -1, NOT TRUE, etc.)
    return EvaluateUnaryOperation(value, a_unary.GetToken());
}
/* public Value VisitUnaryOperator(UnaryOperator a_unary); */

/**/
/*
Interpreter.VisitCall()
```

NAME

VisitCall - visits and evaluates a function call, creating its own scope in the process.

SYNOPSIS

```
public Value VisitCall(Call a_call);
    a_call    -> the function call to visit and evaluate.
```

DESCRIPTION

Evaluates the callee of the function call and ensures it is a function. It then evaluates the arguments of the function call and ensures they match the number of parameters the function has (the arity). If they do, it executes the function, either a built-in or user-defined function.

For built-ins, we do not create a local scope, rather we execute the Java function. For user-defined functions, we create a new scope and put the parameters names into the scope with the arguments as their values. We then execute the body of the function.

RETURNS

If a built-in, the result of the built-in function. If user-defined, the Value of the return statement, or NULL if there is no return statement.

```
*/
/**/
@Override
public Value VisitCall(Call a_call) {
    // Get the callee and make sure it is a function
    Value callee = a_call.GetCallee().Accept(this);
    EnforceCallable(callee, a_call.GetLineNumber());
    Function function = callee.AsFunction();

    // Short-circuit out if parameters and arguments are not the same size
    EnforceSameArity(function.GetParameterCount(), a_call.GetArguments().size(), a_call.GetLineNumber());

    // Evaluate all the arguments to get their values
    List<Value> arguments = new ArrayList<>();
    for (Node argument : a_call.GetArguments()) { arguments.add(argument.Accept(this)); }

    // For built-in functions, we need to execute their Java code - it is not a body to traverse
    // Help received: IDE suggestion for ternary
    return function.IsBuiltIn()
        ? ExecuteBuiltInFunction(function.GetBuiltInFunction(), arguments, a_call.GetLineNumber())
        : ExecuteUserDefinedFunction(function, arguments);
}
/* public Value VisitCall(Call a_call); */

/**/
/*
Interpreter.VisitIdentifier()
```

NAME

VisitIdentifier - visits and evaluates an identifier, returning the value it is bound to.

SYNOPSIS

```
public Value VisitIdentifier(Identifier a_identifier);
    a_identifier    -> the identifier to visit and evaluate.
```

DESCRIPTION

Returns the value of the identifier by looking it up in the current scope. If the identifier is not found, the symbol table will throw an Undefined Identifier error to inform the user that we don't have any information on that variable.

RETURNS

Returns the Value of the identifier, if it was defined.

```
*/
/**/
@Override
public Value VisitIdentifier(Identifier a_identifier) {
    return m_currentScope.GetValue(a_identifier.GetVariableName(), a_identifier.GetLineNumber());
}
/* public Value VisitIdentifier(Identifier a_identifier); */

/**/
/*
Interpreter.VisitLiteral()
```

NAME

VisitLiteral - visits and returns the value of a literal.

SYNOPSIS

```
public Value VisitLiteral(Literal a_literal);
    a_literal    -> the literal to visit and evaluate.
```

DESCRIPTION

Returns the value of the literal. Literals are the base values of the language, like numbers, strings, and booleans. No actual evaluation gets done.

RETURNS

The Value of the literal, e.g. NULL, TRUE, "HELLO", 1.

```
*/
/**/
@Override
public Value VisitLiteral(Literal a_literal) {
    // Handles all literals (e.g. 1, 2.3, "Hello", FALSE, etc.)
```

```

        return a_literal.GetValue();
    }
    /* public Value VisitLiteral(Literal a_literal); */

    /**/
    /*
    Interpreter.VisitGroup()

```

NAME

VisitGroup - visits and evaluates a group, returning the value of the expression inside.

SYNOPSIS

```

public Value VisitGroup(Group a_group);
    a_group    -> the group to visit and evaluate.

```

DESCRIPTION

Evaluates the expression inside the group and returns the result. Groups are used to enforce the order of operations in the language, just as parentheses do. For example (1 + 2) is a group expression.

RETURNS

The Value of the expression inside the group.

```

    */
    /**/
    @Override
    public Value VisitGroup(Group a_group) {
        // Handles all parentheses (e.g. (1 + 2) * 3)
        return a_group.GetExpression().Accept(this);
    }
    /* public Value VisitGroup(Group a_group); */

    /**/
    /*
    Interpreter.VisitListExpression()

```

NAME

VisitListExpression - visits and evaluates a list expression, returning the list Value.

SYNOPSIS

```

public Value VisitListExpression(ListExpression a_listExpression);
    a_listExpression    -> the list expression to visit and evaluate.

```

DESCRIPTION

Evaluates the expressions inside the list expression and returns it as a list Value. As we can define lists with expressions inside each element e.g. [1 + 2, 3 * 4], we need to evaluate them first before making them a literal.

RETURNS

The evaluated list Value.

```
*/
/**/
@Override
public Value VisitListExpression(ListExpression a_listExpression) {
    // Handles all list expressions (e.g. [1, 2, 3], ["Hello", "World"], etc.)
    return EvaluateListExpression(a_listExpression.GetExpressions());
}
/* public Value VisitListExpression(ListExpression a_listExpression); */

/* PRIVATE */

/** Members */

private SymbolTable m_currentScope;           // The current scope of the block we are in

/** Private Helper Functions */

/*
 *      Binary Expressions - Performing operations on two values
 */

/**/
/*
Interpreter.EvaluateBinaryOperator()
```

NAME

EvaluateBinaryOperator - evaluates a binary operation on two values.

SYNOPSIS

```
private Value EvaluateBinaryOperator(Value a_left, Token a_token,
                                     Value a_right);
    a_left      -> the left value of the binary operation.
    a_token     -> the token/operator of the binary operation.
    a_right     -> the right value of the binary operation.
```

DESCRIPTION

Evaluates the binary operation on the left and right values. The operation is determined by the token/operator. If the operation is not valid (e.g. adding a string and a number), a type error is thrown. It

is the center of the interpreter, it performs the actual operations. In essence, it translates our values to Java values and performs the operations on them.

For numbers the operation is done with `BigDecimal`s as they contain the set of integers. They will be converted back to integers if the starting values were integers. This avoids if-else chains for the same operation on integers and floats. Java has generics, but they aren't as powerful as C++ templates as no operator overloading.

The use of lambdas was intentional to avoid multiple functions that would only be used once and duplicate the same 3-4 lines of code every time. This way, the java operation done can be determined within the switch and use a more generic function to apply it.

Our switch expression is used to determine the operation to perform, and the lambda is used to apply the operation to the left and right values. It should make it clear what operation is being done and how it is being done. In case of an invalid operator, an `IllegalStateException` is thrown as something is seriously wrong.

RETURNS

The result of the binary operation, if the types are correct. For example, `1 + 2` would return the Value of 3, `2 ^ 2` would be 4, and `"Hello" + "World"` would be `"HelloWorld"`.

```
*/
/**/
private Value EvaluateBinaryOperator(Value a_left, Token a_token, Value a_right) {
    OperationCode operator = a_token.GetOpcode();
    int lineNumber = a_token.GetLineNumber();

    // Lambdas are used to reduce code duplication and make it clear what operation is being done
    // Help received for lambdas:
    //   https://stackoverflow.com/a/13604748
    //   https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/function/package-summary.html
    //   https://stackoverflow.com/questions/17913409/what-is-a-sam-type-in-java
    //   IDE suggestion for switch expression
    return switch (operator) {
        /* Logical */
        case AND -> ApplyLogical(a_left, a_right, (a, b) -> a && b, lineNumber, operator);
        case OR -> ApplyLogical(a_left, a_right, (a, b) -> a || b, lineNumber, operator);

        /* Equality */
        case EQUALITY -> ApplyEquality(a_left, a_right, Value::Equals, lineNumber, operator);
        case INEQUALITY -> ApplyEquality(a_left, a_right, (a, b) -> !a.Equals(b), lineNumber, operator);

        /* Comparison */
        // We compare to 0 as BigDecimal.compareTo returns -1 if less than, 0 if equal and 1 if greater than
        // Help received: https://stackoverflow.com/a/34677687
```

```

case LESS_THAN -> ApplyComparison(a_left, a_right, (a) -> a < 0, lineNumber, operator);
case LESS_THAN_EQUAL -> ApplyComparison(a_left, a_right, (a) -> a <= 0, lineNumber, operator);
case GREATER_THAN -> ApplyComparison(a_left, a_right, (a) -> a > 0, lineNumber, operator);
case GREATER_THAN_EQUAL -> ApplyComparison(a_left, a_right, (a) -> a >= 0, lineNumber, operator);

/* Arithmetic */
// Addition has special cases (string and list concatenation)
case ADD -> ApplyAddition(a_left, a_right, lineNumber, operator);
case SUBTRACT -> ApplyNumericOperation(a_left, a_right, BigDecimal::subtract, lineNumber, operator);
case MULTIPLY -> ApplyNumericOperation(a_left, a_right, BigDecimal::multiply, lineNumber, operator);

// Division and modulo cannot have a right value of 0 (e.g. 3 / 0 and 3 % 0) - want our error, not Java's.
// Also need to set precision as irrational numbers will result in an arithmetic exception
// Help received: https://docs.oracle.com/javase/8/docs/api/java/math/MathContext.html
case DIVIDE -> ApplyDivmod(a_left, a_right, (a, b) -> a.divide(b, Value.PRECISION), lineNumber, operator);
case MODULO -> ApplyDivmod(a_left, a_right, (a, b) -> a.remainder(b, Value.PRECISION), lineNumber, operator);

// No standard operator for power in java so using external library to do calculations
case POWER -> ApplyPower(a_left, a_right, lineNumber, operator);

// Should never be able to happen, something is seriously wrong
default -> throw new IllegalStateException("Invalid operator: " + operator.LABEL);
};
}
/* private Value EvaluateBinaryOperator(Value a_left, Token a_token, Value a_right); */

/**/
/*
Interpreter.ApplyLogical()

```

NAME

ApplyLogical - applies a logical operation to two boolean values.

SYNOPSIS

```

private Value ApplyLogical(Value a_left, Value a_right,
                           BiPredicate<Boolean, Boolean> a_operation,
                           int a_lineNumber, OperationCode a_operator);
a_left           -> the left boolean value.
a_right          -> the right boolean value.
a_operation       -> the logical operation/function to apply.
a_lineNumber     -> the line number the operation was found on
a_operator       -> the operation code of the operation.

```

DESCRIPTION

Applies the logical operation to the left and right boolean values. The operation is determined by the lambda passed in. If the values are not booleans, a type error is thrown. It is used for the AND and OR operations in the language.

The lambda passed in is used to apply the operation to the two boolean values. It is a BiPredicate as it takes two arguments and returns a Java boolean.

RETURNS

The result of the logical operation, converted back to our Value. For example, TRUE AND FALSE would return FALSE, and TRUE OR FALSE would return TRUE.

```
*/
/**/
private Value ApplyLogical(Value a_left, Value a_right, BiPredicate<Boolean, Boolean> a_operation,
                           int a_lineNumber, OperationCode a_operator) {

    // Must be booleans - 5 AND 7 doesn't make much sense (unless you are C)
    EnforceSameTypes(DataType.BOOLEAN, a_left, a_right, a_lineNumber, a_operator);

    // Apply the AND/OR operation
    return new Value(a_operation.test(a_left.AsBoolean(), a_right.AsBoolean()));
}
/* private Value ApplyLogical(Value a_left, Value a_right,
                             BiPredicate<Boolean, Boolean> a_operation,
                             int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyEquality()
```

NAME

ApplyEquality - applies an equality operation to two values.

SYNOPSIS

```
private Value ApplyEquality(Value a_left, Value a_right,
                           BiFunction<Value, Value, Boolean> a_operation,
                           int a_lineNumber, OperationCode a_operator);

a_left      -> the left value of the equality operation.
a_right     -> the right value of the equality operation.
a_operation  -> the equality operation/function to apply.
a_lineNumber -> the line number the operation was found on
a_operator   -> the operation code of the operation.
```

DESCRIPTION

Applies the equality operation to the left and right values. The operation is determined by the lambda passed in. If the values are not the same type (or both are not numbers), a type error is thrown. It is used for the equality and inequality operations in the language.

The lambda passed in is used to apply the operation to the two values,

resulting in a Java boolean.

RETURNS

A boolean result of the equality operation, converted back to our Value. For example, 1 == 1 would return TRUE, and 1 == "Hello" would throw a type error.

```
*/
/**/
private Value ApplyEquality(Value a_left, Value a_right, BiFunction<Value, Value, Boolean> a_operation,
                           int a_lineNumber, OperationCode a_operator) {

    // If both are numbers, we can compare them directly
    if (a_left.IsNumber() && a_right.IsNumber()) { return new Value(a_operation.apply(a_left, a_right)); }

    // Can't compare equality of different types if not numbers - 5 == "hello" doesn't make sense
    EnforceSameTypes(a_left.GetDataTypes(), a_left, a_right, a_lineNumber, a_operator);

    // Perform the equality check
    return new Value(a_operation.apply(a_left, a_right));
}
/* private Value ApplyEquality(Value a_left, Value a_right,
                               BiFunction<Value, Value, Boolean> a_operation,
                               int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyComparison()
```

NAME

ApplyComparison - applies a comparison operation to two values.

SYNOPSIS

```
private Value ApplyComparison(Value a_left, Value a_right,
                              IntPredicate a_operation,
                              int a_lineNumber, OperationCode a_operator);

a_left      -> the left value of the comparison operation.
a_right     -> the right value of the comparison operation.
a_operation  -> the comparison operation/function to apply.
a_lineNumber -> the line number the operation was found on
a_operator   -> the operation code of the operation.
```

DESCRIPTION

Applies the comparison operation to the left and right values. The operation is determined by the IntPredicate passed in. If the values are not numbers, a type error is thrown. It is used for the comparison operations in the language.

The IntPredicate passed in is used to apply the operation to the two

values, resulting in a Java boolean. The comparison is done by converting the BigDecimals to floats and then comparing them to avoid else-if chains for the same operation on integers and floats.

RETURNS

A boolean result of the comparison operation, converted back to our Value. For example, `1 < 2` would return `TRUE`, and `1 < "Hello"` would throw a type error.

```
*/
/**/
private Value ApplyComparison(Value a_left, Value a_right, IntPredicate a_operation,
                              int a_lineNumber, OperationCode a_operator) {

    // Must be numbers, can compare integers and floats - 5 < 7 and 5.5 < 7. But can't compare strings or lists
    // - "hello" < "world" doesn't make sense
    EnforceNumberTypes(a_left, a_right, a_lineNumber, a_operator);

    // As stated BigDecimal.compareTo return an int depending on the comparison - see lambda for more info
    // Help received:
https://stackoverflow.com/questions/34677644/how-to-use-comparison-operators-like-on-bigdecimal
    int comparison = a_left.AsFloat().compareTo(a_right.AsFloat());
    return new Value(a_operation.test(comparison));
}
/* private Value ApplyComparison(Value a_left, Value a_right,
    IntPredicate a_operation,
    int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyNumericOperation()
```

NAME

`ApplyNumericOperation` - applies a numerical operation to two values.

SYNOPSIS

```
private Value ApplyNumericOperation(Value a_left, Value a_right,
    BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
    int a_lineNumber, OperationCode a_operator);
a_left      -> the left value of the numerical operation.
a_right     -> the right value of the numerical operation.
a_operation  -> the numerical operation/function to apply.
a_lineNumber -> the line number the operation was found on.
a_operator   -> the operation code of the operation.
```

DESCRIPTION

Applies the numerical operation to the left and right values. The operation is determined by the BiFunction passed in. If the values are not numbers, a type error is thrown. It is used for the numerical

operations in the language, like subtraction and multiplication.

The BiFunction passed in is used to apply the operation to the two values, resulting in a BigDecimals and will be converted back to integers if the starting two values were integers.

RETURNS

The result of the numerical operation, converted back to our Value. For example, 2 - 1 would return the Value of 1, and 2 * 3 would return 6.

```
*/
/**/
private Value ApplyNumericOperation(Value a_left, Value a_right,
                                    BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
                                    int a_lineNumber, OperationCode a_operator) {

    // As it's a general numerical operation, we need to enforce that both are numbers
    EnforceNumberTypes(a_left, a_right, a_lineNumber, a_operator);

    // Do the arithmetic operation, and convert back to the correct type (int or float) if both are integers
    BigDecimal result = a_operation.apply(a_left.AsFloat(), a_right.AsFloat());
    return FixBinaryResultType(a_left, a_right, result);
}
/* private Value ApplyNumericOperation(Value a_left, Value a_right,
    BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
    int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyAddition()
```

NAME

ApplyAddition - applies an addition operation to two values.

SYNOPSIS

```
private Value ApplyAddition(Value a_left, Value a_right,
                            int a_lineNumber, OperationCode a_operator);

a_left      -> the left value of the addition operation.
a_right     -> the right value of the addition operation.
a_lineNumber -> the line number the operation was found on.
a_operator  -> the operation code of the operation.
```

DESCRIPTION

Addition is a special case of binary operations as it can be used for string concatenation, list extension, and normal number arithmetic. It is the most complex of the binary operations due to this, and as a result gets its own function.

The left and right values must be numbers for arithmetic, doing implicit

conversion if adding integers to floats. These values will be converted back to an integer if the starting two were so to begin with. If they are strings, they are concatenated. If they are lists, they are extended. If they are not any of these cases (e.g. adding a string and a number), a type error is thrown.

RETURNS

The result of the addition operation, converted back to our Value. For example, 1 + 2 would return the Value of 3, "Hello" + "World" would return "HelloWorld", and [1, 2] + [3, 4] would return [1, 2, 3, 4].

```
*/
/**/
private Value ApplyAddition(Value a_left, Value a_right,
                           int a_lineNumber, OperationCode a_operator) {

    // Both are numbers, so normal arithmetic, e.g. 3 + 2 or 1.5 + 7
    if (a_left.IsNumber() && a_right.IsNumber()) {
        return FixBinaryResultType(a_left, a_right, a_left.AsFloat().add(a_right.AsFloat()));
    }

    // Normal string concatenation, e.g. "hello" + "world"
    if (a_left.IsString() && a_right.IsString()) {
        return new Value(a_left.AsString() + a_right.AsString());
    }

    // Lists we extend, e.g. [1, 2, 3] + [4, 5, 6] = [1, 2, 3, 4, 5, 6]
    if (a_left.IsList() && a_right.IsList()) {
        return new Value(AddLists(a_left.AsList(), a_right.AsList()));
    }

    // Catch if we have different types
    EnforceSameTypes(a_left.GetDataTypes(), a_left, a_right, a_lineNumber, a_operator);

    // Should not be reachable as EnforceSameType() should catch
    assert false : "Could not add values of different types";
    return Value.NULL;
}

/* private Value ApplyAddition(Value a_left, Value a_right,
                               int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyDivmod()
```

NAME

ApplyDivmod - applies a division or modulo operation to two values.

SYNOPSIS

```
private Value ApplyDivmod(Value a_left, Value a_right,
                          BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
```

```

        int a_lineNumber, OperationCode a_operator);
a_left      -> the left value of the operation.
a_right     -> the right value of the operation.
a_operation  -> the division/modulo operation to apply.
a_lineNumber -> the line number the operation was found on.
a_operator   -> the operator code of the operation.

```

DESCRIPTION

Applies the division or modulo operation to the left and right values. The operation is determined by the BiFunction passed in. If the values are not numbers, a type error is thrown. If the right value is 0, a division by zero error is thrown. It is used for the division and modulo operations in the language. Is in its own function due to the special case of division by zero.

The BiFunction passed in is used to apply the operation to the two values, resulting in a BigDecimals and will be converted back to integers if the starting two values were integers. So $2 / 3$ would be 0 as we have two integers, and $1 / 2.0$ would be 0.5 as we cast to a float to get the correct result.

RETURNS

The result of the division or modulo operation, converted back to our Value.

```

*/
/**/
private Value ApplyDivmod(Value a_left, Value a_right, BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
    int a_lineNumber, OperationCode a_operator) {

    // Division and modulo can only be done on numbers - 5 / "hello" doesn't make sense
    EnforceNumberTypes(a_left, a_right, a_lineNumber, a_operator);

    // Make into floats now to avoid repeated function calls
    BigDecimal left = a_left.AsFloat(); BigDecimal right = a_right.AsFloat();

    // Can't divide or modulo by 0, doesn't make sense
    EnforceDivmodByZero(right, a_lineNumber, a_operator);

    // Perform operation and fix the result type back to integer if needed
    BigDecimal result = a_operation.apply(left, right);
    return FixBinaryResultType(a_left, a_right, result);
}
/* private Value ApplyDivmod(Value a_left, Value a_right,
    BiFunction<BigDecimal, BigDecimal, BigDecimal> a_operation,
    int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.ApplyPower()

```

NAME

ApplyPower - applies a power operation to two values.

SYNOPSIS

```
private Value ApplyPower(Value a_left, Value a_right, int a_lineNumber,
                        OperationCode a_operator);
    a_left          -> the left value of the power operation.
    a_right         -> the right value of the power operation.
    a_lineNumber    -> the line number the operation was found on.
    a_operator      -> the operator code of the operation.
```

DESCRIPTION

Applies the power operation to the left and right values. If the values are not numbers, a type error is thrown. If the result of the power operation is too large to handle, an overflow error is thrown. It is used for the power operation in the language. Is in its own function due to the special case of overflow, and the need for an external library.

The power operation is done using an external library as Java does not have a standard power operation. The result is a BigDecimal, which will be converted back to an integer if the starting two values were integers. So 2^3 would be 8 as we have two integers, and $2^{3.0}$ would be 8.0 as we cast to a float to get the correct result.

RETURNS

The result of the power operation, converted back to our Value.

```
*/
/**/
private Value ApplyPower(Value a_left, Value a_right, int a_lineNumber, OperationCode a_operator) {

    // Power can only be done on numbers - 5 ^ "hello" doesn't make sense
    EnforceNumberTypes(a_left, a_right, a_lineNumber, a_operator);

    // Power can result in overflow, so we need to catch that. Very complicated to fix and above the scope of
    // this project to implement. Library is used to calculate as no standard one exists in Java.
    // Library: https://github.com/eobermuhlner/big-math
    try {
        BigDecimal result = BigDecimalMath.pow(a_left.AsFloat(), a_right.AsFloat(), Value.PRECISION);
        return FixBinaryResultType(a_left, a_right, result);
    }
    // We've overflowed, show our error instead of Java's
    catch (ArithmeticException e) {
        throw new Error("Overflow, result of power operation is too large to handle",
                        ErrorType.ARITHMETIC_ERROR,
                        a_lineNumber);
    }
}

/* private Value ApplyPower(Value a_left, Value a_right, int a_lineNumber,
```

```
        OperationCode a_operator); */
```

```
/**/
```

```
/*
```

```
Interpreter.AddLists()
```

NAME

AddLists - adds two lists together, returning the result.

SYNOPSIS

```
private List<Value> AddLists(List<Value> a_left,  
                             List<Value> a_right);  
    a_left      -> the left list to add.  
    a_right     -> the right list to add.
```

DESCRIPTION

Adds the two lists together, keeping the order of the elements. It is used for the addition operation in the language when adding two lists together. It is a helper function to avoid code duplication and make the code more readable.

RETURNS

A List<Value> object that is the result of adding the two lists.

```
*/
```

```
/**/
```

```
private List<Value> AddLists(List<Value> a_left, List<Value> a_right) {  
    // Help Received: https://stackoverflow.com/questions/189559/how-do-i-join-two-lists-in-java  
    List<Value> result = new ArrayList<>();  
    // Add left then right, as we want to keep the order  
    result.addAll(a_left);  
    result.addAll(a_right);  
    return result;  
}
```

```
/**/
```

```
/*
```

```
Interpreter.FixBinaryResultType()
```

NAME

FixBinaryResultType - converts the result of a binary operation back to an integer if the left and right values were integers.

SYNOPSIS

```
private Value FixBinaryResultType(Value a_left, Value a_right,  
                                   BigDecimal a_result);  
    a_left      -> the left value of the binary operation.
```


a_right -> the right value of the binary operation.
a_result -> the Java value result of the binary operation.

DESCRIPTION

Converts the result of the binary operation back to an integer if the left and right values were integers. This avoids if-else chains for the same operation on integers and floats. It is used for all binary operations in the language.

RETURNS

If the left and right values are integers, the converted BigDecimal to an integer Value. If they are not, the result is returned as is.

```
*/  
/**/  
private Value FixBinaryResultType(Value a_left, Value a_right, BigDecimal a_result) {  
    // If left and right values are integers, then go back to an integer. Best way to avoid long if-else chains to  
    //     perform same operation on float or integer while retaining accuracy.  
    // Help received: IDE suggestion for ternary  
    return a_left.IsInteger() && a_right.IsInteger() ? new Value(a_result.toBigInteger()) : new Value(a_result);  
}  
/* private Value FixBinaryResultType(Value a_left, Value a_right,  
    BigDecimal a_result); */  
  
/*  
 *   List and String Access - Accessing subscript and index. We are capped at Java's INTEGER_MAX_VALUE for lists  
 *   and strings, so will report if index goes beyond that.  
 *  
 *   To note: I'm aware of code dupe for strings and lists, as well as index and range subscripts. No way to  
 *   really avoid it without making the code more complex and harder to read. Lists and strings have  
 *   different methods for length and subscripting so use of Java's generics doesn't work without lambdas.  
 *   As for index and range, they are different operations once again, substring() versus charAt() and  
 *   sublist() versus get(). Pains me to have it like this, but as only 2 types are subscriptable, so I'm ok  
 *   with it. Add anymore and I would have to refactor it.  
 */  
  
/**/  
/*  
Interpreter.AccessIndexSubscript()
```

NAME

AccessIndexSubscript - accesses a subscript of a list or string using an index.

SYNOPSIS

```
private Value AccessIndexSubscript(Value a_accessee, Value a_index,  
    int a_lineNumber);  
a_accessee     -> the list or string to access.  
a_index        -> the index to access at.
```

a_lineNumber -> the line number the operation was found on.

DESCRIPTION

Accesses the subscript of a list or string using an index. If the accessee is not a list or string, a type error is thrown. If the index is out of bounds, an index out of bounds error is thrown. It is used for the index subscript operation in the language. We are 0 based indexing, so 0 is the first element, 1 is the second, etc. The index must also be an integer. We are capped at Java's list and string length (the integer max) so will report if index goes beyond that.

RETURNS

The Value of the element at the index in the list or string. For example "Hello":0 would evaluate to "H", and [1, 2, 3]:2 would evaluate to 3.

```
*/
/**/
private Value AccessIndexSubscript(Value a_accessee, Value a_index, int a_lineNumber) {
    // Can only subscript lists and strings - 5[0] doesn't make sense
    EnforceAccessType(a_accessee, a_lineNumber);

    int index = GetSafeIndex(a_index, a_lineNumber);

    if (a_accessee.IsString()) {
        String string = a_accessee.AsString();
        EnforceIndexBounds(index, string.length(), a_lineNumber);
        // Help received: https://stackoverflow.com/a/11230042
        return new Value(String.valueOf(string.charAt(index)));
    }
    else if (a_accessee.IsList()) {
        List<Value> list = a_accessee.AsList();
        EnforceIndexBounds(index, list.size(), a_lineNumber);
        return list.get(index);    // As a literal, not a list
    }

    // Should not be reachable as EnforceAccessType() should catch it, throw real error
    assert false : "Could not access index subscript";
    return Value.NULL;
}
/* private Value AccessIndexSubscript(Value a_accessee, Value a_index,
                                     int a_lineNumber); */

/**/
/*
Interpreter.AccessRangedSubscript()
```

NAME

AccessRangedSubscript - accesses a range of a list or string using a start and end index.

SYNOPSIS

```
private Value AccessRangedSubscript(Value a_accessee,
                                   Value a_startIndex, Value a_endIndex,
                                   int a_lineNumber);
a_accessee      -> the list or string to access.
a_startIndex    -> the start index of the range to access.
a_endIndex      -> the end index of the range to access.
a_lineNumber    -> the line number the operation was found on.
```

DESCRIPTION

Accesses the range of a list or string using a start and end index. If the accessee is not a list or string, a type error is thrown. If the range is out of bounds, with the range ending at 1 - a_endIndex. If these conditions are not met an index out of bounds error is thrown. It is used for the range subscript operation in the language. The indices must also be integers. Once again we are capped at Java's max integer for indexing, so will report if index goes beyond that.

RETURNS

The Value of the range of elements in the list or string. For example "Hello":0:2 would evaluate to "He", and [1, 2, 3]:1:3 would evaluate to [2, 3].

```
*/
/**/
private Value AccessRangedSubscript(Value a_accessee, Value a_startIndex, Value a_endIndex, int a_lineNumber) {
    // Can only subscript lists and strings - 5[0] doesn't make sense
    EnforceAccessType(a_accessee, a_lineNumber);

    // Get our start and end indices with overflow protection
    int startIndex = GetSafeIndex(a_startIndex, a_lineNumber);
    int endIndex = GetSafeIndex(a_endIndex, a_lineNumber);

    // Check if the range is out of bounds then range subscript the value
    if (a_accessee.IsString()) {
        String string = a_accessee.AsString();
        EnforceSubscriptBounds(startIndex, endIndex, string.length(), a_lineNumber);
        return new Value(string.substring(startIndex, endIndex));
    }
    else if (a_accessee.IsList()) {
        List<Value> list = a_accessee.AsList();
        EnforceSubscriptBounds(startIndex, endIndex, list.size(), a_lineNumber);
        // Help received: https://stackoverflow.com/a/32227674
        return new Value(list.subList(startIndex, endIndex)); // As a list, not literal
    }

    // Should not be reachable as EnforceAccessType() should catch it
    assert false : "Could not access ranged subscript";
    return Value.NULL;
}
```

```
/* private Value AccessRangedSubscript(Value a_accessee,
                                     Value a_startIndex, Value a_endIndex,
                                     int a_lineNumber); */
```

```
/**/
/*
Interpreter.GetSafeIndex()
```

NAME

GetSafeIndex - gets the index as an integer, ensuring it is within Java's integer limit.

SYNOPSIS

```
private int GetSafeIndex(Value a_index, int a_lineNumber);
    a_index          -> the index to get as an integer.
    a_lineNumber     -> the line number the operation was found on.
```

DESCRIPTION

Gets the index as an integer, ensuring it is within Java's integer limit. If the index is not an integer, a type error is thrown. We need to do this as we don't have overflow, but are limited to how we can represent the index because of Java. We could make our own list object, but that is an entirely different project in its own right.

RETURNS

The index as a Java integer, ensuring it is within Java's integer limit, if it can.

```
*/
/**/
private int GetSafeIndex(Value a_index, int a_lineNumber) {
    // Can't access with non-integer type, nor can we access outside Java's integer limit
    EnforceGenericType(DataType.INTEGER, a_index, a_lineNumber);
    EnforceIndexLimit(a_index.AsInteger(), a_lineNumber);
    return a_index.AsInteger().intValue();
}
/* private int GetSafeIndex(Value a_index, int a_lineNumber); */
```

```
/*
 *      Unary Expressions - Performing operations on one value
 */
```

```
/**/
/*
Interpreter.EvaluateUnaryOperation()
```

NAME

EvaluateUnaryOperation - evaluates a unary operation on a value.

SYNOPSIS

```
private Value EvaluateUnaryOperation(Value a_value, Token a_token);
    a_value      -> the value to perform the operation on.
    a_token      -> the token containing the operation.
```

DESCRIPTION

Evaluates a unary operation on a value. If the value is not the correct type for the operation, a type error is thrown. It is used for the unary operations in the language, like negation and logical negation. Supported operations are ADD, SUBTRACT, and NOT.

RETURNS

The result of the unary operation, converted back to our Value. For example, -5 would return -5, and NOT TRUE would return FALSE.

```
*/
/**/
private Value EvaluateUnaryOperation(Value a_value, Token a_token) {
    OperationCode operator = a_token.GetOpcode();
    int lineNumber = a_token.GetLineNumber();

    // Help received: IDE suggestion for switch expression
    return switch (operator) {
        // Add (+1) doesn't do much to a value - done for completeness
        case ADD -> a_value;
        case SUBTRACT -> ApplyNumericalNegation(a_value, lineNumber);
        case NOT -> ApplyLogicalNegation(a_value, lineNumber);

        // Should never be able to happen, something is seriously wrong
        default -> throw new IllegalStateException("Invalid operator for unary: " + operator.LABEL);
    };
}
/* private Value EvaluateUnaryOperation(Value a_value, Token a_token); */

/**/
/*
Interpreter.ApplyNumericalNegation()
```

NAME

ApplyNumericalNegation - applies a numerical negation to a value.

SYNOPSIS

```
private Value ApplyNumericalNegation(Value a_value, int a_lineNumber);
    a_value      -> the number value to apply the negation to.
    a_lineNumber -> the line number the operation was found on.
```

DESCRIPTION

Applies a numerical negation to a value. If the value is not a number, a type error is thrown. It is used for the negation operation in the language. We use BigDecimals to avoid if-else chains for the numbers. If our value was an integer to begin with, we convert it back.

RETURNS

The result of the numerical negation, converted back to our Value. For example, --5 would return 5, and -5.5 would return -5.5.

```
*/
/**/
private Value ApplyNumericalNegation(Value a_value, int a_lineNumber) {
    // Can only negate (meaning making negative in this case) on numbers - -"hello" doesn't make sense
    EnforceNumberType(a_value, a_lineNumber);

    // Negate and fix the result type back to integer if needed
    BigDecimal negatedValue = a_value.AsFloat().negate();
    return FixUnaryResultType(a_value, negatedValue);
}
/* private Value ApplyNumericalNegation(Value a_value, int a_lineNumber); */

/**/
/*
Interpreter.ApplyLogicalNegation()
```

NAME

ApplyLogicalNegation - applies a logical negation to a value.

SYNOPSIS

```
private Value ApplyLogicalNegation(Value a_value, int a_lineNumber);
    a_value          -> the boolean value to apply the negation to.
    a_lineNumber     -> the line number the operation was found on.
```

DESCRIPTION

Applies a logical negation to a value. If the value is not a boolean, a type error is thrown. It is used for the logical negation operation in the language, so !TRUE would return FALSE, and !FALSE would return TRUE.

RETURNS

The result of the Java logical negation, converted back to our Value.

```
*/
/**/
private Value ApplyLogicalNegation(Value a_value, int a_lineNumber) {
    // Logical negation (NOT) can only be done on booleans - NOT 5 doesn't make sense (again unless you are C)
    EnforceGenericType(DataType.BOOLEAN, a_value, a_lineNumber);
```

```

        return new Value(!a_value.AsBoolean());
    }
    /* private Value ApplyLogicalNegation(Value a_value, int a_lineNumber); */

    /**/
    /*
    Interpreter.FixUnaryResultType()

```

NAME

FixUnaryResultType - converts the result of a unary operation back to an integer if the value was an integer.

SYNOPSIS

```

private Value FixUnaryResultType(Value a_value, BigDecimal a_result);
    a_value      -> the value that was operated on.
    a_result     -> the Java value result of the unary operation.

```

DESCRIPTION

Converts the result of the unary operation back to an integer if the value was an integer. This avoids if-else chains for the same operation on integers and floats. Similar to FixBinaryResultType, but for only 1 value.

RETURNS

If the value was an integer, the converted BigDecimal to an integer Value. If it was not, the result is returned as is.

```

    */
    /**/
private Value FixUnaryResultType(Value a_value, BigDecimal a_result) {
    // Same as binary result, but for only 1 value where we need to fix the type back to integer if needed
    // Help received: IDE suggestion for ternary
    return a_value.IsInteger() ? new Value(a_result.toBigInteger()) : new Value(a_result);
}

/*
 *      List Expressions - Performing operations on lists
 */

/**/
/*
Interpreter.EvaluateListExpression()

```

NAME

EvaluateListExpression - evaluates a list expression.

SYNOPSIS

```
private Value EvaluateListExpression(List<Node> a_unevaluatedList);  
    a_unevaluatedList    -> the list of unevaluated nodes to evaluate.
```

DESCRIPTION

Evaluates a list expression. It evaluates each element in the list and makes them into values. If an element is a list, it will automatically recurse and evaluate itself. For example, [1+2, 3*4] would evaluate to [3, 12].

RETURNS

The list as our Value object, with each element evaluated and made into a literal.

```
*/  
/**/  
private Value EvaluateListExpression(List<Node> a_unevaluatedList) {  
    List<Value> values = new ArrayList<>();  
  
    // Evaluate each element in the list and make them into values. If an element is a list, it will automatically  
    //      recurse and evaluate itself.  
    for (Node element : a_unevaluatedList) { values.add(element.Accept(this)); }  
  
    return new Value(values);  
}  
/* private Value EvaluateListExpression(List<Node> a_unevaluatedList); */  
  
/*  
 *      Calling Functions - Making sure we can call them and executing them  
 */  
  
/**/  
/*  
Interpreter.ExecuteBuiltInFunction()
```

NAME

EvaluateFunctionCall - evaluates a built-in function call.

SYNOPSIS

```
private Value ExecuteBuiltInFunction(BuiltInFunctions.BuiltIn a_function,  
                                     List<Value> a_arguments,  
                                     int a_lineNumber);  
    a_function    -> the built-in function to execute.  
    a_arguments   -> the arguments to pass to the function.  
    a_lineNumber  -> the line number the function was called on.
```

DESCRIPTION

Runs the Java code associated with the built-in function (passed in by `a_function`) and returns the result. If there is an error within the built-in, it will be caught and rethrown as our error to include the line number it occurred on. The arguments are passed to the built-in function to execute.

If the built-in function is not found, an error is thrown.

RETURNS

The result of the built-in function, if no error appears.

```
*/
/**/
private Value ExecuteBuiltInFunction(BuiltInFunctions.BuiltIn a_function, List<Value> a_arguments,
                                     int a_lineNumber) {

    // Try to execute built-in and return its value if no error
    try { return a_function.Execute(a_arguments); }

    // There was likely something wrong with how built-in was called, so rethrow what the message was
    catch (BuiltInFunctions.BuiltInFunctionException BuiltInError) {
        throw new Error(BuiltInError.getMessage(),
                        ErrorType.BUILT_IN_ERROR,
                        a_lineNumber);
    }
}

/**/
/*
Interpreter.ExecuteUserDefinedFunction()
```

NAME

`ExecuteUserDefinedFunction` - executes a user-defined function.

SYNOPSIS

```
private Value ExecuteUserDefinedFunction(Function a_function,
                                         List<Value> a_arguments);

a_function      -> the user-defined function to execute.
a_arguments     -> the arguments to pass to the function.
```

DESCRIPTION

Executes a user-defined function. It creates a new scope for the function and puts all arguments into it with their respective variable names, and executes the function's body. If the function returns a value, it is caught and returned.

RETURNS

The Value within the ReturnException thrown by the function, if it had one. If it didn't, it returns NULL - every function returns something.

```

*/
/**/
private Value ExecuteUserDefinedFunction(Function a_function, List<Value> a_arguments) {
    // Ensure the number of parameters and arguments are the same, if not it's a caller error
    List<Identifier> parameters = a_function.GetParameters();
    assert parameters.size() == a_arguments.size();

    // Create a new scope for the function
    SymbolTable previousScope = m_currentScope;
    m_currentScope = new SymbolTable(m_currentScope);

    // Put all arguments into new scope with their respective variables
    for (int index = 0; index < a_function.GetParameterCount(); index++) {
        m_currentScope.PutIdentifier(parameters.get(index).GetVariableName(), a_arguments.get(index));
    }

    // Functions return or NULL if no return statement, so we set our return to such
    Value returnValue = Value.NULL;

    // Execute the function's body, at any point it returns a value, we catch it and return it
    try { a_function.GetBody().Accept(this); }
    catch (Jump.ReturnException data) { returnValue = data.GetValue(); }

    // Restore the previous scope
    m_currentScope = previousScope;

    return returnValue;
}
/* private Value ExecuteUserDefinedFunction(Function a_function,
                                           List<Value> a_arguments); */

```

```

/*
 *      Type Enforcement - Laying down the law of type safety
 */

```

```

/**/
/*
Interpreter.EnforceSameTypes()

```

NAME

EnforceSameTypes - ensures two values are the same type.

SYNOPSIS

```

private void EnforceSameTypes(Value.DataType a_expectedType, Value a_left,
                               Value a_right, int a_lineNumber,
                               OperationCode a_operator);
    a_expectedType -> the expected type of the values.

```

a_left -> the left value to compare.
 a_right -> the right value to compare.
 a_lineNumber -> the line number the operation was found on.
 a_operator -> the operator code of the operation.

DESCRIPTION

Ensures two values are the same type. If they are not, a type error is thrown. It is used for binary operations where the types must be the same, e.g. subtracting two numbers or adding two strings.

RETURNS

Nothing, but throws an error if the types are not the expected.

```

*/
/**/
private void EnforceSameTypes(Value.DataType a_expectedType, Value a_left, Value a_right,
                             int a_lineNumber, OperationCode a_operator) {
    // If both our values are not the expected type, we have a type error
    if (!(a_left.GetDataType() == a_expectedType && a_right.GetDataType() == a_expectedType)) {
        throw new Error("Cannot perform '" + a_operator.LABEL + "' to types '" +
            a_left.GetDataType() + "' and '" + a_right.GetDataType() + "'",
            ErrorType.TYPE_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceSameTypes(Value.DataType a_expectedType,
    Value a_left, Value a_right,
    int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.EnforceNumberTypes()

```

NAME

EnforceNumberTypes - ensures two values are numbers.

SYNOPSIS

```

private void EnforceNumberTypes(Value a_left, Value a_right,
                             int a_lineNumber, OperationCode a_operator);
    a_left           -> the left value to compare.
    a_right          -> the right value to compare.
    a_lineNumber    -> the line number the operation was found on.
    a_operator       -> the operator code of the operation.

```

DESCRIPTION

Ensures two values are numbers. If they are not, a type error is thrown. It is used for binary operations where the values must be numbers, e.g. dividing two numbers or multiplying two numbers.

RETURNS

Nothing, but throws an error if the values are not numbers.

```
*/
/**/
private void EnforceNumberTypes(Value a_left, Value a_right, int a_lineNumber, OperationCode a_operator) {
    // If both not a number, we have a type error
    if (!(a_left.IsNumber() && a_right.IsNumber())) {
        throw new Error("Expected two number types for '" + a_operator.LABEL + "'", got "'" +
            a_left.GetDataTypes() + "' and '" + a_right.GetDataTypes() + "'",
            ErrorType.TYPE_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceNumberTypes(Value a_left, Value a_right,
    int a_lineNumber, OperationCode a_operator); */

/**/
/*
Interpreter.EnforceAccessType()
```

NAME

EnforceAccessType - ensures the value is subscriptable.

SYNOPSIS

```
private void EnforceAccessType(Value a_accessee, int a_lineNumber);
    a_accessee      -> the value to check if it is subscriptable.
    a_lineNumber    -> the line number the accessee was found on.
```

DESCRIPTION

Ensures the value is subscriptable. If it is not a string or list, a type error is thrown.

RETURNS

Nothing, but throws an error if the value is not subscriptable.

```
*/
/**/
private void EnforceAccessType(Value a_accessee, int a_lineNumber) {
    // If the accessee is not a string or list, we can't access it
    if (!(a_accessee.IsString() || a_accessee.IsList())) {
        throw new Error("Type '" + a_accessee.GetDataTypes() + "' is not subscriptable",
            ErrorType.TYPE_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceAccessType(Value a_accessee, int a_lineNumber); */
```

```
/**/
/*
Interpreter.EnforceGenericType()
```

NAME

EnforceGenericType - ensures the value is the expected type.

SYNOPSIS

```
private void EnforceGenericType(Value.DataType a_expectedType,
                                Value a_value, int a_lineNumber);
    a_expectedType  -> the expected type of the value.
    a_value         -> the value to check if it is the expected type.
    a_lineNumber    -> the line number the value was found on.
*/
/**/
private void EnforceGenericType(Value.DataType a_expectedType, Value a_value, int a_lineNumber) {
    // If the expected type does not match our values type, we have a type error
    if (a_value.GetDataType() != a_expectedType) {
        throw new Error("Expected '" + a_expectedType + "', got '" + a_value.GetDataType() + "'",
                        ErrorType.TYPE_ERROR,
                        a_lineNumber);
    }
}
/* private void EnforceGenericType(Value.DataType a_expectedType,
                                Value a_value, int a_lineNumber); */
```

```
/**/
/*
Interpreter.EnforceNumberType()
```

NAME

EnforceNumberType - ensures the value is a number.

SYNOPSIS

```
private void EnforceNumberType(Value a_value, int a_lineNumber);
    a_value         -> the value to check if it is a number.
    a_lineNumber    -> the line number the value was found on.
```

DESCRIPTION

Ensures the value is a number. If it is not, a type error is thrown.

RETURNS

Nothing, but throws an error if the value is not a number.

```
*/
/**/
```

```

private void EnforceNumberType(Value a_value, int a_lineNumber) {
    // If the value is not a number, we have a type error
    if (!a_value.IsNumber()) {
        throw new Error("Expected number, got '" + a_value.GetDataType() + "'",
            ErrorType.TYPE_ERROR,
            a_lineNumber);
    }
}
}
/* private void EnforceNumberType(Value a_value, int a_lineNumber); */

/**/
/*
Interpreter.EnforceLoopTypes()

```

NAME

EnforceLoopTypes - ensures the values in a for loop are integers.

SYNOPSIS

```

private void EnforceLoopTypes(Value a_assignment, Value a_endValue,
    Value a_increment, int a_lineNumber);
a_assignment    -> the assignment value in the for loop.
a_endValue       -> the end value in the for loop.
a_increment      -> the increment value in the for loop.
a_lineNumber     -> the line number the for loop was found on.

```

DESCRIPTION

Ensures the values in a for loop are integers. If they are not, a type error is thrown. It is used for the for loop in the language. Does it in a way that will show all errors at once to the user, making it easier to debug.

RETURNS

Nothing, but throws an error if the values are not integers.

```

*/
/**/
private void EnforceForLoopTypes(Value a_assignment, Value a_endValue, Value a_increment, int a_lineNumber) {
    List<String> errors = new ArrayList<>();

    // If any of the values are not integers, we have a type error
    if (!a_assignment.IsInteger()) { errors.add("assignment (got '" + a_assignment.GetDataType() + "')"); }
    if (!a_endValue.IsInteger()) { errors.add("end value (got '" + a_endValue.GetDataType() + "')"); }
    if (!a_increment.IsInteger()) { errors.add("increment (got '" + a_increment.GetDataType() + "')"); }

    // If there are no errors, we are good to go
    if (errors.isEmpty()) { return; }

    // Help received: https://www.geeksforgeeks.org/java-string-join-examples/
    String message = "Expected '" + DataType.INTEGER + "' for " + String.join(", ", errors);
}

```

```

// Print out the errors with commas and an 'and' at the end if there are more than 1 to help user identify
// what they did wrong. Makes it more specific and easier to understand for user.
// Help received: https://www.w3schools.com/java/ref\_string\_lastindexof.asp
int lastComma = message.lastIndexOf(", ");
if (lastComma != -1) {
    message = message.substring(0, lastComma) + " and" + message.substring(lastComma + 1);
}

throw new Error(message,
    ErrorType.TYPE_ERROR,
    a_lineNumber);
}
/* private void EnforceForLoopTypes(Value a_assignment, Value a_endValue,
    Value a_increment, int a_lineNumber); */

```

```

/*
 *      Function Enforcement - Making sure functions are called correctly
 */

```

```

/**/
/*
Interpreter.EnforceCallable()

```

NAME

EnforceCallable - ensures the value is a function.

SYNOPSIS

```

private void EnforceCallable(Value a_function, int a_lineNumber);
    a_function      -> the value to check if it is a function.
    a_lineNumber    -> the line number the function was found on.

```

DESCRIPTION

Ensures the value is a function. If it is not, a type error is thrown.

RETURNS

Nothing, but throws an error if the value is not a function.

```

*/
/**/
private void EnforceCallable(Value a_function, int a_lineNumber) {
    // If the value is not a function, we can't call it
    if (!a_function.IsFunction()) {
        throw new Error("Cannot call non-function type",
            ErrorType.TYPE_ERROR,
            a_lineNumber);
    }
}

```

```
/* private void EnforceCallable(Value a_function, int a_lineNumber); */
```

```
/**/
```

```
/*
```

```
Interpreter.EnforceSameArity()
```

NAME

EnforceSameArity - ensures the number of parameters and arguments are the same for a call and function.

SYNOPSIS

```
private void EnforceSameArity(int a_functionParameters,
                              int a_callArguments,
                              int a_lineNumber);
a_functionParameters -> the number of parameters in the function.
a_callArguments      -> the number of arguments in the call.
a_lineNumber         -> the line number the call was found on.
```

DESCRIPTION

Ensures the number of parameters and arguments are the same for a call and function. If they are not, a runtime is thrown. It is used for function calls where the number of parameters and arguments must be the same, e.g. calling a function with 3 arguments when it only takes 2.

RETURNS

Nothing, but throws an error if the number of parameters and arguments are not the same.

```
*/
```

```
/**/
```

```
private void EnforceSameArity(int a_functionParameters, int a_callArguments, int a_lineNumber) {
    // If the expected and actual number of arguments are not the same, we have a type error
    if (a_functionParameters != a_callArguments) {
        throw new Error("Expected " + a_functionParameters + " argument(s), got " + a_callArguments,
            ErrorType.RUNTIME_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceSameArity(int a_functionParameters,
                                int a_callArguments,
                                int a_lineNumber); */
```

```
/*
```

```
 *      Jump Enforcement - Making jump statements are in the correct place
```

```
*/
```

```
/**/
```

```
/*
```


Interpreter.ErrorOnJump()

NAME

ErrorOnJump - throws an error for a jump statement outside of a loop.

SYNOPSIS

```
private void ErrorOnJump(String a_statement, int a_lineNumber);
    a_statement      -> the jump statement that was found.
    a_lineNumber     -> the line number the jump statement was found on.
```

DESCRIPTION

Throws an error for a jump statement outside of a loop. Very specific case where syntax is correct, as something like BREAK can be within a nested function definition inside a loop, but can't actually be evaluated due to the nest.

RETURNS

Nothing, but throws an error for a jump statement outside of a loop.

```
*/
/**/
private void ErrorOnJump(String a_statement, int a_lineNumber) {
    throw new Error(a_statement + " statement outside of loop",
        ErrorType.RUNTIME_ERROR,
        a_lineNumber);
}

/*
 *      Index and Subscript Enforcement - Making sure we don't go out of bounds
 */

/**/
/*
Interpreter.EnforceIndexBounds()
```

NAME

EnforceIndexBounds - ensures the index is within the bounds of the list or string.

SYNOPSIS

```
private void EnforceIndexBounds(int a_index, int a_size, int a_lineNumber);
    a_index          -> the index to check if it is within bounds.
    a_size           -> the size of the list or string.
    a_lineNumber     -> the line number the index was found on.
```

DESCRIPTION

Ensures the index is within the bounds of the list or string. If it is not, an index out of bounds error is thrown. It is used for subscripting a list or string with an index.

RETURNS

Nothing, but throws an error if the index is out of bounds.

```
*/
/**/
private void EnforceIndexBounds(int a_index, int a_size, int a_lineNumber) {
    // If the index is out of bounds not in range, can't subscript it
    if (a_index < 0 || a_index >= a_size) {
        throw new Error("Index " + a_index + " is not in range for length " + a_size,
            ErrorType.OUT_OF_BOUNDS_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceIndexBounds(int a_index, int a_size,
    int a_lineNumber); */

/**/
/*
Interpreter.EnforceSubscriptBounds()
```

NAME

EnforceSubscriptBounds - ensures the range is within the bounds of the list or string.

SYNOPSIS

```
private void EnforceSubscriptBounds(int a_startIndex, int a_endIndex,
    int a_size, int a_lineNumber);
a_startIndex    -> the start index of the range to check.
a_endIndex      -> the end index of the range to check.
a_size          -> the size of the list or string.
a_lineNumber    -> the line number the range was found on.
```

DESCRIPTION

Ensures the range is within the bounds of the list or string. If it is not, an index out of bounds error is thrown. It is used for subscripting a list or string with a range.

RETURNS

Nothing, but throws an error if the range is out of bounds.

```
*/
/**/
private void EnforceSubscriptBounds(int a_startIndex, int a_endIndex, int a_size, int a_lineNumber) {
```

```

// If not in range and end is not less than start, can't subscript it
if (a_startIndex < 0 || (a_endIndex > a_size || a_endIndex < a_startIndex)) {
    throw new Error("Range " + a_startIndex + " to " + a_endIndex + " is not in range for length " +
        a_size,
        ErrorType.OUT_OF_BOUNDS_ERROR,
        a_lineNumber);
}
}
/* private void EnforceSubscriptBounds(int a_startIndex, int a_endIndex,
    int a_size, int a_lineNumber); */

```

```

/**/
/*
Interpreter.EnforceIndexLimit()

```

NAME

EnforceIndexLimit - ensures the index is within Java's integer limit.

SYNOPSIS

```

private void EnforceIndexLimit(BigInteger a_index, int a_lineNumber);
    a_index          -> the index to check if it is within Java's integer
        limit.
    a_lineNumber     -> the line number the index was found on.

```

DESCRIPTION

Ensures the index is within Java's integer limit. If it is not, an index out of bounds error is thrown. It is used for subscripting a list or string with an index.

RETURNS

Nothing, but throws an error if the index is out of bounds.

```

*/
/**/
private void EnforceIndexLimit(BigInteger a_index, int a_lineNumber) {
    // If the index is too large to handle, can't subscript it
    if (a_index.compareTo(BigInteger.valueOf(Integer.MAX_VALUE)) > 0) {
        throw new Error("Index " + a_index + " is too large to handle",
            ErrorType.OUT_OF_BOUNDS_ERROR,
            a_lineNumber);
    }
}
/* private void EnforceIndexLimit(BigInteger a_index, int a_lineNumber); */

/*
 *      Arithmetic Errors - Handling bad arithmetic operations
 */

```

```
/**/
/*
Interpreter.EnforceDivmodByZero()
```

NAME

EnforceDivmodByZero - ensures the denominator is not 0 for division or modulo.

SYNOPSIS

```
private void EnforceDivmodByZero(BigDecimal a_denominator,
                                int a_lineNumber,
                                OperationCode a_operator);
a_denominator  -> the denominator to check if it is 0.
a_lineNumber   -> the line number the operation was found on.
a_operator      -> the operator code of the operation.
```

DESCRIPTION

Ensures the denominator is not 0 for division or modulo. If it is, an arithmetic error is thrown. It is used for division and modulo operations in the language. What does it mean to divide by 0?

RETURNS

Nothing, but throws an error if the denominator is 0.

```
*/
/**/
private void EnforceDivmodByZero(BigDecimal a_denominator, int a_lineNumber, OperationCode a_operator) {
    // If the denominator is 0, we can't divide or modulo by it
    if (a_denominator.compareTo(BigDecimal.ZERO) == 0) {
        throw new Error("Cannot " + a_operator.LABEL + " by 0",
                        ErrorType.ARITHMETIC_ERROR,
                        a_lineNumber);
    }
}
/* private void EnforceDivmodByZero(BigDecimal a_denominator,
                                int a_lineNumber,
                                OperationCode a_operator); */
}
```

SymbolTable.java

```
///
/// SymbolTable - The structure that stores all the identifiers and their values. It is a tree, with each node
/// representing a scope. The root node is the global scope, and each child node is a local scope. It is traversed
/// from local till upwards to find the identifier, and if not found, it is undefined. Any time we put an
/// identifier in the table, it is put in the most local scope possible. Uses a hashmap for quick access to the
```

```

///     values.
///

package Interpreter;

import Language.Error;
import Language.Error.ErrorType;

import java.util.HashMap;

public class SymbolTable {
    /* PUBLIC */

    /** Constructors */

    /**
     *
     * Interpreter.SymbolTable.SymbolTable()
     *
     * NAME
     *
     *     SymbolTable - constructor for the global scope.
     *
     * SYNOPSIS
     *
     *     SymbolTable();
     *
     * DESCRIPTION
     *
     *     Initializes the global scope of the symbol table. The global scope is
     *     the root of the tree, and has no outer scope.
     *
     * RETURNS
     *
     *     Nothing - it's a constructor.
     */
    /**
     *
     * public SymbolTable() {
     *     m_localScope = new HashMap<>();
     *     // No outer scope for global, so can't set it to anything
     *     m_outerScope = null;
     * }
     */
    /* SymbolTable() */

    /**
     *
     * Interpreter.SymbolTable.SymbolTable()
     *
     * NAME
     *
     *     SymbolTable - constructor for a local scope.

```

SYNOPSIS

```
SymbolTable(SymbolTable a_outerScope);  
    a_outerScope    -> the immediate outer scope to the current scope.
```

DESCRIPTION

Initializes a local scope of the symbol table. The local scope is a child of the outer scope, and has no children of its own (currently). It is used to store identifiers and their values in the most local scope possible.

RETURNS

Nothing - it's a constructor.

```
*/  
/**/  
public SymbolTable(SymbolTable a_outerScope) {  
    m_localScope = new HashMap<>();  
    m_outerScope = a_outerScope;  
}  
/* SymbolTable(SymbolTable a_outerScope) */
```

```
/** Accessors **/
```

```
/**/  
/*  
Interpreter.SymbolTable.GetValue()
```

NAME

GetValue - gets the value of the identifier.

SYNOPSIS

```
Value GetValue(String a_identifier, int a_lineNumber);  
    a_identifier    -> the identifier to get the value of.  
    a_lineNumber    -> the line number the identifier was found on.
```

DESCRIPTION

Gets the value of the identifier in the symbol table. It first checks the local scope, and if it's not there, it recursively checks the outer scope until it hits the global scope. If it's not found anywhere, it's undefined, so we throw an error to inform the user.

RETURNS

The Value of the identifier if it is defined.

```
*/
```

```

/**/
public Value GetValue(String a_identifier, int a_lineNumber) {
    // Return closest scope first
    if (InLocalScope(a_identifier)) { return m_localScope.get(a_identifier); }

    // Recursively backup to the outer scope to see if there, until we hit global
    if (!IsOuterGlobalScope()) { return m_outerScope.GetValue(a_identifier, a_lineNumber); }

    // Couldn't find it anywhere within static tree, so it's undefined (easier to put here than in Interpreter)
    throw new Error("Undefined identifier '" + a_identifier + "'",
        ErrorType.UNDEFINED_IDENTIFIER,
        a_lineNumber);
}
/* public Value GetValue(String a_identifier, int a_lineNumber) */

/** Mutators **/

// Puts the identifier in the most local scope possible
public void PutIdentifier(String a_identifier, Value a_value) {
    m_localScope.put(a_identifier, a_value);
}

/* PRIVATE */

/** Members **/

private final HashMap<String, Value> m_localScope; // Current scope we are at (i.e. local scope)
private final SymbolTable m_outerScope;          // The enclosing scope of the current scope (null if global)

/** Private Helper Functions **/

// Returns true if the identifier is in the current scope
private boolean InLocalScope(String a_identifier) {
    return m_localScope.containsKey(a_identifier);
}

// Returns true if the current scope is the global scope, i.e. no more enclosing scopes
private boolean IsOuterGlobalScope() {
    return m_outerScope == null;
}
}

```

Value.java

```

///
/// Value - Represents and stores a value in the language. Includes data types of an integer, float, boolean, string,

```

```

///    list. Integers are stored as BigIntegers and floats as BigDecimal to avoid overflow within the language. Class
///    performs implicit conversion between floats and integers, and provides access to equality checks and string
///    formatting. All values are immutable, so only copies are returned (no references).
///

package Interpreter;

import AbstractSyntaxTree.Statements.Function;
import Lexer.Lexer;
import Lexer.OperationCode;

import java.math.BigDecimal;
import java.math.BigInteger;
import java.math.MathContext;
import java.util.ArrayList;
import java.util.List;

public final class Value {
    /* PUBLIC */

    /** Constants */

    // Using an enum to represent types as it is more readable and easier to understand. Making the types polymorphic
    // to an abstract class makes it much more difficult to use and creates unnecessary complexity. If we were to
    // add classes and a lot more types, it would be more appropriate.
    public enum DataType {
        INTEGER,    // Whole numbers
        FLOAT,      // Decimal numbers
        BOOLEAN,    // True or false, that's it
        STRING,     // Text surrounded by quotes
        LIST,       // Collection of values of any type - can't be indexed past Java's INT_MAX
        FUNCTION,   // A body of code with parameters

        NULL        // No value, way easier to handle than Java's null
    }

    // Precision of floating point numbers, set to 16 decimal places. Needed as irrational numbers can't be represented.
    // Here to keep information together and avoid magic numbers, but also used in interpreter for calculations.
    // Help received: https://stackoverflow.com/questions/4591206/setting-the-precision-of-a-bigdecimal
    static public final MathContext PRECISION = MathContext.DECIMAL64;
    // Error message for unknown type, used in error handling
    static public final String UNKNOWN_TYPE = "Unknown value type: ";
    // Constant for our NULL to make it more clear when we are using it, instead of new Value() everywhere
    static public final Value NULL = new Value();

    /** Constructors */

    // Initialize with a value of type integer - BigInteger as we don't want overflow
    public Value(BigInteger a_int) {
        m_type = DataType.INTEGER;
        m_value = a_int;
    }

```



```

}

// Initialize with a value of type float - BigDecimal used for same reason as BigInteger (overflow)
public Value(BigDecimal a_float) {
    m_type = DataType.FLOAT;
    // Can't represent irrational numbers, so we need to round to a set precision
    m_value = a_float.round(PRECISION);
}

// Initialize with a value of type boolean
public Value(boolean a_boolean) {
    m_type = DataType.BOOLEAN;
    m_value = a_boolean;
}

// Initialize with a value of type string
public Value(String a_string) {
    m_type = DataType.STRING;
    m_value = a_string;
}

// Initialize with a value of type list
public Value(List<Value> a_list) {
    m_type = DataType.LIST;
    // Assume it's just empty if null
    if (a_list == null) { a_list = new ArrayList<>(); }
    m_value = a_list;
}

// Initialize with a function (includes its name, parameters, and body)
public Value(Function a_function) {
    m_type = DataType.FUNCTION;
    m_value = a_function;
}

// No value is our NULL value
public Value() {
    m_type = DataType.NULL;
    m_value = null;
}

/** Accessors */

// Returns the data type of the value
public DataType GetDataType() {
    return m_type;
}

// Returns if the value is of type integer
public boolean IsInteger() {
    return m_type == DataType.INTEGER;
}

```

```

// Returns if the value is of type float
public boolean IsFloat() {
    return m_type == DataType.FLOAT;
}

// Returns if the value is of type number (integer or float)
public boolean IsNumber() {
    return IsInteger() || IsFloat();
}

// Returns if the value is of type boolean
public boolean IsBoolean() {
    return m_type == DataType.BOOLEAN;
}

// Returns if the value is of type string
public boolean IsString() {
    return m_type == DataType.STRING;
}

// Returns if the value is of type list
public boolean IsList() {
    return m_type == DataType.LIST;
}

// Returns if the value is of type function
public boolean IsFunction() {
    return m_type == DataType.FUNCTION;
}

// Returns if the value is of type null
public boolean IsNull() {
    return m_type == DataType.NULL;
}

/** Methods */

// Returns the integer value, will coerce into integer if it is a float
public BigInteger AsInteger() {
    return (BigInteger) AsNumber(DataType.INTEGER);
}

// Returns the float value, will coerce into float if it is an integer
public BigDecimal AsFloat() {
    return (BigDecimal) AsNumber(DataType.FLOAT);
}

// Returns the proper Java boolean, if it is one
public boolean AsBoolean() {
    ConfirmType(DataType.BOOLEAN);
    return (boolean) m_value;
}

```

```

}

// Returns the proper Java string, if it is one. Casting is left up to FormatValueToString(), just want raw value.
public String AsString() {
    ConfirmType(DataType.STRING);
    return (String) m_value;
}

/**
 *
 Interpreter.Value.AsList()

NAME

    AsList - gets the Value as a list of Values.

SYNOPSIS

    private List<Value> AsList();

DESCRIPTION

    Returns the Value as a list of Values. If the Value is not a list, it
    will throw an assertion error. If the Value is a list, it will return a
    deep copy of the list. This is done to ensure that the list is immutable
    and that changes to the list do not affect the Value.

RETURNS

    A list of Values that is a deep copy of the list stored in the Value.

 */
/**/
public List<Value> AsList() {
    // Don't want to implicitly cast when not a list
    ConfirmType(DataType.LIST);

    List<Value> listCopy = new ArrayList<>();

    // Type erasure means that the compiler doesn't know the type of the list, so we need to check each element
    // Help received: https://stackoverflow.com/questions/339699/java-generics-type-erasure-when-and-what-happens
    for (Object element : (List<?>) m_value) {
        // Serious issue, should never be able to happen
        assert element instanceof Value : UNKNOWN_TYPE + element.getClass().getName();

        // Can safely cast to Value
        Value value = (Value) element;

        // If we have a nested list, we need to deep copy it
        if (value.GetDataType() == DataType.LIST) { listCopy.add(new Value(value.AsList())); }
        else { listCopy.add(value); }
    }
}

```

```

        return listCopy;
    }
    /* private List<Value> AsList(); */

    // Returns the function value, will throw an assertion error if it is not a function
    public Function AsFunction() {
        ConfirmType(DataType.FUNCTION);
        return (Function) m_value;
    }

    /**/
    /*
    Interpreter.Value.FormatValueToString()

```

NAME

FormatValueToString - formats the Value to a 'pretty' string to output.

SYNOPSIS

```
public String FormatValueToString();
```

DESCRIPTION

Depending on the type of the Value, it will format it to a pretty string to output. Integers and floats will be formatted as numbers, booleans as all caps, strings with quotes, lists with brackets, and functions with the word 'FUNCTION' and the function name. If the Value is null, it will be formatted as 'NULL'. This is done to make the output more readable and easier to understand.

RETURNS

A string representation of the Value that is formatted to be 'pretty'.

```

    */
    /**/
    public String FormatValueToString() {
        // Help received: IDE suggestion for switch expression
        return switch (m_type) {
            case INTEGER -> FormatInteger(AsInteger());
            case STRING -> FormatString(AsString());
            case BOOLEAN -> FormatBoolean(AsBoolean());
            case FLOAT -> FormatFloat(AsFloat());
            case LIST -> FormatList(AsList());
            case FUNCTION -> FormatFunction(AsFunction());
            case NULL -> OperationCode.NULL.LABEL;

            // Should never happen, something very wrong
            default -> throw new IllegalStateException(UNKNOWN_TYPE + m_type);
        };
    }
    /* public String FormatValueToString(); */

```

```
/**/
/*
Interpreter.Value.Equals()
```

NAME

Equals - compares the Value object to another Value object.

SYNOPSIS

```
public boolean Equals(Value a_other);
    a_other    -> the other Value to compare to.
```

DESCRIPTION

Compares the Value object to another Value object to see if they are equal to one another. Java nulls can't be compared, two numbers (integer or float) can be compared regardless of same type. Everything else must be the same type to be equal. If the Value is a list, it will recursively compare each element to see if they are equal. If the Value is a function, it will compare the references to see if they are the same.

RETURNS

True if the Value objects are equal, false otherwise. Java null are not equal to anything, so they are false.

```
*/
/**/
// Needed to be done in value since it is important for actual object comparison (makes things much easier).
public boolean Equals(Value a_other) {
    // Can't compare a Java null
    if (a_other == null) { return false; }

    // Can compare two numbers if they are the same type
    // Help received: https://stackoverflow.com/a/34677687 for comparing BigDecimals/BigIntegers
    if (this.IsNumber() && a_other.IsNumber()) { return this.AsFloat().compareTo(a_other.AsFloat()) == 0; }

    // But short circuit out if we are with other types and not numbers (using this for better distinction)
    if (this.m_type != a_other.m_type) { return false; }

    // Compare will depend based on the Values type (compareTo for numbers, equals for everything else)
    // Help received: IDE suggestion for switch expression
    return switch (this.m_type) {
        case INTEGER -> this.AsInteger().compareTo(a_other.AsInteger()) == 0;
        case FLOAT -> this.AsFloat().compareTo(a_other.AsFloat()) == 0;
        case BOOLEAN -> this.AsBoolean() == a_other.AsBoolean();
        case STRING -> this.AsString().equals(a_other.AsString());
        case LIST -> AreListsEqual(this.AsList(), a_other.AsList());
        // Equal if same reference
        case FUNCTION -> this.AsFunction().equals(a_other.AsFunction());
    };
}
```

```

        // NULL == NULL (can't be any other piece of data)
        case NULL -> true;

        // Should never be reachable, something very wrong
        default -> throw new IllegalStateException(UNKNOWN_TYPE + m_type);
    };
}
/* public boolean Equals(Value a_other); */

/* PRIVATE */

/** Members */

DataType m_type;    // The type of the value (integer, float, boolean, string, list)
Object m_value;    // The actual value, stored as object to allow for different java types

/** Private Helper Functions */

/**/
/*
Interpreter.Value.AsNumber()

NAME

    AsNumber - gets the Value as a number.

SYNOPSIS

    private Object AsNumber(DataType a_targetType);
        a_targetType    -> the type to convert the Value to.

DESCRIPTION

    Gets the Value as a number. If the Value is not a number, it will throw
    an assertion error. Depending on the target type, it will convert the
    Value to an integer or a float. If the Value is an integer, it will
    convert it to a float. If the Value is a float, it will convert it to an
    integer. This is done to allow for implicit conversion between integers
    and floats.

    Any other type will throw an assertion error as it is not a number. If
    not caught by assertion (impossible) we will throw an illegal state

RETURNS

    The Value as a number, either a BigInteger or a BigDecimal.

*/
/**/
private Object AsNumber(DataType a_targetType) {
    // Ensure we got a number

```

```

ConfirmNumberType();

// Coerce the number to the target type
// Help received: IDE suggestion for switch expression and ternary's
return switch (m_type) {
    case INTEGER -> a_targetType == DataType.INTEGER ? m_value : new BigDecimal((BigInteger) m_value);
    case FLOAT -> a_targetType == DataType.INTEGER ? ((BigDecimal) m_value).toBigInteger() : m_value;

    // Should never be reachable, something very wrong
    default -> throw new IllegalStateException("Can't coerce '" + m_type + "' to '" + a_targetType + "'");
};
}
/* private Object AsNumber(DataType a_targetType); */

/*
 *      Formatting Functions - Used to convert the value to a string for good-looking output
 */

// BigInteger.toString() works well, added as function for completion
private String FormatInteger(BigInteger a_int) {
    return a_int.toString();
}

// Floats can be shown as integers in BigDecimal class, want to ensure user knows it's a float
private String FormatFloat(BigDecimal a_float) {
    String result = a_float.stripTrailingZeros().toPlainString();
    // If it is being represented as an int (can happen with division) make it look like a float
    if (!result.contains(String.valueOf(Lexer.DECIMAL))) { return result + Lexer.DECIMAL + "0"; }
    return result;
}

// We want all caps to mimic syntax design, ensuring how we represent it in our opcodes
private String FormatBoolean(boolean a_boolean) {
    // Help received: IDE suggestion for ternary
    return a_boolean ? OperationCode.TRUE.LABEL : OperationCode.FALSE.LABEL;
}

// Add quotes to end of string according to how we represent it within syntax
private String FormatString(String a_string) {
    return OperationCode.QUOTE.LABEL + a_string + OperationCode.QUOTE.LABEL;
}

/**/
/*
Interpreter.Value.FormatList()

NAME

    FormatList - formats a list of Values to a string.

SYNOPSIS

```

```
private String FormatList(List<Value> a_list);
    a_list    -> the list of Values to format.
```

DESCRIPTION

Formats a list of Values to a string. It will add brackets to the start and end of the list, and commas and spaces between each element. If the list is empty, it will return empty brackets. If the list is not empty, it will recursively call FormatValueToString() on each element to format it to a string. This is done to make the output more readable and easier to understand.

RETURNS

A string representation of the list of Values that is formatted to be 'pretty', e.g. ["hello", 1.2, [1, TRUE], ...].

```
*/
/**/
private String FormatList(List<Value> a_list) {
    StringBuilder builder = new StringBuilder();

    // Add our list starter according to how we represent it
    builder.append(OperationCode.LBRACKET.LABEL);

    // Deep 'stringify' every element - list will recursively call
    // Will look like this: [element1, element2, ...]
    for (Value element : a_list) {
        builder.append(element.FormatValueToString());
        builder.append(OperationCode.COMMA.LABEL);
        builder.append(" ");
    }

    // Remove the last comma and space if there is one
    if (!a_list.isEmpty()) { builder.delete(builder.length() - 2, builder.length()); }

    // Same thing for ender
    builder.append(OperationCode.RBRACKET.LABEL);

    return builder.toString();
}
/* private String FormatList(List<Value> a_list); */

// Just want to show that it is a function, and the name of the function
private String FormatFunction(Function a_function) {
    return "FUNCTION '" + a_function.GetFunctionName() + "'";
}

/**/
/*
Interpreter.Value.AreListsEqual()
```

NAME

AreListsEqual - compares two lists of Values to see if they are equal.

SYNOPSIS

```
private boolean AreListsEqual(List<Value> a_left, List<Value> a_right);  
    a_left      -> the first list of Values to compare.  
    a_right     -> the second list of Values to compare.
```

DESCRIPTION

Compares two lists of Values to see if they are equal. When the lists are the same size it will recursively compare each element to see if they are equal. If at least one of the values is not equal, it will return false.

RETURN

True if the deep compare on the lists are equal, false otherwise.

```
*/  
/**/  
private boolean AreListsEqual(List<Value> a_left, List<Value> a_right) {  
    // Short circuit out if not same size  
    if (a_left.size() != a_right.size()) { return false; }  
  
    // Deep compare each value  
    for (int index = 0; index < a_left.size(); index++) {  
        // At any point if they are not equal, it means the list aren't  
        if (!a_left.get(index).Equals(a_right.get(index))) { return false; }  
    }  
    return true;  
}  
/* private boolean AreListsEqual(List<Value> a_left, List<Value> a_right); */  
  
/*  
 *      Correct Type Conversions - So developers don't do something they shouldn't  
 */  
  
// Used to tell developer not to use a type that is not the one they are expecting  
private void ConfirmType(DataType a_type) {  
    assert m_type == a_type : "Value is not a " + a_type;  
}  
  
// Confirm that it is an integer or a float, used for developers to know what type they are working with  
private void ConfirmNumberType() {  
    assert IsNumber() : "Value is not a " + DataType.INTEGER + " or a " + DataType.FLOAT;  
}  
}
```

Tests

Cases Directory

libTest.bb

```
..
Library to test the RUN command. And for some more testing.
..

' Store count as [passed, failed]
LET count = [0, 0]

' Helper function to test built-in functions
' a_message: The message of the test
' a_result: The result of the function
' a_expectedResult: The expected result of the function
' RETURN: [passed, failed]
DEF Test(a_message, a_result, a_expectedResult)
    PRINTLN("")

    ' Print the test results
    PRINT("Testing "); PRINTLN(a_message)
    PRINT("Result: "); PRINTLN(a_result)
    PRINT("Expected Result: "); PRINTLN(a_expectedResult)

    ' Check if the result is equal to the expected result
    PRINT("PASSED: ")
    IF a_result == a_expectedResult THEN
        PRINTLN("✓")
        RETURN [count:0 + 1, count:1]
    ELSE
        PRINTLN("✗")
        RETURN [count:0, count:1 + 1]
    END

END

' Print out the results
DEF PrintResults()
    PRINTLN("")
    PRINTLN("Results: ")
    PRINT("Passed: "); PRINTLN(count:0)
    PRINT("Failed: "); PRINTLN(count:1)
END
```

libRUN.bb

```
..
```

```

Library for testing RUN built-in
..
DEF add(val1, val2)
    RETURN val1 + val2
END

```

values.bb

```

..
Tests all values (integers, floats, strings, booleans, lists, and null) and assignment.
..

' Get our testing function from the library, we set count (variable to track tests results) to the result of the Test
'   function. Can't change global variables inside scope, only access them so we have to set count to the result of
'   of the return.
RUN("libTest.bb")

PRINTLN(""); PRINT("Value Tests"); PRINTLN("")

' Integer
LET count = Test("INTEGER 5", 5, 5)
LET count = Test("INTEGER -5", -5, -5)

' Float
LET count = Test("FLOAT 5.5", 5.5, 5.5)
LET count = Test("FLOAT -5.5", -5.5, -5.5)

' Boolean
LET count = Test("BOOLEAN TRUE", TRUE, TRUE)
LET count = Test("BOOLEAN FALSE", FALSE, FALSE)

' String
LET count = Test("STRING \"Hello\"", "Hello", "Hello")
LET count = Test("STRING \"Hello, World\"", "Hello, World", "Hello, World")

' List
LET count = Test("LIST [1, 2, 3]", [1, 2, 3], [1, 2, 3])
LET count = Test("LIST []", [], [])

' Simple access
LET list = [1, 2, 3]
LET string = "Hello, World"
LET count = Test("Access list:0", list:0, 1)
LET count = Test("Access list:1", list:1, 2)
LET count = Test("Access list:2", list:2, 3)
LET count = Test("Access string:0", string:0, "H")
LET count = Test("Access string:7", string:7, "W")
LET count = Test("Access string:11", string:11, "d")

```

```

' Ranged Access
LET count = Test("Ranged Access list:0|2", list:0|2, [1, 2])
LET count = Test("Ranged Access list:1|2", list:1|2, [2])
LET count = Test("Ranged Access string:0|5", string:0|5, "Hello")
LET count = Test("Ranged Access string:7|12", string:7|12, "World")

' NULL
LET count = Test("NULL", NULL, NULL)

' Assignment
LET count = Test("Assignment LET a = 5", LET a = 5, a)
LET count = Test("Assignment LET b = 5.5", LET b = 5.5, b)
LET count = Test("Assignment LET c = TRUE", LET c = TRUE, c)
LET count = Test("Assignment LET d = \"Hello\"", LET d = "Hello", d)
LET count = Test("Assignment LET e = [1, 2, 3]", LET e = [1, 2, 3], e)
LET count = Test("Assignment LET f = NULL", LET f = NULL, f)
LET count = Test("Assignment LET g", LET g, NULL)

' Assigning functions
LET count = Test("Function PRINT", LET print = PRINT, print)

PrintResults()

```

expressions.bb

```

..
Tests all expressions (binary, unary, etc.). Obviously not exhaustive, but it test every operator at least once. It
also
tests grouping and precedence, so we can assume that the operators are working correctly.
..

' Get our testing function from the library, we set count (variable to track tests results) to the result of the Test
'   function. Can't change global variables inside scope, only access them so we have to set count to the result of
'   of the return.
RUN("libTest.bb")

..
ARITHMETIC OPERATORS
..
PRINT("Arithmetic Operators"); PRINTLN("")

' Addition
LET count = Test("Adding 3 + 7", 3 + 7, 10)
LET count = Test("Adding 4.2 + 5.8", 4.2 + 5.8, 10.0)
LET count = Test("Adding 6 + 3.4", 6 + 3.4, 9.4)
LET count = Test("Adding -3 + 7", -3 + 7, 4)
LET count = Test("Adding 3 + -7", 3 + -7, -4)

' Subtraction
LET count = Test("Subtracting 9 - 4", 9 - 4, 5)
LET count = Test("Subtracting 7.5 - 2.5", 7.5 - 2.5, 5.0)

```

```

LET count = Test("Subtracting 8 - 3.2", 8 - 3.2, 4.8)
LET count = Test("Subtracting -5 - 3", -5 - 3, -8)
LET count = Test("Subtracting 5 - -3", 5 - -3, 8)

' Multiplication
LET count = Test("Multiplying 2 * 5", 2 * 5, 10)
LET count = Test("Multiplying 3.5 * 2.0", 3.5 * 2.0, 7.0)
LET count = Test("Multiplying 4 * 1.5", 4 * 1.5, 6.0)
LET count = Test("Multiplying -2 * 4", -2 * 4, -8)
LET count = Test("Multiplying 2 * -4", 2 * -4, -8)

' Division
LET count = Test("Dividing 8 / 2", 8 / 2, 4.0)
LET count = Test("Dividing 9.0 / 3.0", 9.0 / 3.0, 3.0)
LET count = Test("Dividing 7 / 2.5", 7 / 2.5, 2.8)
LET count = Test("Dividing -6 / 3", -6 / 3, -2)
LET count = Test("Dividing 6 / -3", 6 / -3, -2)

' Modulus
LET count = Test("Modulus 10 % 3", 10 % 3, 1)
LET count = Test("Modulus 7.5 % 2.5", 7.5 % 2.5, 0.0)
LET count = Test("Modulus 9 % 2.5", 9 % 2.5, 1.5)
LET count = Test("Modulus -10 % 3", -10 % 3, -1)
LET count = Test("Modulus 10 % -3", 10 % -3, 1)

' Exponentiation
LET count = Test("Exponentiating 3 ^ 2", 3 ^ 2, 9)
LET count = Test("Exponentiating 2.5 ^ 3.0", 2.5 ^ 3.0, 15.625)
LET count = Test("Exponentiating 4 ^ 1.5", 4 ^ 1.5, 8.0)
LET count = Test("Exponentiating -2 ^ 3", -2 ^ 3, -8)
LET count = Test("Exponentiating 2 ^ -3", 2 ^ -3, 0)

' Grouping
LET count = Test("Grouping (3 + 4) * 2", (3 + 4) * 2, 14)
LET count = Test("Grouping 3 + (4 * 2)", 3 + (4 * 2), 11)
LET count = Test("Grouping (2 + 3) * 4 + 1", (2 + 3) * 4 + 1, 21)
LET count = Test("Grouping 2 + (3 * 4) + 1", 2 + (3 * 4) + 1, 15)
LET count = Test("Grouping 1 + 2 * (3 + 4)", 1 + 2 * (3 + 4), 15)
LET count = Test("Grouping 1 + 2 * 3 + 4", 1 + 2 * 3 + 4, 11)

PRINTLN("")

' Unary minus
LET count = Test("Unary minus -3", -3, -3)
LET count = Test("Unary minus -4.5", -4.5, -4.5)
LET count = Test("Unary minus -2 + 5", -2 + 5, 3)
LET count = Test("Unary minus -3.5 + 2.5", -3.5 + 2.5, -1.0)
LET count = Test("Unary minus -4 - 1", -4 - 1, -5)

' Unary plus
LET count = Test("Unary plus +3", +3, 3)
LET count = Test("Unary plus +4.5", +4.5, 4.5)
LET count = Test("Unary plus +2 + 5", +2 + 5, 7)

```

```

LET count = Test("Unary plus +3.5 + 2.5", +3.5 + 2.5, 6.0)
LET count = Test("Unary plus +4 + 1", +4 + 1, 5)

' Unary logical NOT
LET count = Test("Unary logical NOT TRUE", NOT TRUE, FALSE)
LET count = Test("Unary logical NOT FALSE", NOT FALSE, TRUE)

..

COMPARISON OPERATORS
..

PRINTLN(""); PRINTLN("Comparison Operators"); PRINTLN("")

' Equal
LET count = Test("3 == 3", 3 == 3, TRUE)
LET count = Test("4.5 == 4.5", 4.5 == 4.5, TRUE)
LET count = Test("3 == 4.5", 3 == 4.5, FALSE)
LET count = Test("Hello == Hello", "Hello" == "Hello", TRUE)
LET count = Test("[1, 2, 3] == [1, 2, 3]", [1, 2, 3] == [1, 2, 3], TRUE)

' Not equal
LET count = Test("3 != 4", 3 != 4, TRUE)
LET count = Test("4.5 != 5.5", 4.5 != 5.5, TRUE)
LET count = Test("3 != 4.5", 3 != 4.5, TRUE)
LET count = Test("Hello != World", "Hello" != "World", TRUE)
LET count = Test("[1, 2, 3] != [1, 2, 4]", [1, 2, 3] != [1, 2, 4], TRUE)

' Less than
LET count = Test("3 < 4", 3 < 4, TRUE)
LET count = Test("4.5 < 5.5", 4.5 < 5.5, TRUE)
LET count = Test("3 < 4.5", 3 < 4.5, TRUE)

' Less than or equal
LET count = Test("3 <= 4", 3 <= 4, TRUE)
LET count = Test("4.5 <= 5.5", 4.5 <= 5.5, TRUE)
LET count = Test("3 <= 4.5", 3 <= 4.5, TRUE)

' Greater than
LET count = Test("3 > 4", 3 > 4, FALSE)
LET count = Test("4.5 > 5.5", 4.5 > 5.5, FALSE)
LET count = Test("3 > 4.5", 3 > 4.5, FALSE)

' Greater than or equal
LET count = Test("3 >= 4", 3 >= 4, FALSE)
LET count = Test("4.5 >= 5.5", 4.5 >= 5.5, FALSE)
LET count = Test("3 >= 4.5", 3 >= 4.5, FALSE)

..

LOGICAL OPERATORS
..

PRINTLN(""); PRINTLN("Logical Operators"); PRINTLN("")

```

```

' Logical AND
LET count = Test("TRUE AND TRUE", TRUE AND TRUE, TRUE)
LET count = Test("TRUE AND FALSE", TRUE AND FALSE, FALSE)
LET count = Test("FALSE AND TRUE", FALSE AND TRUE, FALSE)
LET count = Test("FALSE AND FALSE", FALSE AND FALSE, FALSE)

' Logical OR
LET count = Test("TRUE OR TRUE", TRUE OR TRUE, TRUE)
LET count = Test("TRUE OR FALSE", TRUE OR FALSE, TRUE)
LET count = Test("FALSE OR TRUE", FALSE OR TRUE, TRUE)
LET count = Test("FALSE OR FALSE", FALSE OR FALSE, FALSE)

' Logical NOT
LET count = Test("NOT TRUE", NOT TRUE, FALSE)
LET count = Test("NOT FALSE", NOT FALSE, TRUE)

PrintResults()

```

statements.bb

```

''
Tests all statements (if, while, etc.). Once again not exhaustive, but it tests all the different types of statements.
It also tests nesting and breaking out of loops.
''

' Get our testing function from the library, we set count (variable to track tests results) to the result of the Test
'   function. Can't change global variables inside scope, only access them so we have to set count to the result of
'   of the return.
RUN("libTest.bb")

''

IF STATEMENTS
''

PRINTLN(""); PRINT("If Statements"); PRINTLN("")

' Simple if
LET result
IF TRUE THEN
    LET result = TRUE
END
LET count =Test("Simple if TRUE branch", result, TRUE)

LET result = FALSE
IF FALSE THEN
    LET result = TRUE
END
LET count = Test("Simple if FALSE branch", result, FALSE)

' If-else
LET result
IF TRUE THEN
    LET result = TRUE

```

```

ELSE
    LET result = FALSE
END
LET count = Test("If-else TRUE branch", result, TRUE)

LET result
IF FALSE THEN
    LET result = TRUE
ELSE
    LET result = FALSE
END
LET count = Test("If-else FALSE branch", result, FALSE)

' Nested if
LET result
IF (TRUE) THEN
    IF (TRUE) THEN
        LET result = TRUE
    END
END
LET count = Test("Nested if TRUE branch", result, TRUE)

LET result
IF TRUE THEN
    IF (FALSE) THEN
        LET result = TRUE
    ELSE
        LET result = FALSE
    END
END
LET count = Test("Nested if FALSE branch", result, FALSE)

' If-else-if
LET result
IF FALSE THEN
    LET result = FALSE
ELSEIF TRUE THEN
    LET result = TRUE
ELSE
    LET result = FALSE
END
LET count = Test("If-else-if TRUE branch", result, TRUE)

' Multiple else-if chain
LET result
IF FALSE THEN
    LET result = FALSE
ELSEIF FALSE THEN
    LET result = FALSE
ELSEIF TRUE THEN

```



```

    LET result = TRUE
ELSE
    LET result = FALSE
END
LET count = Test("Multiple else-if chain TRUE branch", result, TRUE)

```

```

''
WHILE LOOPS
''
PRINTLN(""); PRINT("While Loops"); PRINTLN("")

```

```

' Simple while
LET i = 0
WHILE i < 3 THEN

    LET i = i + 1
END
LET count = Test("Simple while", i, 3)

```

```

' While with break
LET i = 0
WHILE (i < 3) THEN
    IF (i == 1) THEN
        BREAK
    END
    LET i = i + 1
END
LET count = Test("While with break", i, 1)

PRINTLN("")

```

```

' While with continue
LET i = 0
WHILE (i < 3) THEN
    LET i = i + 1
    IF (i == 2) THEN
        CONTINUE
    END
END
LET count = Test("While with continue", i, 3)

```

```

''
FOR LOOPS
''
PRINTLN(""); PRINT("For Loops"); PRINTLN("")

```

```

' Simple for
FOR LET i = 0 TO 2 STEP 1 THEN
    CONTINUE
END

```

```

LET count = Test("Simple for", i, 2)

' For with break
FOR LET i = 0 TO 2 STEP 1 THEN
    IF (i == 1) THEN
        BREAK
    END
END
LET count = Test("For with break", i, 1)

' For with continue
FOR LET i = 0 TO 2 STEP 1 THEN
    IF (i == 1) THEN
        CONTINUE
    END
END
LET count = Test("For with continue", i, 2)

' Nested
LET result = 0
FOR LET i = 0 TO 2 STEP 1 THEN
    FOR LET j = 0 TO 2 STEP 1 THEN
        LET result = result + i + j
    END
END
LET count = Test("Nested for", result, 18)

' Show results
PrintResults()

```

functions.bb

```

..

Tests functions. Though they are statements, they are large enough to test on
their own. Once again not exhaustive, but it tests all the different types of
functions. It also tests recursion and early returns.
..

' Get our testing function from the library, we set count (variable to track tests results) to the result of the Test
'   function. Can't change global variables inside scope, only access them so we have to set count to the result of
'   of the return.
RUN("libTest.bb")

PRINT("Function Tests"); PRINTLN("")

' Function definition
DEF Add(a, b)
    RETURN a + b
END

' Function call

```

```

LET count = Test("Add(3, 4)", Add(3, 4), 7)

' Function call with expression
LET count = Test("Add(3 + 4, 5)", Add(3 + 4, 5), 12)

' Function call with function call
LET count = Test("Add(Add(3, 4), 5)", Add(Add(3, 4), 5), 12)

' Recursive function
DEF Factorial(n)
  IF n == 0 THEN
    RETURN 1
  ELSE
    RETURN n * Factorial(n - 1)
  END
END
LET count = Test("Factorial(5)", Factorial(5), 120)

' Function with no return
DEF NoReturn()
  PRINT("NoReturn() executed")
END
LET count = Test("NoReturn()", NoReturn(), NULL)

' Returning a function
DEF ReturnFunction()
  RETURN Add
END
LET count = Test("(ReturnFunction())(3, 4)", (ReturnFunction())(3, 4), 7)

' Nested function
DEF Outer()
  DEF Inner()
    PRINTLN("Inner() executed")
  END
  PRINTLN("Outer() executed")
  Inner()
END
LET count = Test("Outer()", Outer(), NULL)

' Calling other functions defined previously in function body
DEF Other()
  RETURN Add(3, 4)
END
LET count = Test("Other(), calling Add(3, 4) inside", Other(), 7)

' Returning early
DEF EarlyReturn()
  RETURN 5
  PRINT("This should not be printed")

```

```

END
LET count = Test("EarlyReturn()", EarlyReturn(), 5)

' Nested if in function
DEF NestedIf()
  IF TRUE THEN
    IF TRUE THEN
      RETURN TRUE
    END
  END
END
LET count = Test("NestedIf()", NestedIf(), TRUE)

PrintResults()

```

built-ins.bb

```

''
Tests all the built-in functions within the language. Also will test built-in value keywords. Obviously not exhaustive,
but it tests every built-in function with various types of arguments.
''

' Get our testing function from the library, we set count (variable to track tests results) to the result of the Test
'   function. Can't change global variables inside scope, only access them so we have to set count to the result of
'   of the return.
RUN("libTest.bb")

PRINT("Built-in Functions Tests"); PRINTLN("")

' IS_INTEGER function
LET count = Test("IS_INTEGER(5)", IS_INTEGER(5), TRUE)
LET count = Test("IS_INTEGER(5.5)", IS_INTEGER(5.5), FALSE)
LET count = Test("IS_INTEGER(TRUE)", IS_INTEGER(TRUE), FALSE)
LET count = Test("IS_INTEGER([1, 2, 3])", IS_INTEGER([1, 2, 3]), FALSE)
LET count = Test("IS_INTEGER(NULL)", IS_INTEGER(NULL), FALSE)
LET count = Test("IS_INTEGER(\"5\")", IS_INTEGER("5"), FALSE)

' IS_FLOAT function
LET count = Test("IS_FLOAT(5)", IS_FLOAT(5), FALSE)
LET count = Test("IS_FLOAT(5.5)", IS_FLOAT(5.5), TRUE)
LET count = Test("IS_FLOAT(TRUE)", IS_FLOAT(TRUE), FALSE)
LET count = Test("IS_FLOAT([1, 2, 3])", IS_FLOAT([1, 2, 3]), FALSE)
LET count = Test("IS_FLOAT(NULL)", IS_FLOAT(NULL), FALSE)
LET count = Test("IS_FLOAT(\"5.5\")", IS_FLOAT("5.5"), FALSE)

' IS_NUMBER function
LET count = Test("IS_NUMBER(5)", IS_NUMBER(5), TRUE)
LET count = Test("IS_NUMBER(5.5)", IS_NUMBER(5.5), TRUE)
LET count = Test("IS_NUMBER(TRUE)", IS_NUMBER(TRUE), FALSE)
LET count = Test("IS_NUMBER([1, 2, 3])", IS_NUMBER([1, 2, 3]), FALSE)

```

```

LET count = Test("IS_NUMBER(NULL)", IS_NUMBER(NULL), FALSE)
LET count = Test("IS_NUMBER(\"5\")", IS_NUMBER("5"), FALSE)

' IS_STRING function
LET count = Test("IS_STRING(\"Hello\")", IS_STRING("Hello"), TRUE)
LET count = Test("IS_STRING(5)", IS_STRING(5), FALSE)
LET count = Test("IS_STRING(5.5)", IS_STRING(5.5), FALSE)
LET count = Test("IS_STRING(TRUE)", IS_STRING(TRUE), FALSE)
LET count = Test("IS_STRING([1, 2, 3])", IS_STRING([1, 2, 3]), FALSE)
LET count = Test("IS_STRING(NULL)", IS_STRING(NULL), FALSE)

' IS_BOOLEAN function
LET count = Test("IS_BOOLEAN(TRUE)", IS_BOOLEAN(TRUE), TRUE)
LET count = Test("IS_BOOLEAN(FALSE)", IS_BOOLEAN(FALSE), TRUE)
LET count = Test("IS_BOOLEAN(5)", IS_BOOLEAN(5), FALSE)
LET count = Test("IS_BOOLEAN(5.5)", IS_BOOLEAN(5.5), FALSE)
LET count = Test("IS_BOOLEAN(\"Hello\")", IS_BOOLEAN("Hello"), FALSE)
LET count = Test("IS_BOOLEAN([1, 2, 3])", IS_BOOLEAN([1, 2, 3]), FALSE)
LET count = Test("IS_BOOLEAN(NULL)", IS_BOOLEAN(NULL), FALSE)

' IS_LIST function
LET count = Test("IS_LIST([1, 2, 3])", IS_LIST([1, 2, 3]), TRUE)
LET count = Test("IS_LIST()", IS_LIST(), TRUE)
LET count = Test("IS_LIST(5)", IS_LIST(5), FALSE)
LET count = Test("IS_LIST(5.5)", IS_LIST(5.5), FALSE)
LET count = Test("IS_LIST(TRUE)", IS_LIST(TRUE), FALSE)
LET count = Test("IS_LIST(\"Hello\")", IS_LIST("Hello"), FALSE)
LET count = Test("IS_LIST(NULL)", IS_LIST(NULL), FALSE)

' IS_FUNCTION function
LET count = Test("IS_FUNCTION(PRINT)", IS_FUNCTION(PRINT), TRUE)
LET count = Test("IS_FUNCTION(5)", IS_FUNCTION(5), FALSE)
LET count = Test("IS_FUNCTION(5.5)", IS_FUNCTION(5.5), FALSE)
LET count = Test("IS_FUNCTION(TRUE)", IS_FUNCTION(TRUE), FALSE)
LET count = Test("IS_FUNCTION(\"Hello\")", IS_FUNCTION("Hello"), FALSE)
LET count = Test("IS_FUNCTION([1, 2, 3])", IS_FUNCTION([1, 2, 3]), FALSE)
LET count = Test("IS_FUNCTION(NULL)", IS_FUNCTION(NULL), FALSE)

' IS_NULL function
LET count = Test("IS_NULL(NULL)", IS_NULL(NULL), TRUE)
LET count = Test("IS_NULL(5)", IS_NULL(5), FALSE)
LET count = Test("IS_NULL(5.5)", IS_NULL(5.5), FALSE)
LET count = Test("IS_NULL(TRUE)", IS_NULL(TRUE), FALSE)
LET count = Test("IS_NULL(\"Hello\")", IS_NULL("Hello"), FALSE)

' TO_INTEGER function
LET count = Test("TO_INTEGER(\"5\")", TO_INTEGER("5"), 5)
LET count = Test("TO_INTEGER(5.5)", TO_INTEGER(5.5), 5)
LET count = Test("TO_INTEGER(5)", TO_INTEGER(5), 5)

' TO_FLOAT function
LET count = Test("TO_FLOAT(\"5.5\")", TO_FLOAT("5.5"), 5.5)
LET count = Test("TO_FLOAT(5)", TO_FLOAT(5), 5.0)

```

```

LET count = Test("TO_FLOAT(1.0)", TO_FLOAT(1.0), 1.0)

' TO_STRING function
LET count = Test("TO_STRING(5)", TO_STRING(5), "5")
LET count = Test("TO_STRING(5.5)", TO_STRING(5.5), "5.5")
LET count = Test("TO_STRING(TRUE)", TO_STRING(TRUE), "TRUE")
LET count = Test("TO_STRING([1, 2, 3])", TO_STRING([1, 2, 3]), "[1, 2, 3]")
LET count = Test("TO_STRING(NULL)", TO_STRING(NULL), "NULL")
LET count = Test("TO_STRING(PRINT)", TO_STRING(PRINT), "FUNCTION 'PRINT'")

' APPEND function
LET count = Test("APPEND([1, 2], 3)", APPEND([1, 2], 3), [1, 2, 3])
LET count = Test("APPEND([], 1)", APPEND([], 1), [1])

' PREPEND function
LET count = Test("PREPEND([2, 3], 1)", PREPEND([2, 3], 1), [1, 2, 3])
LET count = Test("PREPEND([], 1)", PREPEND([], 1), [1])

' INSERT function
LET count = Test("INSERT([1, 3], 1, 2)", INSERT([1, 3], 1, 2), [1, 3, 1])
LET count = Test("INSERT([1, 2], 3, 2)", INSERT([1, 2], 3, 2), [1, 2, 3])

' POP function
LET count = Test("POP([1, 2, 3])", POP([1, 2, 3]), [1, 2])
LET count = Test("POP([1])", POP([1]), [])

' REMOVE function
LET count = Test("REMOVE([1, 2, 3], 1)", REMOVE([1, 2, 3], 1), [1, 3])
LET count = Test("REMOVE([1, 2, 3], 2)", REMOVE([1, 2, 3], 2), [1, 2])

' EXTEND function
LET count = Test("EXTEND([1, 2], [3, 4])", EXTEND([1, 2], [3, 4]), [1, 2, 3, 4])
LET count = Test("EXTEND([], [1, 2])", EXTEND([], [1, 2]), [1, 2])

' LENGTH function
LET count = Test("LENGTH([1, 2, 3])", LENGTH([1, 2, 3]), 3)
LET count = Test("LENGTH([])", LENGTH([], 0))

LET message = "Hello, World!"
LET path = "test.txt"

' WRITE function
LET count = Test("WRITE(" + path + ", " + message + ")", WRITE(path, message), NULL)

' READ function
LET count = Test("READ(" + path + ")", READ(path), message)

' INPUT function
PRINTLN("")
PRINTLN("Testing INPUT function, please enter a value:")
LET count = Test("INPUT()", LET in = INPUT(), in)

```

```

LET path = "libRUN.bb"
' RUN function
LET count = Test("RUN(" + path + ")", RUN(path), NULL)
LET count = Test("From libRUN.bb: add(1, 2)", add(1, 2), 3)

' PRINT and PRINTLN functions have clearly been tested in libTest.bb

' Show how many we passed and failed (should be all passed)
PrintResults()

```

Demo Directory

live1.bb

```

..
Everything we do in the CLI we can do in a file!

Lets calculate the letter grade based on a score
..

' Or calculate the letter grade based on input
DEF letter_grade(score)
  IF NOT IS_INTEGER(score) THEN
    PRINTLN("Incorrect input, score must be an integer!")
    RETURN
  END

  IF score >= 90 THEN
    RETURN "A"
  ELSEIF score >= 80 THEN
    RETURN "B"
  ELSEIF score >= 70 THEN
    RETURN "C"
  ELSEIF score >= 60 THEN
    RETURN "D"
  ELSE
    RETURN "F"
  END
END

PRINTLN("Enter your score to get your letter grade")
LET input = INPUT()
LET grade = letter_grade(TO_INTEGER(input))
PRINTLN("Based on your score of " + input + ", you got a " + grade)

```

live2.bb

```

..
Calculating fibonacci using recursion and iteration

Formula = Fn = Fn-1 + Fn-2, where n >= 0

```

```

        if n = 0, result = 0
        if n = 2, 3, result = 1
    ..

' Get the users input and cast to an integer
PRINTLN("Enter you input value for Fibonacci")
LET n = TO_INTEGER(INPUT()) - 1

' Fibonacci with recursion
' n -> input number to get to
DEF fib(n)
    ' Formula doesn't allow for values less than 0
    IF n < 0 THEN
        PRINTLN("Incorrect input, n must be larger than 0!")
    ' Base case, 1 or 2 means result is 1
    ELSEIF n < 2 THEN
        RETURN 1
    END

    ' Recursive case, fib(n-1) + fib(n-2)
    RETURN fib(n - 1) + fib(n - 2)
END

PRINTLN("Result of Fibonacci with recursion: " + TO_STRING(fib(n)))

' Now with iteration
' Our previous numbers to add to each other
LET lhs = 0
LET rhs = 1
' Next number to calculate
LET next = rhs

' Iterate till we hit n
FOR LET count = 1 TO n STEP 1 THEN
    LET lhs = rhs
    LET rhs = next

    LET next = lhs + rhs
END
PRINTLN("Result of Fibonacci with iteration: " + TO_STRING(next))

```

live3.bb

```

..
Let's do some operations on the 2D list
..
LET list = [[10, 10, 10, 10, 10],
            [10, 10, 10, 10],
            [10, 10, 9],
            [],
            [10, 10]]

```



```

DEF sum_no_nines(a_2d_list)
  LET sum = 0
  ' Go through each outer list
  FOR LET i = 0 TO LENGTH(a_2d_list) - 1 STEP 1 THEN

    ' Perhaps a break? Lets break after we encounter an empty list
    IF LENGTH(a_2d_list:i) == 0 THEN
      BREAK
    END

    ' Go through each element in the inner and add
    FOR LET j = 0 TO LENGTH(a_2d_list:i) - 1 STEP 1 THEN
      ' Lets see some continue action
      IF a_2d_list:i:j == 9 THEN
        CONTINUE
      END
      LET sum = sum + a_2d_list:i:j
    END
  END

  END

  RETURN sum
END

```

```

PRINTLN("The sum of the 2D list is: " + TO_STRING(sum_no_nines(list)))

```

live4.bb

```

''

```

Let's do a binary search on a sorted list

Algorithm:

1. Set low to 0 and high to the length of the list - 1
 2. While low is less than or equal to high
 - a. Calculate the mid point
 - b. If the mid point is the target, return the index
 - c. If the mid point is less than the target, set low to mid + 1
 - d. If the mid point is greater than the target, set high to mid - 1
 3. Return -1 if the target is not found
- ```

''

```

```

DEF binary_search(a_list, target)
 LET low = 0
 LET high = LENGTH(a_list) - 1
 ' While low is less than or equal to high
 WHILE low <= high THEN
 ' Calculate the mid point
 LET mid = low + (high - low) / 2

 ' If the mid point is the target, return the index
 IF a_list:mid == target THEN

```

```

 RETURN mid

 ' If the mid point is less than the target, set low to mid + 1
 ELSEIF a_list:mid < target THEN
 LET low = mid + 1

 ' If the mid point is greater than the target, set high to mid - 1
 ELSE
 LET high = mid - 1
 END
END

' Return -1 if the target is not found
RETURN -1
END

' Lets test the binary search
LET list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
LET target = 5
LET index = binary_search(list, target)
PRINTLN("The index of " + TO_STRING(target) + " is " + TO_STRING(index))

LET target = 11
LET index = binary_search(list, target)
PRINTLN("The index of " + TO_STRING(target) + " is " + TO_STRING(index))

```

## ***live5.bb***

```

''
Lets reverse a composite value
''

' We're going to use an external library for this one with the 'RUN' command
RUN("reverse.bb")

' Works on strings
LET duck = "quack"
LET str = "Reversed string " + duck + ": " + reverse_string(duck)

' Now lets write it to a file
WRITE("reversed.txt", str + "\n We just wrote it to a file")

' And print it out what we've read
PRINTLN(READ("reversed.txt"))

```

## ***reverse.bb***

```

''
Reversing a string
''

```

```
DEF reverse_string(a_composite)
 LET reversed = a_composite:0

 ' Loop through each element, and prepend to the reversed string
 FOR LET i = 1 TO LENGTH(a_composite) STEP 1 THEN
 LET reversed = a_composite:i + reversed
 END

 ' Return our result
 RETURN reversed
END
```

## VSCode Extension

### *extension.js*

```
const vscode = require('vscode');
const path = require('path');

// Activates the extension, more specifically the run button in the status bar
// Help received from:
// https://code.visualstudio.com/api/get-started/your-first-extension
// https://www.youtube.com/watch?v=cHQo26fdx_o
function activate() {
 let runCommand = vscode.commands.registerCommand('runbbasic.run', function () {
 // Ensure we are in a valid workspace
 const editor = vscode.window.activeTextEditor;
 if (editor) {
 // Get the name of the file and the directory it is in
 const filePath = editor.document.fileName;
 const directory = path.dirname(filePath);
 const document = path.basename(editor.document.fileName);

 // Create a new terminal to run from
 const terminal = vscode.window.createTerminal('BasicallyBASIC');
 // Enter the right directory and run the file
 terminal.sendText(`cd "${directory}"`);
 terminal.sendText(`bbasic ${document}`);
 terminal.show();
 }
 });
}
```

### *bbasic.tmLanguage.json*

```
{
 "$schema": "https://raw.githubusercontent.com/martinring/tmlanguage/master/tmlanguage.json",
 "name": "BasicallyBASIC",
 "patterns": [
 {
 "include": "#keywords"
 },
 {
 "include": "#strings"
 },
 {
 "include": "#comments"
 },
 {
 "include": "#numbers"
 },
 {
 "include": "#operators"
 }
],
```

```

 {
 "include": "#identifiers"
 }
],
 "repository": {
 "keywords": {
 "patterns": [
 {
 "name": "keyword.control.bbasic",
 "match": "\\b(IF|WHILE|FOR|ELSEIF|ELSE|END|DEF|BREAK|CONTINUE|THEN|STEP|RETURN)\\b"
 },
 {
 "name": "keyword.declaration.bbasic",
 "match": "\\b(LET)\\b"
 },
 {
 "name": "keyword.other.bbasic",
 "match": "\\b(TRUE|FALSE|NULL|AND|OR|NOT)\\b"
 }
]
 },
 "strings": {
 "name": "string.quoted.double.bbasic",
 "begin": "\"",
 "end": "\"",
 "patterns": [
 {
 "name": "constant.character.escape.bbasic",
 "match": "\\\\"
 }
]
 },
 "comments": {
 "patterns": [
 {
 "name": "comment.block.bbasic",
 "begin": "'",
 "end": "'"
 },
 {
 "name": "comment.line.bbasic",
 "match": ".*"
 },
 {
 "name": "punctuation.terminator.bbasic",
 "match": ";"
 }
]
 },
 "numbers": {
 "patterns": [
 {
 "name": "constant.numeric.integer.bbasic",

```

```

 "match": "\\b-?\\d+\\b"
 },
 {
 "name": "constant.numeric.float.bbasic",
 "match": "\\b-?\\d+\\.\\d+\\b"
 }
]
 },
 "operators": {
 "patterns": [
 {
 "name": "keyword.operator.comparison.bbasic",
 "match": "(=|!=|>|=|<|>|=)"
 },
 {
 "name": "keyword.operator.arithmetic.bbasic",
 "match": "(\\+|\\-|*|\\/|\\^)"
 }
]
 },
 "identifiers": {
 "patterns": [
 {
 "name": "entity.name.function.bbasic",
 "match": "\\b[a-zA-Z_][a-zA-Z0-9_]*\\s*\\("
 },
 {
 "name": "variable.other.bbasic",
 "match": "\\b[a-zA-Z_][a-zA-Z0-9_]*\\b"
 }
]
 }
},
"scopeName": "source.bb"
}

```

## ***language-configuration.json***

```

{
 "comments": {
 "lineComment": "//",
 "blockComment": ["/*", "*/"]
 },
 "brackets": [
 ["[", "]"],
 ["(", ")"]
],
 "autoClosingPairs": [
 ["[", "]"],
 ["(", ")"],
 ["\"", "\""],
 ["'", "'"]
],

```

```

 "surroundingPairs": [
 ["[", "]"],
 ["(", ")"],
 ["\\", "\\"],
 ["'", "'"]
]
 }
}

```

## package.json

```

{
 "name": "bb",
 "displayName": "BasicallyBASIC Enhanced",
 "description": "Syntax highlighting and running BasicallyBASIC files",
 "icon": "icon128x128.png",
 "version": "1.0.0",
 "publisher": "Ryan Podzielny",
 "engines": {
 "vscode": "^1.93.0"
 },
 "categories": [
 "Programming Languages"
],
 "main": "./out/extension",
 "activationEvents": [
 "onCommand:runbbasic.run"
],
 "contributes": {
 "languages": [{
 "id": "bbasic",
 "aliases": ["BasicallyBASIC", "bbasic"],
 "extensions": [".bb"],
 "configuration": "./language-configuration.json"
 }],
 "grammars": [{
 "language": "bbasic",
 "scopeName": "source.bb",
 "path": "./syntaxes/bbasic.tmLanguage.json"
 }],
 "commands": [
 {
 "command": "runbbasic.run",
 "title": "Run BasicallyBASIC File",
 "icon": "$(play)"
 }
],
 "menus": {
 "editor/title": [
 {
 "command": "runbbasic.run",
 "when": "resourceLangId==bbasic",
 "group": "navigation"
 }
]
 }
 }
}

```

}  
}  
}  
]